
OpenSTREAM Documentation

*Open Solvers for Two-phase flow Research,
Engineering Analysis and Modeling*

Sep 09, 2026



© 2024–2026 The OpenSTREAM Team
Licensed under the MIT License
<https://github.com/OpenSTREAM-solvers/openstream>

CONTENTS

1	Welcome	1
1.1	What's inside?	1
1.2	Why OpenSTREAM?	1
1.3	Links	1
1.4	About this documentation	2
2	Usage	3
2.1	Getting started	3
2.1.1	Prerequisites	3
2.1.2	Install OpenSTREAM	3
2.1.3	MATLAB and Python compatibility	4
2.1.4	Configure MATLAB and Python	4
2.1.5	Install the Python dependencies	6
2.1.6	Verify the installation	6
2.1.7	Next steps	7
2.2	Run sample script	7
2.3	Intended scope and limitations	8
2.3.1	Simulation frameworks	9
2.3.2	Dimensional scope	9
2.3.3	Geometric scope	10
2.3.4	Fluid and flow scope	10
2.3.5	Flow regimes	11
2.3.6	Thermal treatment	11
2.3.7	Closure-model dependence	11
2.3.8	Numerical formulation	12
2.3.9	Numerical sensitivity	13
2.3.10	Convergence	13
2.3.11	Steady-state and transient calculations	13
2.3.12	Validation status	14
2.3.13	Verification and testing	14
2.3.14	Uncertainty	15
2.3.15	Open research software	15
2.3.16	Intended use	15
2.3.17	Reproducibility of reported results	16
2.3.18	User responsibilities	17
2.3.19	Further information	17
2.4	Release numbering and versioning	18
2.4.1	Release identifiers	18
2.4.2	Prerelease identifiers	18
2.4.3	Stable and development versions	19

2.4.4	Release documentation	19
2.4.5	Compatibility status	20
2.4.6	Tested configurations	20
2.4.7	Inspecting the active environment	20
2.4.8	Continuous integration	21
2.4.9	OpenSTREAM-database compatibility	22
2.4.10	Saved objects and persisted results	22
2.4.11	Backward compatibility	23
2.4.12	Deprecation	23
2.4.13	Reproducibility	23
2.4.14	Citing a release	24
2.4.15	Migration between versions	24
2.4.16	Development and release responsibilities	24
3	Guides	27
3.1	Theory overview	27
3.1.1	Solver capabilities	27
3.1.2	Detailed theory pages	28
3.2	Generated files and session directories	40
3.2.1	Input files	41
3.2.2	Session directories	41
3.2.3	Overwriting an existing session	42
3.2.4	Logging	42
3.2.5	Saving solver objects	43
3.2.6	OpenSTREAM-database outputs	43
3.2.7	Sandbox calculations	44
3.2.8	Test outputs	44
3.2.9	Version-control policy	44
3.2.10	Moving or deleting generated files	45
3.2.11	Recommended workflow	45
3.2.12	Troubleshooting	45
3.2.13	Further information	46
3.3	Glossary	46
3.3.1	Recommended terminology	49
3.3.2	Style notes	49
3.4	Notation	50
3.4.1	General conventions	50
3.4.2	Primary variables	50
3.4.3	Phase and field subscripts	51
3.4.4	Source-term notation	51
3.4.5	Closure model notation	51
3.4.6	Units	51
3.5	Global reference list	52
3.6	Testing and verification	52
3.6.1	Test organization	53
3.6.2	Running all tests	53
3.6.3	Running individual test categories	53
3.6.4	Continuous integration	54
3.6.5	Reference solutions	55
3.6.6	Adding new tests	55
3.6.7	Unit test guidelines	55
3.6.8	Integration test guidelines	56
3.6.9	Environment validation	56
3.6.10	Generated outputs	57

3.6.11	Troubleshooting	57
3.6.12	Contributor checklist	57
3.7	Troubleshooting	58
3.7.1	Initial diagnostic checklist	58
3.7.2	MATLAB cannot locate OpenSTREAM	59
3.7.3	MATLAB cannot locate OpenSTREAM-database	59
3.7.4	Python cannot be loaded	60
3.7.5	CoolProp cannot be imported	60
3.7.6	Environment tests fail	60
3.7.7	An input file cannot be read	61
3.7.8	An input identifier cannot be found	61
3.7.9	OpenSTREAM reports inconsistent or missing inputs	61
3.7.10	A session directory already exists	62
3.8	Tutorials	62
3.8.1	1. Quick Start	62
3.8.2	2. Mixture Solver Capabilities	63
3.8.3	3. Two-Fluid Solver Capabilities	63
3.8.4	4. Three-Field Solver Capabilities	63
3.8.5	5. Four-Field Solver Capabilities	63
3.8.6	6. Input Files	63
3.8.7	7. Numerical Convergence and Solver Diagnostics	63
3.8.8	8. Mesh and Time-Step Sensitivity	63
3.8.9	9. Advanced Visualization and Post-Processing	64
4	Applications	65
4.1	Contributing to the database	65
4.1.1	Types of contributions	65
4.1.2	Dataset organization	66
4.1.3	Dataset template	66
4.1.4	Naming convention	67
4.1.5	Source data	69
4.1.6	Lightweight datasets	70
4.1.7	Dataset fields	70
4.1.8	Data provenance	72
4.1.9	Dataset class	73
4.1.10	OpenSTREAM dependency	74
4.1.11	Application projects	74
4.1.12	Comparison and validation	75
4.1.13	Plotting and post-processing	76
4.1.14	Testing and verification	76
4.1.15	Documentation requirements	77
4.1.16	Commit messages	78
4.1.17	Submitting a contribution	78
4.1.18	Contributor checklist	79
4.2	OpenSTREAM-database	80
4.2.1	Purpose	80
4.2.2	Repository organization	80
4.2.3	Dataset source files	81
4.2.4	Lightweight in-memory datasets	82
4.2.5	Units	82
4.2.6	Data preparation and provenance	82
4.2.7	Application projects	83
4.2.8	Requirements	83
4.2.9	Installation	84

4.2.10	MATLAB setup	84
4.2.11	Typical workflow	84
4.2.12	Generated files	85
4.2.13	Data provenance and citation	85
4.2.14	Validation and interpretation	85
4.2.15	Testing and verification	86
4.2.16	Development status	86
4.2.17	Contributing	86
4.3	Application projects	87
4.3.1	NURETH21	87
4.3.2	NUTHOS15_MRM	88
4.4	Publications	88
4.4.1	Overviews	88
4.4.2	Mixture model	89
4.4.3	Two-fluid model	89
4.4.4	Three-field model	89
4.4.5	Four-field model	89
5	Packages	91
5.1	InputEnums	91
5.2	Inputs	101
5.3	Session	118
5.4	Solver	120
5.4.1	Four-field solver	121
5.4.2	Mixture solver	138
5.4.3	Three-field solver	160
5.4.4	Two-fluid solver	169
6	ExternalPackages	201
6.1	CoolPropWrapper	201
7	Community	203
7.1	Authors	203
7.1.1	Maintainers	203
7.1.2	Contributors and users	203
7.2	Contribute to the code	204
7.2.1	How to contribute	204
7.2.2	Coding standards	205
7.2.3	Documentation	206
7.2.4	Testing	206
7.2.5	Reporting issues	208
7.2.6	Code of conduct	208
7.2.7	Communication	208
7.2.8	Contributor checklist	209
7.3	Contribute to the documentation	209
7.3.1	Set up Sublime Text as a text editor	209
7.3.2	Configure Python and Sphinx	210
7.3.3	Start writing	211
7.3.4	Writing guidelines	211
7.3.5	Build the documentation	214
7.3.6	Documentation-maintenance checklist	216
7.3.7	Submitting documentation changes	219
7.3.8	Continuous integration	219
7.3.9	Reporting documentation issues	219

7.4	OpenSTREAM FAQ	220
7.4.1	Getting started	220
7.4.2	Running simulations	225
7.4.3	Choosing a solver	228
7.4.4	Numerical convergence	230
7.4.5	Results and post-processing	233
7.4.6	Physical scope and limitations	240
7.4.7	Extending and contributing	247
7.5	Support	249
	Bibliography	251
	MATLAB Module Index	255
	Index	257

WELCOME

OpenSTREAM (**O**pen **S**olvers for **T**wo-phase flow **R**esearch, **E**ngineering **A**nalysis and **M**odeling) is an open-source, object-oriented computational environment designed for simulating one-dimensional, multi-field, two-phase flows in straight geometries (Le Corre *et al.* [2025a]) (Le Corre *et al.* [2025b]). It supports phenomena such as wall boiling, phase interactions and non-equilibrium effects, making it a relevant tool for researchers working in thermal-hydraulics and two-phase flow systems.

1.1 What's inside?

OpenSTREAM offers a suite of solver frameworks tailored to different modeling needs:

- A **mixture solver** – with thermal non-equilibrium capabilities
- A generic **two-fluid solver** – for separate-phase modeling
- A **three-field solver** – for annular two-phase flow
- An advanced **four-field solver** – for annular two-phase flow explicitly capturing disturbance waves

These solvers are designed to simulate single-component, thermally expandable, steady-state and transient boiling two-phase flows in straight multi-wall channels, which may be uniformly or non-uniformly heated. The implementation relies on a set of reasonable simplifying assumptions to ensure computational efficiency and stability. Each solver includes a set of baseline closure models, which can be customized by modifying the corresponding class methods in the source code.

1.2 Why OpenSTREAM?

OpenSTREAM is built to support open, transparent research and to lower the barrier to entry for developing and validating fundamental models in two-phase flow simulations. Its goal is to make advanced thermal-hydraulic modeling more accessible to engineers, scientists, and students alike. Whether you're conducting research or learning the fundamentals, OpenSTREAM provides a collaborative and extensible environment for model development, performance evaluation, and cross-institutional validation, advancing the state of the art in thermal-hydraulic simulation.

1.3 Links

OpenSTREAM is developed openly on GitHub:

- **Source code:** [OpenSTREAM-solvers/openstream](#)
- **Report an issue:** [Issue tracker](#)
- **Contribute:** see the [Community](#) section

1.4 About this documentation

This guide will walk you through:

- Installation and setup instructions
 - The design philosophy behind OpenSTREAM
 - How to run simulations and interpret results
 - Example applications and use cases
 - How to contribute
-

1. Jean-Marie Le Corre, Jason Chan, Elisabeth Walter, Evan T. Hurlburt, and Roman W. Morse. OpenSTREAM: an open-source platform for two-phase flow modeling and simulation. In *Proceedings of the 21st International Topical Meeting on Nuclear Reactor Thermal Hydraulics (NURETH-21)*. Busan, Korea, August 31–September 5 2025a. URL: https://openstream-solvers.github.io/openstream/_static/publications/LeCorre2025OpenSTREAM.pdf.
2. Jean-Marie Le Corre, Jason Chan, Elisabeth Walter, Evan T. Hurlburt, and Roman W. Morse. OpenSTREAM: a new open-source platform for two-phase flow model development. In *Proceedings of the 12th International Conference on Multiphase Flow (ICMF 2025)*. Toulouse, France, May 12–16 2025b. URL: https://openstream-solvers.github.io/openstream/_static/publications/LeCorre2025ICMF.pdf.

2.1 Getting started

OpenSTREAM is a MATLAB program for simulating one-dimensional two-phase flows. Fluid properties are calculated using the Python version of [CoolProp](#) through [CoolPropWrapper](#), a MATLAB interface to CoolProp.

This guide explains how to obtain OpenSTREAM, configure MATLAB and Python, install the required dependencies, and verify the installation.

2.1.1 Prerequisites

Before installing and configuring OpenSTREAM, ensure that the following software is available:

- MATLAB.
- A Python version compatible with the installed MATLAB release.
- Git or GitHub Desktop.

CoolProp and its associated Python dependencies are installed later in this guide.

No additional MathWorks toolbox is currently required by the core OpenSTREAM solver frameworks.

2.1.2 Install OpenSTREAM

OpenSTREAM is hosted in the [OpenSTREAM GitHub repository](#).

There are two principal ways to obtain and maintain a local copy of the repository:

- The [Git command-line interface](#).
- [GitHub Desktop](#).

Install using the Git command-line interface

Clone the repository and its submodules using SSH:

```
git clone --recursive git@github.com:OpenSTREAM-solvers/openstream.git
```

Alternatively, clone the repository using HTTPS:

```
git clone --recursive https://github.com/OpenSTREAM-solvers/openstream.git
```

The `--recursive` option also clones the repository submodules, including [CoolPropWrapper](#).

For information about authentication, see:

- [Connecting to GitHub with SSH](#).

- [Caching GitHub credentials](#).
- [Working with Git submodules](#).

Enter the cloned repository:

```
cd openstream
```

Install using GitHub Desktop

Open GitHub Desktop, sign in, and select **Clone repository**. Select the OpenSTREAM repository and choose the local destination folder.

After cloning, verify that the `CoolPropWrapper` submodule contains its expected files. If the submodule is empty, initialize it from a terminal opened in the repository root:

```
git submodule update --init --recursive
```

2.1.3 MATLAB and Python compatibility

OpenSTREAM requires MATLAB. No additional MathWorks toolbox is currently required by the core solver frameworks. Fluid properties are calculated externally using the Python version of CoolProp through CoolPropWrapper.

OpenSTREAM is tested with selected MATLAB and Python version combinations through its continuous-integration workflow.

Table 2.1: CI-tested MATLAB and Python configurations

MATLAB release	Python version
R2024b	3.11
R2025b	3.12
R2026a	3.13

These combinations represent configurations tested by the OpenSTREAM continuous-integration workflow. Other MATLAB and Python combinations may work but are not necessarily tested. Python 3.14 is not currently included in the tested CI configuration.

The tested software configurations apply to the OpenSTREAM version or development revision documented on this site. See [Release numbering and versioning](#) for the project versioning convention.

Compatibility policy

OpenSTREAM distinguishes between tested and untested software configurations. A tested configuration is a combination of MATLAB, Python, CoolProp, and associated dependencies that is exercised by the OpenSTREAM continuous-integration workflow.

The configurations listed above represent the environments tested for the current OpenSTREAM development version or release. Other software combinations may work but are not guaranteed and may not be covered by automated testing.

Compatibility information may change as new MATLAB, Python, and CoolProp versions become available. Consult the compatibility information corresponding to the OpenSTREAM version being used.

2.1.4 Configure MATLAB and Python

Before using OpenSTREAM, verify that a suitable Python version is installed and that MATLAB is configured to use the corresponding Python executable.

The general compatibility requirements are available in the [MathWorks Python compatibility table](#). The **MATLAB Interface** column identifies the Python versions supported by each MATLAB release.

The following configuration instructions are adapted from the [CoolPropWrapper repository](#).

Inspect the configured Python environment

In MATLAB, inspect the current Python environment:

```
pythonEnvironment = pyenv;  
disp(pythonEnvironment)
```

If no Python environment is configured, or if the configured version is not suitable for the installed MATLAB release, install a compatible Python version and configure MATLAB to use it.

Install a compatible Python version

Windows

Download the required version from [Python.org](#) and follow the installation instructions.

Linux

Check whether Python is already available:

```
python3 --version
```

If necessary, install an appropriate version using the operating-system package manager. For example, on a Debian-based distribution:

```
sudo apt-get install python3.x
```

Replace `x` with the required minor version.

macOS

Check the available Python version:

```
python3 --version
```

If an appropriate version is not available, install one using Homebrew:

```
brew install python
```

Alternatively, download the required version from [Python.org](#).

Configure the Python executable in MATLAB

Configure MATLAB to use the required Python executable:

```
pyenv('Version', '<path-to-python-executable>')
```

Replace `<path-to-python-executable>` with the complete path to the Python executable.

Typical locations include:

- **Windows:** `C:\Users\Username\AppData\Local\Programs\Python\Python311\python.exe`
- **Linux:** `/usr/bin/python3.11`
- **macOS:** `/usr/local/bin/python3` or `/opt/homebrew/bin/python3`

On Linux and macOS, commands such as `which python3` can help locate the executable.

2.1.5 Install the Python dependencies

The recommended approach is to install the dependencies from the requirements file supplied with CoolPropWrapper.

Alternatively, install the dependencies used by the current continuous-integration environment directly:

```
python -m pip install CoolProp==8.0.0 nanobind
```

In the commands above, `python` must refer to the same Python executable configured through `pyenv`. If necessary, replace `python` with the complete path to that executable.

The current OpenSTREAM continuous-integration environment installs `CoolProp==8.0.0` together with `nanobind`. Other CoolProp versions may work but are not necessarily tested.

When several Python installations are available, ensure that the dependencies are installed using the same Python executable configured in MATLAB.

2.1.6 Verify the installation

After configuring MATLAB and Python, verify that MATLAB is using the intended Python environment and that CoolProp is accessible through CoolPropWrapper.

Inspect the Python environment

```
pythonEnvironment = pyenv;  
disp(pythonEnvironment)
```

Confirm that the reported executable and Python version correspond to the intended installation.

Verify the Python packages

From a terminal, use the same Python executable configured in MATLAB:

```
python -m pip show CoolProp  
python -m pip show nanobind
```

Create a CoolPropWrapper object

In MATLAB, create a fluid-property object:

```
fluidProperties = ...  
    CoolPropWrapper.CoolPropWrapper('WATER');
```

Successful construction confirms that MATLAB can access Python, import CoolProp, and initialize the requested fluid.

Evaluate a representative fluid property

Evaluate the saturation temperature of water at 6 MPa:

```
saturationTemperature = ...  
    fluidProperties.temperature('P',6.0e6,'Q',0);  
disp(saturationTemperature)
```

A finite numerical result confirms that the MATLAB-Python-CoolProp interface is functioning.

Run the automated tests

From the OpenSTREAM tests folder, run the complete automated test suite:

```
runOpenSTREAMTests
```

The complete suite includes environment, unit, and integration tests. The environment tests verify Python availability, package installation, and basic CoolProp functionality. The same complete test suite is executed by the MATLAB continuous-integration workflow when changes are pushed to the repository.

Run the sample script

Run the *sample script* to perform a complete OpenSTREAM calculation and confirm that the installation is functioning correctly.

The sample calculation uses the mixture solver to simulate boiling two-phase flow in a uniformly heated tube followed by an adiabatic section. It constructs the required input objects, evaluates the fluid properties, solves the case, generates representative axial plots, and saves the solver object.

Run the commands from the OpenSTREAM repository root, as described on the sample-script page.

2.1.7 Next steps

Congratulations! At this point, you should have a local copy of OpenSTREAM and a functioning MATLAB, Python, and CoolProp configuration.

Continue with the *OpenSTREAM tutorials*, beginning with Tutorial 1: Quick Start. Tutorial 1 constructs the input objects, runs the example case with the mixture, two-fluid, three-field, and four-field solver frameworks, generates representative plots, and saves the solver objects.

You can then continue with the solver-specific and workflow-oriented tutorials according to your interests.

2.2 Run sample script

After installation, you're all set to launch your first simulation!

As a first exercise, open MATLAB and run the commands listed below from within the `openstream` folder, which will simulate an example of boiling two-phase flow in a uniformly heated tube followed by an adiabatic region. The **OpenSTREAM** mixture model will be used, which defaults to the Homogeneous Equilibrium Model (HEM) when the physical models are not modified by the user.

This is the quickest and easiest way to start exploring what OpenSTREAM can do, and to confirm everything is working smoothly under the hood.

```
% Clear existing variables from workspace
clearvars

% Import the necessary classes for inputs and solvers
import Inputs.*
import Solvers.*
import Solvers.Mixture.*

% Turn off warnings during input setup
warning off

% Create the input set using paths to model, options, geometry, and boundary condition.
↪ files
inputSet = InputSet( ...
```

(continues on next page)

(continued from previous page)

```

modelFilePath      = './inputs/models.inp', modelID      = 'TUTORIAL1', ...
optionsFilePath    = './inputs/options.inp', optionsID   = 'DEFAULT', ...
geometryFilePath   = './inputs/geom.inp',   geometryID  = 'TUTORIAL1', ...
bcFilePath         = './inputs/tutorial1.inp', ...
sessionParentDir   = fullfile(pwd,'outputs'), ...
overwriteSessionFiles = true, ...
LOGMODE           = 'BOTH');

% Create a mixture solver object
mixSolver = MixtureSolver(inputSet);

% Solve
mixSolver.solve();

% Default axial plots
mixSolver.plotz();

% Save results
mixSolver.save();

```

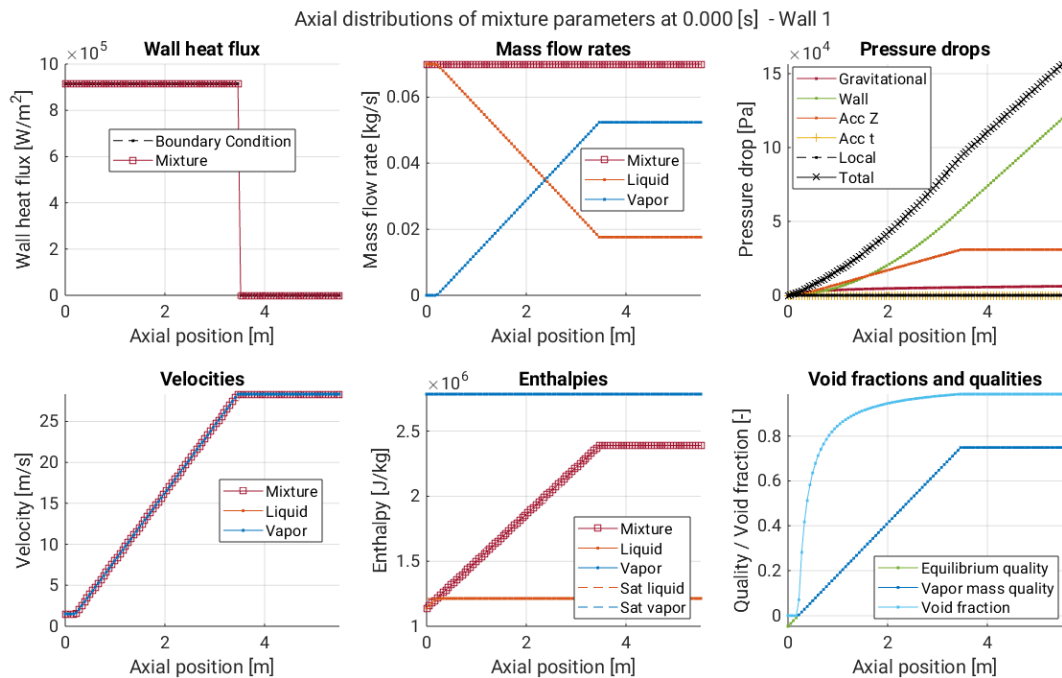


Figure 2.1: Sample figure of axial distributions of mixture parameters.

2.3 Intended scope and limitations

OpenSTREAM is an open-source computational environment for the development, assessment, and application of one-dimensional two-phase-flow models.

The software is primarily intended to support:

- Fundamental model development.
- Implementation and assessment of closure relations.
- Comparison of alternative two-phase-flow formulations.
- Verification of numerical implementations.
- Validation against experimental data.
- Numerical sensitivity and uncertainty studies.
- Reproducible research and publication companion workflows.
- Education and collaboration in thermal-hydraulics and two-phase-flow modeling.

OpenSTREAM provides a flexible research environment rather than a single fixed modeling methodology. The validity and accuracy of a calculation depend on the selected solver framework, closure models, numerical options, input data, and conditions under consideration.

2.3.1 Simulation frameworks

OpenSTREAM includes four solver frameworks with increasing levels of physical resolution:

- A mixture solver.
- A two-fluid solver.
- A three-field solver for annular two-phase flow.
- A four-field solver that separately represents the base film and disturbance waves.

The frameworks are intended for different modeling objectives. A more detailed formulation does not necessarily provide a more accurate result for every application. Increasing the number of fields also introduces additional closure relations, initialization requirements, numerical couplings, and sources of uncertainty.

A summary of the current solver capabilities is provided in the theory overview. Detailed assumptions, governing equations, and references are provided on the individual solver theory pages.

2.3.2 Dimensional scope

OpenSTREAM solves one-dimensional conservation equations along the principal flow direction.

The one-dimensional formulation assumes that relevant cross-sectional behavior can be represented using averaged quantities or closure relations. Radial, azimuthal, and local three-dimensional effects are not resolved directly.

The formulation is therefore most appropriate when:

- Axial transport is dominant.
- Cross-sectional quantities can be represented using averaged values.
- Local geometric effects can be represented through effective geometry, loss coefficients, or closure models.
- Three-dimensional flow structures are not the principal quantities of interest.

OpenSTREAM does not directly resolve:

- Cross-sectional velocity or void-fraction distributions.
- Local turbulent structures.
- Three-dimensional mixing between parallel channels.
- Detailed spacer-grid or obstruction flow fields.
- Local recirculation or flow separation.

- Multidimensional interface geometry.
- Computational-fluid-dynamics-scale phenomena.

The absence of these resolved effects does not necessarily prevent an OpenSTREAM calculation, but their influence must be represented through appropriate inputs, effective parameters, or closure relations.

2.3.3 Geometric scope

The current solver frameworks are designed for straight channels with one or more wall surfaces.

Supported representations can include:

- Circular tubes.
- Annuli.
- Rectangular channels.
- Simplified rod-bundle geometries.
- Multi-wall channels with different wall-heating conditions.
- Uniformly or nonuniformly heated channels.

The separate specification of flow area, wetted or heated perimeter, wall mesh, and wall-power distribution allows the representation of several cross-sectional shapes within the one-dimensional formulation.

The current geometry representation does not directly model:

- Arbitrary three-dimensional flow paths.
- Branches, junctions, plena, or interconnected system networks.
- Detailed spacer-grid geometry.
- Local changes in cross-sectional shape that require multidimensional resolution.
- Crossflow between subchannels.
- Complex components whose behavior cannot be represented through one-dimensional geometry and closure relations.

Users must determine whether an effective one-dimensional representation is adequate for the intended application.

2.3.4 Fluid and flow scope

The current OpenSTREAM frameworks are primarily intended for single-component, thermally expandable, two-phase flows.

The available models have mainly been developed and demonstrated for co-current boiling flows under conditions relevant to thermal-hydraulic experiments and water-cooled reactor applications.

Applicability to other fluids or operating conditions depends on:

- Availability and validity of the required thermophysical properties.
- Applicability of the selected closure relations.
- Compatibility of the flow regime with the selected solver framework.
- Numerical behavior under the investigated conditions.

A thermophysical-property calculation may be available for a fluid even when the selected closure relations have not been assessed for that fluid. Property availability must therefore not be interpreted as evidence that the complete model configuration is applicable.

2.3.5 Flow regimes

The mixture and two-fluid frameworks provide general representations of single-phase and two-phase transport under their respective assumptions.

The three-field and four-field frameworks are intended specifically for annular two-phase flow. Their separate liquid-film and droplet equations are initialized at the predicted onset of annular flow using the upstream mixture solution.

Results from the annular-flow frameworks depend on the modeling of:

- Onset of annular flow.
- Initial division of liquid between film and droplets.
- Film entrainment.
- Droplet deposition.
- Wall boiling and film evaporation.
- Field momentum transfer.
- Liquid-film dryout.
- Disturbance-wave development in the four-field framework.

The annular-flow formulations should not be assumed to apply outside the flow regimes and conditions supported by their selected closure models.

2.3.6 Thermal treatment

The mixture and two-fluid frameworks include thermal-equilibrium and thermal-nonequilibrium capabilities, depending on the selected models.

The current three-field and four-field annular-flow formulations generally assume thermal equilibrium while resolving hydrodynamic nonequilibrium between the represented flow fields.

In the four-field framework, the base film and disturbance waves have separate transport behavior. The model can represent nonequilibrium development of disturbance-wave properties, including wave amplitude, velocity, and frequency. This hydrodynamic nonequilibrium should not be confused with thermal nonequilibrium between phases.

The thermal assumptions of the selected solver and closure models must be reviewed before interpreting calculated temperatures, phase change, or post-CHF behavior.

2.3.7 Closure-model dependence

OpenSTREAM separates the conservation-equation frameworks from the physical and closure models used to complete them.

Calculated results can depend strongly on the selected models for:

- Void fraction and phase slip.
- Wall and interfacial friction.
- Wall heat transfer.
- Thermal nonequilibrium.
- Interfacial phase change.
- Onset of annular flow.
- Film entrainment.
- Droplet deposition.

- Film and wave momentum transfer.
- Base-film equilibrium thickness.
- Disturbance-wave frequency.
- Critical boiling transition and film dryout.

The availability of a closure model in the software does not imply that the model has been comprehensively validated for every fluid, geometry, pressure, mass flux, flow regime, or heating condition.

Users must review:

- The theoretical basis of each selected model.
- The conditions for which the model was developed.
- The available validation evidence.
- Any empirical coefficients or nondefault parameters.
- Sensitivity of the results to alternative model selections.

A favorable result obtained using one model combination must not be interpreted as general validation of the solver framework or of other model combinations.

2.3.8 Numerical formulation

The current governing equations are discretized using a one-dimensional finite-volume formulation.

The standard numerical implementation uses:

- First-order upwind treatment of axial advection.
- Fully implicit backward-Euler time integration.
- Fixed-point iterations for nonlinear closures and field coupling.
- Pseudo-time advancement to obtain steady-state solutions.
- Physical-time advancement for transient calculations.

These methods are designed to provide a robust and transparent baseline for model development and application. They also introduce limitations.

First-order spatial and temporal discretizations can introduce numerical diffusion and can reduce the resolution of sharp gradients or rapidly varying behavior. Results may depend on:

- Axial mesh resolution.
- Physical or pseudo-time-step selection.
- Point-iteration tolerances.
- Temporal convergence criteria.
- Under-relaxation factors.
- Interpolation options.
- Initialization and field-transition procedures.

A converged nonlinear or temporal solution is not necessarily independent of the mesh, time step, or numerical options.

2.3.9 Numerical sensitivity

Calculations used for quantitative conclusions should include numerical sensitivity studies appropriate to the application.

Relevant studies can include:

- Axial mesh refinement.
- Physical time-step refinement.
- Pseudo-time-step sensitivity.
- Point-iteration tolerance sensitivity.
- Temporal convergence tolerance sensitivity.
- Under-relaxation sensitivity.
- Sensitivity to field initialization.
- Sensitivity to the location of modeled flow-regime transitions.
- Comparison of available interpolation or discretization options.

A calculation should not be considered numerically verified solely because the solver reports a converged state.

2.3.10 Convergence

OpenSTREAM distinguishes between:

- Pointwise convergence of coupled variables at a location and time step.
- Pseudo-time convergence toward a steady-state solution.
- Physical-time advancement in a transient calculation.

Convergence difficulties can occur near:

- Onset of boiling.
- Onset of annular flow.
- Field initialization.
- Film dryout or rewetting.
- Sharp changes in wall heating.
- Strong interfacial-transfer regions.
- Discontinuities in geometry or local-loss definitions.

Under-relaxation, smaller pseudo-time steps, mesh refinement, or improved initialization can assist convergence. Such numerical adjustments should be documented and shown not to alter the intended converged physical solution materially.

A calculation that does not meet the selected convergence criteria should not be presented as a converged OpenSTREAM result.

2.3.11 Steady-state and transient calculations

The OpenSTREAM solver frameworks support steady-state and transient calculations.

Steady-state solutions are generally obtained through pseudo-time advancement. The pseudo-time history is a numerical procedure for reaching a steady solution and should not be interpreted as physical transient behavior.

Transient calculations use physical time and require appropriate initial conditions, time-step selection, and time-dependent boundary conditions.

The availability of transient execution does not imply that every closure model has been validated for transient applications. Models based on steady-state or local-equilibrium assumptions require particular care when used in rapidly changing conditions.

2.3.12 Validation status

OpenSTREAM includes models and solver frameworks that have been compared with selected experimental datasets and reference calculations.

Validation is necessarily limited to the investigated:

- Fluids.
- Geometries.
- Flow regimes.
- Pressures.
- Mass fluxes.
- Thermal conditions.
- Boundary conditions.
- Measured quantities.
- Solver and closure-model configurations.

Inclusion of a comparison in a publication, tutorial, dataset package, or application project does not constitute comprehensive validation outside the examined conditions.

The OpenSTREAM-database repository provides dataset implementations and calculated-versus-measured workflows for model assessment. Dataset-specific documentation should be reviewed for provenance, measurement uncertainty, processing methods, supported comparisons, and known limitations.

Publication companion workflows reproduce or illustrate selected calculations and figures from the corresponding publications. The companions do not represent complete records of the underlying research programmes. Additional cases, sensitivity studies, closure-model development, uncertainty analyses, and intermediate investigations may have been performed but are not necessarily included.

2.3.13 Verification and testing

OpenSTREAM includes automated environment, unit, integration, and regression tests.

The tests provide evidence that the implemented and tested functionality behaves as expected in the tested software environments. The automated test suite does not currently exercise every:

- Class or method.
- Solver path.
- Closure-model combination.
- Input option.
- Error condition.
- Transient application.
- Post-processing capability.

Passing the available tests does not constitute complete verification or validation of OpenSTREAM.

Contributors and users remain responsible for performing verification appropriate to their changes and applications.

2.3.14 Uncertainty

OpenSTREAM calculations can be affected by several categories of uncertainty:

- Experimental uncertainty.
- Source-data transcription or digitization uncertainty.
- Thermophysical-property uncertainty.
- Closure-model uncertainty.
- Model-parameter uncertainty.
- Numerical uncertainty.
- Boundary-condition uncertainty.
- Geometric simplification.
- Measurement-to-calculation alignment.

Calculated-versus-measured agreement should be interpreted in relation to the relevant uncertainties. Small numerical differences do not necessarily indicate meaningful model differences, while apparent agreement does not necessarily demonstrate that the underlying physical mechanisms are represented correctly.

2.3.15 Open research software

OpenSTREAM is developed as an open and extensible research platform. The source code is available for inspection, modification, testing, and extension.

The open implementation improves transparency but does not remove the need for engineering judgment, verification, validation, peer review, or documentation.

Users who modify solver equations, closure models, default parameters, numerical methods, or reference solutions should:

- Document the changes.
- Add or update relevant tests.
- Review numerical differences.
- Reassess applicable validation evidence.
- Record the software version or commit used.
- Avoid presenting modified results as results from an unmodified OpenSTREAM release.

2.3.16 Intended use

OpenSTREAM is well suited to:

- Fundamental thermal-hydraulic model development.
- Closure-model implementation and comparison.
- One-dimensional solver development.
- Experimental-data assessment.
- Reproducible calculation workflows.

- Sensitivity and uncertainty studies.
- Educational demonstrations.
- Research collaboration.
- Development of models for later integration into more comprehensive simulation tools.

OpenSTREAM should be used with particular caution when:

- Three-dimensional effects dominate the phenomenon.
- The geometry cannot be represented adequately as a straight channel.
- Crossflow or network behavior is important.
- The selected closure relations are applied outside their documented ranges.
- Experimental validation is unavailable for the conditions of interest.
- Numerical sensitivity has not been assessed.
- Results are strongly dependent on uncertain or nondefault parameters.

2.3.17 Reproducibility of reported results

Calculations reported in publications, technical reports, presentations, or other research outputs should retain sufficient information to reproduce the reported results.

The reproducibility record should identify, as applicable:

- The OpenSTREAM version, release, or commit.
- The OpenSTREAM-database version, release, or commit, when used.
- The MATLAB version.
- The Python and CoolProp versions.
- The selected OpenSTREAM solver framework.
- The input files, input identifiers, dataset records, or lightweight in-memory case definition.
- The selected physical and closure models.
- All nondefault model parameters.
- The selected numerical options.
- The axial mesh and physical or pseudo-time-step settings.
- The pointwise and temporal convergence criteria.
- The final convergence status.
- Any under-relaxation or initialization settings introduced to obtain convergence.
- The scripts or Live Scripts used to perform the calculations and generate the reported figures.
- Any filtering, interpolation, smoothing, averaging, or other post-processing applied.
- The distinction between directly measured, derived, and calculated quantities.
- Any experimental cases, sensitivity studies, or intermediate analyses intentionally omitted from the published companion workflow.

Where practical, reported results should be accompanied by an executable script or MATLAB Live Script. The workflow should generate the reported quantities and figures from the documented inputs rather than relying only on stored output files.

A publication companion workflow may focus on the calculations and figures selected for publication. It should not be presented as a complete record of the underlying research activities, which may include additional test cases, closure-model development, sensitivity studies, uncertainty analyses, numerical investigations, and intermediate results.

Generated solver outputs alone are not sufficient for reproducibility. The retained record must include the software versions, inputs, model selections, numerical settings, convergence information, and post-processing procedure needed to regenerate the results.

2.3.18 User responsibilities

Users are responsible for:

- Selecting an appropriate solver framework.
- Selecting physically applicable closure models.
- Reviewing assumptions and validity ranges.
- Defining consistent geometry and boundary conditions.
- Confirming unit consistency.
- Reviewing generated inputs.
- Confirming numerical convergence.
- Performing appropriate mesh and time-step sensitivity studies.
- Evaluating experimental, model, parameter, and numerical uncertainty.
- Reviewing available validation evidence.
- Recording the software version or commit used.
- Documenting nondefault models, parameters, and numerical settings.
- Interpreting results within the limits of the selected formulation.

For research and publication work, users should retain sufficient information to reproduce the calculation, including inputs, model selections, numerical options, software versions, convergence status, and post-processing procedures.

2.3.19 Further information

For additional details, consult:

- The theory overview.
- The individual solver theory pages.
- The tutorials.
- The numerical convergence tutorial.
- The OpenSTREAM package reference.
- The *OpenSTREAM-database* documentation.
- The available publications and publication companion workflows.

The detailed formulation and documented assumptions of the selected solver and closure models take precedence over this high-level summary.

2.4 Release numbering and versioning

OpenSTREAM uses a hybrid calendar-based and semantic version-numbering scheme. Release identifiers have the following general form:

```
vYYYY.N
```

where YYYY identifies the release year and N identifies the successive release within that year.

Examples include:

- **v2026.0**: the first principal OpenSTREAM release associated with 2026.
- **v2026.1**: a subsequent OpenSTREAM release published in 2026.
- **v2026.1-alpha**: an alpha prerelease associated with the planned **v2026.1** release.

Principal releases collect reviewed code, model, interface, testing, and documentation changes. Releases are published when justified by the accumulated changes rather than according to a fixed schedule.

2.4.1 Release identifiers

The release-year component identifies when a version is published. It does not indicate the validity range of the implemented physical models or the software environment required to run the release.

The successive-release component is incremented when an additional release is published within the same year.

The version number does not by itself distinguish among feature, correction, documentation, or compatibility releases. The nature and scope of a release are described in its release notes.

2.4.2 Prerelease identifiers

A suffix may be added when a version is made available before its final release. For example:

```
v2026.1-alpha
```

identifies an alpha prerelease associated with the planned **v2026.1** release.

Prerelease identifiers may include:

alpha

Identifies an early prerelease that may contain incomplete, experimental, or insufficiently tested functionality.

beta

Identifies a prerelease whose principal functionality is available but may still require broader testing and correction.

rc

Identifies a release candidate intended for final verification before publication of the corresponding stable release.

A numerical suffix may be added when several prereleases of the same type are published:

```
v2026.1-alpha.1  
v2026.1-beta.1  
v2026.1-rc.1
```

Prerelease versions must not be interpreted as stable releases. Their interfaces, defaults, numerical behavior, documentation, and reference solutions may change before the corresponding final release.

2.4.3 Stable and development versions

A tagged release without a prerelease suffix is considered a stable OpenSTREAM release.

The development branch may contain changes that are not available in the latest stable release. These changes may include:

- New or modified solver capabilities.
- New physical and closure models.
- Changes to input parameters or accepted values.
- Changes to default models or numerical options.
- Corrections affecting calculated results.
- Changes to saved-object formats.
- Updated tests and numerical reference solutions.
- Documentation for functionality not available in the latest release.

Users working from a development branch should record the complete Git commit identifier rather than referring only to the branch name. A branch name can refer to different source revisions over time, whereas a commit identifier identifies a specific repository state.

The active source revision can be obtained using:

```
git rev-parse HEAD
```

A concise source description can be obtained using:

```
git describe --tags --always --dirty
```

The `dirty` suffix indicates that the working tree contains local changes that are not included in the identified commit. Results obtained from a modified working tree should be reported accordingly.

2.4.4 Release documentation

Each released OpenSTREAM version should be accompanied by release notes summarizing, as applicable:

- New solver and physical-model capabilities.
- Corrections to existing implementations.
- Changes to governing equations or closure relations.
- Changes to input parameters or accepted values.
- Changes to default physical models or numerical options.
- Numerical-method changes.
- Changes affecting calculated results.
- Changes to saved solver objects or persisted results.
- Added or modified automated tests.
- Changes to approved numerical reference solutions.
- Documentation and tutorial updates.
- Tested MATLAB, Python, and CoolProp configurations.
- Known limitations and unresolved issues.

- Migration information for existing input files or user-developed models.
- Compatibility considerations for OpenSTREAM-database.

A correction that changes numerical results should be identified explicitly. The release notes should distinguish intentional physical or numerical changes from differences caused only by formatting, documentation, or repository maintenance.

2.4.5 Compatibility status

OpenSTREAM depends on MATLAB and, for thermophysical properties, a compatible Python and CoolProp environment.

Compatibility information should distinguish among:

Tested

The configuration is exercised by automated continuous integration or by a documented release-verification procedure.

Supported

The configuration is expected to be usable and is within the intended support range, although every combination of functionality may not be exercised automatically.

Untested

The configuration has not been included in the documented verification process. It may work, but compatibility has not been established.

Incompatible

The configuration is known not to satisfy the OpenSTREAM requirements or to produce a documented compatibility failure.

Absence from the tested configuration matrix should be interpreted as untested rather than automatically incompatible.

2.4.6 Tested configurations

Each release should identify the MATLAB, Python, and CoolProp configurations included in its verification process.

The compatibility table should be maintained with the released documentation using the following structure:

OpenSTREAM version	MATLAB	Python	CoolProp	Status
Current release	See release notes	See release notes	See release notes	Tested configurations are listed with the release

The release notes and continuous-integration configuration are the authoritative sources for the combinations tested for a particular release.

A successful test result for one MATLAB, Python, and CoolProp combination does not establish compatibility for every possible combination of those components.

2.4.7 Inspecting the active environment

Users should record the software environment used for calculations that support publications, technical reports, validation studies, or numerical reference solutions.

The MATLAB version can be displayed using:

```
version
```

The complete MATLAB product information can be displayed using:

```
ver
```

The Python environment selected by MATLAB can be inspected using:

```
pyenv
```

The Python version can be displayed using:

```
py.sys.version
```

The active CoolProp version can be displayed using:

```
py.CoolProp.CoolProp.get_global_param_string('version')
```

The reported environment should include, as applicable:

- OpenSTREAM release or Git commit.
- MATLAB release.
- Python version.
- CoolProp version.
- Operating system.
- OpenSTREAM-database release or Git commit.
- Local modifications affecting the calculation.

2.4.8 Continuous integration

OpenSTREAM uses continuous integration to exercise selected software environments and run the available automated test suite.

The continuous-integration configuration provides evidence for the tested combinations associated with the corresponding source revision. It does not demonstrate compatibility with configurations that are not included in the workflow.

The automated tests also do not exercise every:

- Solver path.
- Physical or closure model.
- Input combination.
- Transient application.
- Error condition.
- Post-processing capability.
- MATLAB, Python, and CoolProp combination.

A successful continuous-integration run therefore confirms that the implemented tests passed in the tested environments. It does not constitute complete verification or validation of OpenSTREAM.

2.4.9 OpenSTREAM-database compatibility

OpenSTREAM and OpenSTREAM-database are maintained as separate repositories.

An OpenSTREAM-database calculation should use an OpenSTREAM version compatible with the dataset classes, project workflows, input fields, and solver interfaces used by the calculation.

Publication and validation workflows should record both repository versions or commits:

```
OpenSTREAM:          <release or commit>
OpenSTREAM-database: <release or commit>
```

A project that runs with the current development versions of both repositories may not run unchanged with an earlier release if public interfaces, input options, result structures, or model identifiers have changed.

When compatibility between specific releases is important, the compatible OpenSTREAM and OpenSTREAM-database versions should be identified in the project documentation or release notes.

2.4.10 Saved objects and persisted results

Saved solver objects and persisted result structures can depend on the OpenSTREAM, MATLAB, Python, and CoolProp versions used to create them.

Changes to any of the following can affect compatibility:

- MATLAB class definitions.
- Property names and access rules.
- Enumeration values.
- Result-object organization.
- Input-object organization.
- Python object references.
- CoolProp interfaces.
- Serialization behavior.

Saved objects should not be treated as the only reproducibility record for a calculation.

For long-term reproducibility, retain:

- The original input files or dataset records.
- The solver and model selections.
- Nondefault model parameters.
- Numerical options.
- The OpenSTREAM version or commit.
- The OpenSTREAM-database version or commit, when applicable.
- The MATLAB, Python, and CoolProp versions.
- The script or Live Script required to regenerate the result.

When loading objects created with an earlier version, review the resulting warnings and verify that the object structure and calculated quantities remain consistent with the current implementation.

2.4.11 Backward compatibility

OpenSTREAM aims to preserve existing user workflows where practical. However, backward compatibility is not guaranteed for every development or prerelease revision.

An incompatible change may be necessary when:

- Correcting an erroneous implementation.
- Improving an established public interface.
- Replacing an ambiguous input convention.
- Changing a model formulation.
- Updating a dependency interface.
- Restructuring saved results or solver objects.
- Removing obsolete functionality.

Incompatible changes should be documented in the release notes together with migration guidance where practical.

2.4.12 Deprecation

Functionality intended for removal should, where practical, be deprecated before being removed from a stable release.

A deprecation notice should identify:

- The deprecated functionality.
- The recommended replacement.
- Any required migration steps.
- The release in which the deprecation is introduced.
- The planned removal release, when known.

Users should update deprecated workflows before relying on a later OpenSTREAM release.

2.4.13 Reproducibility

When citing, distributing, or reproducing an OpenSTREAM calculation, record the release identifier or complete source revision used for the calculation.

For reported numerical results, also record:

- The OpenSTREAM-database version or commit, when used.
- The MATLAB version.
- The Python version.
- The CoolProp version.
- The selected solver framework.
- The input files, identifiers, dataset records, or in-memory case definition.
- The selected physical and closure models.
- Nondefault model parameters.
- Numerical options.
- Mesh and time-step settings.
- Convergence criteria and final convergence status.

- Post-processing procedures.
- Any local source modifications.

A release identifier is preferable when the calculation uses an unmodified stable release. A complete Git commit should be reported when the calculation uses a development revision or a commit that is not associated with a release.

The wider reproducibility requirements for reported calculations are described in the [Intended scope and limitations](#) page.

2.4.14 Citing a release

A publication should cite the relevant OpenSTREAM publication and identify the software release or source revision used.

A release-based statement can use the following form:

```
Calculations were performed using OpenSTREAM vYYYY.N.
```

A development-revision statement can use:

```
Calculations were performed using OpenSTREAM commit <commit-id>.
```

When OpenSTREAM-database is used, identify its release or commit separately.

The repository release archive or assigned persistent identifier should be used when available so that the cited software version remains findable.

2.4.15 Migration between versions

Before reproducing an earlier calculation using a newer OpenSTREAM version:

1. Review the release notes for every intervening release.
2. Identify changes to inputs, defaults, models, numerical methods, and result structures.
3. Confirm that the original physical-model configuration remains available.
4. Regenerate the OpenSTREAM inputs from the authoritative source.
5. Run the calculation using the documented numerical settings.
6. Confirm convergence.
7. Compare the new and original calculated results.
8. Investigate and document any numerical differences.

Updating a calculation to a newer software version creates a new computational result. Agreement with the earlier result should be checked rather than assumed.

2.4.16 Development and release responsibilities

Before publishing a release, maintainers should:

- Confirm that the intended source revision is identified by the release tag.
- Run the complete available automated test suite.
- Review continuous-integration results.
- Build and review the HTML documentation.
- Build and review the PDF documentation.

- Confirm that documentation corresponds to the released source.
- Review dependency and compatibility information.
- Review changes to approved numerical reference solutions.
- Prepare release notes.
- Document known limitations and migration requirements.
- Confirm that generated outputs and temporary files are not included.
- Confirm that the release archive contains the required licensing and citation information.

Users should rely on the released documentation corresponding to the selected version rather than assuming that the current online documentation describes an earlier release.

3.1 Theory overview

This page provides a high-level overview of the theoretical foundations of the simulation frameworks implemented in **OpenSTREAM**, together with the relevant references.

The frameworks are designed to simulate one-dimensional, two-phase flow in straight channels. They provide increasing levels of physical resolution, from a mixture formulation to the explicit representation of liquid films, entrained droplets, and disturbance waves.

The appropriate framework depends on the flow regime, quantities of interest, and assumptions acceptable for the intended application. Detailed equations, derivations, assumptions, and implementation notes are provided on the individual theory pages linked below.

3.1.1 Solver capabilities

Framework	Principal fields	Thermal treatment	Time dependence	Principal applications
Mixture	Mixture, with liquid and vapor properties	Thermal equilibrium or nonequilibrium	Steady-state and transient	Initialization, system-level boiling, CHF, and post-CHF calculations
Two-fluid	Separate liquid and vapor fields	Thermal equilibrium or nonequilibrium	Steady-state and transient	Separate-phase transport and interfacial exchange
Three-field	Vapor, liquid film, and entrained droplets	Thermal equilibrium in the current annular-flow formulation	Steady-state and transient	Annular flow, film transport, entrainment, deposition, and dryout
Four-field	Vapor, base film, disturbance waves, and entrained droplets	Thermal equilibrium in the current annular-flow formulation	Steady-state and transient	Wave-resolved annular flow and developing wave behavior

The table summarizes the current solver formulations. Availability of a solver framework does not imply that every physical model, closure relation, flow regime, or application has been comprehensively validated.

OpenSTREAM is primarily intended for fundamental model development, assessment, and validation in one-dimensional straight-channel configurations. The applicability of a calculation depends on the selected closure models, their validity ranges, the numerical settings, and the conditions under consideration.

For a broader discussion of geometric, physical, numerical, and validation limitations, see Intended scope and limitations.

3.1.2 Detailed theory pages

Mixture model

- **Purpose:** Provide initialization and robust predictions under relevant simplifications.
- **Assumptions:** Represent the flow using a single mixture field, with optional hydrodynamic and thermal nonequilibrium models.
- **Equations:** Conserve mixture mass, momentum, and energy. Optionally (MRM), conserve vapor mass and energy.

Two-fluid model

- **Purpose:** Resolve hydrodynamic and thermal nonequilibrium between the liquid and vapor phases.
- **Assumptions:** Represent the liquid and vapor as separate fields coupled through interfacial exchange models.
- **Equations:** Conserve mass, momentum, and energy separately for the liquid and vapor phases.

Three-field model

- **Purpose:** Model annular two-phase flow using separate vapor, liquid-film, and entrained-droplet fields.
- **Assumptions:** Apply the current annular-flow formulation under thermal-equilibrium conditions up to liquid-film dryout.
- **Equations:** Conserve mass and momentum for the liquid film and entrained droplets together with the mixture-model vapor solution.

Four-field model

- **Purpose:** Resolve the continuous base film and disturbance waves as separate liquid fields in annular flow.
- **Assumptions:** Apply the current annular-flow formulation under thermal-equilibrium conditions up to liquid-film dryout.
- **Equations:** Conserve mass and momentum for the base film, disturbance waves, and entrained droplets together with the mixture-model vapor solution and a disturbance-wave number-density transport equation.

Select one of the theory pages above for the detailed formulation, implemented assumptions, closure relations, and references.

Mixture model

The mixture simulation framework in **OpenSTREAM** provides a simplified yet powerful approach to modeling two-phase flows. Based on a three-equation formulation, it treats the phases as a single continuum with averaged properties. This abstraction enables efficient simulation of systems where phase separation is minimal or where a fully resolved multi-field model is not required.

The mixture model serves two key roles within OpenSTREAM:

- **Robust initialization** for more complex solvers, ensuring robust starting conditions.
- **Efficient simulation** of flows with hydrodynamically well-coupled phases, where the mixture approximation remains physically meaningful.
- **Industry relevance** due to its simplicity and robustness, offering a practical approach for simulating averaged flow behavior in complex thermal-hydraulic systems.

In addition, a Mixture Relaxation Model (MRM) is available (Le Corre *et al.* [2026]). This extended formulation introduces vapor mass and energy conservation equations, allowing for thermal non-equilibrium between phases and expanding the model's applicability to more dynamic flow regimes.

By reducing complexity while preserving essential dynamics, the mixture model offers a practical entry point for both model development and exploratory analysis in thermal-hydraulic systems.

An overview of the mixture model implemented in OpenSTREAM is provided below. A more detailed derivation and theoretical background can be found in Le Corre *et al.* [2025a].

Governing equations

The mixture model solves the following conservation equations:

1. Mass conservation

$$\frac{\partial}{\partial t} \left(\frac{W}{u} \right) + \frac{\partial W}{\partial z} = 0$$

where:

- W is the mixture mass flow rate
- u is the mixture (static) velocity

2. Momentum conservation

$$\frac{\partial W}{\partial t} + \frac{\partial}{\partial z} (u^+ \cdot W) = -A \cdot \left(\frac{\partial p}{\partial z} + \frac{\partial p_K}{\partial z} + \cos \theta \cdot g \cdot \rho \right) - \sum \Pi_{wall}^n \cdot \tau_{wall}^n$$

where:

- ρ is the mixture density
- u^+ is mixture (advection) velocity
- A is the cross-sectional area
- p is the pressure
- K relates to obstruction form loss
- θ is the inclination angle
- g is the gravitational acceleration
- n is the wall index
- Π_{wall}^n is the wall perimeter for wall index n
- τ_{wall}^n is the wall shear stress for wall index n

3. Energy conservation

$$\frac{\partial}{\partial t} \left(h \cdot \frac{W}{u} \right) + \frac{\partial}{\partial z} (h^+ \cdot W) = \sum \Pi_{wall}^n \cdot q''_{wall}^n$$

where:

- h is the mixture (static) specific enthalpy
- h^+ is the mixture (advection) specific enthalpy
- q''_{wall}^n is the wall heat flux for wall index n

Mixture relaxation model

When selecting the Mixture Relaxation Model (MRM), two supplementary conservation equations are solved for the vapor phase. These equations provide the information required to determine the vapor mass fraction and vapor enthalpy while accounting for delayed interfacial heat and mass transfer. The resulting five-equation framework remains computationally efficient while allowing departures from thermodynamic equilibrium in subcooled boiling, saturated boiling, and post-boiling transition conditions.

1. Vapor mass conservation

$$\frac{\partial}{\partial t} \left(\frac{W_v}{u_v} \right) + \frac{\partial W_v}{\partial z} = A \cdot a_i \cdot (\Gamma - \Lambda) + \sum \Pi_{wall}^n \cdot \Gamma_{wb}^n$$

where:

- W_v is the vapor mass flow rate
- u_v is the vapor velocity
- a_i is the volumetric interfacial area
- Γ is the interfacial evaporation mass flux
- Λ is the interfacial condensation mass flux
- Γ_{wb}^n is the wall boiling mass flux for wall index n

2. Vapor energy conservation

$$\frac{W_v}{u_v} \cdot \frac{\partial}{\partial t} (h_v) + W_v \cdot \frac{\partial}{\partial z} (h_v) = A \cdot a_i \cdot (h_l - h_v) \cdot \Gamma + \sum \Pi_{wall}^n \cdot q''_{wall,v}$$

where:

- h_v is the vapor specific enthalpy
- $q''_{wall,v}$ is the wall heat flux vapor for wall index n

3. Time relaxation approximation

The fundamental assumption of the MRM is to consider that the interfacial mass exchanges are governed by the deviation from thermal equilibrium under a time relaxation principle:

$$A \cdot a_i \cdot (\Gamma - \Lambda) = \frac{x_{eq} \cdot W - W_v}{u_v \cdot t_{Relax}}$$

where:

- x_{eq} is the thermodynamic equilibrium quality
- t_{Relax} is the relaxation time for the interfacial mass transfer (evaporation or condensation)

Closure models

To complete the conservation equations, several closure models are required:

- Wall shear stress: τ_w^n
- Form pressure losses: $\frac{\partial p_K}{\partial z}$
- Wall heat transfer models
- Relaxation time for the interfacial mass transfer (required for the MRM model only): t_{Relax}

For each simulation, the selected closure models are defined in the OpenSTREAM model file, chosen from the available options listed in [InputEnums](#). If not explicitly specified by the user, default models are applied as defined in [Inputs.Model](#). All relevant closure models are implemented in [Solvers.Mixture.Mixture](#), which the users can modify to suit specific simulation needs.

In addition, the thermodynamic properties for each phase are computed using [CoolProp](#), an open-source thermophysical property library that provides accurate equations of state and transport properties for a wide range of fluids.

Features and assumptions

- Supports both steady-state and transient simulations in straight channels
- Supports uniform and non-uniform wall heat flux distribution
- Can include thermal non-equilibrium modeling (subcooled boiling or post Critical Heat Flux) via closure models or MRM
- Allows phase velocity slip using drift flux models
- Neglects minor contributions such as frictional heating and temporal pressure gradient contributions

Role in OpenSTREAM

The mixture model provides a robust tool for simulating homogeneous two-phase flow, including cases with thermal non-equilibrium. It also provides an initialization framework for more complex solvers (i.e., two-fluid, three-field, four-field). Currently, its pressure gradient solution is reused across all solver frameworks to enhance robustness and numerical stability.

Implementation notes

Package

- [*Solvers*](#)

Module

- [*Solvers.Mixture*](#)

Mixture solver class

- [*Solvers.Mixture.MixtureSolver*](#)

Field and phase classes:

- [*Solvers.Mixture.Mixture*](#)
- [*Solvers.Mixture.Liquid*](#)
- [*Solvers.Mixture.Vapor*](#)

Key solver methods

- [*Solvers.Mixture.MixtureSolver.solve\(\)*](#)

Key field properties:

- [*Solvers.Mixture.Mixture.W*](#)
- [*Solvers.Mixture.Mixture.P*](#)
- [*Solvers.Mixture.Mixture.H*](#)

Two-fluid model

The two-fluid simulation framework in **OpenSTREAM** provides a detailed representation of two-phase flow by solving six conservation equations—mass, momentum, and energy for both liquid and vapor phases. This formulation enables the simulation of key non-equilibrium effects and is foundational in nuclear reactor thermal-hydraulic code systems.

The two-fluid model serves several key roles within OpenSTREAM:

- **Full separate phase treatment:** Liquid and vapor are modeled as distinct continua, each with its own set of governing equations.
- **Non-equilibrium dynamics:** Captures both thermal and hydrodynamic non-equilibrium, essential for accurate transient and boiling flow simulations.
- **Industry relevance:** Commonly used in reactor system codes to simulate phenomena such as evaporation, condensation, and phase separation.

By explicitly modeling both liquid and vapor phases, the two-fluid approach preserves critical non-equilibrium dynamics while offering a flexible foundation for detailed model development and advanced thermal-hydraulic analysis.

An overview of the two-fluid model implemented in OpenSTREAM is provided below. A more detailed derivation and theoretical background can be found in Le Corre *et al.* [2025a] and Walter [2024].

Governing equations

1. Mass conservation

$$\text{Liquid: } \frac{\partial}{\partial t} \left(\frac{W_l}{u_l} \right) + \frac{\partial W_l}{\partial z} = -Aa_i(\Gamma - \Lambda) - \sum \Pi_{wall}^n \Gamma_{wb}^n$$

$$\text{Vapor: } \frac{\partial}{\partial t} \left(\frac{W_v}{u_v} \right) + \frac{\partial W_v}{\partial z} = Aa_i(\Gamma - \Lambda) + \sum \Pi_{wall}^n \Gamma_{wb}^n$$

where:

- W_l, W_v are the liquid and vapor mass flow rates
- u_l, u_v are the liquid and vapor velocities
- a_i is the volumetric interfacial area
- Γ is the interfacial evaporation mass flux
- Λ is the interfacial condensation mass flux
- Γ_{wb}^n is the wall boiling mass flux for wall index n

2. Momentum conservation

$$\text{Liquid: } \rho_l A_l \left(\frac{\partial u_l}{\partial t} + u_l \frac{\partial u_l}{\partial z} \right) = -Aa_i \Lambda (u_l - u_v) - A_l \left(\frac{\partial p}{\partial z} + \cos \theta g \rho_l \right) + Aa_i \tau_{v,l} - \sum \Pi_{wall}^n \tau_{wall,l}^n$$

$$\text{Vapor: } \rho_v A_v \left(\frac{\partial u_v}{\partial t} + u_v \frac{\partial u_v}{\partial z} \right) = (Aa_i \Gamma + \sum \Pi_{wall}^n \Gamma_{wb}^n) (u_l - u_v) - A_v \left(\frac{\partial p}{\partial z} + \cos \theta g \rho_v \right) - Aa_i \tau_{v,l} - \sum \Pi_{wall}^n \tau_{wall,v}^n$$

where

- A_l, A_v are the liquid and vapor cross-sectional areas
- $\tau_{v,l}$ is the interfacial shear stress
- $\tau_{wall,l}, \tau_{wall,v}$ are the liquid and vapor wall shear stresses

3. Energy conservation

$$\text{Liquid: } \rho_l A_l \left(\frac{\partial h_l}{\partial t} + u_l \frac{\partial h_l}{\partial z} \right) = -Aa_i \Lambda (h_l - h_v) + \sum \Pi_{wall}^n (q''_{wall,l} - (h_v - h_l) \Gamma_{wb}^n)$$

$$\text{Vapor: } \rho_v A_v \left(\frac{\partial h_v}{\partial t} + u_v \frac{\partial h_v}{\partial z} \right) = Aa_i \Gamma (h_l - h_v) + \sum \Pi_{wall}^n q''_{wall,v}$$

where

- h_l, h_v are the liquid and vapor specific enthalpies
- $q''^n_{wall,l}, q''^n_{wall,v}$ are the wall heat flux to liquid and vapor for wall index n

Closure models

To complete the conservation equations, several closure models are required:

- Volumetric interfacial area: a_i
- Interfacial evaporation and condensation mass fluxes: Γ, Λ
- Wall boiling mass flux: Γ_{wb}^n
- Interfacial shears stress: $\tau_{v,l}$
- Wall shear stress for each phase: $\tau_{wall,l}, \tau_{wall,v}$
- Wall heat flux for each phase: $q''^n_{wall,l}, q''^n_{wall,v}$
- Wall heat transfer models

These relations depend on the local flow regime, which is determined by an additional flow regime identification model.

The selected closure models are defined in the OpenSTREAM model file, chosen from the available options listed in [InputEnums](#). If not explicitly specified by the user, default models are applied as defined in [Inputs.Model](#). All closure models are implemented in [Solvers.TwoFluid.Liquid](#) and [Solvers.TwoFluid.Vapor](#), which the users can modify to suit specific simulation needs.

In addition, the thermodynamic properties for each phase are computed using [CoolProp](#), an open-source thermophysical property library that provides accurate equations of state and transport properties for a wide range of fluids.

Features and assumptions

- Supports both steady-state and transient simulations in straight channels
- Supports uniform and non-uniform wall heat flux distribution
- Captures phase-specific velocities and temperatures
- Includes interfacial mass, momentum, and energy exchange
- Supports wall boiling and condensation
- Neglects minor contributions such as frictional heating and temporal pressure gradient contributions

Role in OpenSTREAM

The two-fluid model is the legacy framework for simulating non-equilibrium two-phase flows. It is validated against system codes like TRACE and supports both steady-state and transient simulations.

Implementation notes

Package

- [Solvers](#)

Module

- [Solvers.TwoFluid](#)

Two-fluid solver class

- `Solvers.TwoFluid.TwoFluidSolver`

Phase classes

- `Solvers.TwoFluid.Liquid`
- `Solvers.TwoFluid.Vapor`

Flow regime enumeration class

- `Solvers.TwoFluid.REGIMES`

Key solver methods

- `Solvers.TwoFluid.TwoFluidSolver.solve()`

Key field properties:

- `Solvers.TwoFluid.Liquid.W`
- `Solvers.TwoFluid.Liquid.U`
- `Solvers.TwoFluid.Liquid.H`
- `Solvers.TwoFluid.Vapor.W`
- `Solvers.TwoFluid.Vapor.U`
- `Solvers.TwoFluid.Vapor.H`

Three-field model

The three-field simulation framework in **OpenSTREAM** provides a structured representation of annular two-phase flow by solving relevant conservation equations for the liquid film and droplet fields, with the vapor field derived from the mixture solver. This formulation is tailored for saturated flow conditions within a theoretical framework used in other codes, such as in subchannel analysis for Boiling Water Reactor (BWR) fuel simulations (Adamsson and Le Corre [2011])(Adamsson and Le Corre [2014]).

The three distinct flow fields: vapor, entrained droplets and liquid film, are illustrated in Figure 3.1.

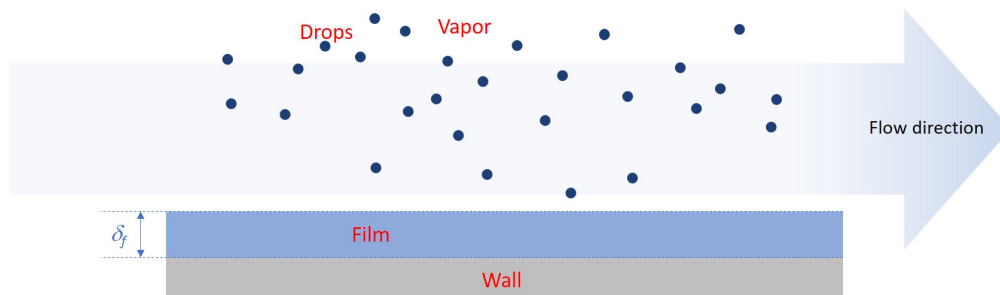


Figure 3.1: Field geometrical characteristics in the three-field simulation framework (vapor, drops and film).

The three-field model serves several key roles within OpenSTREAM:

- **Field separation:** Distinguishes between vapor, entrained droplets, and liquid film, enabling detailed modeling of annular flow dynamics.
- **Film and droplet transport:** Captures deposition and entrainment processes critical to predicting film dryout and droplet behavior.
- **Thermal equilibrium assumption:** Simplifies energy conservation while retaining essential mass and momentum exchanges.

By explicitly modeling the liquid film and droplet fields, the three-field framework enables accurate simulation of saturated annular flow regimes in single channels and supports advanced thermal-hydraulic analysis for various industrial applications.

An overview of the three-field model implemented in OpenSTREAM is provided below. A more detailed derivation and theoretical background can be found in Le Corre *et al.* [2025a].

Governing equations

The conservation equations are formulated at the wall level, indexed by n , to support multi-wall geometries with different heating rates. These equations are solved starting from the onset of annular two-phase flow. Upstream of this onset, the solution from the mixture solver is used to initialize the flow fields.

1. Mass conservation

$$\text{Film: } \frac{\partial}{\partial t} \left(\frac{W_f^n}{u_f^n} \right) + \frac{\partial W_f^n}{\partial z} = \Pi_{all}^n (D - E^n - \Gamma_{wb}^n)$$

where:

- W_f^n is the liquid film mass flow rate for wall index n
- u_f^n is the liquid film velocity for wall index n
- D is the drop deposition mass flux
- E is the film entrainment mass flux
- Γ_{wb}^n is the wall boiling mass flux for wall index n

The drop mass flow rate, W_d , is simply computed by subtracting W_f from the total liquid mass flow rate obtained from the mixture model.

2. Momentum conservation

$$\text{Liquid Film: } \rho_{ls} \delta_f^n \left(\frac{\partial u_f^n}{\partial t} + u_f^n \frac{\partial u_f^n}{\partial z} \right) = (u_d - u_f^n) D - \delta_f^n \left(\frac{\partial p}{\partial z} + \cos \theta g \rho_{ls} \right) + \tau_{v,f}^n - \tau_{wall,f}^n$$

$$\text{Droplets: } \rho_{ls} \left(\frac{\partial u_d}{\partial t} + u_d \frac{\partial u_d}{\partial z} \right) = \left(\sum \Pi_{wall}^n (u_f^n - u_d) E^n \right) \frac{\rho_{ls} u_d}{W_d} - \left(\frac{\partial p}{\partial z} + \cos \theta g \rho_{ls} \right) + \frac{A_d}{V_d} \tau_{v,d}$$

where:

- ρ_{ls} is the saturated liquid density
- δ_f^n is the liquid film thickness for wall index n
- u_d is the drop velocity
- $\tau_{v,f}^n$ is the vapor/film interfacial shear stress for wall index n
- $\tau_{v,d}^n$ is the vapor/drop interfacial shear stress
- $\tau_{wall,f}^n$ is the wall shear stress on the liquid film for wall index n
- A_d, V_d are the drop interfacial area and volume (based on surface averaging)

3. Energy conservation

$$\text{Under thermal equilibrium: } \Gamma_{wb}^n = \frac{q''_w{}^n}{h_{vs} - h_{ls}}$$

where:

- h_{vs}, h_{ls} are the saturated vapor and liquid specific enthalpies
- $q''_w{}^n$ is the wall heat flux for wall index n

Closure models

To complete the conservation equations, several closure models are required:

- Onset of annular two-phase flow
- Film/drop mass flow rate split at onset of annular two-phase flow
- Drop deposition mass flux: D
- Film entrainment mass flux: E
- Vapor/film interfacial shear stress: $\tau_{v,f}^n$
- Vapor/drop interfacial shear stress: $\tau_{v,d}^n$
- Wall shear stress on the liquid film: $\tau_{w,f}^n$
- Drop interfacial area and volume: A_d, V_d

The selected closure models are defined in the OpenSTREAM model file, chosen from the available options listed in [InputEnums](#). If not explicitly specified by the user, default models are applied as defined in [Inputs.Model](#). All closure models are implemented in [Solvers.ThreeField.Film](#) and [Solvers.ThreeField.Drop](#), which the users can modify to suit specific simulation needs.

In addition, the thermodynamic properties for each phase are computed using [CoolProp](#), an open-source thermophysical property library that provides accurate equations of state and transport properties for a wide range of fluids.

Features and assumptions

- Supports both steady-state and transient simulations in straight channels
- Supports uniform and non-uniform wall heat flux distribution
- Applicable up to film dryout
- Models drop deposition and film entrainment
- Assumes thermal equilibrium (no temperature difference between phases)
- Supports multi-wall geometries (e.g., annuli)
- Neglects minor contributions such as frictional heating and temporal pressure gradient contributions

Role in OpenSTREAM

The three-field model is the preferred framework for simulating annular two-phase flow up to the point of liquid film dryout. It offers detailed modeling of liquid film and droplet dynamics, making it particularly well-suited for advanced thermal-hydraulic analyses in Boiling Water Reactor (BWR) applications.

Implementation notes

Package

- [Solvers](#)

Module

- [Solvers.ThreeField](#)

Three-field solver class

- [Solvers.ThreeField.ThreeFieldSolver](#)

Field classes

- `Solvers.ThreeField.Film`
- `Solvers.ThreeField.Drop`

Key solver methods

- `Solvers.ThreeField.ThreeFieldSolver.solve()`

Key field properties:

- `Solvers.ThreeField.Film.W`
- `Solvers.ThreeField.Film.U`
- `Solvers.ThreeField.Drop.W`
- `Solvers.ThreeField.Drop.U`

Four-field model

The four-field simulation framework in **OpenSTREAM** extends the traditional three-field model by explicitly representing disturbance waves, in addition to vapor, droplets, and the base liquid film. This modeling approach was originally developed in Le Corre [2022a] and Le Corre [2022b]. It provides improved resolution of annular two-phase flow dynamics, enabling more accurate and detailed simulations.

The four distinct flow fields: vapor, entrained droplets, base liquid film and disturbance waves, are illustrated in Figure 3.2.

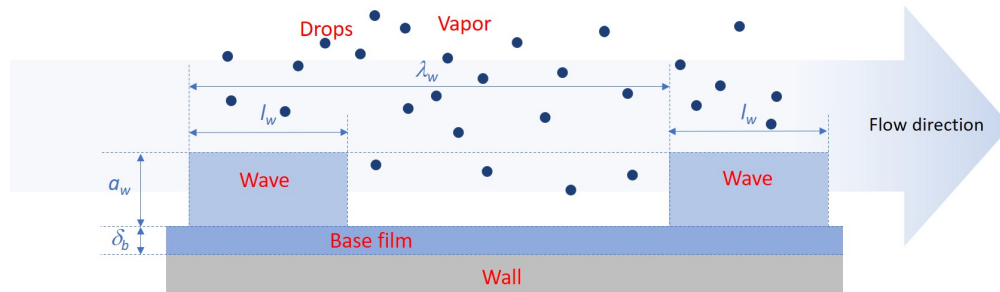


Figure 3.2: Field geometrical characteristics in the four-field simulation framework (vapor, drops, base film and waves).

The four-field model serves several key roles within OpenSTREAM:

- **Wave-resolved modeling:** Separates the liquid film into base film and disturbance waves, capturing their distinct transport behaviors and interactions.
- **Non-equilibrium dynamics:** Includes a Boltzmann-type wave number density transport equation to simulate wave formation, merging, and dissipation.
- **Enhanced predictive capability:** Enables detailed simulation of film dryout, wave-driven mass transport, and hydrodynamic transitions in developing annular flow.

By explicitly modeling disturbance waves and their interactions with other flow fields, the four-field framework offers state-of-the-art capabilities for simulating complex annular flow phenomena, including intermittent film dryout.

An overview of the four-field model implemented in OpenSTREAM is provided below. A more detailed derivation and theoretical background can be found in Le Corre [2022a] and Le Corre *et al.* [2025a].

Governing equations

The conservation equations are formulated at the wall level, indexed by n , to support multi-wall geometries with different heating rates. These equations are solved starting from the onset of annular two-phase flow. Upstream of this onset, the solution from the mixture solver is used to initialize the flow fields.

1. Mass conservation

$$\text{Base Film: } \frac{\partial}{\partial t} \left(\frac{W_b^n}{u_b^n} \right) + \frac{\partial W_b^n}{\partial z} = \Pi_{wall}^n (D_b - \Gamma_{wb,b}^n + \Psi_w^n - \Psi_b^n)$$

$$\text{Disturbance Waves: } \frac{\partial}{\partial t} \left(\frac{W_w^n}{u_w^n} \right) + \frac{\partial W_w^n}{\partial z} = \Pi_{wall}^n (D_w - E^n - \Gamma_{wb,w}^n - \Psi_w^n + \Psi_b^n)$$

where:

- W_b^n, W_w^n are the base film and wave mass flow rates for wall index n
- u_b^n, u_w^n are the base film and wave velocities for wall index n
- D_b, D_w are the drop deposition fluxes on the base film and wave fields
- $\Gamma_{wb,b}^n, \Gamma_{wb,w}^n$ are the base film and wave wall boiling mass fluxes for wall index n
- Ψ_w^n, Ψ_b^n are exchange mass fluxes between film and wave fields for wall index n

2. Momentum conservation

$$\text{Base Film: } \rho_{ls} \delta_b^n \left(\frac{\partial u_b^n}{\partial t} + u_b^n \frac{\partial u_b^n}{\partial z} \right) = D_b (u_d - u_b^n) + \Psi_w^n (u_w^n - u_b^n) - \delta_b^n \left(\frac{\partial p}{\partial z} + g \rho_{ls} \right) + \beta_b^n \tau_{v,b}^n + (1 - \beta_b^n) \tau_{w,b}^n - \tau_{wall,b}^n$$

$$\text{Disturbance Waves: } \rho_{ls} \delta_w^n \left(\frac{\partial u_w^n}{\partial t} + u_w^n \frac{\partial u_w^n}{\partial z} \right) = D_w (u_d - u_w^n) + \Psi_b^n (u_b^n - u_w^n) - \delta_w^n \left(\frac{\partial p}{\partial z} + g \rho_{ls} \right) + (1 - \beta_b^n) (\tau_{v,w}^n - \tau_{w,b}^n)$$

where

- ρ_{ls} is the saturated liquid density
- δ_b^n, δ_w^n are the base film and wave (equivalent) thicknesses for wall index n
- u_d is the drop velocity
- β_b^n is the base film interfacial fraction
- $\tau_{v,b}^n, \tau_{v,w}^n$ are the vapor/base film and vapor/wave interfacial shear stresses for wall index n
- $\tau_{w,b}^n$ is the base film/wave interfacial shear stress
- $\tau_{wall,b}^n$ is the wall shear stress on the base film for wall index n

3. Energy conservation

$$\text{Base Film under thermal equilibrium: } \Gamma_{wb,b}^n = \frac{q''_{wall,b}^n}{h_{vs} - h_{ls}}$$

$$\text{Disturbance Waves under thermal equilibrium: } \Gamma_{wb,w}^n = \frac{q''_{wall,w}^n}{h_{vs} - h_{ls}}$$

where:

- h_{vs}, h_{ls} are the saturated vapor and liquid specific enthalpies
- $q''_{wall,b}^n, q''_{wall,w}^n$ are the wall heat flux to the base film and wave fields for wall index n

4. Wave number density transport

$$\frac{\partial N_w^n}{\partial t} + \frac{\partial}{\partial z} (u_w^n N_w^n) = \frac{N_w^{eq,n} - N_w^n}{\tau_{Relax}^n}$$

where:

- N_w^n is the wave number density (spatial density of waves)
- $N_w^{eq,n}$ is the equilibrium wave number density

- t_w^{Relax} is the relaxation time associated with wave formation, merging, and dissipation.

Closure models

To complete the conservation equations, several closure models are required:

- Same closure models as for the *three-field model*
- Base film/wave mass flow rate split at onset of annular two-phase flow
- Drop deposition split between base film and wave fields
- Exchange mass fluxes between film and waves: Ψ_w^n, Ψ_b^n
- Wall boiling split between base film and wave fields
- Base film interfacial fraction: β_b^n
- Vapor/base film and vapor/wave interfacial shear stresses: $\tau_{v,b}^n, \tau_{v,w}^n$
- Base film/wave interfacial shear stress: $\tau_{w,b}^n$
- Wall shear stress on the base film: $\tau_{wall,b}^n$
- Equilibrium wave number density: $N_w^{eq,n}$
- Relaxation time associated with wave formation, merging, and dissipation: t_w^{Relax}

The selected closure models are defined in the OpenSTREAM model file, chosen from the available options listed in *InputEnums*. If not explicitly specified by the user, default models are applied as defined in *Inputs.Model*. All closure models are implemented in *Solvers.FourField.Wave* and *Solvers.FourField.Base*, which the users can modify to suit specific simulation needs.

In addition, the thermodynamic properties for each phase are computed using *CoolProp*, an open-source thermophysical property library that provides accurate equations of state and transport properties for a wide range of fluids.

Features and assumptions

- Supports both steady-state and transient simulations in straight channels
- Supports uniform and non-uniform wall heat flux distribution
- Models intermittent wave transport and non-equilibrium wave dynamics
- Includes wave-film exchange and wave number density evolution
- Assumes thermal equilibrium (no temperature difference between phases)
- Supports multi-wall geometries (e.g., annuli)
- Neglects minor contributions such as frictional heating and temporal pressure gradient contributions

Role in OpenSTREAM

The four-field model provides state-of-the-art simulation capabilities for annular two-phase flow, including in developing flow regions. It captures wave dynamics and their impact on mass and momentum transfer.

Implementation notes

Package

- *Solvers*

Module

- *Solvers.FourField*

Four-field solver class

- *Solvers.FourField.FourFieldSolver*

Field classes

- *Solvers.FourField.Wave*
- *Solvers.FourField.Base*
- *Solvers.FourField.Drop*

Key solver methods

- *Solvers.FourField.FourFieldSolver.solve()*

Key field properties:

- *Solvers.FourField.Wave.W*
- *Solvers.FourField.Wave.U*
- *Solvers.FourField.Wave.FREQUENCY*
- *Solvers.FourField.Base.W*
- *Solvers.FourField.Base.U*
- *Solvers.ThreeField.Drop.W*, inherited by *Solvers.FourField.Drop*
- *Solvers.ThreeField.Drop.U*, inherited by *Solvers.FourField.Drop*

3.2 Generated files and session directories

OpenSTREAM calculations can generate input files, session directories, logs, saved solver objects, exported data, and figures.

Understanding where these files are created is important for:

- keeping source repositories clean;
- avoiding accidental overwriting of previous calculations;
- reproducing reported results;
- separating temporary output from approved numerical references;
- preventing generated files from being committed to Git.

This page describes the general file-management conventions used by OpenSTREAM. Individual tutorials, tests, and OpenSTREAM-database projects may define more specific output locations.

3.2.1 Input files

An OpenSTREAM calculation is constructed from model, numerical-option, geometry, and boundary-condition inputs. These inputs can be read from existing files or generated by another workflow, such as OpenSTREAM-database.

Typical input paths include:

```
modelFilePath
optionsFilePath
geometryFilePath
bcFilePath
```

The complete paths should be passed explicitly when constructing an `Inputs.InputSet` object. Avoid relying on the current MATLAB working directory to locate maintained input files.

For example:

```
inputSet = Inputs.InputSet( ...
    modelFilePath = modelFilePath, ...
    modelID = modelID, ...
    optionsFilePath = optionsFilePath, ...
    optionsID = optionsID, ...
    geometryFilePath = geometryFilePath, ...
    geometryID = geometryID, ...
    bcFilePath = bcFilePath, ...
    sessionParentDir = sessionParentDir);
```

Maintained input files should remain separate from generated calculation output.

3.2.2 Session directories

Each calculation uses a session directory to store files associated with that solver execution.

The parent location is selected through:

```
sessionParentDir = '<output-folder>'
```

OpenSTREAM creates the calculation-specific session directory below this parent directory.

The session directory can contain, depending on the selected workflow and logging configuration:

- calculation logs;
- solver output;
- copied or generated session input files;
- exported numerical data;
- saved solver objects;
- files required for restarting or reviewing the calculation.

Users should select `sessionParentDir` explicitly for maintained workflows. This avoids creating output in unexpected locations and makes it easier to remove or archive completed calculations.

A local exploratory calculation can use:

```
sessionParentDir = fullfile(pwd, 'outputs');
```

A maintained application should normally use a project-specific output location rather than depending implicitly on `pwd`.

3.2.3 Overwriting an existing session

The `overwriteSessionFiles` option controls whether an existing session directory can be replaced.

For example:

```
inputSetOpts = { ...  
    'overwriteSessionFiles', true, ...  
    'LOGMODE', 'BOTH'};
```

When `overwriteSessionFiles` is `true`, OpenSTREAM can remove an existing session directory before creating the new calculation session.

A warning similar to the following can be displayed:

```
Warning: <session-directory> was removed.
```

This message indicates that the existing calculation directory was deliberately replaced. It does not by itself indicate a solver failure.

Use `overwriteSessionFiles = true` when a workflow is intended to be rerun reproducibly and earlier generated output can be replaced.

Use overwriting with caution when:

- previous results have not been reviewed or archived;
- several calculations share the same session location;
- a session contains manually added files;
- the output is required as an approved numerical reference;
- another MATLAB process may be using the directory.

A workflow should not rely on overwriting as a substitute for unique and well-organized output locations.

3.2.4 Logging

The `InputSet` logging configuration is passed through `LOGMODE`.

For example:

```
inputSetOpts = { ...  
    'overwriteSessionFiles', true, ...  
    'LOGMODE', 'BOTH'};
```

The selected logging mode determines how calculation information is made available during solver execution. Use the same logging configuration when comparing or reproducing calculations whenever the log content forms part of the review procedure.

Logging settings do not replace convergence checks. A calculation should be evaluated using the final solver state and the relevant pointwise, steady-state, or transient convergence information.

3.2.5 Saving solver objects

OpenSTREAM solver objects can be saved when a persistent result is required.

A saved solver object can retain convenient access to calculated fields, input information, and post-processing functionality. However, saved objects can depend on:

- the OpenSTREAM source revision;
- MATLAB class definitions;
- Python integration;
- CoolProp compatibility;
- result-object organization;
- serialization behavior.

A saved solver object should therefore not be the only record retained for a calculation.

For reproducibility, also retain:

- the original input files or dataset records;
- the solver framework;
- the physical and closure models;
- nondefault model parameters;
- numerical settings;
- convergence criteria and status;
- the OpenSTREAM version or commit;
- the supporting script or Live Script.

3.2.6 OpenSTREAM-database outputs

OpenSTREAM-database generates OpenSTREAM input and result folders for the selected dataset case.

A dataset package can contain generated folders such as:

```
+DatasetName/
├── +inputs/
│   └── case-000001/
├── +results/
│   └── case-000001/
```

The generic Dataset implementation provides the generated model, geometry, numerical-option, and boundary-condition file paths to `Inputs.InputSet`. The result folder is used as the session parent directory for the calculation.

The dataset source file, dataset class, plotting methods, and README remain maintained source content. Generated `+inputs` and `+results` folders are temporary calculation artifacts and should normally be excluded from version control.

Lightweight OpenSTREAM-database cases do not use a dataset-specific XML or JSON package. Their generated-file location should be defined by the workflow so that output is not created unexpectedly in the repository root.

The OpenSTREAM-database `runCase` wrapper also provides a `saveResultsToFile` option. This controls whether the calculated solver object is saved after execution. It is an OpenSTREAM-database workflow option and should not be confused with the core OpenSTREAM `overwriteSessionFiles` setting.

3.2.7 Sandbox calculations

The OpenSTREAM sandbox folder is intended for temporary exploratory calculations.

Generated input files, session directories, logs, figures, saved solver objects, and local scripts created in the sandbox should remain local and should not be committed.

Copy and rename any `*_default.m` template before modifying it.

Reusable work should be moved to the appropriate maintained location:

- general learning workflows belong in the tutorials;
- automated verification belongs in the tests;
- reusable implementation changes belong in the corresponding source package;
- publication and experimental-validation workflows belong in OpenSTREAM-database.

3.2.8 Test outputs

Automated tests can generate temporary files, logs, session directories, CSV files, and saved solver objects under test-specific output folders.

Generated test output should not normally be committed.

Approved numerical reference solutions are different from temporary test output. Reference solutions form part of the regression baseline and should be stored only in the designated reference location.

A numerical reference should be updated only after the calculated difference has been understood, justified, and reviewed.

3.2.9 Version-control policy

Generated files should normally be excluded using repository `.gitignore` rules.

Before committing a change, review tracked, untracked, and ignored files:

```
git status
git status --short
git status --ignored --short
```

Do not commit:

- temporary session directories;
- generated input or result folders;
- logs;
- MATLAB autosave files;
- local sandbox scripts;
- saved solver objects that are not approved references;
- exported figures or data that can be regenerated;
- obsolete output copies.

Do not use `git add .` without first reviewing all modified and untracked files.

3.2.10 Moving or deleting generated files

Generated output can normally be deleted after the calculation has been reviewed, provided that the inputs and workflow required to regenerate it have been retained.

Before deleting a session, confirm that it does not contain:

- the only copy of an input modification;
- an approved numerical reference;
- manually prepared post-processing data;
- results required for a publication or technical report;
- diagnostic information needed to investigate a failure.

When a calculation must be archived, store the associated input files, software versions, settings, convergence information, and execution workflow together with any retained results.

3.2.11 Recommended workflow

For a reproducible calculation:

1. Select an explicit session parent directory.
2. Keep maintained inputs separate from generated outputs.
3. Decide whether an existing session may be overwritten.
4. Run the calculation with the intended logging configuration.
5. Review the final convergence state.
6. Review generated inputs and outputs.
7. Retain the information needed to regenerate the calculation.
8. Archive only results that cannot be regenerated easily or that form an approved reference.
9. Remove or ignore temporary output before committing repository changes.

3.2.12 Troubleshooting

Unexpected output location

If generated files appear in an unexpected folder:

1. Inspect `sessionParentDir`.
2. Check whether any input or output path depends on `pwd`.
3. Display the current working directory using `pwd`.
4. Use `fullfile` and explicit repository or project paths.
5. Confirm that a lightweight OpenSTREAM-database workflow defines an appropriate generated-file location.

Existing session was removed

If OpenSTREAM reports that a directory was removed, check whether:

```
overwriteSessionFiles = true
```

was supplied to `Inputs.InputSet`.

The message indicates replacement of the previous session output. Set a different session parent directory or session identifier if the previous results must be retained.

Existing session cannot be replaced

If an existing session prevents execution:

- review whether overwriting is appropriate;
- select a new session parent directory;
- remove or archive the existing directory manually;
- confirm that no other MATLAB process is using the same session;
- verify write permission for the output location.

Output should not be committed

If a generated file is already tracked but should remain locally:

```
git rm --cached <path>
```

Then add the path to `.gitignore`, commit the removal from the index, and retain the local working copy.

3.2.13 Further information

For additional guidance, consult:

- The getting-started and input tutorials.
- The numerical convergence tutorial.
- The testing documentation.
- The contribution guidelines.
- The *OpenSTREAM-database* documentation.
- The release numbering and versioning page.

3.3 Glossary

This glossary defines terminology used throughout the **OpenSTREAM** documentation. The intent is to make the documentation more consistent and to clarify the distinction between closely related concepts used in one-dimensional, multi-field, two-phase flow modeling.

Annular flow

A two-phase flow regime in which a continuous vapor core is surrounded by a liquid film along the wall. Liquid droplets may also be entrained in the vapor core. In OpenSTREAM, annular flow models may represent vapor, droplets, liquid film, and, in advanced formulations, disturbance waves as separate fields.

Boundary condition

A prescribed condition imposed at the boundary of the computational domain. Typical boundary conditions specify inlet flow properties (mass flow, enthalpy), outlet pressure, and wall heat flux.

Closure model

A model used to close the system of conservation equations by providing relationships for physical processes that are not resolved directly. Examples include wall heat transfer, interfacial drag, interfacial heat transfer, phase change, entrainment, deposition, and wall friction.

Conservation equation

A governing equation expressing the conservation of a physical quantity, such as mass, momentum, or energy. In OpenSTREAM, conservation equations are formulated for the mixture, phase, or field variables depending on the selected solver.

Deposition

The transfer of liquid droplets from the vapor core back to the wall film. Deposition is typically represented through a closure model in annular flow formulations.

Disturbance wave

A coherent liquid structure traveling along the liquid film in annular flow. Disturbance waves can influence wall wetting, film mass transport, entrainment, deposition, and interfacial momentum transfer.

Droplet field

A dispersed liquid field representing entrained droplets transported within the vapor core. In annular flow modeling, the droplet field is distinct from the wall liquid film field.

Entrainment

The transfer of liquid from the wall film into the vapor core as droplets. Entrainment is typically represented through a closure model in annular flow formulations.

Field

A computationally resolved constituent with its own set of transported variables or conservation equations. A field is not always identical to a thermodynamic phase. For example, liquid droplets and a wall liquid film are both liquid phase constituents, but they may be represented as separate fields.

Film dryout

The disappearance of the continuous wall liquid film. In annular flow modeling, film dryout marks the loss of wall wetting by the liquid film and is often associated with a change in heat transfer behavior.

Four-field model

A multi-field model for annular two-phase flow in which vapor, droplets, liquid film, and disturbance waves are represented explicitly.

Governing equation

A mathematical equation describing the evolution of a conserved quantity or transported variable. Governing equations in OpenSTREAM are based on one-dimensional conservation laws with source terms.

Hydrodynamic equilibrium

A modeling assumption in which the phases or fields share the same velocity, or in which relative motion is neglected. When hydrodynamic non-equilibrium is considered, different phases or fields may have different velocities.

Hydrodynamic non-equilibrium

A condition in which two or more phases or fields have different velocities or momentum balances. Two-fluid and multi-field formulations commonly account for hydrodynamic non-equilibrium.

Interfacial area

The area of contact between phases or fields per unit volume. Interfacial area appears in closure models for interfacial transfer of mass, momentum, and energy.

Interfacial transfer

Exchange of mass, momentum, or energy between phases or fields. Examples include evaporation, condensation, interfacial drag, and interfacial heat transfer.

Liquid film

A continuous liquid layer flowing along the wall in annular flow. The liquid film is treated separately from entrained liquid droplets in multi-field annular flow models.

Mixture model

A reduced two-phase flow model in which the phases are represented through mixture quantities. Depending on

the implementation, the mixture model may include additional relations to account for thermal non-equilibrium.

Multi-field model

A model in which different constituents of the flow are represented as separate computational fields. Multi-field models allow different phases or flow structures, such as vapor, droplets, liquid film, and disturbance waves, to be described with different transported variables.

Multi-wall channel

A channel representation that may include more than one heated or unheated wall surface. This allows wall-specific thermal or hydraulic conditions to be represented in the model formulation.

Non-equilibrium

A condition in which one or more variables are not shared among phases or fields. Non-equilibrium may be thermal, hydrodynamic, mechanical, or chemical depending on the variables considered.

One-dimensional model

A model in which flow variables vary primarily along one spatial coordinate, usually the axial direction. Cross-sectional effects are represented through averaged quantities and closure models rather than resolved explicitly.

Phase

A physically distinct thermodynamic state of matter, such as liquid or vapor. A phase may be represented by one or more fields in a multi-field model.

Phase change

Conversion of mass from one phase to another, such as evaporation or condensation. In two-phase thermal-hydraulic models, phase change is usually represented as an interfacial or wall source term.

Relaxation model

A model that drives a variable toward an equilibrium or target state over a characteristic relaxation time. Relaxation models are often used to represent unresolved non-equilibrium processes in a numerically robust way.

Relaxation time

A characteristic timescale over which a variable approaches an equilibrium or target value in a relaxation model. Smaller relaxation times correspond to faster return toward the target state.

Solver

A computational implementation of a mathematical model and numerical method used to solve a specific set of governing equations. In OpenSTREAM, different solvers correspond to different levels of physical detail and modeling assumptions.

Source term

A term in a conservation or transport equation representing production, destruction, or exchange of a quantity. Source terms may arise from wall interaction, interfacial transfer, external forces, or phase change.

Straight channel

A flow path without geometric bends or complex three-dimensional features. In one-dimensional modeling, a straight channel is represented along a single axial coordinate.

Thermal equilibrium

A modeling assumption in which phases or fields share the same temperature, or in which temperature differences are neglected.

Thermal non-equilibrium

A condition in which two or more phases or fields may have different temperatures or energy balances. Thermal non-equilibrium is important when heat transfer and phase change cannot be represented accurately by a single shared temperature.

Three-field model

A multi-field model for annular two-phase flow in which vapor, droplets, and liquid film are represented as separate fields.

Two-fluid model

A model in which the liquid and vapor phases are described by separate conservation equations. Two-fluid models can represent hydrodynamic and thermal non-equilibrium between phases.

Two-phase flow

Flow involving two thermodynamic phases, commonly liquid and vapor in the context of thermal-hydraulic applications.

Wall boiling

Boiling that occurs at a heated wall. Wall boiling models typically include mechanisms such as nucleation, evaporation, convection, quenching, and associated wall heat partitioning, depending on the level of model detail.

Wall friction

Momentum loss associated with shear stress at the wall. Wall friction is usually represented by a closure model or friction factor correlation.

Wall heat flux

The heat flux imposed at, or transferred through, a wall boundary. In heated channel simulations, wall heat flux is a key input or result depending on the problem formulation.

3.3.1 Recommended terminology

For consistency across the documentation, the following preferred terms are recommended:

Preferred term	Use when referring to	Avoid or use only with care
OpenSTREAM	The project, code, and documentation set	OpenStream, openstream
two-phase flow	Liquid-vapor flow modeled by OpenSTREAM	multiphase flow, unless more than two phases are intended
field	Computationally resolved constituent	phase, when the distinction is important
phase	Thermodynamic state, such as liquid or vapor	field, when referring to physical state
closure model	Implemented model used to close governing equations	closure law, correlation, constitutive law, unless a specific distinction is intended
thermal non-equilibrium	Different temperatures or energy balances among phases or fields	thermal disequilibrium, nonequilibrium thermal effects
hydrodynamic non-equilibrium	Different velocities or momentum balances among phases or fields	velocity slip, unless specifically referring to relative velocity
source term	Additive term in a conservation or transport equation	forcing term, production term, unless more specific
straight channel	One-dimensional channel without bends or complex 3D geometry	straight geometry, if channel-specific wording is clearer

3.3.2 Style notes

- Use `OpenSTREAM` consistently as the project name.
- Use `two-phase flow` when referring specifically to liquid-vapor applications.
- Use `multi-field` with a hyphen when it modifies a noun, for example `multi-field model`.
- Use `one-dimensional` with a hyphen when it modifies a noun, for example `one-dimensional solver`.
- Use `thermal non-equilibrium` and `hydrodynamic non-equilibrium` consistently.

- Prefer `closure` model for implemented models and reserve `constitutive relation` for broader physical relationships.
- Distinguish carefully between `phase` and `field` in theory pages.

3.4 Notation

This page summarizes the notation conventions used in the **OpenSTREAM** documentation. The theory pages, solver descriptions, closure model documentation, and implementation notes are kept consistent across the mixture, two-fluid, three-field, and four-field formulations.

3.4.1 General conventions

OpenSTREAM uses one-dimensional, cross-section averaged conservation equations. Unless otherwise stated, the independent variables are time and the axial coordinate:

$$t \quad \text{and} \quad z$$

where t is time and z is the axial coordinate along the flow channel.

The following conventions are used throughout the documentation:

- z for the axial coordinate.
- t for time.
- k as a generic field or phase index when no specific constituent is intended.
- Subscripts identify phases, fields, interfaces, and walls.
- Superscripts identify time levels, iteration counters, or model-specific states when needed.
- All symbols are defined immediately after the equation in which they first appear.

3.4.2 Primary variables

Symbol	Suggested name	Meaning
t	time	Time coordinate.
z	axial coordinate	One-dimensional spatial coordinate along the flow path.
A	flow area	Cross-sectional flow area.
P_w	wetted perimeter	Wall perimeter in contact with the fluid or liquid film.
D_h	hydraulic diameter	Hydraulic diameter of the channel.
α_k	volume fraction	Volume fraction of field or phase k .
ρ_k	density	Density of field or phase k .
u_k	velocity	Axial velocity of field or phase k .
p	pressure	Pressure (shared among fields).
T_k	temperature	Temperature of field or phase k .
h_k	specific enthalpy	Specific enthalpy of field or phase k .
e_k	specific internal energy	Specific internal energy of field or phase k .
E_k	total specific energy	Total specific energy of field or phase k , if used.
x	vapor quality	Vapor mass fraction in a two-phase mixture.
q''_w	wall heat flux	Heat flux imposed at, or transferred through, the wall.
g	gravitational acceleration	Gravitational acceleration.

3.4.3 Phase and field subscripts

The notation distinguishes between thermodynamic phases and computational fields. A phase is a physical state of matter, such as liquid or vapor. A field is a computationally resolved constituent with its own transported variables.

Subscript	Meaning	Notes
l	liquid phase	Used for liquid phase.
g	vapor phase	Used consistently for vapor or gas.
f	liquid film	Used for the wall liquid-film field in annular flow models.
d	droplets	Used for entrained liquid droplets in the vapor core.
$wall$	wall	Used for wall quantities, such as T_{wall} or q''_{wall} .
i	interface	Used for interfacial quantities, such as a_i or htc_i .
w	disturbance wave	Use for disturbance wave quantities when a compact subscript is needed.

3.4.4 Source-term notation

Source terms make it clear which conserved quantity is affected and which interaction is represented.

Symbol	Name	Meaning
Γ_k	mass source	Net mass source to field or phase k .
M_k	momentum source	Net momentum source to field or phase k .
Q_k	energy source	Net energy source to field or phase k .
$\Gamma_{a \rightarrow b}$	mass transfer rate	Mass transferred from field a to field b .
$M_{a \rightarrow b}$	momentum transfer	Momentum transferred from field a to field b .
$Q_{a \rightarrow b}$	heat or energy transfer	Energy transferred from field a to field b .

3.4.5 Closure model notation

Closure models use notation that clearly identifies the modeled process. Examples include:

Symbol	Suggested meaning	Typical use
C_D	drag coefficient	Interfacial or droplet drag model.
f_{wall}	wall friction factor	Wall momentum loss.
htc_{wall}	Wall heat-transfer coefficient	Wall heat transfer.
htc_i	Interfacial heat-transfer coefficient	Interfacial heat transfer.
a_i	interfacial area concentration	Interfacial transfer models.
τ_{wall}	wall shear stress	Wall friction and pressure drop.
τ_i	interfacial shear stress	Momentum exchange between fields.
t_{relax}	relaxation time	Relaxation models.

3.4.6 Units

SI units are used consistently unless a specific model, correlation, or input file requires otherwise.

Quantity	Unit	Notes
Length	m	Use for z , hydraulic diameter, and channel dimensions.
Time	s	Use for t , time steps, and relaxation times.
Pressure	Pa	Use for absolute pressure unless otherwise stated.
Temperature	K	Use kelvin in equations. Celsius may be used in user-facing examples if clearly stated.
Density	kg/m ³	Use for phase, field, and mixture density.
Velocity	m/s	Use for axial velocities.
Mass flow rate	kg/s	Use for total or field-specific mass flow rates.
Mass flux	kg/m ² /s	Use for area-normalized mass flow rate.
Heat flux	W/m ²	Use for wall heat flux.
Specific enthalpy	J/kg	Use for phase or field enthalpy.

3.5 Global reference list

This page gathers all the sources cited throughout the **OpenSTREAM** documentation in one convenient place. Whether you're tracing a model back to its theoretical roots or looking for further reading, this is your go-to resource for exploring the academic and technical foundations behind the code.

3.6 Testing and verification

OpenSTREAM includes an automated testing framework to support code verification, numerical consistency, and software environment compatibility.

The test suite is intended to:

- Detect regressions when modifying existing functionality.
- Verify representative solver workflows against reference solutions.
- Confirm that required external dependencies are available and functional.
- Support reproducible development.

The current test suite focuses primarily on core infrastructure and a subset of solver capabilities. Additional tests will be added over time to improve coverage of numerical models, closure laws, input processing, and post-processing functionality.

Note

OpenSTREAM's automated test suite is under active development.

The current tests verify selected functionality, solver workflows, numerical results, and software dependencies, but do not provide comprehensive coverage of the complete OpenSTREAM code base or every implemented physical model and numerical option.

Passing all tests indicates that the covered functionality behaves as expected for the tested cases. It should not be interpreted as complete verification or validation of OpenSTREAM.

3.6.1 Test organization

The OpenSTREAM tests are organized into three categories:

Folder	Purpose
tests/ environment	Verification of the MATLAB-Python environment and required external packages, including basic CoolProp functionality.
tests/unit	Focused tests of individual OpenSTREAM classes, methods, and utilities.
tests/ integration	End-to-end solver workflows, physical consistency checks, numerical regression comparisons, and solver persistence tests.

Unit tests should execute quickly and focus on individual components. Integration tests construct and solve representative OpenSTREAM cases and compare selected calculated quantities with approved reference solutions.

3.6.2 Running all tests

The recommended way to execute the complete test suite is to run `runOpenSTREAMTests.m` from the `tests` folder:

```
runOpenSTREAMTests
```

The test runner:

- Recursively discovers test classes in the `tests` folder and its subfolders.
- Executes the environment, unit, and integration tests, including the numerical regression comparisons.
- Displays detailed progress and failure diagnostics.
- Displays a summary table containing the test class, test method, and result.
- Reports the total execution time of the complete test suite.
- Raises an error if any test fails or does not complete.

After execution, the MATLAB test results and generated summary table are available in the workspace as `testResults` and `testSummary`.

A successful run finishes without errors and reports all tests as passed.

3.6.3 Running individual test categories

Individual test categories can also be run separately from the `tests` folder.

Environment validation tests:

```
suite = matlab.unittest.TestSuite.fromFolder('environment');  
runner = matlab.unittest.TestRunner.withTextOutput;  
results = runner.run(suite);
```

Unit tests:

```
suite = matlab.unittest.TestSuite.fromFolder('unit');  
runner = matlab.unittest.TestRunner.withTextOutput;  
results = runner.run(suite);
```

Integration tests:

```
suite = matlab.unittest.TestSuite.fromFolder('integration');  
runner = matlab.unittest.TestRunner.withTextOutput;  
results = runner.run(suite);
```

The command `results = runner.run(suite)` returns an array of `matlab.unittest.TestResult` objects, with one element for each test method executed. Each element records the test name, pass, failure, and incomplete status, execution duration, and additional framework details.

The results can be inspected directly or converted to a table:

```
disp(results)  
resultTable = table(results);  
disp(resultTable)
```

Failed or incomplete tests can be identified using:

```
failed = results([results.Failed]);  
incomplete = results([results.Incomplete]);  
notPassed = results(~[results.Passed]);
```

3.6.4 Continuous integration

The complete available test suite is executed automatically by the MATLAB CI workflow when changes are pushed to the repository.

After pushing a change, review the MATLAB CI result in GitHub Actions. The workflow fails if any automated test fails or does not complete.

The documentation build and MATLAB CI are separate workflows. A successful documentation build or GitHub Pages deployment does not indicate that the MATLAB tests passed. The status of each workflow should therefore be reviewed independently.

When MATLAB CI fails:

1. Open the failed MATLAB CI workflow.
2. Identify the failing test class and test method in the diagnostics.
3. Reproduce the failure locally using `runOpenSTREAMTests`.
4. Determine whether the failure results from a software defect, an unintended numerical change, a changed environment, or an intentional modification that requires an approved update to the test or reference solution.
5. Run the complete test suite locally before pushing the correction.

3.6.5 Reference solutions

Integration tests compare selected generated results with approved reference solutions.

Reference data are stored under:

```
tests/integration/references/<session-directory>/
```

The reference solutions represent the expected solver behavior for defined test cases. They are version-controlled and form part of the approved regression baseline.

Reference solutions should only be updated when:

- A defect has been corrected.
- A model formulation has intentionally changed.
- Numerical behavior has been intentionally modified and reviewed.
- The approved test environment has changed in a way that justifiably changes the calculated reference results.

Reference solutions should **not** be updated merely to make a failing test pass. Before replacing a reference solution, the numerical differences and their cause should be understood and reviewed.

3.6.6 Adding new tests

When introducing new functionality:

1. Add unit tests whenever possible.
2. Add integration tests if the change affects solver results.
3. Verify that all existing tests continue to pass.
4. Update the documentation if user-visible behavior changes.

As a general rule:

- New testable classes and methods should have focused unit tests.
- New numerical and closure models should have unit tests where possible and integration tests demonstrating their solver-level behavior.
- New external dependencies should have environment tests.

3.6.7 Unit test guidelines

Unit tests should:

- Focus on a single class or function.
- Use small and deterministic inputs.
- Avoid unnecessary file I/O.
- Execute quickly.
- Be independent of one another.

A good unit test verifies one piece of behavior and has a clear failure mode.

3.6.8 Integration test guidelines

Integration tests should, where practical:

- Exercise representative solver workflows.
- Use representative and reproducible input files.
- Verify that solvers reach their expected states.
- Verify propagation of prescribed boundary conditions.
- Check relevant conservation or physical-consistency relationships.
- Compare important calculated quantities with approved reference values.
- Verify that saved solver objects can be reloaded consistently.
- Produce deterministic results suitable for automated execution.

The current solver integration tests verify selected behavior including:

- Final solver states.
- Mixture-solver inlet boundary conditions.
- The mixture steady-state energy balance.
- Transfer of the final pseudo-time initialization state to the first stored physical state.
- Field mass flow rate distributions.
- Field velocity distributions.
- Field enthalpy distributions.
- Solver save-and-load behavior.

Calculated field distributions are compared with the approved reference values using a strict relative tolerance. This avoids false failures caused by insignificant floating-point differences while retaining sensitivity to numerical regressions. Reference values equal to zero remain subject to exact comparison because no absolute tolerance is currently applied.

Future integration tests may additionally verify pressures, void fractions, film thicknesses, exchange terms, transition locations, and other solver-specific quantities.

Not all quantities, solver options, model combinations, physical regimes, error conditions, or post-processing capabilities are currently tested.

3.6.9 Environment validation

Environment tests verify external software requirements.

The current environment tests verify:

- Python availability through MATLAB.
- Python-version accessibility.
- Installation and import of required Python packages.
- Basic CoolProp functionality.

Environment tests help identify installation problems before running larger simulations.

3.6.10 Generated outputs

Some tests generate temporary files, session directories, logs, and saved solver objects.

Generated outputs are stored in category-specific `outputs` directories, including:

```
tests/unit/outputs/  
tests/integration/outputs/
```

Unit-test artifacts are removed automatically during test teardown. Integration-test outputs are retained after execution to support inspection and troubleshooting, and are recreated or replaced by a subsequent test run.

Generated outputs should not normally be committed to the repository. Repository `.gitignore` rules exclude directories named `outputs` from version control.

Approved reference solutions are stored separately under `tests/integration/references` and remain version-controlled as part of the regression baseline.

3.6.11 Troubleshooting

A test failure does not necessarily indicate a software defect.

When a test fails:

1. Read the diagnostic message carefully.
2. Identify the failing test class and method.
3. Determine whether the failure is caused by:
 - A coding error.
 - An unintended numerical change.
 - An intentional model or numerical change.
 - An outdated or inappropriate reference solution.
 - A software-environment or dependency problem.
 - A platform-dependent floating-point difference.
4. Re-run the failing test class or test category locally.
5. Inspect generated outputs and compare the actual and expected values.
6. Verify the expected behavior manually when needed.
7. Update reference solutions only when the change is intentional, understood, justified, and reviewed.
8. Run the complete test suite before pushing the correction.

3.6.12 Contributor checklist

Before submitting a pull request:

- The complete local test suite passes.
- New or modified functionality includes appropriate tests.
- Numerical differences from approved references have been reviewed.
- Reference solutions have been updated only when justified.
- Documentation has been updated when user-visible behavior changes.
- Generated outputs are not committed.

- The MATLAB CI workflow passes after the changes are pushed.

3.7 Troubleshooting

This page provides initial diagnostic guidance for common OpenSTREAM setup, execution, convergence, output, plotting, testing, and documentation problems.

OpenSTREAM calculations depend on several interacting components:

- The OpenSTREAM source revision.
- MATLAB and its active search path.
- The MATLAB-Python interface.
- Python and CoolProp.
- OpenSTREAM input files and selected identifiers.
- The selected solver framework and physical models.
- Numerical settings and convergence criteria.
- Session-directory and file-system permissions.

When investigating a problem, record the relevant error message, software versions, input selections, solver state, and steps required to reproduce the behavior.

3.7.1 Initial diagnostic checklist

Before changing physical models or numerical settings:

1. Confirm the current MATLAB working directory.
2. Confirm that MATLAB locates the intended OpenSTREAM installation.
3. Confirm that the expected Python environment is active.
4. Confirm that CoolProp can be imported.
5. Confirm that all input files and identifiers are correct.
6. Review the complete error message and calculation log.
7. Confirm that generated files are written to the intended location.
8. Check whether the problem is reproducible using an unmodified tutorial or sample case.
9. Record the OpenSTREAM version, release, or Git commit.

Useful commands include:

```
pwd
path
which Inputs.InputSet
which Solvers.Mixture.MixtureSolver
version
pyenv
py.sys.version
py.CoolProp.CoolProp.get_global_param_string('version')
```

When using a development revision, record the Git commit:

```
git rev-parse HEAD
git describe --tags --always --dirty
```

The dirty suffix indicates that the working tree contains local changes that are not part of the identified commit.

3.7.2 MATLAB cannot locate OpenSTREAM

Typical symptoms include:

- MATLAB reports an undefined package, class, function, or variable.
- `Inputs.InputSet` cannot be found.
- A solver class cannot be constructed.
- MATLAB resolves a class from an unintended repository copy.

Add the OpenSTREAM repository root to the MATLAB path:

```
addpath('<path-to-openstream>')
```

Then verify the resolved location:

```
which Inputs.InputSet
which Solvers.Mixture.MixtureSolver
```

The returned paths should refer to the intended OpenSTREAM working copy.

If an unexpected copy is found:

1. Inspect the MATLAB search path.
2. Remove obsolete or duplicate repository paths.
3. Add the intended repository root.
4. Run `rehash` if recently added or renamed files are not detected.
5. Restart MATLAB if cached class definitions remain inconsistent with the source files.

Multiple OpenSTREAM copies on the MATLAB path can produce confusing behavior when classes from different revisions are loaded together.

3.7.3 MATLAB cannot locate OpenSTREAM-database

OpenSTREAM-database is maintained as a separate repository. Both repository roots must be available on the MATLAB path.

```
addpath('<path-to-openstream>')
addpath('<path-to-openstream-database>')
```

Verify the generic dataset class:

```
which Dataset
```

Verify a dataset-specific class using its package-qualified name:

```
which Wurtz1978.Wurtz1978
```

The returned locations should refer to the intended OpenSTREAM-database working copy.

A dataset package should locate its authoritative source file using `getSourceFilePath` rather than relying on the current MATLAB working directory.

3.7.4 Python cannot be loaded

Inspect the Python environment selected by MATLAB:

```
pyenv
```

Display the active Python version:

```
py.sys.version
```

If the selected Python environment is not the intended environment, review the OpenSTREAM installation instructions and the MATLAB documentation for configuring `pyenv`.

After changing the Python environment, MATLAB may need to be restarted before the new environment is used consistently.

Record both the Python executable and version when reporting an environment problem.

3.7.5 CoolProp cannot be imported

Test whether MATLAB can import CoolProp:

```
py.importlib.import_module('CoolProp');
```

Display the active CoolProp version:

```
py.CoolProp.CoolProp.get_global_param_string('version')
```

If the import fails:

- Confirm that CoolProp is installed in the Python environment selected by MATLAB.
- Confirm that the installation uses a Python version compatible with the selected OpenSTREAM environment.
- Confirm that MATLAB is not connected to another Python environment.
- Restart MATLAB after changing the Python installation.
- Run the OpenSTREAM environment tests.

A working Python installation does not by itself confirm that CoolProp is installed in the same environment.

3.7.6 Environment tests fail

Run the complete available test suite from the `tests` folder:

```
runOpenSTREAMTests
```

If an environment test fails:

1. Review the reported MATLAB, Python, and CoolProp versions.
2. Confirm that MATLAB uses the intended Python executable.
3. Test the failing dependency directly.
4. Compare the active environment with the configurations documented for the selected OpenSTREAM release.

5. Review the continuous-integration configuration for tested combinations.

A configuration that is not included in the tested environment matrix should be treated as untested rather than automatically incompatible.

3.7.7 An input file cannot be read

Confirm that the complete file path exists:

```
isfile(modelFilePath)
isfile(optionsFilePath)
isfile(geometryFilePath)
isfile(bcFilePath)
```

Avoid relying on the current working directory. Construct paths using `fullfile` and known repository or project roots:

```
modelFilePath = fullfile( ...
    openstreamPath, ...
    'inputs', ...
    'models.inp');
```

Review:

- File spelling and capitalization.
- File extensions.
- Access permissions.
- Network-drive availability.
- Whether the file is empty or incomplete.
- Whether the path refers to a file from another branch or repository copy.

If the file was generated by another workflow, review that workflow for errors before modifying the OpenSTREAM reader.

3.7.8 An input identifier cannot be found

OpenSTREAM input files can contain several named input sets. Confirm that the selected identifier exists in the corresponding file.

Review each file and identifier independently:

- Model file and model identifier.
- Numerical-option file and option identifier.
- Geometry file and geometry identifier.
- Boundary-condition file and case identifier.

An identifier from one file revision may not exist in another revision. Confirm that the source code, documentation, tutorials, and input files belong to compatible OpenSTREAM revisions.

3.7.9 OpenSTREAM reports inconsistent or missing inputs

Review the generated or maintained input files before changing the solver.

Confirm that:

- Required geometry quantities are present.

- Boundary conditions use SI units.
- Absolute temperatures use kelvin.
- Wall meshes and wall-power distributions have compatible lengths.
- The sum of the wall-mesh intervals is consistent with the modeled length.
- Selected models are valid identifiers.
- Numerical options contain valid values.
- Multi-wall inputs contain consistent information for each wall.

When using OpenSTREAM-database, also review the selected dataset record and the generated input files.

A source-data implementation can load successfully while still containing a physically inconsistent geometry, power, unit conversion, or boundary condition.

3.7.10 A session directory already exists

An existing session can prevent a new calculation from using the same location.

The `overwriteSessionFiles` setting controls whether existing session files can be replaced:

```
inputSetOpts = { ...  
    'overwriteSessionFiles', true, ...  
    'LOGMODE', 'BOTH'};
```

Use overwriting only when the existing output can be replaced safely.

If previous results must be retained:

3.8 Tutorials

Ready to get hands-on with **OpenSTREAM**? These tutorials introduce the principal solver frameworks, input formats, numerical workflows, and post-processing capabilities of OpenSTREAM, progressing from a basic example to more advanced simulation and analysis tasks.

Each tutorial is provided as a MATLAB Live Script in the OpenSTREAM `tutorials` folder. The Live Scripts can be executed, modified, and extended directly in MATLAB. Exported HTML versions are also available through the links below.

Note

The tutorials are intended to be executed interactively as MATLAB Live Scripts. Some tutorials include animations and interactive figures that are displayed in separate pop-up windows when run from the MATLAB Live Editor. These interactive features and their controls are generally not available in the exported HTML versions.

3.8.1 1. Quick Start

Open Tutorial 1: Quick Start

Run a provided OpenSTREAM case using the mixture, two-fluid, three-field, and four-field solver frameworks. This tutorial introduces input loading, solver construction, basic axial plots, inspection of selected results, and saving solver objects. It is the recommended starting point for new users.

3.8.2 2. Mixture Solver Capabilities

Open Tutorial 2: Mixture Solver Capabilities

Explore homogeneous and nonhomogeneous mixture formulations, including fixed-slip and drift-flux models. The tutorial also introduces empirical and relaxation-based thermal-nonequilibrium models, pressure-drop components, wall-friction and local-loss options, and a physical transient calculation.

3.8.3 3. Two-Fluid Solver Capabilities

Open Tutorial 3: Two-Fluid Solver Capabilities

Construct and run two-fluid calculations with separate liquid and vapor fields. Examine phase mass flow rates, velocities, enthalpies, temperatures, hydrodynamic and thermal nonequilibrium, interfacial area, flow-regime indicators, phase exchange terms, pseudo-time convergence, and physical transient behavior.

3.8.4 4. Three-Field Solver Capabilities

Open Tutorial 4: Three-Field Solver Capabilities

Model annular flow using separate liquid-film and entrained-droplet fields together with the mixture-model vapor solution. Compare onset-of-annular-flow initialization methods, momentum formulations, entrainment and deposition closures, film thickness, mass and momentum exchange terms, pseudo-time convergence, and transient behavior.

3.8.5 5. Four-Field Solver Capabilities

Open Tutorial 5: Four-Field Solver Capabilities

Extend the three-field formulation by separating the liquid film into base-film and disturbance-wave fields. Examine film decomposition, wave geometry and frequency, field fractions, onset-of-annular-flow splitting, equilibrium and relaxation models, momentum formulations, field exchange terms, pseudo-time convergence, and a physical transient calculation.

3.8.6 6. Input Files

Open Tutorial 6: Input Files

Learn how OpenSTREAM cases are assembled from physical-model, numerical- option, geometry, and boundary-condition inputs. This tutorial demonstrates input-set selection, inspection of imported objects, loading equivalent model definitions from INP and JSON files, and conversion of native input files to JSON.

3.8.7 7. Numerical Convergence and Solver Diagnostics

Open Tutorial 7: Numerical Convergence and Solver Diagnostics

Distinguish local pointwise convergence, pseudo-time convergence, and physical-time advancement. Inspect solver states and initialization histories, examine the influence of relaxation factors and the pseudo-time step, troubleshoot convergence difficulties, and assess whether a converged solution is numerically consistent.

3.8.8 8. Mesh and Time-Step Sensitivity

Open Tutorial 8: Mesh and Time-Step Sensitivity

Evaluate the sensitivity of mixture-solver results to spatial and temporal discretization. Compare steady-state solutions calculated with different axial meshes and transient responses calculated with different physical time steps. The tutorial also discusses local resolution, peak values, transition timing, computational effort, and the distinction between a sensitivity assessment and a systematic convergence study.

3.8.9 9. Advanced Visualization and Post-Processing

Open Tutorial 9: Advanced Visualization and Post-Processing

Apply the principal OpenSTREAM visualization and post-processing capabilities to a transient solution. Generate animated axial distributions, time histories at selected elevations, and time-elevation surfaces. Extract calculated quantities, assemble MATLAB tables, export selected results to CSV, and save and reload solver objects for additional analysis.

Warning

The physical models, closure relations, numerical settings, and solver options selected throughout these tutorials are intended for educational and demonstration purposes.

Their use in a tutorial does not imply that they are appropriate, recommended, calibrated, verified, or validated for a particular application. Before applying OpenSTREAM to a technical study, assess the assumptions, applicability range, calibration basis, numerical sensitivity, and validation evidence associated with every selected model.

APPLICATIONS

4.1 Contributing to the database

Contributions to OpenSTREAM-database are welcome. Contributions may include new publicly available datasets, corrections to existing dataset implementations, additional application cases, improved plotting and comparison methods, uncertainty information, validation metrics, documentation, and project workflows.

OpenSTREAM-database complements the core OpenSTREAM repository. Contributions related to solver frameworks, physical models, closure models, numerical methods, input handling, or other core functionality should normally be made in the OpenSTREAM repository.

Contributions related to experimental datasets and their application, assessment, validation, or publication companion workflows belong in OpenSTREAM-database.

4.1.1 Types of contributions

Contributions may include:

- New publicly available experimental datasets.
- Additional cases from datasets already implemented.
- Corrections to source-data transcription, digitization, conversion, or metadata.
- Improved dataset documentation and bibliographic information.
- Additional OpenSTREAM solver or model configurations.
- New calculated-versus-measured comparisons.
- Experimental uncertainty information.
- Validation metrics and statistical analyses.
- Improved visualization and post-processing methods.
- MATLAB Live Script application and publication companion projects.
- Corrections to existing application workflows.
- Tests and verification capabilities.
- Development of an automated testing and regression framework.

4.1.2 Dataset organization

Dataset implementations should follow the established MATLAB package organization:

```
+DatasetName/  
├── +src/  
│   └── DatasetName.xml  
├── @DatasetName/  
│   ├── DatasetName.m  
│   └── plotResults.m  
└── README.md
```

Additional dataset-specific methods may be included when required. For example:

```
+DatasetName/  
├── +src/  
│   └── DatasetName.xml  
├── @DatasetName/  
│   ├── DatasetName.m  
│   ├── plotResults.m  
│   └── plotTrends.m  
└── README.md
```

The package components have the following roles:

+src

Contains the authoritative XML or JSON source-data representation. All stored numerical values must use SI units, with absolute temperatures expressed in kelvin.

@DatasetName

Contains the dataset-specific class and separately implemented methods.

DatasetName.m

Defines the dataset-specific class, which inherits from the generic `Dataset` class.

plotResults.m

Provides dataset-specific plotting and calculated-versus-measured comparison functionality.

README.md

Documents the original experimental source, implemented data, units, uncertainty, assumptions, processing, usage, verification, attribution, and limitations.

A dataset package should normally maintain only one authoritative machine-readable source file. If XML and JSON representations are both retained, the dataset README must identify the authoritative representation, explain how the alternative representation is generated, and describe how consistency is verified.

The dataset-specific class should inherit from the generic `Dataset` class and use the shared dataset interface where practical.

4.1.3 Dataset template

OpenSTREAM-database includes a `+Template` package that provides the starting structure for a new dataset implementation.

The template contains:

- A `+src` folder containing guidance for adding the authoritative XML or JSON source-data file.
- An `@Template` class folder.

- A template `Template.m` class that inherits from the generic `Dataset` class.
- A template `plotResults.m` method.
- A template `README.md` containing the expected dataset documentation structure.

The template follows this organization:

```
+Template/
├── +src/
│   └── README.md
├── @Template/
│   ├── Template.m
│   └── plotResults.m
└── README.md
```

To begin implementing a new dataset:

1. Copy the complete `+Template` package.
2. Select a valid dataset name following the naming convention described below.
3. Rename the package folder, class folder, and MATLAB class file consistently.
4. Update the class definition, `addPath` method, comments, and documentation.
5. Add the authoritative XML or JSON source-data file under `+src`.
6. Add all fields required by each supported experimental case.
7. Add and document any required dataset-specific fields.
8. Complete the dataset-specific properties and methods.
9. Complete the dataset `README` using the template headings.
10. Implement dataset-specific result plotting where applicable.
11. Verify source-data loading, input generation, case execution, and post-processing.

The template supplies the expected structure and documentation placeholders. Contributors remain responsible for completing and verifying all dataset-specific source data, metadata, methods, assumptions, unit conversions, uncertainties, plotting functionality, and application workflows.

4.1.4 Naming convention

Dataset packages follow a strict naming convention. The package folder, class folder, class file, class definition, dataset name, and source-data filename must use consistent names.

For a dataset named `AuthorYear`, the expected structure is:

```
+AuthorYear/
├── +src/
│   └── AuthorYear.xml
├── @AuthorYear/
│   ├── AuthorYear.m
│   └── plotResults.m
└── README.md
```

The following names must agree:

Element	Required form
MATLAB package folder	<code>+AuthorYear</code>
MATLAB class folder	<code>@AuthorYear</code>
MATLAB class file	<code>AuthorYear.m</code>
MATLAB class definition	<code>classdef AuthorYear < Dataset</code>
Dataset name assigned in <code>addPath</code>	<code>data.name = 'AuthorYear'</code>
Source-data filename	<code>AuthorYear.xml</code>
Source-data location	<code>+AuthorYear/+src/AuthorYear.xml</code>

The leading + and @ characters are part of MATLAB package and class-folder syntax:

- `+AuthorYear` defines the MATLAB package.
- `@AuthorYear` defines the class folder within that package.
- `AuthorYear.m` contains the class definition.
- The class is referenced using the package-qualified name `AuthorYear.AuthorYear`.

For publication-based datasets, the package name should generally combine the surname of the first author and the four-digit publication year, following the convention used by existing dataset packages.

If this combination is already used by another package, add a concise qualifier that unambiguously identifies the publication. Prefer the second author's surname or a short title keyword, for example `AuthorCoauthor2020` or `Author2020CHF`.

The suffixes a, b, and so forth may be used when closely related publications are conventionally cited as `Author2020a`, `Author2020b`, and so forth. The assignment of these suffixes must be documented and remain stable.

The capitalization of author names and title keywords must be used consistently throughout the package name, class name, source filename, and documentation. For example, do not mix `Author2020`, `author2020`, and `AUTHOR2020`.

The dataset name must be a valid MATLAB identifier. Spaces, hyphens, punctuation, and other invalid identifier characters must not be used.

When copying the template, update every occurrence of `Template` in:

1. The package-folder name.
2. The class-folder name.
3. The class filename.
4. The `classdef` declaration.
5. `data.name` in `addPath`.
6. The source-data filename.
7. Code examples and headings in `README.md`.
8. Project scripts that construct or refer to the dataset class.

For example:

```
classdef AuthorYear < Dataset
    % AUTHORYEAR Dataset implementation for Author et al. (Year).

    methods

        function addPath(data)
```

(continues on next page)

(continued from previous page)

```

% ADDPATH Define the dataset name and source-data file.

data.name = 'AuthorYear';
data.path = ...
    data.getSourceFilePath('AuthorYear.xml');
end

end

end

```

Use `getSourceFilePath` rather than a hard-coded relative or absolute path. This allows the dataset package to locate its source file independently of the current MATLAB working directory.

Separately implemented methods in the `@AuthorYear` folder do not need redundant declarations in the main class file.

The resulting object is constructed using the package-qualified class name:

```
data = AuthorYear.AuthorYear(...);
```

A mismatch between package, class, or source-data names may prevent MATLAB from locating the class or may cause the dataset implementation to reference the wrong source-data file.

4.1.5 Source data

Dataset source files are stored under the package `+src` directory.

XML and JSON are supported source formats. A dataset package should normally maintain only one authoritative machine-readable source representation.

If XML and JSON representations are both retained, the dataset README must state:

- Which representation is authoritative.
- How the alternative representation is generated.
- Whether contributors may edit the alternative representation.
- How consistency between the representations is verified.

Equivalent XML and JSON files should not be maintained independently without a documented synchronization and verification procedure.

All numerical data stored in OpenSTREAM-database source files must use SI units. Absolute temperatures must be expressed in kelvin. This requirement applies even when the original experimental publication reports data using another system of units.

The source-data implementation should preserve the original experimental information as faithfully as possible. Any transcription, correction, character recognition, digitization, filtering, interpolation, reconstruction, unit conversion, or derivation must be documented.

Missing numerical information must be represented explicitly and must not be interpreted as zero unless zero is the reported experimental value. The missing-value convention must be documented in the dataset README and handled explicitly by the dataset-specific implementation.

The dataset object reads and interprets the corresponding XML or JSON source file during construction. Reading the source file is therefore part of dataset-object construction and is not a separate step in the application workflow.

4.1.6 Lightweight datasets

OpenSTREAM-database can also be used without an XML or JSON source-data file.

A case can be defined directly using a one-row MATLAB table and passed to a lightweight Dataset object:

```
data = Dataset( ...
    1, ...
    'isLightWeight',true, ...
    'lightWeightEntryData',entryData);
```

This workflow is suitable for:

- Publication demonstration cases.
- Concise examples and tutorials.
- Exploratory calculations.
- User-defined cases that do not require a maintained dataset package.
- Comparisons of several OpenSTREAM solver frameworks using a common case definition.

The table must contain the geometry, boundary-condition, wall-heating, and fluid information required by the selected OpenSTREAM workflow. All numerical values must follow the same SI-unit convention as file-based datasets.

A lightweight case should not replace a maintained dataset package when experimental provenance, measurement uncertainty, repeated case selection, or long-term reuse must be documented.

4.1.7 Dataset fields

Each experimental case in the XML or JSON source file must define the fields required by its intended OpenSTREAM workflow. Additional dataset-specific fields may be added when required to represent the available experimental information.

Mandatory fields

The following fields are normally required for a complete heated-channel case:

Field	Description	SI unit
Fluid	Working-fluid identifier used by OpenSTREAM.	Not applicable
Pressure	System pressure.	Pa
MassFlow	Inlet mass flow rate.	kg/s
InletEnthalpy	Inlet specific enthalpy.	J/kg
Length	Axial length of the modeled channel.	m
Area	Channel flow area.	m ²
Perimeter	Channel perimeter used by the OpenSTREAM geometry.	m
Power	Total applied power.	W
WallMesh	Axial wall-interval lengths associated with the wall-power distribution.	m
WallPower	Relative axial wall-power distribution.	Dimensionless

Field names are case-sensitive and must be written exactly as expected by the input-generation implementation.

The mandatory fields must satisfy the following requirements:

- Every supported case must define the fields required by its workflow.
- All numerical values must use SI units.

- Absolute temperatures must be expressed in kelvin.
- Fluid must use an identifier supported by the OpenSTREAM fluid-property interface.
- WallMesh and WallPower must have compatible lengths and ordering.
- The sum of WallMesh must be consistent with the modeled channel length.
- Geometry fields must represent the channel modeled by OpenSTREAM.
- Boundary-condition fields must represent the intended experimental case.
- Any conversion, reconstruction, or interpretation used to obtain a field must be documented in the dataset README.

Fields that are not required for a particular workflow may be omitted only when the generic input-generation implementation and dataset-specific methods support their absence.

Dataset-specific fields

Additional fields may be included when required to represent information specific to the dataset.

Dataset-specific fields may describe:

- Measured quantities.
- Instrument and measurement locations.
- Experimental uncertainties.
- Facility or test-section characteristics not represented by the mandatory geometry fields.
- Inlet, outlet, or local experimental conditions.
- Flow-regime or transition information.
- Dataset-specific case classifications.
- Information required for calculated-versus-measured comparisons.
- Quantities derived during source-data preparation or post-processing.

Each dataset-specific field must be documented in the dataset README. The documentation should state:

- The exact field name.
- The physical meaning of the field.
- Whether the field is required or optional.
- Confirmation that all numerical data use SI units.
- The unit used for the implemented quantity whenever clarification is useful.
- The unit reported in the original experimental source, when different.
- The source table, figure, page, appendix, or data file.
- Any unit conversion, transcription, character recognition, digitization, interpolation, derivation, or other processing applied.
- The reported measurement uncertainty, where available.
- The behavior when the field is absent or unavailable.
- Whether the quantity is measured, prescribed, transcribed, digitized, reconstructed, derived, or calculated.

A general SI-unit statement is sufficient where the representation is unambiguous. Units must be stated explicitly where clarification is needed. Absolute temperatures must be identified as being expressed in kelvin.

A field name must not imply an incorrect physical unit. For example, a field named `InletSubcooling` may represent an enthalpy difference in J/kg rather than a temperature difference. Such conventions must be stated explicitly.

Dataset-specific fields must not replace, rename, or change the meaning of the mandatory fields.

When the same physical quantity is represented by several dataset implementations, a common field name and representation should be used where practical.

Source-file requirements

The XML or JSON source representation should use a consistent structure across all implemented cases.

Before completing a dataset implementation, confirm that:

- Every supported case contains the fields required by its workflow.
- Mandatory and dataset-specific field names are used consistently.
- Numerical fields contain valid SI values.
- Absolute temperatures are expressed in kelvin.
- Related array dimensions and ordering are mutually consistent.
- `WallMesh` and `WallPower` describe compatible axial distributions.
- Case identifiers are unique.
- Missing experimental information is represented explicitly.
- Missing numerical information is not represented as zero unless zero is the reported value.
- Undocumented replacement values are not introduced.
- Derived, reconstructed, or calculated fields are identified and documented.
- Unsupported or incomplete cases are identified clearly.

The dataset class should validate the fields required by its implementation and report missing, inconsistent, or unsupported data with a clear diagnostic.

4.1.8 Data provenance

Every dataset contribution must identify the authoritative experimental source.

The dataset README should include:

- The complete bibliographic reference.
- A link to the publication or public data source, when available.
- A description of the experimental facility.
- The relevant test section and geometry.
- The measured and prescribed quantities.
- The range of experimental conditions.
- Confirmation that all stored numerical data use SI units, with absolute temperatures expressed in kelvin.
- The reported measurement uncertainties.
- Known limitations or qualifications of the data.

- The relationship between the original data and the implemented source-data file.
- Any licensing, redistribution, attribution, or citation requirements.

The source-data preparation method must be stated explicitly. Applicable methods include:

- Manual transcription from publication tables.
- Character recognition from scanned reports.
- Figure digitization.
- Import from publicly available machine-readable data.
- Unit conversion.
- Interpolation or reconstruction.
- Derivation of additional quantities required for OpenSTREAM input generation or post-processing.

Where applicable, document the checks performed to improve accuracy and internal consistency, including:

- Geometry consistency.
- Mass-flow, mass-flux, and area consistency.
- Power reconstruction.
- Energy-balance checks.
- Unit-conversion checks.
- Array-length and ordering checks.
- Comparison of repeated or related cases.
- Review against the original tables or figures.

The original publication or public data source remains authoritative for the experimental configuration, measurements, uncertainties, and interpretation.

The OpenSTREAM-database implementation must not imply greater accuracy, completeness, or certainty than is supported by the published information.

4.1.9 Dataset class

The dataset-specific MATLAB class should:

- Inherit from the generic `Dataset` class.
- Read and interpret its source-data representation during object construction.
- Define its package name and source-data file through `addPath`.
- Use `getSourceFilePath` to locate the authoritative source file.
- Preserve the generic dataset interface where practical.
- Identify the available experimental cases and quantities.
- Generate valid OpenSTREAM inputs.
- Operate consistently using SI units.
- Handle missing experimental values explicitly.
- Avoid hard-coded local file-system paths.
- Avoid redundant declarations for methods already implemented separately in the `@DatasetName` folder.

- Use clear property and method names.
- Document assumptions and transformations.
- Validate inputs and report unsupported selections clearly.
- Support reproducible execution across the intended environments.

Functionality common to several datasets should be considered for implementation in the generic `Dataset` class rather than duplicated across dataset-specific classes.

4.1.10 OpenSTREAM dependency

OpenSTREAM is maintained as a separate repository and is not included as a Git submodule of OpenSTREAM-database.

Dataset implementations and application projects should rely on the required OpenSTREAM classes being available on the MATLAB path. They should not assume a fixed relative directory arrangement between the two repositories.

Installation and MATLAB path instructions are provided on the [OpenSTREAM-database](#) page.

4.1.11 Application projects

Application projects are stored under the repository `projects` folder. They may use a dataset package or a lightweight in-memory `Dataset` object.

A contributed project should provide a reproducible workflow for:

1. Constructing a dataset-specific object, which reads and interprets the corresponding XML or JSON source file, or constructing a lightweight `Dataset` object from an in-memory table.
2. Selecting one or more experimental or demonstration cases.
3. Selecting an OpenSTREAM solver and model configuration.
4. Generating the required OpenSTREAM input files.
5. Running the selected cases.
6. Reviewing solver convergence.
7. Extracting calculated and measured quantities.
8. Comparing the results.
9. Visualizing and interpreting the comparison.
10. Documenting assumptions, numerical settings, and limitations.

Projects should avoid hard-coded local paths and should state the expected working directory or MATLAB path configuration clearly.

MATLAB Live Scripts are recommended when explanatory text, executable code, figures, calculated results, convergence information, and interpretation form part of the documented application workflow.

Projects intended to support publications or technical reports should follow the reproducibility guidance provided in the OpenSTREAM scope and limitations page.

Publication companion workflows

A publication companion Live Script should reproduce or illustrate selected calculations, figures, and results presented in the corresponding publication.

The companion workflow must not be described as a complete record of the research programme. Research activities generally include additional test cases, closure-model development, sensitivity studies, uncertainty analysis, numerical investigations, intermediate results, and unsuccessful model variants that are not included in the published figures.

The project documentation should identify:

- The corresponding publication.
- The figures or results reproduced.
- The experimental datasets used.
- Important nondefault physical and numerical settings.
- Calculations or comparisons from the publication that are not reproduced.
- The limitations of the companion workflow.

The MATLAB Live Script is the authoritative executable version. A completed execution may also be exported to HTML for viewing without MATLAB. Interactive Live Editor features may not be preserved in the HTML export.

4.1.12 Comparison and validation

Calculated-versus-measured comparisons should identify:

- The experimental quantity being compared.
- Whether the experimental value is directly measured or derived.
- The corresponding OpenSTREAM quantity.
- The location and time associated with the comparison.
- The SI unit used for the comparison.
- The selected solver and physical models.
- The numerical settings.
- The spatial and temporal discretization.
- The experimental uncertainty, where available.
- The comparison metric, where applicable.
- Any filtering, averaging, smoothing, interpolation, or alignment procedure.

A favorable comparison for one case, quantity, dataset, or model selection must not be described as general validation outside the investigated conditions.

Contributors should distinguish between:

- Verification that an implementation behaves as intended.
- Validation against experimental observations.
- Numerical sensitivity to mesh, time step, convergence settings, and solver options.
- Sensitivity to closure-model and parameter selections.
- Experimental, model, parameter, and numerical uncertainty.

4.1.13 Plotting and post-processing

Dataset-specific plotting methods should:

- Use SI units.
- Use kelvin for absolute temperatures.
- Identify measured, derived, and calculated quantities clearly.
- Include readable axis labels, units, legends, and captions.
- Distinguish solvers and model configurations consistently.
- Represent experimental uncertainty when available and relevant.
- Avoid implying agreement beyond the precision or uncertainty of the available data.
- Support reproducible use from the corresponding application project.
- Ensure that smoothing used for visualization does not modify the stored source data.
- Keep original measurement points visible or otherwise traceable when smoothed curves are shown.

Common plotting or comparison behavior should be implemented in shared functionality where practical.

4.1.14 Testing and verification

OpenSTREAM-database does not currently include an automated test suite.

Until automated testing is implemented, contributors should verify changes using reproducible checks appropriate to the contribution.

These checks should include, where applicable:

- Confirming that the source-data files can be loaded correctly.
- Confirming that required fields are present.
- Confirming that all numerical source data use SI units.
- Confirming that absolute temperatures use kelvin.
- Reviewing unit conversions against the original experimental source.
- Comparing manually transcribed, character-recognized, or digitized values against the original source.
- Performing relevant geometry, power, unit, and energy-balance checks.
- Verifying that the intended experimental cases can be selected.
- Reviewing the generated OpenSTREAM input files.
- Running the affected application cases.
- Confirming that the selected OpenSTREAM calculations complete and converge as expected.
- Comparing calculated and measured quantities.
- Confirming that missing measurements are not interpreted as zeros.
- Confirming that direct measurements and derived quantities are distinguished.
- Reviewing generated figures and post-processing results.
- Confirming that existing application workflows are not adversely affected.
- Comparing results with previous approved executions, when available.

The verification procedure and its results should be described in the pull request.

Automated testing is a planned improvement. Future tests may cover:

- Source-data parsing.
- SI-unit conversion.
- Dataset metadata.
- Required-field validation.
- Case selection.
- OpenSTREAM input generation.
- Dataset-specific plotting and comparison methods.
- Application workflows.
- Numerical regression against approved results.

4.1.15 Documentation requirements

Each new dataset contribution should include a package README describing:

- The original experimental source and complete bibliographic reference.
- The scope of the implemented data.
- The experimental facility and relevant geometry.
- The implemented experimental conditions and measured quantities.
- The mandatory and dataset-specific fields.
- Confirmation that all numerical data use SI units.
- Confirmation that absolute temperatures use kelvin.
- Any unit conversions, manual transcription, character recognition, digitization, interpolation, filtering, reconstruction, or other data processing.
- Any sanity checks or consistency checks performed.
- The reported measurement uncertainties, where available.
- Known limitations, ambiguities, or missing information.
- Supported OpenSTREAM application workflows.
- A minimal reproducible usage example.
- Citation, licensing, attribution, and redistribution requirements.

User-visible changes should also be reflected in the OpenSTREAM Applications documentation when they affect:

- The OpenSTREAM-database description.
- Available datasets.
- Publication companion workflows.
- Exported project workflows.
- Publications.
- Installation or path requirements.
- Recommended application procedures.

4.1.16 Commit messages

Commit messages must follow the conventions documented in the [OpenSTREAM commit message style guide](#).

Contributors should organize changes into focused commits. Source-data corrections should normally be committed separately from code and documentation changes so that their provenance and verification remain clear in the repository history.

4.1.17 Submitting a contribution

Before submitting a contribution:

1. Confirm that OpenSTREAM and OpenSTREAM-database are both available on the MATLAB path.
2. Perform the relevant reproducible verification checks.
3. Run the affected application projects.
4. Review generated OpenSTREAM inputs and calculated results.
5. Confirm that all numerical source data use SI units.
6. Confirm that absolute temperatures use kelvin.
7. Review unit conversions and retained numerical precision.
8. Verify bibliographic references and dataset provenance.
9. Review applicable licensing, attribution, and redistribution conditions.
10. Identify one authoritative machine-readable source representation for each dataset package.
11. Confirm that generated outputs are not included in the commit.
12. Confirm that source PDFs and other copyrighted publications are not included unless redistribution is explicitly permitted.
13. Remove MATLAB autosave files, obsolete copies, generated outputs, local sandbox scripts, and temporary processing files.
14. Update the documentation and project exports when required.
15. Describe intentional numerical changes in the pull request.
16. Organize changes into focused commits using the required message style.

The pull-request description should identify:

- The purpose and scope of the contribution.
- The original experimental source.
- Added or modified cases and quantities.
- Data-processing and unit-conversion steps.
- Affected OpenSTREAM solvers and models.
- The verification procedure performed.
- The results of the verification.
- Numerical differences from previous results.
- Remaining limitations or unresolved questions.

4.1.18 Contributor checklist

Before submitting a pull request, confirm that:

- The contribution follows the established dataset package organization.
- The dataset template was used or the equivalent required structure was followed.
- The strict naming convention is followed.
- `getSourceFilePath` is used to locate the dataset source file.
- One authoritative machine-readable source representation is identified.
- The original experimental source is identified and cited.
- Applicable licensing and redistribution conditions have been reviewed.
- Source PDFs and other copyrighted publications are not unintentionally committed.
- Every implemented case defines the fields required by its supported workflow.
- Generic field names use the exact required spelling and capitalization.
- All mandatory and dataset-specific numerical fields use SI units.
- Absolute temperatures use kelvin.
- `WallMesh` and `WallPower` are mutually consistent.
- Dataset-specific fields are documented in the dataset README.
- Dataset-specific fields do not replace or rename mandatory fields.
- Measured, prescribed, transcribed, digitized, reconstructed, derived, and calculated quantities are distinguished.
- Unit conversions are documented.
- Source-data preparation methods are documented.
- Measurement uncertainties are included where available.
- Dataset assumptions and processing steps are documented.
- Relevant sanity checks and consistency checks have been performed.
- Missing numerical information is not interpreted as zero.
- The affected dataset and application workflows have been verified.
- The verification procedure and results are documented.
- Generated OpenSTREAM inputs and calculated results have been reviewed.
- Any numerical differences from previous results are understood and justified.
- Documentation and application projects have been updated.
- Publication companion workflows are not presented as complete research records.
- Generated input, result, session, and output directories are not committed.
- Temporary files, MATLAB autosaves, obsolete copies, local scripts, and processing files are not committed.
- Commit messages follow the OpenSTREAM style guide.

4.2 OpenSTREAM-database

OpenSTREAM-database provides an application and validation environment for **OpenSTREAM**, enabling users to configure, run, and evaluate OpenSTREAM solvers using publicly available experimental datasets.

OpenSTREAM-database complements the core OpenSTREAM repository:

- **OpenSTREAM** provides the solver frameworks, physical and closure models, numerical methods, input handling, visualization capabilities, tutorials, documentation, and core automated tests.
- **OpenSTREAM-database** provides dataset implementations, application workflows, calculated-versus-measured comparisons, publication companion workflows, and an environment for model assessment and validation.

The two repositories are maintained separately. OpenSTREAM is not included as a Git submodule of OpenSTREAM-database. A compatible OpenSTREAM working copy must be installed independently and made available on the MATLAB path.

4.2.1 Purpose

OpenSTREAM-database supports:

- Application of OpenSTREAM solvers to experimental cases.
- Reproducible generation of OpenSTREAM input files from dataset definitions.
- Execution of individual cases or selected groups of cases.
- Comparison of calculated and measured quantities.
- Development and assessment of physical and closure models.
- Numerical sensitivity and uncertainty studies.
- Documentation of dataset provenance, assumptions, and processing.
- Publication companion workflows illustrating selected calculations, figures, and results.
- Application- and validation-oriented research and solver development.

Separating application datasets from the core solver repository allows OpenSTREAM and OpenSTREAM-database to evolve independently while retaining a common application interface.

4.2.2 Repository organization

The principal repository structure is:

```
openstream-database/  
├── +DatasetName/  
├── +Template/  
├── @Dataset/  
├── projects/  
├── sandbox/  
├── LICENSE  
├── README.md  
└── functionSignatures.json
```

Each dataset is implemented as a MATLAB package that generally contains:

```
+DatasetName/
├── +src/
│   └── DatasetName.xml
├── @DatasetName/
│   ├── DatasetName.m
│   └── plotResults.m
└── README.md
```

Additional dataset-specific methods may be provided when required, for example for post-processing or trend analysis.

The package components have the following roles:

+src

Contains the structured experimental data and the information required to define OpenSTREAM cases.

@DatasetName

Contains the dataset-specific class and associated methods.

plotResults.m

Provides dataset-specific visualization and calculated-versus-measured comparison functionality.

README.md

Documents the original experimental source, implemented data, units, uncertainties, assumptions, processing methods, supported workflows, and known limitations.

The generic `Dataset` class provides functionality shared by the individual dataset implementations, including source-data handling, case selection, OpenSTREAM input generation, case execution, convergence review, and result storage.

The `+Template` package provides the recommended starting structure for new dataset contributions.

The `projects` folder contains publication companion workflows and other reviewed application projects.

The `sandbox` folder provides a local workspace for exploratory OpenSTREAM-database calculations. Temporary sandbox scripts, generated inputs, results, logs, figures, and saved objects are not maintained as repository content.

4.2.3 Dataset source files

Experimental cases and associated information are normally stored in a structured XML or JSON file under the dataset package's `+src` directory.

XML and JSON are supported source formats. A dataset package should normally maintain only one authoritative machine-readable source file. If equivalent XML and JSON representations are both retained, the dataset README must identify the authoritative source, explain how the alternative representation is generated, and describe how consistency is verified.

Each case can contain two categories of information:

1. **OpenSTREAM case-definition information**, whose field names follow a strict naming convention so that the generic `Dataset` functionality can identify geometry, boundary conditions, wall heating, and other information passed to OpenSTREAM.
2. **Dataset-specific information**, whose field names and contents can be defined according to the needs of the dataset implementation.

Dataset-specific information can include experimental measurements, metadata, classifications, uncertainties, derived quantities, and other information used by the dataset class or application workflows. For example, a dataset-specific `plotResults` method can use these fields to construct calculated-versus-measured comparisons.

The reserved OpenSTREAM field names are case-sensitive and must use the spelling expected by the input-generation implementation. Other fields can be added freely, provided that they do not conflict with reserved names and are interpreted consistently by the corresponding dataset class.

Scalar quantities are stored as individual values. Array quantities are represented by repeated XML elements or JSON arrays. Related arrays must have compatible lengths, ordering, and physical meaning.

Missing numerical measurements must be represented consistently and must not be interpreted as zero. The convention used for a dataset should be documented in its README and handled explicitly by dataset-specific methods.

Detailed source-file conventions should be documented in the corresponding dataset README and class implementation.

4.2.4 Lightweight in-memory datasets

OpenSTREAM-database can also be used without an XML or JSON source-data file.

A case can be defined directly using a one-row MATLAB table and passed to a lightweight Dataset object:

```
data = Dataset( ...  
    1, ...  
    'isLightWeight',true, ...  
    'lightWeightEntryData',entryData);
```

This in-memory workflow is useful for:

- publication demonstration cases;
- concise examples and tutorials;
- exploratory calculations;
- user-defined cases that do not require a maintained dataset package;
- comparisons of multiple OpenSTREAM solver frameworks using a common case definition.

The table must contain the geometry, boundary-condition, wall-heating, and fluid fields required by the selected OpenSTREAM workflow. Numerical values must follow the same SI-unit conventions as file-based datasets.

A lightweight case is not a substitute for a maintained dataset package when experimental provenance, uncertainty, repeated case selection, or long-term reuse must be documented.

4.2.5 Units

OpenSTREAM uses SI units for all inputs, calculated quantities, and stored results.

OpenSTREAM-database follows the same convention. All numerical data stored in dataset source files or supplied through lightweight in-memory records must use SI units. Absolute temperatures are expressed in kelvin.

Data converted from other unit systems must be checked and documented. Units may be stated explicitly for individual fields or established through a clear dataset-wide convention. Units must be stated whenever clarification is useful, particularly for quantities whose representation could otherwise be ambiguous.

For example, a field called `InletSubcooling` may represent an enthalpy difference in J/kg rather than a temperature difference. Such conventions must be stated explicitly in the dataset README.

The original publications should be consulted for the reported quantities, measurement definitions, original units, and experimental uncertainties.

4.2.6 Data preparation and provenance

Dataset values may be obtained from publication tables, supplementary data, public data repositories, or digitized figures.

The dataset README must document the preparation method, including, as applicable:

- manual transcription from tables;

- character recognition from scanned reports;
- figure digitization;
- unit conversion;
- interpolation or reconstruction;
- corrections applied after review;
- derived quantities added for OpenSTREAM input generation;
- sanity checks and consistency checks.

Relevant checks can include geometry consistency, mass-flow and mass-flux consistency, power reconstruction, energy balances, array-length checks, unit checks, and comparison of related experimental cases.

The original publication or public data source remains authoritative. Machine-readable dataset implementations can contain transcription, recognition, digitization, conversion, or processing errors even after review.

Direct measurements must be distinguished from quantities derived during source-data preparation, calculation, or post-processing.

4.2.7 Application projects

The `projects` folder contains reviewed MATLAB Live Scripts that accompany OpenSTREAM publications and illustrate selected computational workflows, figures, and results presented in those publications.

A companion Live Script can include:

- explanatory text;
- selected experimental cases;
- physical-model and numerical-option selections;
- executable OpenSTREAM calculations;
- convergence information;
- calculated-versus-measured comparisons;
- figures and post-processing;
- interpretation of the selected results.

The companion workflows do not represent complete records of the research activities supporting the publications. Research and model-development activities generally include additional experimental cases, closure-model evaluations, sensitivity studies, uncertainty analyses, numerical investigations, intermediate results, and unsuccessful model variants that are not included in the published figures or companion workflows.

The MATLAB Live Scripts are the authoritative executable versions of the publication companion workflows. Completed executions may also be exported as static HTML documents containing explanatory text, code, calculated output, and figures. HTML exports can be viewed without MATLAB, although interactive Live Editor features may not be preserved.

Further information about the available projects is provided on the [application projects](#) page.

4.2.8 Requirements

OpenSTREAM-database requires:

- A compatible MATLAB installation.
- A working OpenSTREAM installation.

- The Python and CoolProp environment required by OpenSTREAM.
- Access to the OpenSTREAM-database repository.

OpenSTREAM is maintained as a separate repository and must be installed independently. Consult the [OpenSTREAM installation guide](#) for the current MATLAB, Python, CoolProp, and installation requirements.

4.2.9 Installation

Clone OpenSTREAM and OpenSTREAM-database as separate repositories.

A convenient local organization is:

```
projects/  
├── openstream/  
└── openstream-database/
```

The repositories do not need to share the same parent directory, provided that both repository roots are available on the MATLAB path.

4.2.10 MATLAB setup

Add both repository roots to the MATLAB path:

```
addpath('<path-to-openstream>')  
addpath('<path-to-openstream-database>')
```

Verify that MATLAB can locate functionality from both repositories:

```
which Inputs.InputSet  
which Dataset
```

The returned paths should refer to the intended OpenSTREAM and OpenSTREAM-database working copies.

4.2.11 Typical workflow

A typical file-based dataset workflow consists of:

1. Constructing a dataset-specific object from its source-data file.
2. Selecting one or more experimental cases.
3. Initializing the corresponding OpenSTREAM inputs.
4. Selecting an OpenSTREAM solver and model configuration.
5. Generating the OpenSTREAM input files.
6. Running the selected calculations.
7. Reviewing solver convergence.
8. Extracting calculated quantities.
9. Comparing calculated and measured results.
10. Visualizing and interpreting the comparison.

A lightweight workflow instead constructs a generic `Dataset` object from an in-memory MATLAB table and then follows the same input-generation, execution, convergence-review, and post-processing sequence.

The exact workflow depends on the selected dataset and solver. Consult the dataset README and any corresponding project file before running an application.

4.2.12 Generated files

OpenSTREAM-database may generate input packages and output folders containing:

- OpenSTREAM input files;
- solver session files;
- solver results;
- convergence logs;
- saved solver objects;
- processed comparisons;
- figures;
- other post-processing outputs.

Generated artifacts are excluded from version control through the repository `.gitignore` rules. Source-data files, MATLAB classes, reviewed project source files, templates, and documentation remain version-controlled.

Generated files should not be used as the only record of a calculation. Reproducible work should retain the source dataset, model and numerical configuration, software version or commit, and workflow required to regenerate the results.

4.2.13 Data provenance and citation

The datasets implemented in OpenSTREAM-database are derived from publicly available experimental sources.

When using a dataset:

- Consult and cite the original experimental publication or public data source.
- Review the experimental conditions, measurement definitions, original units, and reported uncertainties.
- Review any conversion, transcription, character recognition, digitization, processing, interpolation, filtering, reconstruction, or assumptions introduced by the implementation.
- Distinguish direct measurements from derived or calculated quantities.
- Identify the OpenSTREAM and OpenSTREAM-database versions or commits used.
- Cite OpenSTREAM and OpenSTREAM-database as appropriate.

Use of an OpenSTREAM-database implementation does not replace citation of the original experimental source.

Licensing of OpenSTREAM-database software and documentation does not supersede rights, licenses, attribution requirements, or redistribution conditions associated with the original publications and experimental data.

4.2.14 Validation and interpretation

Inclusion of a dataset, comparison method, or application workflow does not imply that every OpenSTREAM solver, closure model, model combination, or calculated quantity has been comprehensively verified or validated against that dataset.

Users remain responsible for evaluating:

- The applicability and quality of the experimental data.
- The assumptions and validity ranges of the selected models.
- The effects of dataset processing and derived quantities.
- Numerical convergence and discretization sensitivity.

- Sensitivity to closure-model and numerical-option selections.
- Experimental, model, parameter, and numerical uncertainty.
- The suitability of the selected comparison quantities and metrics.
- The interpretation of calculated-versus-measured differences.

A favorable comparison for one case, quantity, dataset, or model selection must not be interpreted as general validation outside the conditions examined.

4.2.15 Testing and verification

OpenSTREAM-database does not currently include a repository-wide automated test suite.

Dataset implementations and application workflows should therefore be verified through documented and reproducible execution of the affected cases. Verification should include, as applicable:

- successful loading of the source-data records;
- review of source-data provenance and processing;
- consistency checks on geometry, boundary conditions, units, and power;
- review of generated OpenSTREAM inputs;
- confirmation of solver convergence;
- comparison of calculated and experimental quantities;
- confirmation that missing measurements are not interpreted as zeros;
- confirmation that derived quantities are identified as derived;
- sensitivity to relevant physical models and numerical settings.

Detailed review and contribution requirements are provided in the [contribution guidelines](#).

4.2.16 Development status

OpenSTREAM-database and its application workflows are under active development. Dataset implementations, source-file formats, project workflows, comparison methods, documentation, and verification capabilities may continue to evolve.

Users reproducing earlier calculations should review:

- the dataset-specific README;
- the corresponding publication companion, when available;
- the OpenSTREAM and OpenSTREAM-database repository histories;
- the versions or commits used for the original calculation.

4.2.17 Contributing

Contributions of new datasets, source-data corrections, application workflows, publication companions, documentation, comparison methods, and verification capabilities are welcome.

Before contributing, review the [OpenSTREAM-database contribution guidelines](#).

4.3 Application projects

OpenSTREAM-database includes MATLAB Live Script projects that accompany OpenSTREAM publications and illustrate the principal computational workflows and selected results presented in those publications.

The project source files are located in the OpenSTREAM-database `projects` folder:

```
projects/
├── NURETH21.mlx
├── NUTHOS15_MRM.mlx
```

Each Live Script combines explanatory text, model selections, executable OpenSTREAM calculations, convergence information, figures, and interpretation. The objective is to provide a transparent and reproducible workflow for the selected calculations and results presented in the corresponding publication.

The companion workflows do not represent the complete research activities supporting the publications. Model development and qualification generally involve additional experimental cases, closure-model evaluations, sensitivity studies, numerical investigations, and intermediate analyses that are not included in the published figures or companion Live Scripts.

The MATLAB Live Scripts are the authoritative executable versions of the published companion workflows. Completed executions may also be provided as static HTML exports containing the calculated output and figures. The HTML exports can be viewed without MATLAB, although interactive Live Editor features may not be preserved.

Before running or extending a project:

- Install OpenSTREAM and OpenSTREAM-database separately and make both repository roots available on the MATLAB path.
- Review the corresponding publication and associated dataset documentation.
- Review the selected experimental cases and generated OpenSTREAM inputs.
- Review the physical-model and numerical-option selections.
- Confirm that the calculations converge as documented.
- Distinguish direct experimental measurements from derived quantities.
- Record the OpenSTREAM and OpenSTREAM-database versions or commits used.

Further installation and configuration information is provided on the [OpenSTREAM-database](#) page.

For calculations used in publications or technical reports, follow the reproducibility guidance.

OpenSTREAM-database does not currently include an automated test suite. Project workflows are therefore verified through reproducible execution, review of convergence, and documented comparison of calculated and experimental quantities.

4.3.1 NURETH21

The NURETH21 project is provided as a MATLAB Live Script:

```
projects/NURETH21.mlx
```

The Live Script is a reproducible computational companion to the NURETH-21 publication:

The workflow reproduces selected OpenSTREAM calculations and results presented in the paper:

- Figures 3 and 4: comparison of the mixture, two-fluid, three-field, and four-field solver frameworks.
- Figure 6: validation of fully developed base-film and disturbance-wave properties using the Wurtz1978 dataset.

- Figure 7: validation of developing disturbance-wave behavior using the Sawai1989 dataset.

The demonstration case also illustrates the lightweight OpenSTREAM-database workflow. The geometry, boundary conditions, and heating conditions are defined in a one-row MATLAB table and passed directly to lightweight Dataset objects. No XML or JSON source-data file is required.

The validation calculations use the dataset packages to select experimental cases, generate OpenSTREAM inputs, run the FourField solver, check convergence, and compare calculated and measured annular-flow properties.

The Live Script focuses on the calculations required to illustrate the figures and conclusions presented in the publication. It does not include all cases, model-development activities, closure-model evaluations, sensitivity studies, or numerical investigations performed during the underlying research.

Figure 5 is not reproduced because the TRACE reference results required for that comparison are not included in the companion workflow.

A completed execution of the workflow is available as an HTML export:

[Open the completed NURETH21 companion workflow](#)

4.3.2 NUTHOS15_MRM

The NUTHOS15_MRM project is provided as a MATLAB Live Script:

```
projects/NUTHOS15_MRM.mlx
```

The Live Script is intended to provide a reproducible computational companion to the corresponding NUTHOS-15 publication. It will document the selected OpenSTREAM calculations and results used to illustrate the application and evaluation of the MRM model, including:

- the selected experimental cases;
- the physical-model and numerical-option selections;
- the executable OpenSTREAM calculations;
- convergence information;
- calculated-versus-measured comparisons;
- interpretation of the principal results.

The companion workflow will focus on the calculations and figures presented in the publication. It will not constitute a complete record of the model-development process, sensitivity studies, closure-model evaluations, or additional cases considered during the research.

The workflow and its HTML export will be linked from this page after the project has been completed and reviewed.

4.4 Publications

This page features a curated list of publications where the **OpenSTREAM solvers** have been documented, developed, or applied to research activities in two-phase flow simulation and thermal-hydraulic modeling.

4.4.1 Overviews

1. Jean-Marie Le Corre, Jason Chan, Elisabeth Walter, Evan T. Hurlburt, and Roman W. Morse. Open-STREAM: an open-source platform for two-phase flow modeling and simulation. In *Proceedings of the 21st International Topical Meeting on Nuclear Reactor Thermal Hydraulics (NURETH-21)*. Busan, Korea, August 31–September 5 2025a. URL: https://openstream-solvers.github.io/openstream/_static/publications/LeCorre2025OpenSTREAM.pdf.

2. Jean-Marie Le Corre, Jason Chan, Elisabeth Walter, Evan T. Hurlburt, and Roman W. Morse. OpenSTREAM: a new open-source platform for two-phase flow model development. In *Proceedings of the 12th International Conference on Multiphase Flow (ICMF 2025)*. Toulouse, France, May 12–16 2025b. URL: https://openstream-solvers.github.io/openstream/_static/publications/LeCorre2025ICMF.pdf.
-

4.4.2 Mixture model

1. Jean-Marie Le Corre, Fabrice Francois, and Henryk Anglart. A mixture relaxation model for thermal non-equilibrium in convective flow boiling. In *Proceedings of the 15th International Topical Meeting on Nuclear Reactor Thermal-Hydraulic, Operation and Safety (NUTHOS-15)*. Kashiwa-no-ha, Japan, September 2026.
-

4.4.3 Two-fluid model

1. Elisabeth Walter, Jason Chan, Roman W. Morse, Haipeng Li, and Jean-Marie Le Corre. One-dimensional, two-fluid, two-phase flow simulation framework in the open-source platform OpenSTREAM. In *Proceedings of the 12th International Conference on Multiphase Flow (ICMF 2025)*. Toulouse, France, May 12–16 2025. URL: https://openstream-solvers.github.io/openstream/_static/publications/Walter2025.pdf.
 2. Elisabeth K. Walter. Derivation and open source implementation of a simplified one-dimensional two-fluid model. Master's thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2024. URL: <https://www.diva-portal.org/smash/record.jsf?pid=diva2:1888131>.
-

4.4.4 Three-field model

1. Jason Chan, Roman W. Morse, Nathan Hogan, Evan T. Hurlburt, and Jean-Marie Le Corre. One-dimensional, advanced annular two-phase flow simulations with the open-source platform OpenSTREAM. In *Proceedings of the 12th International Conference on Multiphase Flow (ICMF 2025)*. Toulouse, France, May 12–16 2025. URL: https://openstream-solvers.github.io/openstream/_static/publications/Chan2025.pdf.
 2. Cole Dunbar, Juliana P. Duarte, Roman W. Morse, and Jean-Marie Le Corre. Multi-field simulations of dry-out and rewet cycles in rod bundle geometry. In *Proceedings of the 21st International Topical Meeting on Nuclear Reactor Thermal Hydraulics (NURETH-21)*. Busan, Korea, August 31–September 5 2025. URL: https://openstream-solvers.github.io/openstream/_static/publications/Dunbar2025.pdf.
-

4.4.5 Four-field model

1. Jason Chan, Roman W. Morse, Nathan Hogan, Evan T. Hurlburt, and Jean-Marie Le Corre. One-dimensional, advanced annular two-phase flow simulations with the open-source platform OpenSTREAM. In *Proceedings of the 12th International Conference on Multiphase Flow (ICMF 2025)*. Toulouse, France, May 12–16 2025. URL: https://openstream-solvers.github.io/openstream/_static/publications/Chan2025.pdf.
2. Howard Y. K. Li, Ketan Ajay, David R. Novog, and Jean-Marie Le Corre. Four-field modeling of disturbance waves at high pressure near the onset of annular two-phase flow. In *Proceedings of the 15th International Topical Meeting on Nuclear Reactor Thermal-Hydraulic, Operation and Safety (NUTHOS-15)*. Kashiwa-no-ha, Japan, September 2026.

PACKAGES

5.1 InputEnums

The *InputEnums* package defines a set of enumeration classes used to specify valid options for physical models in **OpenSTREAM**. These enums help ensure consistency and clarity when selecting model configurations, making the simulation setup more robust and less error-prone.

Typical examples include choosing between void fraction models, interfacial transfer options, or flow regime assumptions, each represented by a clearly defined enum.

class InputEnums.**BTMTM**

BTMTM Enumeration of post-boiling transition wall momentum transfer models

This class defines the available *Inputs.Model.BTMTM* models for calculating the post-boiling transition wall momentum transfer, used by the mixture solver.

Models:

- TPFM — Wall momentum transfer based on two-phase friction multiplier, using multiplier selected in *Inputs.Model.TPFM*
- VAPOR — Wall momentum transfer based on vapor phase only, using single-phase momentum transfer model selected in *Inputs.Model.SPMTM*

class InputEnums.**EQSTROUHAL**

EQSTROUHAL Enumeration of wave equilibrium Strouhal number models

This class defines the available *Inputs.Model.EQSTROUHAL* models for calculating the wave equilibrium Strouhal number, used by the four-field solver.

Models:

- RISO — Based on RISO dataset (Le Corre [2022a])
- SAWAI — Based on SAWAI dataset (Le Corre [2022a])
- MFVAL — MFVAL model (under development)
- CUSTOM — Custom model using user-defined *Inputs.Model.EQSTROUHALCOEF*
- CSTFRQ — Custom constant frequency using user-defined *Inputs.Model.CSTWAVEFREQ*

class InputEnums.**INTAREA**

INTAREA Enumeration of volumetric interfacial area calculation models

This class defines the available *Inputs.Model.INTAREA* models for calculating the volumetric interfacial area, used by the two-fluid solver.

Models:

- DISPGAS2DISPLIQ — Transition from spherical bubbles to spherical droplets (Walter [2024])

class InputEnums.FLUIDPROPERTIES

FLUIDPROPERTIES Enumeration of fluid property calculation assumptions

This class defines the available [Inputs.Model.PROPERTIES](#) assumptions to calculate fluid properties, applicable to all solvers.

Models:

- SATURATED — Properties based on saturated conditions at system pressure
- PSYSTEM — Properties based on system pressure and local phase enthalpy

class InputEnums.WAVEBASEINT

WAVEBASEINT Enumeration of wave to base film interfacial momentum transfer models

This class defines the available [Inputs.Model.WAVEBASEINT](#) models for calculating the interfacial momentum transfer between wave and base film regions, used by the four-field solver.

Models:

- VAPORSHEAR — Vapor shear across wave area (consistent with Le Corre [2022a], Eq. 46).
- VAPORSHEARDROPMASS — Also includes drop deposition momentum transfer, ensuring consistency between wave and base film velocities for thin films.

class InputEnums.SPMTM

SPMTM Enumeration of single-phase wall momentum transfer models

This class defines the available [Inputs.Model.SPMTM](#) models for calculating the single-phase wall momentum transfer (liquid or gas), used by the mixture solver.

Models:

- BLASIUS — Blasius wall momentum transfer model ($f = C(1)Re^{C(2)} + C(3)$) using user-defined [Inputs.Model.FRCTION](#) coefficients

class InputEnums.MFBT

MFBT Enumeration of Minimum Film Boiling Transition (MFBT) models

This class defines the available [Inputs.Model.MFBT](#) models for evaluating the Minimum Film Boiling Transition, used by the mixture solver.

Models:

- NONE — No MFBT transition applied
- CONSTANT — Constant deviation from the saturation temperature using user-defined [Inputs.Model.DTMFB](#)

class InputEnums.NEARWALLH

NEARWALLH Enumeration of near-wall equilibrium enthalpy models

This class defines the available [Inputs.Model.NEARWALLH](#) models for setting the near-wall enthalpy to mixture enthalpy ratio, used by the mixture solvers.

Models:

- RATIO — Constant near-wall enthalpy to mixture enthalpy ratio using user-defined [Inputs.Model.NEARWALLHRATIO](#)
- FILM — Mass flux-dependent input for annular two-phase flow

class InputEnums.**INTTRANSH**

INTTRANSH Enumeration of interfacial enthalpy transfer models

This class defines the available [Inputs.Model.INTTRANSH](#) models for calculating the phase enthalpy associated with interfacial mass transfer, used by the mixture and two-fluid solvers.

Models:

- BULK — Uses bulk phase enthalpy
- SATURATED — Uses saturated phase enthalpy

class InputEnums.**DEPOSITION**

DEPOSITION Enumeration of droplet deposition models

This class defines the available [Inputs.Model.DEPOSITION](#) models for calculating the droplet deposition mass flux, used by the three-field and four-field solvers.

NOTE: Entrainment and deposition correlations are coupled. It is recommended to use the same model for both entrainment and deposition mass flux to ensure consistency.

Models:

- NONE — No deposition applied
- GOVAN — Hewitt and Govan correlation (Hewitt and Govan [1990])
- OKAWA — Okawa et al. correlation (Okawa *et al.* [2003])

class InputEnums.**NEARWALLRELAX**

NEARWALLRELAX Enumeration of near-wall energy transfer time relaxation models

This class defines the available [Inputs.Model.NEARWALLRELAX](#) models for calculating the near-wall energy transfer time relaxation, used in mixture solver.

Models:

- QUALITY — Simple model relating relaxation time to equilibrium quality using user-defined [Inputs.Model.NEARWALLRELAXX](#) and [Inputs.Model.NEARWALLRELAXT](#)
- VOID — Physical model based on local phase volumetric fraction using user-defined [Inputs.Model.NEARWALLRELAXCOEF](#)
- FILM — Model applicable to annular two-phase flow

class InputEnums.**OAFENTRAINED**

OAFENTRAINED Enumeration of entrained drop models at onset of annular flow

This class defines the available [Inputs.Model.OAFENTRAINED](#) models for splitting the droplet and film mass flow rates at the onset of annular flow, used by the three-field and four-field solvers.

Models:

- RATIO — Ratio of droplet to liquid mass flow rates using user-defined [Inputs.Model.OAFDROPRATIO](#)
- EQUILIBRIUM — Equilibrium assumption model ($M_{\text{ent}} = M_{\text{dep}}$) (Adamsson and Le Corre [2011], Equations 15 and 16)

class InputEnums.**BTHTM**

BTHTM Enumeration of post-boiling transition wall heat transfer models

This class defines the available [Inputs.Model.BTHTM](#) models for calculating the post-boiling transition wall heat transfer, used by the mixture solver.

Models:

- VAPOR — Wall heat transfer to the vapor phase only, based on single-phase heat transfer model selected in [Inputs.Model.SPHTM](#)
- DOUGALL — Dougall-Rohsenow wall heat transfer model (Dougall and Rohsenow [1963])
- BISHOP — Bishop-Sandberg-Tong wall heat transfer model (Bishop *et al.* [1965])
- MOECK — Groeneveld-Moeck wall heat transfer model (Groeneveld and Moeck [1969])
- DELORME — Groeneveld-Delorme wall heat transfer model (Groeneveld and Delorme [1976])
- CONDIEIV — Condie-Bengston IV wall heat transfer model (Morris *et al.* [1982])

class InputEnums.**WAVEFREQUENCY**

WAVEFREQUENCY Enumeration of wave number density transport models

This class defines the available [Inputs.Model.WAVEFREQUENCY](#) models for solving the wave number density (or wave frequency) transport equation, used in the four-field solver.

Models:

- EQUILIBRIUM — Assumes instantaneous equilibrium between wave generation and dissipation.
- RELAXATION — Full transport model including time relaxation effects set by user-defined RELAXTW (Le Corre [2022a])

class InputEnums.**THINFILMFRIC**

THINFILMFRIC Enumeration of thin film friction models

This class defines the available [Inputs.Model.THINFILMFRIC](#) models for calculating wall friction under thin film conditions, used by the three-field and four-field solvers.

Models:

- LAMINAR — Laminar flow model ($16/Re_{film}$)
- TURBULENT — Turbulent flow model

class InputEnums.**TPFM**

TPFM Enumeration of two-phase wall friction multiplier models

This class defines the available [Inputs.Model.TPFM](#) models for calculating the two-phase wall friction multiplier used by the mixture solver.

Models:

- HOMOGENEOUS — Assumes no slip between phases; both phases move at the same velocity
- SLIP — Accounts for velocity differences (slip) between liquid and vapor phases using user-defined [Inputs.Model.SLIP](#)
- EPRI — Empirical model developed by the Electric Power Research Institute (EPRI) (Reddy and others [1982])

class InputEnums.**MOMENTWAVE**

MOMENTWAVE Enumeration of film momentum conservation models

This class defines the available [Inputs.Model.MOMENTWAVE](#) models for solving the momentum conservation equation for the wave field, used to compute wave velocity in the four-field solver.

Models:

- ALGEBRAIC — Simple algebraic model (Adamsson and Le Corre [2014], Equations 21 or 23)
- EQUILIBRIUM — Equilibrium model

- FULL — Full non-equilibrium momentum model (Le Corre [2022a])

class InputEnums.TPHTM

TPHTM Enumeration of two-phase wall heat transfer models

This class defines the available `Inputs.Model.TPHTM` models for calculating the two-phase wall heat transfer, used by the mixture solver.

Models:

- THOM — Thom wall heat transfer model (Rohsenow *et al.* [1998])

class InputEnums.MOMENTDROP

MOMENTDROP Enumeration of droplet momentum conservation models

This class defines the available `Inputs.Model.MOMENTDROP` models for solving the droplet momentum conservation equation, used to compute droplet velocity in the three-field and four-field solvers.

Models:

- ALGEBRAIC — Simple algebraic model consistent with mixture solver
- SLIP — Slip ratio model (droplet/gas velocity)
- EQUILIBRIUM — Equilibrium model
- EQUILIBRIUMS — Simplified equilibrium model based on force balance ($F_{\text{drag}} + F_{\text{gravity}} + F_{\text{buoyancy}} = 0$)
- FULL — Full non-equilibrium momentum model (Le Corre *et al.* [2025a])

class InputEnums.BUBBLEDRAG

BUBBLEDRAG Enumeration of bubble drag coefficient models

This class defines the available `Inputs.Model.BUBBLEDRAG` models for calculating the bubble drag coefficient, used by the two-fluid solver.

Models:

- CONSTANT — Constant drag coefficient using user-defined `Inputs.Model.BUBBLEDRAGCOEF`
- STOKES — Stokes flow regime model ($\frac{24}{Re_p}$)
- VISCOUS — Viscous regime model (Walter [2024])
- DISTORTED — Model for distorted bubbles (non-spherical) (Walter [2024])

class InputEnums.MOMENTBASE

MOMENTBASE Enumeration of base film momentum conservation models

This class defines the available `Inputs.Model.MOMENTBASE` models for solving the base film momentum conservation equation, used to compute the base film velocity in the four-field solver.

Models:

- ALGEBRAIC — Simple algebraic model (Adamsson and Le Corre [2014], Equations 21 or 23)
- EQUILIBRIUM — Equilibrium model
- EQUILIBRIUMS — Simplified equilibrium model based on force balance ($F_{\text{wall}} + F_{\text{vapor}} = 0$)
- FULL — Full non-equilibrium model (Le Corre [2022a])
- FULLNOP — Full non-equilibrium model excluding pressure terms (gravity and buoyancy)

class InputEnums.**INTLENGTH**

INTLENGTH Enumeration of interfacial length scale models

This class defines the available [Inputs.Model.INTLENGTH](#) models for calculating the interfacial length scale, used by the two-fluid solver.

Models:

- **CONSTANT** — Assumes a constant interfacial length scale using user-defined [Inputs.Model.INTLENGTHVCS](#) and [Inputs.Model.INTLENGTHLCST](#) for the dispersed gas and liquid phases, respectively

class InputEnums.**SPHTM**

SPHTM Enumeration of single-phase wall heat transfer models

This class defines the available [Inputs.Model.SPHTM](#) models for calculating the single-phase wall heat transfer (liquid or gas), used by the mixture solver.

Models:

- **DITTUSBOELTER** — Dittus-Boelter wall heat transfer model for heating ($Nu = 0.023 \cdot Re^{0.8} \cdot Pr^{0.4}$) (Dittus and Boelter [1930], McAdams [1942])
- **DITTUSBOELTERGEN** — Generalized Dittus-Boelter wall heat transfer model using user-defined [Inputs.Model.DITTUSBOELTERCOEF](#)
- **SIEDERTATE** — Sieder-Tate wall heat transfer model ($Nu = 0.027 \cdot Re^{0.8} \cdot Pr^{1/3} \cdot \left(\frac{\mu_b}{\mu_w}\right)^{0.14}$) (Sieder and Tate [1936])
- **GNIELINSKI** — Gnielinski wall heat transfer model (Gnielinski [1976])

class InputEnums.**DEPENHANCEMENT**

DEPENHANCEMENT Enumeration of droplet deposition enhancement models

This class defines the available [Inputs.Model.DEPENHANCEMENT](#) models for calculating local droplet deposition enhancement, used by the three-field and four-field solvers.

Models:

- **NONE** — No enhancement applied
- **WINDECKER** — Model adapted from Windecker [1999], as documented in Le Corre [2024], using user-defined [Inputs.Model.KTUNING](#)

class InputEnums.**BASEEQTHICK**

BASEEQTHICK Enumeration of base film equilibrium thickness models

This class defines the available [Inputs.Model.BASEEQTHICK](#) models for calculating the equilibrium base film thickness, used by the four-field solver.

Models:

- **RISO** — Based on RISO dataset (Le Corre [2022a])
- **COEFS** — Generalized RISO model using user-defined [Inputs.Model.BASEEQTHICKCOEF](#)
- **YPLUS** — Based on dimensionless wall distance (y+) using user-defined [Inputs.Model.BASEYPLUS](#)
- **MFVAL** — MFVAL model (under development)

class InputEnums.DROPDRAG

DROPDRAG Enumeration of droplet drop drag coefficient models

This class defines the available [Inputs.Model.DROPDRAG](#) models for calculating the droplet drag coefficient, used by the two-fluid solver.

Models:

- CONSTANT — Constant drag coefficient using user-defined [Inputs.Model.DROPDRAGCOEF](#)
- STOKES — Stokes flow regime model ($\frac{24}{Re_p}$)
- VISCOUS — Viscous regime model (Walter [2024])
- DISTORTED — Model for distorted (non-spherical) droplets (Walter [2024])

class InputEnums.CBT

CBT Enumeration of Critical Boiling Transition (CBT) models

This class defines the available [Inputs.Model.CBT](#) models for evaluating the Critical Boiling Transition, used by the mixture solver.

Models:

- NONE — No CBT transition applied
- BIASI — Biasi correlation model (Biasi *et al.* [1966])
- BEZRUKOV — Bezrukov correlation model (Bezrukov *et al.* [1976])
- ELEVATION — CBT determined from specified elevation using user-defined [Inputs.Model.CBTELEVATION](#)

class InputEnums.ENTRAINMENT

ENTRAINMENT Enumeration of droplet entrainment models

This class defines the available [Inputs.Model.ENTRAINMENT](#) models for calculating the droplet entrainment mass flux, used by the three-field and four-field solvers.

NOTE: Entrainment and deposition correlations are coupled. It is recommended to use the same model for both entrainment and deposition mass flux to ensure consistency.

Models:

- NONE — No entrainment
- GOVAN — Hewitt and Govan correlation (Hewitt and Govan [1990])
- OKAWA2003 — Okawa *et al.* correlation (Okawa *et al.* [2003])
- OKAWA2004 — Okawa *et al.* correlation (Okawa *et al.* [2004])
- OKAWA2004MOD — Modified Okawa (2004) model from (Adamsson and Le Corre [2011])
- OKAWAGEN — Generic Okawa-based model using user-defined [Inputs.Model.OKAWACOEFS](#)

class InputEnums.TPKM

TPKM Enumeration of two-phase local loss multiplier models

This class defines the available [Inputs.Model.TPKM](#) models for calculating the two-phase local loss multiplier used by the mixture model

Models:

- HOMOGENEOUS — Assumes no slip between phases; both phases move at the same velocity.

- SLIP — Accounts for velocity differences (slip) between liquid and vapor phases using user-defined [Inputs.Model.SLIP](#).
- ROMIE — Romie model; behaves like the homogeneous model when a homogeneous void fraction model is applied.

class InputEnums.OAF

OAF Enumeration of onset of annular flow models

This class defines the available [Inputs.Model.OAF](#) models for predicting the onset of annular flow, used by the three-field and four-field solvers.

Models:

- WALLIS — Full Wallis correlation (Wallis [1969], Equation 11.1019)
- WALLIS_SIMP — Simplified Wallis correlation (Sanmiguel Gimeno *et al.* [2015], Equation 6), used for compatibility with codes employing this approach
- LEVITAN — Levitan correlation (Levitan and Borevsky [1989])

class InputEnums.VOID

VOID Enumeration of void fraction models

This class defines the available [Inputs.Model.VOID](#) models to calculate the void fraction in the mixture solver.

Models:

- HOMOGENEOUS — Assumes vapor and liquid phases move at the same velocity.
- SLIP — Assumes a slip ratio between vapor and liquid phases using user-defined [Inputs.Model.SLIP](#).
- BESTION — Drift-flux model developed by Bestion (Bestion [1990]).
- EPRI — Drift-flux model developed by the Electric Power Research Institute (EPRI) (Lellouche and Zolotar [1982]).

class InputEnums.OAFFILMSPLIT

OAFFILMSPLIT Enumeration of film mass flow ratio models at onset of annular flow

This class defines the available [Inputs.Model.OAFFILMSPLIT](#) models for splitting the base film and wave mass flow rates at the onset of annular flow, used by the four-field solver.

Models:

- RATIO — Ratio of base film to film mass flow rates using user-defined [Inputs.Model.OAFBASERATIO](#)
- EQUILIBRIUM — Equilibrium model (film thickness = equilibrium film thickness) (Le Corre [2022a])

class InputEnums.MOMENTFILM

MOMENTFILM Enumeration of film momentum conservation models

This class defines the available [Inputs.Model.MOMENTFILM](#) models for solving the momentum conservation equation for the liquid film, used to compute film velocity in the three-field solver.

Models:

- ALGEBRAIC — Simple algebraic model
- EQUILIBRIUM — Equilibrium model
- EQUILIBRIUMS — Simplified equilibrium model based on force balance ($F_{\text{wall}} + F_{\text{vapor}} = 0$)
- FULL — Full non-equilibrium momentum model (Le Corre *et al.* [2025a])

class InputEnums.**MRMTIMEINT**

MRMTIMEINT Enumeration of time integration options for the MRM vapor mass/energy conservation equations

This class defines the available [Inputs.Options.MRMTIMEINT](#) options for solving the vapor mass and energy conservation equations when using the MRM model in the mixture solver.

Models:

- EULER — Backward Euler
- EXPONENTIAL — Exponential

class InputEnums.**LOCRELVEL**

LOCRELVEL Enumeration of local relative velocity models

This class defines the available [Inputs.Model.LOCRELVEL](#) models for calculating the local relative velocity between liquid and vapor phases, used by the two-fluid solver.

Models:

- AREAMEAN — Difference between area-averaged phase velocities (Walter [2024])
- SCALED — Scaled difference using user-defined [Inputs.Model.RELVELCST](#) (Walter [2024])
- DRIFT — Uses drift velocity as the local relative velocity (Walter [2024])
- SIMPLE — Simplified assumptions for bubbly and annular flow regime (Walter [2024])

class InputEnums.**VAPORFRIC**

VAPORFRIC Enumeration of vapor to film interfacial friction models

This class defines the available [Inputs.Model.VAPORFRIC](#) models for calculating the vapor-to-film interfacial friction coefficient used by the three-field and four-field solvers.

Models:

- CONSTANT — Uses a fixed, friction coefficient using user-defined [Inputs.Model.VAPORFRICCST](#)
- WALLIS — Wallis model based on void fraction (Wallis [1969]).
- WALLISTHICK — Wallis model based on film thickness (Adamsson and Le Corre [2011], Equation 35).
- SOLVER_DEPENDENT — Friction model determined by the solver configuration.

class InputEnums.**THERMALRELAX**

THERMALRELAX Enumeration of MRM time relaxation models

This class defines the available [Inputs.Model.THERMALRELAX](#) models for calculating relaxation times in the MRM model, used when [Inputs.Model.THERMALNONEQ](#) is set to *MRM*.

Models:

- FOURIERX — Simple model relating the Fourier number to equilibrium quality using user-defined [Inputs.Model.RELAXX](#), [Inputs.Model.RELAXCONDFO](#) and [Inputs.Model.RELAXEVAPFO](#)
- TIMEX — Simple model relating relaxation time to equilibrium quality using user-defined [Inputs.Model.RELAXX](#), [Inputs.Model.RELAXCONDT](#) and [Inputs.Model.RELAXEVAPT](#)
- VOID — Physical model simplified to Fourier number dependence on phase volumetric fraction and input characteristic length of the dispersed phase at CBT using user-defined [Inputs.Model.RELAXCONDCOE](#) and [Inputs.Model.RELAXEVAPCOEF](#)
- HOMOGENEOUS — Physical model based on homogeneous assumptions and spherical dispersed phase

- FOURIER — Empirical model based on Fourier number dependence on Reynolds number, etc, using user-defined [Inputs.Model.RELAXCONDFOCOEF](#) and [Inputs.Model.RELAXEVAPFOCOEF](#)

class InputEnums.THERMALNONEQ

THERMALNONEQ Enumeration of thermal non-equilibrium models

This class defines the available [Inputs.Model.THERMALNONEQ](#) models for calculating phase thermal non-equilibrium effects, used by the mixture solver.

Models:

- EQUILIBRIUM — Assumes thermal equilibrium ($X = \max(0, X_{EQ})$)
- SAHAZUBER — Saha-Zuber subcooled boiling model (Saha and Zuber [1973])
- EPRI — EPRI subcooled boiling model (Lellouche and Zolotar [1982])
- MRM — Mixture Relaxation Model for subcooled boiling and post-CHF conditions

class InputEnums.INTNU

INTNU Enumeration of interfacial Nusselt number models

This class defines the available [Inputs.Model.INTNU](#) models for calculating the interfacial Nusselt number, used by the two-fluid solver.

Models:

- CONSTANT — Assumes a constant Nusselt number using user-defined [Inputs.Model.INTNUVCST](#) and [Inputs.Model.INTNULCST](#) for the dispersed gas and liquid phases, respectively
- RANZMARSHALL — Ranz-Marshall correlation-based model (Ranz and Marshall [1952]) using user-defined [Inputs.Model.RANZMARSHALLVCST](#) and [Inputs.Model.RANZMARSHALLLCST](#) for the dispersed gas and liquid phases, respectively
- RELAXATION — Time relaxation-based model (Walter [2024])

class InputEnums.MOMENTLIQUID

MOMENTLIQUID Enumeration of liquid momentum conservation models

This class defines the available [Inputs.Model.MOMENTLIQUID](#) models for solving the momentum conservation equation for the liquid phase, used to compute liquid velocity in the two-fluid solver.

Models:

- MIXTURE — Uses the same velocity as in the mixture model
- SLIP — Slip ratio model (gas/liquid velocity) using user-defined [Inputs.Model.SLIP](#)
- FULL — Full non-equilibrium momentum model (Le Corre *et al.* [2025a], Walter [2024])

class InputEnums.MOMENTGAS

MOMENTGAS Enumeration of gas momentum conservation models

This class defines the available [Inputs.Model.MOMENTGAS](#) models for solving the momentum conservation equation for the gas phase, used to compute gas velocity in the two-fluid solver.

Models:

- MIXTURE — Uses the same gas velocity as in the mixture model
- SLIP — Slip ratio model (gas/liquid velocity) using user-defined [Inputs.Model.SLIP](#)
- FULL — Full non-equilibrium momentum model (Le Corre *et al.* [2025a], Walter [2024])

5.2 Inputs

The *Inputs* package defines a set of classes used to configure and import all the necessary parameters for running OpenSTREAM simulations. This includes:

- Boundary conditions: *Inputs.BoundaryConditions*
- System geometry: *Inputs.Geometry*
- Physical models: *Inputs.Model*
- Numerical options: *Inputs.Options*
- Fluid thermophysical properties: *Inputs.FluidProperties*
- Input handling: *Inputs.Input*
- Complete input set management: *Inputs.InputSet*

These input definitions form the backbone of each simulation setup, ensuring consistency, flexibility, and clarity across different solver frameworks.

class *Inputs.Model*(filePath, modelID)

Bases: *Inputs.Input*

MODEL Class for defining and managing physical model configuration

This class encapsulates all physical modeling parameters used in the simulations, including fluid properties, regime transitions, wall and interfacial exchange models for all solvers. It reads model data from input files and handles default values as well as input validation.

Constructor Summary

Model(filePath, modelID)

MODEL Constructor for *Model()* class

Parses model input file and initializes properties. Applies default values and validates entries.

Inputs:

- filePath — Path to model input file
- modelID — Identifier for model configuration

Property Summary

BASEEQTHICK = 'RISO'

Equilibrium base film thickness model selected from *InputEnums.BASEEQTHICK*

BASEEQTHICKCOEF = [5.37E-5-0.641.21]

Equilibrium base film thickness coefficients [-]

BASEYPLUS = 15

Equilibrium base film y+ value [-]

BTHTM = 'VAPOR'

Boiling transition wall heat transfer model selected from *InputEnums.BTHTM*

BTMTM = 'TPFM'

Boiling transition wall momentum transfer model selected from *InputEnums.BTMTM*

BUBBLEDRAG = 'STOKES'

Bubble drag model selected from *InputEnums.BUBBLEDRAG*

BUBBLEDRAGCOEF = 0.45

Bubble constant drag coefficient [-]

CBT = 'NONE'

Critical Boiling Transition model selected from *InputEnums.CBT*

CBTELEVATION = 1

Critical Boiling Transition Elevation [m]

CBTKEFFECT = [00]

Grid effect coefficients ($1 + C(1) \exp(C(2)z)$) [-]

CBTMULT = @z1

Critical boiling Heat flux multiplier function

CSTWAVEFREQ = 100

Imposed constant wave frequency [Hz]

DEPENHANCEMENT = 'NONE'

Drop deposition enhancement model due to local perturbations selected from *InputEnums.DEPENHANCEMENT*

DEPOSITION = 'OKAWA'

Drop deposition model selected from *InputEnums.DEPOSITION*

DITTUSBOELTERCOEF = [0.0230.80.4]

Dittus-Boelter coefficients [-]

DROPDIAM = 1E-3

Drop diameter [m]

DROPDRAG = 'VISCOUS'

Drop drag model selected from *InputEnums.DROPDRAG*

DROPDRAGCOEF = 0.45

Drop constant drag coefficient [-]

DROPSLIP = 1.0

Drop/vapor velocity ratio [-]

DTMFB = 100

Minimum film boiling temperature from saturation [K]

ENTRAINMENT = 'OKAWA2003'

Film entrainment model selected from *InputEnums.ENTRAINMENT*

EQSTROUHAL = 'RISO'

Equilibrium wave Strouhal number model selected from *InputEnums.EQSTROUHAL*

EQSTROUHALCOEF = [1.1236E-40.50.0]

Equilibrium wave Strouhal number coefficients [-]

FLUID = 'WATER'

CoolProp fluid identifier

FRICITION = [0.2-0.20]

Wall friction coefficients [-]

G = 9.81

Gravitational acceleration [m/s²]

ID

Identifier for the model configuration

INTAREA = 'DISPGAS2DISPLIQ'

Interfacial area model selected from *InputEnums.INTAREA*

INTLENGTH = 'CONSTANT'

Interfacial length scale model selected from *InputEnums.INTLENGTH*

INTLENGTHLCST = 2E-3

Dispersed liquid constant interfacial length scale [m]

INTLENGTHVCST = 2E-3

Dispersed gas constant interfacial length scale [m]

INTNU = 'RELAXATION'

Interfacial heat transfer model selected from *InputEnums.INTNU*

INTNULCST = 2.0

Dispersed liquid constant interfacial Nusselt number [-]

INTNUVCST = 2.0

Dispersed gas constant interfacial Nusselt number [-]

INTTRANSH = 'BULK'

Interfacial enthalpy transfer model selected from *InputEnums.INTTRANSH*

KBLOCKRATIO = [00]

Blockage ratios of local perturbations [-]

KLOC = [00]

Elevation of local perturbations [m]

KLOSS = [00]

Pressure loss coefficients of local perturbations [-]

KTRELAX = NaN

Thermal relaxation time at local perturbations [s]

KTUNING = [00]

Drop deposition enhancement tuning coefficients [-]

LOCRELVEL = 'AREAMEAN'

Local relative phase velocity model selected from *InputEnums.LOCRELVEL*

MFBT = 'NONE'

Minimum Film Boiling Transition model selected from *InputEnums.MFBT*

MOECKCOEF = [1.09E-30.9891.41-1.15]

Groeneveld-Moeck post-boiling transition model coefficients [-]

MOMENTBASE = 'FULLNOP'

Base film momentum conservation model selected from *InputEnums.MOMENTBASE*

MOMENTDROP = 'SLIP'

Drop momentum conservation model selected from *InputEnums.MOMENTDROP*

MOMENTFILM = 'ALGEBRAIC'

Film momentum conservation model selected from *InputEnums.MOMENTFILM*

MOMENTGAS = 'MIXTURE'

Gas momentum conservation model selected from *InputEnums.MOMENTGAS*

MOMENTLIQUID = 'MIXTURE'

Liquid momentum conservation model selected from *InputEnums.MOMENTLIQUID*

MOMENTWAVE = 'FULL'

Wave momentum conservation model selected from *InputEnums.MOMENTWAVE*

NEARWALLEQOAF = false

Near-wall equilibrium at onset of annular two-phase flow

NEARWALLH = 'RATIO'

Near-wall equilibrium enthalpy model selected from *InputEnums.NEARWALLH*

NEARWALLHRATIO = 1.0

Near-wall equilibrium enthalpy ratio [-]

NEARWALLRATIO = 0.5

Near-wall mass flow distribution ratio [-]

NEARWALLRELAX = 'QUALITY'

Near-wall energy transfer time relaxation model selected from *InputEnums.NEARWALLRELAX*

NEARWALLRELAXCOEF = [0.1E-31/30.05]

Near-wall energy transfer time relaxation coefficients for void option

NEARWALLRELAXT = [1.00.50.30.10.1]

Near-wall energy transfer relaxation time array [s]

NEARWALLRELAXX = [-0.5-0.25-0.10.01.0]

Near-wall exchange relaxation time thermodynamic quality [-]

NNODES = 100

Number of axial nodes used in the simulation

OAF = 'WALLIS'

Onset of annular flow model selected from *InputEnums.OAF*

OAFBASERATIO = 0.5

Base/Film mass ratio at onset of annular flow [-]

OAFDROPRATIO = 0.7

Drop/Liquid mass ratio at onset of annular flow [-]

OAFENTRAINED = 'EQUILIBRIUM'

Entrained drop model at onset of annular flow selected from *InputEnums.OAFENTRAINED*

OAFFILMSPLIT = 'EQUILIBRIUM'

Film mass flow split model at onset of annular flow selected from *InputEnums.OAFFILMSPLIT*

OAFTRANSITION = [0.100.0]

Annular flow transition function parameters (sigmoid width/location wrt OAF) [m]

OKAWACOEFS = [3200.1114.79E-41]

Coefficients of Okawa entrainment model [-]

POSFILM = true

Positive film flow rate/thickness model

PROPERTIES = 'SATURATED'

Assumption model for fluid properties selected from *InputEnums.FLUIDPROPERTIES*

RANZMARSHALLCST = [20.61/21/3]

Dispersed liquid Ranz-Marshall coefficients [-]

RANZMARSHALLVCST = [20.61/21/3]

Dispersed gas Ranz-Marshall coefficients [-]

RELAXCONDCOEF = [0.1E-31/30.050.0]

Interfacial condensation time relaxation coefficients for THERMALRELAX HOMOGENEOUS model

RELAXCONDFO = [1.01.01.01.01.0].*1E-4

Interfacial condensation Fourier number array for THERMALRELAX FOURIERX model [-]

RELAXCONDFOCOEF = [5E-55E500]

Interfacial condensation time relaxation coefficients for THERMALRELAX FOURIER model

RELAXCONDT = [1.00.50.30.10.1]

Interfacial condensation relaxation time array for THERMALRELAX TIMEX model [s]

RELAXEVAPCOEF = [0.1E-31/31E-50.0]

Interfacial evaporation time relaxation coefficients for THERMALRELAX HOMOGENEOUS model

RELAXEVAPFO = [1.01.01.01.01.0].*5E-5

Interfacial evaporation Fourier number array for THERMALRELAX FOURIERX model [-]

RELAXEVAPFOCOEF = [1E-45E52210E-31]

Interfacial evaporation time relaxation coefficients for THERMALRELAX FOURIER model

RELAXEVAPT = [0.30.30.30.30.3]

Interfacial evaporation relaxation time array for THERMALRELAX TIMEX model [s]

RELAXTB = 0.2

Base film / wave mass exchange relaxation time [s]

RELAXTW = 0.2

Wave number density relaxation time [s]

RELAXX = [-0.5-0.25-0.10.01.0]

Interfacial phase change relaxation time thermodynamic quality [-]

RELVELCST = 1.0

Multiplication factor to the local relative velocity [-]

SHAPEFACTORCOEF = [1.325E5210]

Wave shape factor coefficients [-]

SLIP = 1

Phase velocity ratio [-]

SOLVERDEPENDENTPROPS = struct("VAPORFRIC", ...)

Mapping of solver-dependent properties and their values

SPHTM = 'DITTUSBOELTER'

Single-phase wall heat transfer model selected from *InputEnums.SPHTM*

SPMTM = 'BLASIUS'

Single-phase wall momentum transfer model selected from *InputEnums.SPMTM*

THERMALNONEQ = 'EQUILIBRIUM'

Thermal non-equilibrium model selected from *InputEnums.THERMALNONEQ*

THERMALRELAX = 'TIMEX'

Thermal non-equilibrium time relaxation model selected from *InputEnums.THERMALRELAX*

THINFILMFRIC = 'TURBULENT'

Thin film wall friction model selected from *InputEnums.THINFILMFRIC*

THINFILMTHICK = 1E-4

Minimum thin film thickness [m]

THINWAVETHICK = 1E-5

Minimum thin wave thickness [m]

TPFM = 'HOMOGENEOUS'

Two-phase friction multiplier model selected from *InputEnums.TPFM*

TPHTM = 'THOM'

Two-phase wall heat transfer model selected from *InputEnums.TPHTM*

TPKM = 'HOMOGENEOUS'

Two-phase local loss multiplier model selected from *InputEnums.TPKM*

VAPORFRIC = 'SOLVER_DEPENDENT'

Vapor friction model selected from *InputEnums.VAPORFRIC*

VAPORFRICCST = 0.005

Vapor friction constant [-]

VOID = 'HOMOGENEOUS'

Void fraction model selected from *InputEnums.VOID*

WAVEBASEINT = 'VAPORSHEAR'

Wave / base film interfacial momentum transfer model selected from *InputEnums.WAVEBASEINT*

WAVEDRAGCOEF = [0.021.350E50.437]

Wave drag coefficient [-]

WAVEFREQUENCY = 'RELAXATION'

Wave number density model selected from *InputEnums.WAVEFREQUENCY*

WAVEMIXCOEF = 2

Base film / wave turbulent mixing coefficient

WBOILINGN = 2

Wall boiling function exponent [-]

WBOILINGXSAT = 0

Equilibrium thermodynamic quality at onset of saturated wall boiling [-]

WBOILINGXSUB = -0.2

Equilibrium thermodynamic quality at onset of subcooled wall boiling [-]

Method Summary

static writeInputFile(filePathName, ID, varargin)

WRITEINPUTFILE Writes model configuration data to input file

Inputs: - filePathName — Path to output file - ID — Model identifier - varargin — Additional name-value pairs for model properties

class Inputs.InputSet(opts)

INPUTSET Class for managing a complete set of input objects

This class initializes and stores all input components required for a simulation run. It handles model, options, geometry, and boundary condition inputs, and sets up logging via a Session object. It also supports applying solver-dependent property modifications.

Constructor Summary

InputSet(opts)

INPUTSET Constructor for [InputSet\(\)](#) class

Initializes all input objects and sets up logging session. Parses input files and handles warnings.

Inputs:

- opts — Struct with fields:
 - modelFilePath, modelID
 - optionsFilePath, optionsID
 - geometryFilePath, geometryID
 - bcFilePath
 - LOGMODE, sessionName, sessionDirName
 - sessionParentDir, overwriteSessionFiles

Property Summary

bc

BoundaryConditions input object

geometry

Geometry input object

model

Model input object

options

Options input object

session

Session object for logging and file management

Method Summary**applySolverDependentProps**(*inputSet*, *solverName*)

APPLYSOLVERDEPENDENTPROPS Applies solver-dependent property modifications to all input objects

Useful for customizing inputs based on selected solver

Inputs:

- *inputSet* — InputSet object
- *solverName* — Name of the solver to apply dependencies for

class Inputs.**FluidProperties**(*P*, *modelObj*)

FLUIDPROPERTIES Class for managing thermophysical properties of a fluid

This class encapsulates saturated and generic fluid properties used in thermal-hydraulic simulations. Saturated properties are computed based on system pressure and stored as class properties. Generic properties (e.g., temperature, density, viscosity) are computed dynamically using enthalpy and pressure.

Constructor Summary**FluidProperties**(*P*, *modelObj*)

FLUIDPROPERTIES Constructor for *FluidProperties()* class

Initializes fluid properties at given pressure(s) using CoolProp. Saturated properties are computed and assigned to each object instance.

Inputs:

- *P* — Vector of system pressures [Pa]
- *modelObj* — Model object containing fluid name and property assumptions

Property Summary**ALPHA_F = 1**

Saturated liquid thermal diffusivity [m²/s]

ALPHA_G = 1

Saturated vapor thermal diffusivity [m²/s]

CPF = 1

Saturated liquid constant pressure specific heat [J/kg/K]

CPG = 1

Saturated vapor constant pressure specific heat [J/kg/K]

FLUID = 'WATER'

Coolprop fluid identifier

HF = 1

Saturated liquid enthalpy [J/kg]

HFG = 0

Latent heat of evaporation [J/kg]

HG = 1

Saturated vapor enthalpy [J/kg]

KF = 1
 Saturated liquid thermal conductivity [W/m/K]

KG = 1
 Saturated vapor thermal conductivity [W/m/K]

MUF = 1
 Saturated liquid viscosity [Pa.s]

MUG = 1
 Saturated vapor viscosity [Pa.s]

PCRIT = 1
 Critical pressure [Pa]

PRANDTLF = 1
 Saturated liquid Prandtl number [-]

PRANDTLG = 1
 Saturated vapor Prandtl number [-]

PRESSURE = 1
 System pressure [Pa]

PROPERTIES = 'SATURATED'
 Fluid property assumptions from *Inputs.Model.PROPERTIES*

PTRIPLE = 1
 Triple point pressure [Pa]

QK = 1
 Kutateladze characteristic heat flux [W/m²]

RHOF = 1
 Saturated liquid mass density [kg/m³]

RHOG = 1
 Saturated vapor mass density [kg/m³]

SIGMA = 1
 Surface tension [N/m]

TSAT = 1
 Saturated fluid temperature [K]

UC = 1
 Kutateladze vapor critical velocity [m/s]

Method Summary

ALPHAL(*H*)
ALPHAL() Liquid thermal diffusivity [m²/s]
 Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

ALPHAV(*H*)
ALPHAV() Vapor thermal diffusivity [m²/s]
 Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

CPL(*H*)

CPL() Liquid constant pressure specific heat [J/kg/K]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

CPV(*H*)

CPV() Vapor constant pressure specific heat [J/kg/K]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

H(*T*)

H() Fluid enthalpy at given temperature and system pressure [J/kg]

KL(*H*)

KL() Liquid thermal conductivity [W/m/K]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

KV(*H*)

KV() Vapor thermal conductivity [W/m/K]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

MUL(*H*)

MUL() Liquid dynamic viscosity [Pa.s]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

MUV(*H*)

MUV() Vapor dynamic viscosity [Pa.s]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

PRANDTLL(*H*)

PRANDTLL() Liquid Prandtl number [-]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

PRANDTLV(*H*)

PRANDTLV() Vapor Prandtl number [-]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

RHOL(*H*)

RHOL() Liquid mass density [kg/m³]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

RHOV(*H*)

RHOV() Vapor mass density [kg/m³]

Behavior depends on property assumption model (*Inputs.Model.PROPERTIES*)

T(*H*)

T() Fluid temperature at given enthalpy and system pressure [K]

Enthalpy is clamped between HMIN and HMAX to ensure valid CoolProp input

plot(*H*)

PLOT Plot properties for given enthalpy vector

transient(*param, opt*)

TRANSIENT Generate *transient()* distribution array for parameter param

class Inputs.**Geometry**(*filePath, geometryID*)

Bases: *Inputs.Input*

GEOMETRY Class for defining and managing geometrical input parameters

This class reads and stores geometrical parameters from an input file. It supports calculations of derived quantities such as hydraulic diameter, area-based diameter, and wall perimeter ratios.

Constructor Summary

Geometry(*filePath, geometryID*)

GEOMETRY Constructor for *Geometry()* class

Parses geometry input file and initializes properties. Applies default values if no input is provided.

Inputs:

- *filePath* — Path to geometry input file
- *geometryID* — Identifier for geometry configuration

Property Summary

ANGLE = 0

Inclination angle from vertical [deg]

AREA = 1

Coolant area [m²]

ID

Channel identifier

LENGTH = 1

Axial length [m]

PERIM = 1

Perimeter(s) of the channel walls [m]

Method Summary

ADIAM()

ADIAM() Diameter based on coolant cross-section area [m]

Formula: $2\sqrt{\frac{AREA}{\pi}}$

HDIAM()

HDIAM() Hydraulic diameter [m]

Formula: $4\frac{Area}{Totalperimeter}$

NWALL()

NWALL() Number of wall segments defined by PERIM

RWALL()

RWALL() Relative contribution of each wall segment to total perimeter [-]

static writeInputFile(filePathName, ID, varargin)

Writes geometry input data to file

Inputs:

- filePathName — Path to output file
- ID — Geometry identifier
- varargin — Additional name-value pairs for geometry properties

class Inputs.**BoundaryConditions**(filePath, geometryObjInput)

Bases: [Inputs.Input](#)

BOUNDARYCONDITIONS Class for defining and managing boundary condition parameters

This class provides access to boundary condition inputs and saturated fluid properties at system pressure. It is used by the solvers to initialize and manage inlet conditions, power distributions, and fluid state calculations.

Key Features:

- Parses input files and assigns values to boundary condition fields.
- Interfaces with fluid property libraries to compute saturation and inlet thermodynamic states.
- Validates consistency between geometry and power mesh definitions.
- Supports plotting of boundary condition time series.

Constructor Summary

BoundaryConditions(filePath, geometryObjInput)

BOUNDARYCONDITIONS Constructor for [BoundaryConditions\(\)](#) class

Parses input file and initializes boundary condition parameters. Validates consistency with geometry and applies default values if needed.

Inputs:

- filePath — Path to input file (optional)
- geometryObjInput — Geometry object for validation

Property Summary

HIN = []

Inlet enthalpy [J/kg]

MFLOW = []

Mass flow rate [kg/s]

POWER = 0

Total power [W]

PRESSURE = []

System pressure [Pa]

TIME

Simulation time [s]

WMESH = 1

Node size distribution [m]

WPOWER = 1

Relative power distribution(s) [-]

Method Summary

DHIN(*fluidObj*)

DHIN() Inlet subcooling enthalpy (HF - HIN) [J/kg]

DTIN(*fluidObj*)

DTIN() Inlet subcooling temperature (TSAT - TIN) [K]

HF(*fluidObj*)

HF() Liquid saturated enthalpy at system pressure using fluid object [J/kg]

HG(*fluidObj*)

HG() Vapor saturated enthalpy at system pressure using fluid object [J/kg]

TIN(*fluidObj*)

TIN() Inlet temperature based on inlet enthalpy and pressure [K]

TSAT(*fluidObj*)

TSAT() Saturation temperature at system pressure using fluid object [K]

XIN(*fluidObj*)

XIN() Inlet equilibrium quality [-]

plot(*fluidObj*, *opt*)

PLOT Generates plots of boundary condition parameters over time

Inputs:

- *fluidObj* — FluidProperties object
- *opt* — Struct with fields:
 - *display* — Cell array of parameters to *plot()*
 - *tIdx* — Time indices to include
 - *unitTemp* — Temperature unit ('K' or 'C')

static writeInputFile(*filePathName*, *TIME*, *PRESSURE*, *HIN*, *MFLOW*, *varargin*)

WRITEINPUTFILE Writes boundary condition data to input file

Inputs:

- *filePathName* — Path to output file
- *TIME* — Time value [s]
- *PRESSURE* — Pressure value [Pa]
- *HIN* — Enthalpy value [J/kg]
- *MFLOW* — Mass flow rate [kg/s]
- *varargin* — Additional name-value pairs

class Inputs.Options(*filePath*, *optionsID*)

Bases: *Inputs.Input*

OPTIONS Class for defining and managing numerical solver options

This class encapsulates all numerical parameters used in the simulations, including time stepping, convergence criteria, and relaxation factors for all solvers. It reads options data from input files and handles default values as well as input validation.

Constructor Summary

Options(*filePath*, *optionsID*)

OPTIONS Constructor for *Options()* class

Parses options input file and initializes properties. Applies default values and validates entries.

Inputs:

- *filePath* — Path to options input file
- *optionsID* — Identifier for options configuration

Property Summary

AXIALINTERP = 'next'

Method for interpolating axial power distribution

ENTNUMMAXITER = 100

Maximum number of iterations to find film entrainment number

ENTNUMTOL = 1E-2

Target velocity slip tolerance to find film entrainment number [-]

ERRORFW = 1E-1

Wave frequency error target in inner iterations [Hz]

ERRORH = 1E-1

Enthalpy error target in inner iterations [J/kg]

ERRORP = 1E-1

Pressure error target in inner iterations [Pa]

ERRORU = 1E-4

Velocity error target in inner iterations [m/s]

ERRORUD = 1E-2

Drop velocity error target in inner iterations [m/s]

ERRORUF = 1E-2

Film velocity error target in inner iterations [m/s]

ERRORW = 1E-3

Mass flow rate error target in inner iterations [kg/s]

ERRORWF = 1E-4

Film mass flow rate error target in inner iterations [kg/s/m]

ID

Identifier for the options configuration

MAXITER = 100

Maximum number of inner iterations per time step

MRMTIMEINT = 'EXPONENTIAL'

Time integration for vapor mass/energy conservation equations

OAFEQUILMAXITER = 100

Maximum number of iterations to find equilibrium at onset of annular flow

OAFEQUILTOL = 1E-4

Target film mass flow rate tolerance to find equilibrium at onset of annular flow [kg/s/m]

RELAXFW = 0.5

Relaxation factor for the wave number conservation equation [-]

RELAXHL = 1

Relaxation factor for the liquid energy conservation equation [-]

RELAXHM = 1.0

Relaxation factor for the mixture energy conservation equation [-]

RELAXHV = 0.7

Relaxation factor for the vapor energy conservation equation [-]

RELAXPM = 1.0

Relaxation factor for the mixture momentum conservation equation [-]

RELAXUB = 0.2

Relaxation factor for the base momentum conservation equation [-]

RELAXUD = 0.2

Relaxation factor for the drop momentum conservation equation [-]

RELAXUF = 0.2

Relaxation factor for the film momentum conservation equation [-]

RELAXUL = 0.5

Relaxation factor for the liquid momentum conservation equation [-]

RELAXUV = 0.5

Relaxation factor for the vapor momentum conservation equation [-]

RELAXUW = 0.2

Relaxation factor for the wave momentum conservation equation [-]

RELAXWB = 0.5

Relaxation factor for the base mass conservation equation [-]

RELAXWF = 0.5

Relaxation factor for the film mass conservation equation [-]

RELAXWL = 1

Relaxation factor for the liquid mass conservation equation [-]

RELAXWM = 1.0

Relaxation factor for the mixture mass conservation equation [-]

RELAXWV = 0.7

Relaxation factor for the vapor mass conservation equation [-]

RELAXWW = 0.5

Relaxation factor for the wave mass conservation equation [-]

SSCONVFW = 1E-1

Wave frequency steady-state convergence criterion [Hz]

SSCONVH = 1E+0

Enthalpy steady-state convergence criterion [J/kg]

SSCONVP = 1E+0

Pressure steady-state convergence criterion [Pa]

SSCONVU = 1E-3

Velocity steady-state convergence criterion [m/s]

SSCONVUD = 1E-2

Drop velocity steady-state convergence criterion [m/s]

SSCONVUF = 1E-2

Film velocity steady-state convergence criterion [m/s]

SSCONVW = 1E-3

Mass flow rate steady-state convergence criterion [kg/s]

SSCONVWF = 1E-4

Film mass flow rate steady-state convergence criterion [kg/s/m]

SSMAXITER = 30

Maximum number of steady-state iterations

SSTSTEP = 1.0

Time step size for steady-state iterations [s]

TIMEINTERP = 'linear'

Method for interpolating time-dependent boundary conditions

TSTEP = 0.1

Time step size for transient simulations [s]

UDEQUILMAXITER = 100

Maximum number of iterations to find droplet equilibrium velocity

UDEQUILTOL = 1E-3

Target drop force tolerance to find droplet equilibrium velocity [N/m³]

UFEQUILMAXITER = 100

Maximum number of iterations to find film equilibrium velocity

UFEQUILTOL = 1E-3

Target film force tolerance to find film equilibrium velocity [N/m³]

Method Summary

static writeInputFile(filePathName, ID, varargin)

Writes numerical options configuration to input file

Inputs:

- filePathName — Path to output file
- ID — Options identifier
- varargin — Additional name-value pairs for options properties

class Inputs.Input(inputFilePath, key, val)

Bases: dynamicprops, matlab.mixin.Copyable

INPUT Abstract superclass for input-handling classes

Provides core functionality for reading, validating, and managing structured input data. Supports dynamic property creation, default value handling, and solver-dependent customization.

Constructor Summary

Input (*inputFilePath, key, val*)

INPUT Constructor for [Input\(\)](#) class

Parses input file and optionally filters entries by key-value pair. Adds dynamic property 'inputStruct' to hold parsed data.

Inputs:

- *inputFilePath* — Path to input file (string)
- *key* — Field name to filter by (optional)
- *val* — Value to match for filtering (optional)

Property Summary

extra = struct.empty()

Stores unused input entries for reference or debugging

warnings = struct.empty()

Stores warning messages generated during input parsing

Method Summary

applySolverDependentProperties (*solverName*)

APPLYSOLVERDEPENDENTPROPERTIES Applies solver-specific property values based on dependency flags

Returns a copy of the object with updated properties

static convert2JSON (*inputFilePath*)

CONVERT2JSON Converts input file contents to JSON-formatted string

defaultValueUsedReport (*propNames, propValues*)

DEFAULTVALUEUSEDREPORT Generates report of properties for which default values were used

Can print or return formatted string array

listInputProperties (*opts*)

LISTINPUTPROPERTIES Returns list of protected property names for the object

Optionally excludes specified properties

static validateFunctionHandleInput (*handleString*)

VALIDATEFUNCTIONHANDLEINPUT Validates safety of function handle string

Throws error if restricted keywords are detected

validateInputEntry (*objPropname, opts*)

VALIDATEINPUTENTRY Validates whether a property entry is specified or defaulted

Throws error if required entry is missing or empty Returns flags and default value if applicable

static writeInputFile (*filePathName, fdMode, varargin*)

WRITEINPUTFILE Writes input data to file in standard format

Accepts name-value pairs and optional file mode

5.3 Session

The `Session` package in **OpenSTREAM** provides tools for managing session environments, logging, and file structure during simulations. It helps ensure organized workflows and consistent runtime behavior. This includes:

- Session logging manager `Session.Log`
 - Logging mode: `Session.LogMode`
 - Session management: `Session.Session`
-

`Session.isLegalPath(pathname)`

ISLEGALPATH Checks if the given pathname is valid on the current operating system

This function attempts to convert the input pathname to a Java Path object. If the conversion succeeds, the path is considered legal and the function returns true. If an exception is thrown during conversion, the path is considered illegal and the function returns false.

Input:

- `pathname` — String representing the file or folder path to validate

Output:

- `bool` — Logical true if the path is valid, false otherwise

`class Session.Session(opt)`

Bases: `handle`

SESSION Class for managing simulation session metadata and logging

This class handles session naming, directory creation, and logging setup. It ensures that session directories are valid and manages file overwriting behavior. It also integrates with the `Log` class to control output and warnings.

Constructor Summary

`Session(opt)`

SESSION Constructor for `Session()` class

Initializes session metadata including name, directory, and overwrite behavior.

Inputs:

- `opt.name` — `Session()` name
- `opt.dirName` — Directory name
- `opt.parentDir` — Parent directory path
- `opt.overwriteFiles` — Flag to allow overwriting existing session files

Property Summary

`dirName` = ""

Name of the session directory

`directory`

Full path to the session *directory*

`log`

Log object for managing output and warnings

`name` = ""

Name of the session (used for log file naming)

overwriteFiles = false

Flag to allow overwriting existing session files

parentDir = ""

Parent directory where session folder will be created

showWarnings = true

Flag to control display of warnings

Method Summary

makeSessionDirectory()

MAKESESSIONDIRECTORY Creates the session directory

Validates path legality, handles existing directories, closes open files and diaries if necessary, and creates the directory.

setupLog(session, LOGMODE, opts)

SETUPLOG Initializes the Log object for the session

Inputs:

- LOGMODE — Logging mode (console, file, both, none)
- opts.LOGFID — Optional file ID for logging

class Session.Log(LOGMODE, opts)

Bases: `handle`

LOG Console and file-based logging manager

This class manages logging of messages and warnings to console and/or file. It supports multiple logging modes, integrates with session management, and provides diary functionality for persistent logs.

Constructor Summary

Log(LOGMODE, opts)

LOG Constructor for `Log()` class

Initializes logging mode, session context, and warning settings. Creates log file and session directory if needed.

Inputs:

- LOGMODE — Determines where logs go (console, file, both, or none), using `Session.LogMode`.
- opt.session — Session object for directory and naming context
- opt.LOGFID — File ID for the open log file.
- opt.showWarnings — Whether to display warnings in the console.

Property Summary

LOGFID = -1

log file

LOGMODE

Logging mode (console only, file only, both, or none)

diaryFilePath

Full path to the diary file

diaryIsOn = false

Flag indicating whether MATLAB diary is active

logFileName

Name of the log file (without extension)

logFilePath

Full path to the log file

showWarnings = true

Warnings

Method Summary**closeLog()**

CLOSELOG Closes log file and resets file ID

delete()

DELETE Destructor for log class

Ensures log file and diary are properly closed

diaryOff()

DIARYOFF Turn off diary logging

diaryOn()

DIARYON Turn on diary logging if applicable

TODO: re-evaluate if using the diary is needed. What does the diary do that the log doesn't? If we find a use case, describe it here.

log(varargin)

LOG Logs messages to console and/or file depending on LOGMODE

openLog(opts)

OPENLOG Opens log file for writing

Optionally keeps it open between writes

warning(varargin)

WARNING Logs warnings using MATLAB's built-in *warning()* mechanism

Optionally writes to file if warnings are suppressed

class Session.LogMode

Bases: uint16

LOGMODE Enumeration of supported logging modes

This enumeration defines the available logging configurations for the simulation framework. It is used to control whether messages and warnings are printed to the console, written to a file, both, or suppressed entirely.

5.4 Solver

The *Solvers* package defines the **OpenSTREAM** core solver modules and their associated superclasses. Each solver is tailored to simulate specific regimes of one-dimensional, two-phase flow, with increasing levels of physical detail and complexity.

Explore the available solver frameworks:

5.4.1 Four-field solver

The `Solvers.FourField` module in **OpenSTREAM** provides tools and classes for simulating annular two-phase flows using a four-field approach. In this method, the liquid phase is represented by three distinct fields (base film, disturbance waves, and droplets), each governed by its own set of conservation equations for mass, momentum, and energy. The vapor phase is solved using the `Solvers.Mixture` module. This framework enables detailed modeling of liquid film structures, interfacial interactions, slip velocities, and hydrodynamic non-equilibrium effects, making it particularly suitable for annular boiling two-phase flow regimes where field separation and wave dynamics play a significant role.

This module includes:

- Solver class for four-field-based flow simulations: `Solvers.FourField.FourFieldSolver`
- Base film, wave and drop field classes: `Solvers.FourField.Base`, `Solvers.FourField.Wave`, `Solvers.FourField.Drop`
- Film class: `Solvers.FourField.Film`
- Four-field primary properties: `Solvers.FourField.Base.W`, `Solvers.FourField.Base.U`, `Solvers.FourField.Base.H`, `Solvers.FourField.Wave.W`, `Solvers.FourField.Wave.U`, `Solvers.FourField.Wave.H`
- Key solver class methods: `Solvers.FourField.FourFieldSolver.initializeSolver`, `Solvers.FourField.FourFieldSolver.solve()`, `Solvers.FourField.FourFieldSolver.plotz`
- Field class methods for calculating secondary four-field parameters.

class `Solvers.FourField.FourFieldSolver`(*inputSet*, *mixSolver*)

Bases: `Solvers.ThreeField.ThreeFieldSolver.ThreeFieldSolver`

FOURFIELDSOLVER Solver for initializing, solving, and visualizing four-field flow.

The `FourFieldSolver` class extends `Solvers.ThreeField.ThreeFieldSolver` to handle thermal-hydraulic simulations using a four-field approach.

Responsibilities:

- Initializes solver parameters from an `Inputs.InputSet` object
- Constructs `Solvers.FourField.Film`, `Solvers.FourField.Wave`, `Solvers.FourField.Base`, and `Solvers.FourField.Drop` objects for transient and steady-state analysis
- Provides plotting utilities for spatial and temporal distributions

Key Components:

- Liquid film (base and wave) and drop field construction
- Field mass/momentum/energy transport modeling
- Pressure drop gradient obtained from mixture solver
- Vapor solution obtained from mixture solver
- Visualization of results across time and axial domains

Constructor Summary

FourFieldSolver(*inputSet*, *mixSolver*)

FOURFIELDSOLVER Constructor for the FourFieldSolver() class.

Creates a new instance of the FourFieldSolver() class using the provided *Inputs.InputSet*. This constructor initializes the solver by calling the *Solvers.ThreeField.ThreeFieldSolver* constructor and then sets up all necessary solver parameters and internal data structures via *Solvers.FourField.FourFieldSolver.initializeSolver*

Input:

- *inputSet* — An *Inputs.InputSet* object containing model configuration, geometry, boundary conditions, and solver options.
- *mixSolver* — An optional solved *Solvers.Mixture.MixtureSolver* object

Notes:

- This constructor assumes *Inputs.InputSet* is fully validated.
- A solved mixture-model solution is required for initialization. If a mixture solver is supplied, its solution is reused. If no mixture solver is supplied, one is created and solved automatically.
- Initialization includes film, wave, base, and droplet objects.
- Time/space discretization and boundary condition interpolation initialized by *Solvers.Mixture.MixtureSolver* are used

Method Summary**EQUIL**(*ffSolver*, *flm*, *drp*, *mix*, *zIdx*)

EQUIL() Find entrained ratio at film/drop equilibrium state [-]

Iteratively computes the entrained liquid ratio at the onset of annular flow by balancing droplet deposition and film entrainment rates.

Inputs:

- *ffSolver* — *Solvers.FourField.FourFieldSolver* object
- *flm* — Film field object (*Solvers.FourField.Film*)
- *drp* — Droplet field object (*Solvers.FourField.Drop*)
- *mix* — Mixture object (*Solvers.Mixture.Mixture*)
- *zIdx* — Axial index for evaluation

Algorithm:

- Starts with an initial guess for droplet mass flow (50% of liquid mass)
- Updates droplet and film flow rates iteratively
- Computes difference between deposition and entrainment rates
- Stops when error < 1e-4 kg/s/m or after 100 iterations

Notes:

- Enhanced drop deposition is disabled
- Uses interpolation for improved convergence
- Falls back to ad-hoc update if interpolation fails
- Method update film and drop mass flow rates. Use *flm.copy()* and *drp.copy()* as input arguments if used in post-process

initializeSolver(*ffSolver*)

INITIALIZESOLVER Initializes solver parameters and constructs film, wave, base, and drop objects.

Sets up the solver's internal state using the provided *Inputs.InputSet* and mixture solver solutions. Includes initialization of data structures for transient and steady-state simulations.

Operations performed:

- Initializes film, wave, and base arrays for each time step
- Initializes fluid property objects
- Sets up mixture object
- Assigns initial conditions for field mass flow rates, velocities, enthalpies

Notes:

- Assumes uniform axial discretization.
- Wall heat flux computed from interpolated boundary conditions.

plott(*ffSolver, zIdx, opt*)

PLOTT Plots temporal distributions of four-field parameters at a given axial location.

Generates time-series plots of selected four-field parameters at a specified axial index. The method supports multiple display modes, wall selections, and plot arrangements.

Inputs:

- *ffSolver* — [Solvers.FourField.FourFieldSolver](#) object containing simulation data
- *zIdx* — Axial index (scalar, positive integer)
- *opt.display* — Parameters to display (e.g., 'HFLUX', 'W', 'U', etc.)
- *opt.solveMode* — Solve mode: 'REAL' or 'NULL'
- *opt.wall* — Wall index(es) to plot
- *opt.tIdx* — Time indices to include in the plot
- *opt.reverseTime* — Logical flag to reverse time axis
- *opt.unitTemp* — Temperature unit: 'K' or 'C'
- *opt.arrangement* — Plot arrangement: 'flow', 'vertical', or 'horizontal'
- *opt.resize* — Resize factor for plot scaling

Notes:

- If *opt.display* is set to 'ALL', all supported parameters are plotted.
- The function validates that at least two time indices are provided.
- Temperature unit conversion is applied if 'C' is selected.
- Each wall is plotted in a separate tile with appropriate legends and axis scaling.

plotz(*ffSolver, tIdx, opts*)

PLOTZ Plots spatial (axial) distributions of four-field parameters.

Generates axial plots of selected four-field parameters at specified time indices. The function supports multiple display modes, wall selections, and optional animation over time.

Inputs:

- *ffSolver* — [Solvers.FourField.FourFieldSolver](#) object containing simulation data
- *tIdx* — Time index or indices (vector of positive integers)
- *opts.display* — Parameters to display (e.g., 'HFLUX', 'W', 'U', etc.)
- *opts.solveMode* — Solve mode: 'REAL' or 'NULL'
- *opts.wall* — Wall index(es) to plot
- *opts.zIdx* — Axial indices to include in the plot
- *opts.obstructions* — Logical flag to display obstruction locations
- *opts.annular* — Logical flag to plot annular two-phase flow parameters only from the onset of annular flow
- *opts.unitTemp* — Temperature unit: 'K' or 'C'
- *opts.arrangement* — Plot arrangement: 'flow', 'vertical', or 'horizontal'
- *opts.resize* — Resize factor for plot scaling

Notes:

- If multiple time indices are provided, the function creates an animated plot.
- At least two axial indices must be specified to enable spatial plotting.
- Temperature unit conversion is applied if 'C' is selected.
- Obstruction locations are plotted if *opts.obstructions* is true.
- Each wall is plotted in a separate tile with appropriate legends and axis scaling.

plotzt(*ffSolver, opt*)

PLOTZT Plots 2D time/elevation distributions of four-field parameters.

Generates surface plots of selected four-field related parameters over time and axial (elevation) positions. It supports plotting for base and wave fields, and can handle both real and null (initial) transient

data.

Inputs:

- `ffSolver` — [Solvers.FourField.FourFieldSolver](#) object containing simulation data
- `opt.display` — Parameter(s) to display (e.g., 'HFLUX', 'U', etc.)
- `opt.label` — Corresponding labels for display parameters
- `opt.unit` — Units for each parameter
- `opt.field` — Field to plot: 'film' or 'drop'
- `opt.solveMode` — Solve mode: 'REAL' or 'NULL'
- `opt.wall` — Wall index(es) to plot
- `opt.zIdx` — Axial indices (must include at least 2)
- `opt.tIdx` — Time indices (must include at least 2)
- `opt.reverseTime` — Logical flag to reverse time axis
- `opt.shading` — Surface shading style: 'faceted', 'flat', or 'interp'
- `opt.view` — View angle for 3D plot [azimuth elevation]

Notes:

- If 'ALL' is passed to `opt.display`, all supported parameters are plotted.
- The function validates that at least two axial and time indices are provided to enable meaningful 2D plotting.
- Each wall is plotted in a separate figure window with appropriate titles and subplot grouping.

`solve(ffSolver)`

SOLVE Executes the four-field solver for steady-state and transient simulations.

Runs the full solution process for the [Solvers.FourField.FourFieldSolver](#) object. It manages initialization, time stepping, axial sweeps, inner iterations, convergence checks, and logging.

Workflow:

- Enables logging and opens persistent log file
- Validates solver state before execution
- Runs steady-state initialization (`solveINIT = true`)
- If initial step converges, proceeds with transient simulation
- Handles exceptions and ensures proper log closure

Notes:

- Uses internal *solver* function to handle both steady-state and transient modes
- Applies relaxation factors for phase flows, velocities, and enthalpies
- Supports thermal non-equilibrium modeling
- Logs progress and outputs to session directory

class Solvers.FourField.Drop

Bases: [Solvers.ThreeField.Drop](#)

DROP Class for modeling drop field in four-field solver

This class encapsulates the physical and numerical properties of the drop field, including flow variables and phase interactions. It supports multiple solver models and provides methods for computing derived quantities.

All methods are inherited from [Solvers.ThreeField.Drop](#)

class Solvers.FourField.Film(inputSet, fluid)

Bases: [Solvers.AbstractFilm](#)

FILM Class for modeling liquid film in four-field solver

Represents the physical and numerical properties of the liquid film in a four-field thermal-hydraulic solver. The film consists of two sub-fields: base film and wave film, which interact with vapor and droplet phases.

This class manages initialization, mass flow distribution, velocity and enthalpy calculations, and provides methods for splitting film flow at the onset of annular flow (OAF) according to model options.

Key responsibilities:

- Store solver state (time, axial positions, heat flux, evaporation flux)
- Aggregate flow properties from base and wave components
- Initialize base and wave fields and apply OAF split logic
- Support equilibrium calculations for film thickness and flow ratios

Constructor Summary

Film(*inputSet*, *fluid*)

FILM Creates a *Film*() object

Initializes the film object with input set and fluid properties required by the four-field solver.

Inputs:

- *inputSet* — *Inputs.InputSet* object containing geometry and model options
- *fluid* — *Inputs.FluidProperties* object containing thermophysical properties

Property Summary

DT = 0

Time step size [s] from *Inputs.options.TSTEP*

H

) double {mustBeNumeric} = 1E6 % Enthalpy [J/kg]

Type

(

Type

,

HFLUX = 1.

Film heat flux [W/m^2]

ITR

Iteration tracking

MEVAP = -1.

Evaporation mass flux [kg/s/m^2]

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from *Inputs.Model.NNODES*

TIDX = 1

Time step index [-]

TIME = 0

Time series [s]

U

) double {mustBeNumeric} = 1. % Velocity [m/s]

Type

(

```

    Type
    ,
W
    ) double {mustBeNumeric} = 1. % Mass flow rate [kg/s]
    Type
    (
    Type
    ,
Z = 1.
    Elevation [m]
base = NaN
    Solvers.FourField.Base object
wave = NaN
    Solvers.FourField.Wave object
```

Method Summary

EQUIL(*film*, *Wf*, *zIdx*)

EQUIL() Equilibrium base/wave split ratio [-]

Calculates the equilibrium split ratio of film mass flow between base and wave using base equilibrium thickness, base velocity, perimeter, and liquid density.

Inputs:

- *film* — *Solvers.FourField.Film* object
- *Wf* — Film mass flow rate [kg/s]
- *zIdx* — Axial indices to evaluate (optional)

copyFlowProperties(*srcObj*, *targetObj*, *opts*)

COPYFLOWPROPERTIES Copy flow properties between Film objects

Copies flow-related properties (base and wave fields) from a source Film object to one or more target Film objects. Ensures that spatial meshes match before copying.

Inputs:

- *srcObj* — Source *Solvers.FourField.Film* object
- *targetObj* — Target *Solvers.FourField.Film* object(s)
- *opts.all* — Logical flag to copy all properties (optional, default = false)

distributeOAFW(*film*, *Wf*, *zIdx*)

DISTRIBUTEOAFW Distribute film mass flow at OAF [kg/s]

Distributes the film mass flow rate at the onset of annular flow (OAF) between base and wave fields according to the *Inputs.Model.OAFFILMSPLIT* model option.

Inputs:

- *film* — *Solvers.FourField.Film* object
- *Wf* — Film mass flow rate to distribute [kg/s] (per wall or per z-index)
- *zIdx* — Axial indices to apply distribution (optional)

initializeBaseAndWave(*film*, *WIN*, *ITR*)

INITIALIZEBASEANDWAVE Initialize base and wave fields

Initializes base and wave arrays given inlet film mass flow rate and iteration settings. Performs OAF film split, computes velocities, enthalpies, and sets initial wave frequency using equilibrium correlations.

Inputs:

- film — *Solvers.FourField.Film* object
- WIN — Inlet film mass flow rate [kg/s]
- ITR — Iteration control struct

class *Solvers.FourField.Wave(film)*

Bases: *Solvers.AbstractFilm*

WAVE Class for modeling liquid waves in four-field solver

This class represents the physical and numerical properties of the wave field in a four-field thermal-hydraulic solver. It handles flow variables, heat transfer, and phase interaction mechanisms between the waves and other fields (base film, drop, vapor).

The class supports initialization from an existing film object and provides methods for computing derived quantities such as mass flow fractions, interfacial fractions, and deposition/evaporation fluxes.

Constructor Summary

Wave(film)

WAVE Constructor for *Wave()* class

Initializes the wave object using properties from an existing film object. Copies solver configuration and geometry data, and allocates flow property arrays for multiple walls.

Inputs:

- film — *Solvers.AbstractFilm* object representing the parent film

Property Summary

DT = 0

Time step size [s] from *Inputs.options.TSTEP*

FREQUENCY = 1

Wave frequency [Hz]

H = 1E6

Enthalpy [J/kg]

HFLUX = 1.

Film heat flux [W/m^2]

ITR

Iteration tracking

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from *Inputs.Model.NNODES*

TIDX = 1

Time step index [-]

TIME = 0

Time series [s]

U = 1.

Velocity [m/s]

W = 1.

Mass flow rate [kg/s]

Z = 1.

Elevation [m]

film

Parent *Solvers.AbstractFilm* object

Method Summary

AMPLITUDE(*wave*, *zIdx*)

AMPLITUDE() Wave amplitude [m]

Computes wave amplitude based on mass flow, shape factor, fluid density, and frequency. Limited by half the hydraulic diameter.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

BETA(*wave*, *zIdx*)

BETA() Wave film interfacial fraction [-]

Calculates the fraction of the interface occupied by waves.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

BETAP(*wave*, *zIdx*)

BETAP() Wave film heat flux fraction [-]

Determines the fraction of heat flux directed to the waves.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

DELTA_FREQ(*wave*, *zIdx*)

DELTA_FREQ() Wave frequency source/sink [Hz/m]

Calculates the exchange term for wave frequency relaxation toward equilibrium frequency.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

DRAGCOEF(*wave*, *zIdx*)

DRAGCOEF() Wave drag coefficient [-]

Calculates the drag coefficient for vapor-wave interaction based on Reynolds number and *Inputs.Model.WAVEDRAGCOEF* model coefficients.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

EPSILON(*wave*, *zIdx*)*EPSILON()* Wave mass flow fraction [-]

Computes the fraction of total film mass flow carried by waves.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

EQFREQUENCY(*wave*, *zIdx*)*EQFREQUENCY()* Equilibrium wave frequency [Hz]

Computes the equilibrium wave frequency based on Strouhal number and vapor velocity.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

EQPERIOD(*wave*, *zIdx*)*EQPERIOD()* Equilibrium wave period [s]

Calculates the inverse of equilibrium wave frequency.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

EQSTROUHAL(*wave*, *zIdx*)*EQSTROUHAL()* Equilibrium Strouhal number [-]Calculates the equilibrium Strouhal number using selected *Inputs.Model.EQSTROUHAL* model.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

ETA(*wave*, *zIdx*)*ETA()* Wave film deposition fraction [-]

Computes the fraction of deposition attributed to waves.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

FBASE(*wave*, *drop*, *zIdx*)*FBASE()* Base interfacial force [N/m²]

Computes the interfacial force between wave and base film.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *drop* — *Solvers.FourField.Drop* object
- *zIdx* — Axial indices to evaluate (optional)

FBASEMASS(*wave*, *drop*, *zIdx*)*FBASEMASS()* Wave mass exchange force [N/m²]

Calculates the force due to mass exchange between wave and base film.

Inputs:

- *wave* — *Solvers.FourField.Wave* object

- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

FDEP(*wave, drop, zIdx*)

FDEP() Droplet interaction force [N/m²]

Calculates the force due to deposition of droplets onto the wave field.

Inputs:

- wave — *Solvers.FourField.Wave* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

FDRAG(*wave, zIdx*)

FDRAG() Vapor drag force on wave [N/m²]

Calculates the shear stress due to vapor drag acting on the wave field.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

FSHEAR(*wave, zIdx*)

FSHEAR() Vapor shear stress [N/m²]

Computes the shear stress exerted by vapor on the wave field using friction factor.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

FTOT(*wave, drop, zIdx*)

FTOT() Total interfacial force [N/m²]

Computes the total interfacial force acting on the wave field, including vapor shear, buoyancy, gravity, droplet interaction, and base film forces.

Inputs:

- wave — *Solvers.FourField.Wave* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

FVAPOR(*wave, zIdx*)

FVAPOR() Wave vapor shear stress [N/m²]

Computes the total shear stress exerted by vapor on the wave field.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

FWALL(*wave, zIdx*)

FWALL() Wave wall shear stress [N/m²]

Placeholder method. Wave does not implement wall shear stress.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

MBASE(*wave, drop, zIdx*)

MBASE() Mass flux interaction with base film [kg/m²/s]

Computes the net mass exchange between wave and base film, including turbulent mixing.

Inputs:

- wave — *Solvers.FourField.Wave* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

MDEP(*wave, drop, zIdx*)

MDEP() Wave deposition mass flux [kg/m²/s]

Computes the deposition mass flux from waves to droplets.

Inputs:

- wave — *Solvers.FourField.Wave* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

MENT(*wave, zIdx*)

MENT() Wave entrainment mass flux [kg/m²/s]

Computes the mass flux associated with entrainment from the wave field.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

MEVAP(*wave, zIdx*)

MEVAP() Wave evaporation mass flux [kg/m²/s]

Calculates the evaporation mass flux from the wave field based on heat flux fraction.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

MTOT(*wave, drop, zIdx*)

MTOT() Total mass flux [kg/m²/s]

Calculates the total mass flux for the wave field, including evaporation, entrainment, deposition, and interactions with base film.

Inputs:

- wave — *Solvers.FourField.Wave* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

MTURB(*wave, zIdx*)

MTURB() Turbulent mass exchange [kg/m²/s]

Returns the turbulent mixing mass flux between wave and base film.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

N(*wave, zIdx*)

N() Wave number density [1/m]

Computes the number of waves per unit length (inverse of wave spacing).

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

PERIOD(wave, zIdx)

PERIOD() Wave time period [s]

Calculates the inverse of wave frequency.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

REV(wave, zIdx)

REV() Vapor Reynolds number (relative to waves) [-]

Computes the Reynolds number for vapor flow interacting with waves.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

SHAPEFACTOR(wave, zIdx)

SHAPEFACTOR() Wave shape factor [-]

Computes the dimensionless shape factor for waves based on Reynolds number and *Inputs.Model.SHAPEFACTORCOEF* model.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

SPACING(wave, zIdx)

SPACING() Wave spacing [m]

Computes the axial spacing between waves based on velocity and frequency.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

THICK(wave, zIdx)

THICK() Wave thickness [m]

Computes the wave thickness as the product of amplitude and interfacial fraction. This parameter represents the thickness that the wave field would have if fully overlayed on top of the base film.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

WIDTH(wave, zIdx)

WIDTH() Wave width [m]

Calculates wave width as a function of amplitude and shape factor, limited by wave spacing.

Inputs:

- wave — *Solvers.FourField.Wave* object
- zIdx — Axial indices to evaluate (optional)

WL(*wave*, *zIdx*)

WL() Wave mass flow rate per unit perimeter [kg/s/m]

Calculates the mass flow carried by the wave field at specified axial locations.

Inputs:

- *wave* — *Solvers.FourField.Wave* object
- *zIdx* — Axial indices to evaluate (optional)

class Solvers.FourField.Base(*film*)

Bases: *Solvers.AbstractFilm*

BASE Class for modeling liquid base film in four-field solver

This class represents the physical and numerical properties of the base film field in a four-field thermal-hydraulic solver. It handles flow variables, heat transfer, and phase interaction mechanisms between the base film and other fields (wave, drop, vapor).

The class supports initialization from an existing film object and provides methods for computing derived quantities such as mass flow fractions, interfacial fractions, and deposition/evaporation fluxes.

Constructor Summary

Base(*film*)

BASE Constructor for *Base()* class

Initializes the base film object using properties from an existing film object. Copies solver configuration and geometry data, and allocates flow property arrays for multiple walls.

Inputs:

- *film* — *Solvers.AbstractFilm* object representing the parent film

Property Summary

DT = 0

Time step size [s] from *Inputs.options.TSTEP*

H = 1E6

Enthalpy [J/kg]

HFLUX = 1.

Film heat flux [W/m^2]

ITR

Iteration tracking

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from *Inputs.Model.NNODES*

TIDX = 1

Time step index [-]

TIME = 0

Time series [s]

U = 1.

Velocity [m/s]

W = 1.

Mass flow rate [kg/s]

Z = 1.

Elevation [m]

Method Summary

BETA(*base, zIdx*)

BETA() Base film interfacial fraction [-]

Computes the fraction of interfacial area attributed to the base film.

Inputs:

- *base* — *Solvers.FourField.Base* object
- *zIdx* — Axial indices to evaluate (optional)

BETAP(*base, zIdx*)

BETAP() Base film heat flux fraction [-]

Determines the fraction of heat flux directed to the base film. Applies suppression in pre-annular region to enforce target base film thickness at onset of annular flow (OAF).

Inputs:

- *base* — *Solvers.FourField.Base* object
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Applies suppression in pre-annular region to enforce target base film thickness at onset of annular flow (OAF). The entire film evaporation hence occurs in the wave field.

EPSILON(*base, zIdx*)

EPSILON() Base film mass flow fraction [-]

Calculates the fraction of total film mass flow carried by the base film at specified axial locations.

Inputs:

- *base* — *Solvers.FourField.Base* object
- *zIdx* — Axial indices to evaluate (optional)

EQTHICK(*base, zIdx*)

EQTHICK() Base equilibrium thickness [m]

Computes the equilibrium thickness of the base film based on *Inputs.Model.BASEEQTHICK* model.

Inputs:

- *base* — *Solvers.FourField.Base* object
- *zIdx* — Axial indices to evaluate (optional)

ETA(*base, zIdx*)

ETA() Base film deposition fraction [-]

Computes the fraction of drop deposition allocated to the base film based on mass flow and interfacial fractions.

Inputs:

- *base* — *Solvers.FourField.Base* object
- *zIdx* — Axial indices to evaluate (optional)

FBASEVAPOR(*base, zIdx*)*FBASEVAPOR()* Vapor shear stress on base film [N/m²]

Computes the shear force exerted by the vapor phase on the base film, scaled by interfacial fraction.

Inputs:

- base — *Solvers.FourField.Base* object
- zIdx — Axial indices to evaluate (optional)

FDEP(*base, drop, zIdx*)*FDEP()* Droplet deposition force [N/m²]

Calculates the momentum transfer to the base film from droplet deposition, based on velocity difference and deposition flux.

Inputs:

- base — *Solvers.FourField.Base* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

FDRY(*base, drop, zIdx*)*FDRY()* Dry fraction [-]

Computes the fraction of the exposure period during which the base film is expected to be dry.

Inputs:

- base — *Solvers.FourField.Base* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

FTOT(*base, drop, zIdx*)*FTOT()* Total interfacial force on base film [N/m²]

Aggregates all force contributions acting on the base film, including wall shear, wave interaction, vapor shear, buoyancy, gravity, and droplet deposition based on *Inputs.Model.MOMENTBASE* model.

Inputs:

- base — *Solvers.FourField.Base* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

FWAVE(*base, drop, zIdx*)*FWAVE()* Wave interfacial force [N/m²]

Computes the interfacial shear force exerted by the wave field on the base film based on *Inputs.Model.WAVEBASEINT* model.

Inputs:

- base — *Solvers.FourField.Base* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

FWAVEMASS(*base, drop, zIdx*)*FWAVEMASS()* Base-wave mass exchange force [N/m²]

Calculates the momentum exchange force due to mass transfer between base and wave films, based on velocity difference and MWAVE flux.

Inputs:

- base — *Solvers.FourField.Base* object
- drop — *Solvers.FourField.Drop* object

- `zIdx` — Axial indices to evaluate (optional)

MDEP(*base, drop, zIdx*)

MDEP() Base deposition mass flux [kg/m²/s]

Calculates deposition flux to the base film from drops, scaled by deposition fraction.

Inputs:

- `base` — *Solvers.FourField.Base* object
- `zIdx` — Axial indices to evaluate (optional)

MENT(*base, zIdx*)

MENT() Base entrainment mass flux [kg/m²/s]

Placeholder for future implementation if needed. Currently returns zero entrainment.

Inputs:

- `base` — *Solvers.FourField.Base* object
- `zIdx` — Axial indices to evaluate (optional)

MEVAP(*base, zIdx*)

MEVAP() Base evaporation mass flux [kg/m²/s]

Computes evaporation flux from the base film using heat flux fraction and parent film evaporation rate.

Inputs:

- `base` — *Solvers.FourField.Base* object
- `zIdx` — Axial indices to evaluate (optional)

MTOT(*base, drop, zIdx*)

MTOT() Total mass flux of the base film [kg/m²/s]

Calculates the total mass flux exchange associated with the base film, including evaporation, entrainment, deposition, and wave interaction.

Inputs:

- `base` — *Solvers.FourField.Base* object
- `drop` — *Solvers.FourField.Drop* object
- `zIdx` — Axial indices to evaluate (optional)

MTURB(*base, zIdx*)

MTURB() Turbulent mass exchange [kg/m²/s]

Estimates turbulent mass exchange between base and wave films using wave mixing coefficient *Inputs.Model.WAVEMIXCOEF*.

Inputs:

- `base` — *Solvers.FourField.Base* object
- `zIdx` — Axial indices to evaluate (optional)

MWAVE(*base, drop, zIdx*)

MWAVE() Base film mass flux interaction with wave [kg/m²/s]

Computes the mass flux exchange between the base film and wave field. Includes net exchange term based on relaxation toward equilibrium thickness and turbulent mixing contribution.

Inputs:

- `base` — *Solvers.FourField.Base* object
- `drop` — *Solvers.FourField.Drop* object
- `zIdx` — Axial indices to evaluate (optional)

Outputs:

- Mwave — Mass flux transferred to wave field [kg/m²/s]
- Mnet — Net exchange term before turbulent adjustment [kg/m²/s]

TBASE(*base, zIdx*)

TBASE() Base film exposure time [s]

Calculates the time during which the base film is exposed to vapor between successive waves, based on wave spacing and relative velocity (eq 24 of Le Corre [2022a]).

Inputs:

- base — *Solvers.FourField.Base* object
- zIdx — Axial indices to evaluate (optional)

TDRY(*base, drop, zIdx*)

TDRY() Base film dry-out time [s]

Estimates the time of base film dry out under current evaporation and deposition conditions. Returns zero if dry-out does not occur.

Inputs:

- base — *Solvers.FourField.Base* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

THICKMIN(*base, drop, zIdx*)

THICKMIN() Minimum base film thickness [m]

Calculates the minimum thickness of the base film at the end of its exposure period, based on mass flow rate, liquid density, velocity, and channel perimeter (eq. 28 of Le Corre [2022a]).

Inputs:

- base — *Solvers.FourField.Base* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

WMIN(*base, drop, zIdx*)

WMIN() Minimum base film mass flow rate [kg/s]

Calculates the minimum mass flow rate of the base film at the end of its exposure period (TBASE), accounting for evaporation and deposition.

Inputs:

- base — *Solvers.FourField.Base* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

WMINL(*base, drop, zIdx*)

WMINL() Minimum base film mass flow rate per perimeter [kg/m-s]

Computes the minimum base film mass flow rate normalized by channel perimeter.

Inputs:

- base — *Solvers.FourField.Base* object
- drop — *Solvers.FourField.Drop* object
- zIdx — Axial indices to evaluate (optional)

copyFlowProperties(*srcObj, targetObj, opts*)

COPYFLOWPROPERTIES Copy flow property arrays between Base objects

Copies selected flow properties (W, U, H) from a source object to one or more target Base objects. Supports full copy or partial copy (excluding first axial node) based on options.

Inputs:

- srcObj — Source object containing flow properties
- targetObj — Array of *Solvers.FourField.Base* objects to update
- opts.all — Logical flag; if true, copies all axial nodes, otherwise skips the first node (default: false)

5.4.2 Mixture solver

The *Solvers.Mixture* module in **OpenSTREAM** provides classes and tools for simulating two-phase flows using a mixture approach. In this method, all phases are treated as a single continuum with averaged properties, making it suitable for cases where phase separation is minimal or not explicitly resolved. Additionally, the solver supports a Mixture Relaxation Model (MRM), which introduces thermal non-equilibrium between phases by solving separate mass and energy conservation equations for the vapor phase.

This module includes:

- Solver class for mixture-based flow simulations: *Solvers.Mixture.MixtureSolver*
- Mixture field class: *Solvers.Mixture.Mixture*
- Phase classes: *Solvers.Mixture.Liquid*, *Solvers.Mixture.Vapor*
- Mixture primary properties: *Solvers.Mixture.Mixture.W*, *Solvers.Mixture.Mixture.P*, *Solvers.Mixture.Mixture.H*
- Key solver class methods: *Solvers.Mixture.MixtureSolver.initializeSolver*, *Solvers.Mixture.MixtureSolver.solve()*, *Solvers.Mixture.MixtureSolver.plotz*
- Field class methods for calculating secondary mixture and phase parameters.

class *Solvers.Mixture.MixtureSolver*(*inputSet*)

Bases: *Solvers.AbstractSolver*

MIXTURESOLVER Solver for initializing, solving, and visualizing mixture flow.

The *MixtureSolver* class handles the setup, execution, and visualization of thermal-hydraulic simulations using a two-phase mixture approach or a Mixture Relaxation Model (MRM).

Responsibilities:

- Initializes solver parameters from an *Inputs.InputSet* object
- Interpolates boundary conditions in time and space
- Constructs *Solvers.Mixture.Mixture* objects for transient and steady-state analysis
- Provides plotting utilities for spatial and temporal distributions

Key Components:

- Mixture construction and phase separation (liquid/vapor)
- Mixture mass/momentum/energy transport modeling
- Support for relaxation models and near-wall phenomena
- Visualization of results across time and axial domains

Constructor Summary

MixtureSolver(*inputSet*)

MIXTURESOLVER Constructor for the MixtureSolver() class.

Creates a new instance of the MixtureSolver() class using the provided *Inputs.InputSet*. This constructor initializes the solver by calling the abstract *Solvers.AbstractSolver* superclass constructor and then sets up all necessary solver parameters and internal data structures via *Solvers.Mixture.MixtureSolver.initializeSolver*

Input:

- *inputSet* — An *Inputs.InputSet* object containing model configuration, geometry, boundary conditions, and solver options.

Notes:

- This constructor assumes *Inputs.InputSet* is fully validated.
- Solver initialization includes time/space discretization, boundary condition interpolation, and mixture object setup.

Property Summary

DT = 0

Time step size [s] from *Inputs.Options.TSTEP*

DZ = 0

Axial step size [m]

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from *Inputs.Model.NNODES*

TIME = 0

Time series [s]

Z = 1.

Elevation [m]

boundaryConditions

Boundary conditions object *Inputs.BoundaryConditions*

fluid

Fluid object *Inputs.FluidProperties*

inputSet

Input set object *Inputs.InputSet*

mixture

Mixture object

mixtureInit

Null transient mixture object

Method Summary

axialInterpolate(*mix*, *x*, *y*)

AXIALINTERPOLATE Performs axial interpolation of data using a specified method.

Interpolates the values in *y* at positions defined by *mix.Z* using the interpolation method specified in *Inputs.options.AXIALINTERP*. The interpolation is performed over the domain defined by *x*.

If the interpolation returns NaN (e.g., due to points slightly outside the interpolation range when using methods like ‘next’), linear extrapolation is applied to those specific points to ensure continuity.

This function ensures robust handling of out-of-bound values by using extrapolation and fallback strategies.

initializeSolver(*mixSolver*)

INITIALIZESOLVER Initializes solver parameters and constructs mixture objects.

Sets up the solver’s internal state using the provided [Inputs.InputSet](#). This includes discretization of time and space, interpolation of boundary conditions, and initialization of data structures for transient and steady-state simulations.

Operations performed:

- Computes time steps and axial node positions
- Interpolates boundary conditions in time and space
- Initializes mixture array for each time step
- Constructs data structures for:
 - Pressure drops (DP, DPSUM)
 - Momentum and energy derivatives (MDER)
 - Mixture Relaxation Model (MRM)
 - Near-wall transport (NEARWALL)
 - Iteration tracking (ITR)
- Initializes fluid property objects
- Sets up phase-specific objects (Liquid, Vapor)
- Assigns initial conditions for mass flow rate, pressure, enthalpy

Notes:

- The function assumes uniform axial discretization.
- Wall heat flux is computed from interpolated boundary conditions.
- MRM fields are initialized after enthalpy to ensure correct vapor quality (XEQ) computation.

interpBoundaryConditions(*mixSolver*)

INTERBOUNDARYCONDITIONS Interpolates boundary conditions in time and space.

Expands user-defined [Inputs.BoundaryConditions](#) to match the solver’s spatial grid and time steps. It performs interpolation of relevant boundary condition parameters both temporally and axially, and computes derived quantities such as wall heat flux.

Operations performed:

- Interpolates time-dependent boundary condition parameters: ‘TIME’, ‘PRESSURE’, ‘HIN’, ‘MFLOW’, ‘POWER’
- Interpolates wall power distribution in space (axially)
- Interpolates wall power in time across all walls
- Computes wall heat flux at each axial node and time step

Notes:

- Uses `Solvers.Mixture.MixtureSolver.timeInterpolate` and `Solvers.Mixture.MixtureSolver.axialInterpolate` for interpolation.
- Throws an error if any expected boundary condition parameter is missing.
- The calculation of HFLUX is based on normalized wall power and geometric properties.

plott(*mixSolver*, *zIdx*, *opt*)

PLOTT Plots temporal distributions of mixture parameters at a given axial location.

Generates time-series plots of selected mixture parameters at a specified axial index. The method supports multiple display modes, wall selections, and plot arrangements.

Inputs:

- *mixSolver* — [Solvers.Mixture.MixtureSolver](#) object containing simulation data

- `zIdx` — Axial index (scalar, positive integer)
- `opt.display` — Parameters to display (e.g., 'HFLUX', 'W', 'DP', etc.)
- `opt.solveMode` — Solve mode: 'REAL' or 'NULL'
- `opt.wall` — Wall index(es) to plot
- `opt.tIdx` — Time indices to include in the plot
- `opt.reverseTime` — Logical flag to reverse time axis
- `opt.unitTemp` — Temperature unit: 'K' or 'C'
- `opt.arrangement` — Plot arrangement: 'flow', 'vertical', or 'horizontal'
- `opt.resize` — Resize factor for plot scaling
- `opt.nearWall` — Logical flag to include near-wall parameters

Notes:

- If `opt.display` is set to 'ALL', all supported parameters are plotted.
- The function validates that at least two time indices are provided.
- Temperature unit conversion is applied if 'C' is selected.
- Near-wall data is included if `opt.nearWall` is true.
- Each wall is plotted in a separate tile with appropriate legends and axis scaling.
- In the pressure drop plot, the temporal acceleration includes the slip-induced contribution (if any)

plotz(*mixSolver*, *tIdx*, *opts*)

PLOTZ Plots spatial (axial) distributions of mixture parameters.

Generates axial plots of selected mixture parameters at specified time indices. The function supports multiple display modes, wall selections, near-wall data, and optional animation over time.

Inputs:

- `mixSolver` — [Solvers.Mixture.MixtureSolver](#) object containing simulation data
- `tIdx` — Time index or indices (vector of positive integers)
- `opts.display` — Parameters to display (e.g., 'HFLUX', 'W', 'DP', etc.)
- `opts.solveMode` — Solve mode: 'REAL' or 'NULL'
- `opts.wall` — Wall index(es) to plot
- `opts.zIdx` — Axial indices to include in the plot
- `opts.obstructions` — Logical flag to display obstruction locations
- `opts.unitTemp` — Temperature unit: 'K' or 'C'
- `opts.arrangement` — Plot arrangement: 'flow', 'vertical', or 'horizontal'
- `opts.resize` — Resize factor for plot scaling
- `opts.nearWall` — Logical flag to include near-wall parameters

Notes:

- If multiple time indices are provided, the function creates an animated plot.
- At least two axial indices must be specified to enable spatial plotting.
- Temperature unit conversion is applied if 'C' is selected.
- Obstruction locations are plotted if `opts.obstructions` is true.
- Each wall is plotted in a separate tile with appropriate legends and axis scaling.
- In the pressure drop plot, the temporal acceleration includes the slip-induced contribution (if any)

plotzt(*mixSolver*, *opt*)

PLOTZT Plots 2D time/elevation distributions of mixture parameters.

Generates surface plots of selected mixture-related parameters over time and axial (elevation) positions. It supports plotting for mixture, liquid and vapor fields, and can handle both real and null (initial) transient data.

Inputs:

- `mixSolver` — [Solvers.Mixture.MixtureSolver](#) object containing simulation data
- `opt.display` — Parameter(s) to display (e.g., 'HFLUX', 'U', etc.)
- `opt.label` — Corresponding labels for display parameters
- `opt.unit` — Units for each parameter

- `opt.field` — Field to plot: ‘mixture’, ‘liquid’, or ‘vapor’
- `opt.solveMode` — Solve mode: ‘REAL’ or ‘NULL’
- `opt.wall` — Wall index(es) to plot
- `opt.zIdx` — Axial indices (must include at least 2)
- `opt.tIdx` — Time indices (must include at least 2)
- `opt.reverseTime` — Logical flag to reverse time axis
- `opt.shading` — Surface shading style: ‘faceted’, ‘flat’, or ‘interp’
- `opt.view` — View angle for 3D plot [azimuth elevation]

Notes:

- If ‘ALL’ is passed to `opt.display`, all supported parameters are plotted.
- The function validates that at least two axial and time indices are provided to enable meaningful 2D plotting.
- Each wall is plotted in a separate figure window with appropriate titles and subplot grouping.

`solve`(*mixSolver*)

SOLVE Executes the mixture solver for steady-state and transient simulations.

Runs the full solution process for the [Solvers.Mixture.MixtureSolver](#) object. It manages initialization, time stepping, axial sweeps, inner iterations, convergence checks, and logging.

Workflow:

- Enables logging and opens persistent log file
- Validates solver state before execution
- Runs steady-state initialization (`solveINIT = true`)
- If initial step converges, proceeds with transient simulation
- Handles exceptions and ensures proper log closure

Notes:

- Uses internal *solver* function to handle both steady-state and transient modes
- Applies relaxation factors for flow, pressure, enthalpy, and vapor properties
- Supports thermal non-equilibrium modeling via vapor relaxation
- Logs progress and outputs to session directory

`timeInterpolate`(*mix*, *y*)

TIMEINTERPOLATE Performs temporal interpolation of data using a specified method.

Interpolates the values in *y* at the time points defined by *mix.TIME* using the interpolation method specified in `Inputs.options.TIMEINTERP`.

If the boundary condition time vector (*mix.inputSet.bc.TIME*) is scalar, interpolation is skipped and the original data *y* is returned unchanged.

This function supports flexible interpolation strategies and handles cases where interpolation is unnecessary.

`class Solvers.Mixture.Vapor`(*mix*)

Bases: [Solvers.AbstractPhase](#)

VAPOR Represents the vapor phase in a mixture solver simulation.

The Vapor class provides access to vapor-specific properties and calculations derived from the mixture solution. It encapsulates methods for computing flow, thermodynamic, and transport properties of the vapor phase at each axial node.

Responsibilities:

- Compute vapor mass fraction and void fraction
- Calculate vapor velocity, enthalpy, and temperature

- Evaluate wall heat flux and wall evaporation contributions
- Support derived quantities such as Reynolds number and mass flux

Notes:

- All methods assume access to a valid `Solvers.Mixture.Mixture` object
- Axial indexing is optional; defaults to full axial domain
- Enthalpy calculations adapt to thermal non-equilibrium models
- Velocity and enthalpy calculations include fallback logic to handle single-phase liquid regions and numerical stability

Constructor Summary

Vapor(*mix*)

VAPOR Constructor of the `Vapor()` class

Initializes the `Vapor()` object from one or more instances of the `Solvers.Mixture.Mixture` class. Each `Vapor()` instance stores a reference to its corresponding Mixture object and the number of axial nodes (NZ).

Input:

- *mix* — Array of `Solvers.Mixture.Mixture` objects representing the simulation state

Notes:

- Supports vectorized initialization for transient simulations
- Assumes each `Solvers.Mixture.Mixture` object is fully initialized

Method Summary

C(*vapor*, *zIdx*)

`C()` Vapor mass fraction [-]

FW(*vapor*, *zIdx*)

`FW()` Vapor wall friction factor [-]

Computes the Darcy wall friction factor based on `Inputs.Model.SPMTM` model.

Supported models

- BLASIUS: Blasius model ($f = C(1)Re^{C(2)} + C(3)$) using user-defined `Inputs.Model.FRICTION` coefficients

H(*vapor*, *zIdx*)

`H()` Vapor enthalpy [J/kg]

Calculation depends on `Inputs.Model.THERMALNONEQ` model.

HFLUX(*vapor*, *zIdx*)

`HFLUX()` Wall heat flux to vapor [W/m^2]

HFLUXWALEVAP(*vapor*, *zIdx*)

HFLUX Wall evaporation heat flux [W/m^2]

HWALL(*vapor*, *twall*, *zIdx*)

`HWALL()` Single-phase vapor wall heat transfer coefficient [W/m^2/K]

Computes the wall heat transfer coefficient using vapor thermal conductivity and Nusselt number.

MFLUX(*vapor*, *zIdx*)

`MFLUX()` Vapor mass flux [kg/m^2-s]

NU(*vapor*, *twall*, *zIdx*)

NU() Vapor wall Nusselt number [-]

Computes the Nusselt number for wall heat transfer to vapor based on *Inputs.Model.SPHTM* model.

Q(*vapor*, *zIdx*)

Q() Vapor volumetric flow rate [m³/s]

RE(*vapor*, *zIdx*)

RE() Vapor Reynolds number [-]

T(*vapor*, *zIdx*)

T() Vapor temperature [K]

TAUW(*vapor*, *zIdx*)

TAUW() Vapor wall shear stress [N/m²]

TIME(*vapor*)

TIME() Time series [s]

U(*vapor*, *zIdx*)

U() Vapor velocity [m/s]

VF(*vapor*, *zIdx*)

VF() Void fraction [-]

W(*vapor*, *zIdx*)

W() Vapor mass flow rate [kg/s]

X(*vapor*, *zIdx*)

X() Vapor mass flow fraction [-]

Z(*vapor*, *zIdx*)

Z() Axial nodes [m]

class *Solvers.Mixture.Mixture*(*inputSet*, *fluid*)

Bases: *Solvers.AbstractField*

MIXTURE Class for modeling two-phase mixture field in mixture solver

This class encapsulates the physical and numerical properties of a fluid mixture, including flow variables, phase interactions, pressure losses, heat transfer, and relaxation models. It supports multiple solver models and provides methods for computing derived quantities and handling transitions such as boiling and annular flow.

Constructor Summary

Mixture(*inputSet*, *fluid*)

MIXTURE Constructor for *Mixture()* class

Initializes the mixture object with input configuration and fluid properties. Sets up flow property tracking for simulation.

Inputs:

- *inputSet* — *Inputs.InputSet* object containing model, geometry, and boundary conditions
- *fluid* — *Inputs.FluidProperties* object containing thermophysical fluid data

Property Summary

DP
Saved detailed pressure drops [Pa]

DPSUM
Saved detailed cumulative pressure drops [Pa]

DT = 0
Time step size [s] from `Inputs.options.TSTEP`

DZ = 0
Axial step size [m]

H = 1E6
Enthalpy [J/kg]

HFLUX = 1.
Wall heat flux [W/m^2]

ITR
Iteration tracking

MDER
Saved detailed material derivative terms

MRM
Mixture relaxation terms

NEARWALL
Near-wall terms

NTIME = 0
Number of time steps [-]

NZ = 0
Number of axial steps [-] from `Inputs.Model.NNODES`

P = 7E6
Pressure [Pa]

TIDX = 1
Time step index [-]

TIME = 0
Time series [s]

W = 1.
Mass flow rate [kg/s]

Z = 1.
Elevation [m]

cbt = false
Critical Boiling Transition flag [-]

fluid
`Inputs.FluidProperties` object

inputSet

Inputs.InputSet object

liquid

Liquid phase object

mfbt = false

Minimum Film Boiling Transition flag [-]

vapor

Vapor phase object

Method Summary**AFDISTR**(*mix*, *param1*, *param2*, *zIdx*)

AFDISTR() Computes annular flow distribution over axial positions.

Evaluates a weighted distribution between *param1* and *param2* using the annular flow fraction defined in *Solvers.Mixture.Mixture.AFFNC*.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object containing axial grid and flow fractions
- *param1* — First flow property
- *param2* — Second flow property
- *zIdx* — Axial indices to evaluate (optional; defaults to full axial range)

Notes:

- The annular flow fraction is extracted from *Solvers.Mixture.Mixture.AFFNC* at *zIdx*
- The output is a linear interpolation: $(1 - \text{affnc}) * \text{param1} + \text{affnc} * \text{param2}$
- *mix(1).NZ* is used for default indexing, assuming *mix* may be an array

AFFNC(*mix*, *zIdx*)

AFFNC() Computes annular flow fraction over axial positions.

Evaluates the annular flow fraction using a sigmoid function parameterized by the transition model and geometry. The sigmoid is evaluated via the cached *sigm* method.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object containing model and geometry data
- *zIdx* — Axial indices to evaluate (optional; defaults to full axial range)

Notes:

- Sigmoid parameters are scaled by node density ($\text{NNODES} / \text{LENGTH}$)
- The center of the transition is offset by *mix.OAFIDX()*
- Uses *mix.sigm(...)* to evaluate the sigmoid function

CBT(*mix*, *zIdx*)

CBT() Critical Boiling Transition flag [-], wall dependent

Retrieves precomputed *CBT()* flags by *Solvers.Mixture.Mixture.CBT_CALC*, updated when *mix.H* is set.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to retrieve (optional)

CBTIDX(*mix*)

CBTIDX() Index of first CBT occurrence [-]

Compute the index of first CBT occurrence. If no occurrence is found, NaN is returned.

Inputs

- `mix` — `Solvers.Mixture.Mixture` object

CBTZ(*mix*, *zIdx*)

`CBTZ()` Length from upstream CBT [m]

Compute the distance from the upstream critical boiling transition. Support multiple transitions.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `zIdx` — Axial indices to evaluate (optional)

CHF(*mix*, *zIdx*)

`CHF()` Critical Heat Flux [W/m²], wall dependent

Retrieves precomputed `CHF()` values by `Solvers.Mixture.Mixture.CBT_CALC`, updated when `mix.H` is set.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `zIdx` — Axial indices to retrieve (optional)

DPACCS(*mix*, *MCTold*, *zIdx*)

`DPACCS()` Slip-induced temporal acceleration pressure drop [Pa]

Computes the pressure drop due to the slip-induced temporal acceleration between time steps.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `MCTold` — Previous slip-induced momentum correction [kg/s]
- `zIdx` — Axial indices to evaluate (optional)

DPACCT(*mix*, *UPold*, *zIdx*)

`DPACCT()` Temporal acceleration pressure drop [Pa]

Computes the pressure drop due to temporal acceleration between time steps.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `UPold` — Previous axial (advection) velocity [m/s]
- `zIdx` — Axial indices to evaluate (optional)

DPACCZ(*mix*, *zIdx*)

`DPACCZ()` Spatial acceleration pressure drop [Pa]

Computes the pressure drop due to spatial acceleration of the flow.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `zIdx` — Axial indices to evaluate (optional)

DPGRAV(*mix*, *zIdx*)

`DPGRAV()` Gravitational pressure loss [Pa]

Computes the pressure loss due to gravity along the axial direction.

Inputs:

- `mix` — `Solvers.Mixture.Mixture` object
- `zIdx` — Axial indices to evaluate (optional)

DPK(*mix*, *zIdx*)

DPK() Local pressure loss [Pa]

Computes the pressure loss due to localized effects (e.g., flow mixers).

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

DPPARTS(*mix*, *UPold*, *MCTold*, *zIdx*)

DPPARTS() All pressure loss components [Pa]

Returns a structure containing all individual pressure loss components.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *UPold* — Previous axial (advection) velocity [m/s]
- *zIdx* — Axial indices to evaluate (optional)

Returns:

- *dpparts* — Struct with fields:
 - GRAV : Gravitational loss [Pa]
 - WALL : Wall friction loss [Pa]
 - ACCZ : Spatial acceleration loss [Pa]
 - ACCT : Temporal acceleration loss [Pa]
 - ACCS : Slip-induced temporal acceleration loss [Pa]
 - K : Local loss [Pa]
 - TOT : Total pressure loss [Pa]

DPTOT(*mix*, *UPold*, *MCTold*, *zIdx*)

DPTOT() Total pressure loss [Pa]

Computes the total pressure loss by summing all contributing components.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *UPold* — Previous axial (advection) velocity [m/s]
- *MCTold* — Previous slip-induced momentum correction [kg/s]
- *zIdx* — Axial indices to evaluate (optional)

DPWALL(*mix*, *zIdx*)

DPWALL() Wall friction pressure drop [Pa]

Computes the pressure drop due to wall friction.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

ECT(*mix*, *zIdx*)

ECT() Slip-induced energy correction term [J/m]

Computes the slip-induced energy correction term per unit length.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

FOURIERCOND(*mix*, *zIdx*)

FOURIERCOND() Calculates the non-dimensional relaxation time for wall-dependent interfacial condensation.

Computes the non-dimensional relaxation time (akin to a Fourier number) associated with interfacial condensation at the specified axial indices, based on the thermal relaxation model defined in [Inputs.Model.THERMALRELAX](#).

Inputs:

- *mix* — [Solvers.Mixture.Mixture](#) object containing fluid and model data
- *zIdx* — Axial indices to evaluate (optional; defaults to full axial domain)

Supported Models:

- FOURIERX: Interpolate Fourier number from pre-defined quality-time pairs
- TIMEX: Interpolate relaxation time from pre-defined quality-time pairs
- VOID: Computes Fourier number based on void fraction and fluid properties
- HOMOGENEOUS: Computes Fourier number based on homogeneous approach
- FOURIER: Empirical Fourier number correlation

FOURIEREVAP(*mix*, *zIdx*)

[FOURIEREVAP\(\)](#) Calculates the non-dimensional relaxation time for wall-dependent interfacial evaporation.

Computes the non-dimensional relaxation time (akin to a Fourier number) associated with interfacial evaporation at the specified axial indices, based on the thermal relaxation model defined in [Inputs.Model.THERMALRELAX](#).

Inputs:

- *mix* — [Solvers.Mixture.Mixture](#) object containing fluid and model data
- *zIdx* — Axial indices to evaluate (optional; defaults to full axial domain)

Supported Models:

- FOURIERX: Interpolate Fourier number from pre-defined quality-time pairs
- TIMEX: Interpolate relaxation time from pre-defined quality-time pairs
- VOID: Computes relaxation time based on void fraction and fluid properties
- HOMOGENEOUS: Computes Fourier number based on homogeneous approach
- FOURIER: Empirical Fourier number correlation

FW(*mix*, *zIdx*)

[FW\(\)](#) Darcy wall friction factor [-]

Computes the Darcy wall friction factor based on [Inputs.Model.SPMTM](#) model.

Inputs:

- *mix* — [Solvers.Mixture.Mixture](#) object
- *zIdx* — Axial indices to evaluate (optional)

Supported models

- BLASIUS: Blasius model ($f = C(1)Re^{C(2)} + C(3)$) using user-defined [Inputs.Model.FRICTION](#) coefficients

FWL(*mix*, *zIdx*)

[FWL\(\)](#) Liquid-equivalent Darcy wall friction factor [-]

Computes the Darcy friction factor, using liquid-equivalent Reynolds number, based on [Inputs.Model.SPMTM](#) model.

Inputs:

- *mix* — [Solvers.Mixture.Mixture](#) object
- *zIdx* — Axial indices to evaluate (optional)

Supported models

- BLASIUS: Blasius model ($f = C(1)Re^{C(2)}$) using user-defined [Inputs.Model.FRICTION](#) coefficients

HINT(*mix*, *zIdx*)

HINT() Linear interfacial heat transfer rate [W/m]

Computes the interfacial heat transfer rates due to condensation and evaporation, based on interfacial mass transfer and enthalpy differences.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object with flow and enthalpy data
- *zIdx* — Axial indices to evaluate (optional; defaults to full axial range)

Notes:

- Uses *Solvers.Mixture.Mixture.MINT* for mass transfer and *Solvers.Mixture.Mixture.MRM.HV* for vapor enthalpy
- Behavior depends on *Inputs.Model.INTTRANSH*:
 - ‘*BULK*’: Consideration of bulk enthalpy
 - ‘*SATURATED*’: Consideration of saturated enthalpy

HINTCOND(*mix*, *zIdx*)

HINTCOND() Linear interfacial heat condensation rate [W/m]

Extracts the condensation component from *Solvers.Mixture.Mixture.HINT*.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

HINTEVAP(*mix*, *zIdx*)

HINTEVAP() Linear interfacial heat evaporation rate [W/m]

Extracts the evaporation component from *Solvers.Mixture.Mixture.HINT*.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

HM(*mix*, *zIdx*)

H Static enthalpy [J/kg]

Computes the mass-centered mean mixture enthalpy.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

HTOT(*mix*, *zIdx*)

HTOT() Total linear vapor heat rate [W/m]

Computes the total vapor heat transfer rate by summing interfacial and wall contributions.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

HVTOT(*mix*, *zIdx*)

HVTOT() Total linear vapor specific enthalpy transfer [J/kg/m]

Computes the specific enthalpy input to the vapor per unit mass flow. Uses a lumped wall approach for now.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Uses lumped wall approach: sums WV and Htot across walls
- Set to zero when WV is negligible

HWALEVAP(*mix*, *zIdx*)

HWALEVAP() Linear wall heat evaporation (boiling) rate [W/m]

Computes the heat transfer rate associated with wall boiling.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object with boiling and enthalpy data
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Behavior depends on *Inputs.Model.INTTRANSH*:
 - BULK: Consideration of bulk enthalpy
 - SATURATED: Consideration of saturated enthalpy

HWALHEAT(*mix*, *zIdx*)

HWALHEAT() Linear wall heat to vapor rate [W/m]

Returns the wall heat generation rate (LHGR) at boiling transition nodes.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Applies only at nodes flagged by *Solvers.Mixture.Mixture.CBT* or *Solvers.Mixture.Mixture.MFBT*.

HWALL(*mix*, *twall*, *zIdx*)

HWALL() Wall heat transfer coefficient [W/m²/K]

Computes the effective wall heat transfer coefficient by combining single-phase, boiling and post-CBT models. Supports multiple walls

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

HWALLBOIL(*mix*, *zIdx*)

HWALLBOIL() Boiling wall heat transfer coefficient [W/m²/K]

Computes the boiling wall heat transfer coefficient using selected *Inputs.Model.TPHTM* model.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

Supported models

- THOM: Thom's correlation

HWALLPOSTCBT(*mix*, *twall*, *zIdx*)

HWALLPOSTCBT() Post-CBT wall heat transfer coefficient [W/m²/K]

Computes the effective wall heat transfer coefficient under post-CBT conditions. based on *Inputs.Model.BTHTM* model.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *twall* — Wall temperature
- *zIdx* — Axial indices to evaluate (optional)

Supported wall heat transfer models

- VAPOR: Heat transfer to vapor phase
- DOUGALL: Dougall-Rohsenow model (Dougall and Rohsenow [1963])
- BISHOP: Bishop-Sandberg-Tong model (Bishop *et al.* [1965])
- MOECK: Groeneveld-Moeck model (Groeneveld and Moeck [1969])
- DELORME: Groeneveld-Delorme model (Groeneveld and Delorme [1976])
- CONDIEIV: Condie-Bengston IV model (Morris *et al.* [1982])

JG(*mix*, *zIdx*)

JG() Superficial vapor velocity [m/s]

Computes the superficial velocity of the vapor phase.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

JL(*mix*, *zIdx*)

JL() Superficial liquid velocity [m/s]

Computes the superficial velocity of the liquid phase.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

KDIST(*mix*, *zIdx*)

KDIST() Distance from upstream obstruction [m]

Computes the axial distance from the nearest upstream obstruction or inlet, based on *Inputs.Model.KLOC*.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object with axial grid and obstruction location
- *zIdx* — Axial indices to evaluate (optional)

KIDX(*mix*, *zIdx*)

KIDX() Obstruction index

Returns the axial node index of the obstruction closest to *Inputs.Model.KLOC*.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object with axial grid
- *zIdx* — Axial indices to evaluate (optional)

KLOCZ(*mix*, *zIdx*)

KLOCZ() Obstruction positions [m]

Returns the axial position of the obstruction closest to *Inputs.Model.KLOC*.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object with axial grid
- *zIdx* — Axial indices to evaluate (optional)

KLOSS(*mix*, *zIdx*)

KLOSS() Local pressure loss coefficient [-]

Returns the pressure loss coefficient at the elevation closest to *Inputs.Model.KLOC*.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object

- `zIdx` — Axial indices to evaluate (optional)

KTRELAX(*mix*, *zIdx*)

KTRELAX() Local relaxation time [s]

Returns the relaxation time at the elevation closest to *Inputs.Model.KLOC*. All other axial positions are set to NaN.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object with axial grid and model parameters
- *zIdx* — Axial indices to evaluate (optional; defaults to full axial range)

LHGR(*mix*, *zIdx*)

LHGR() Linear heat generation rate [W/m]

Computes the linear heat generation rate by multiplying wall perimeter with local heat flux.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object with geometry and heat flux data
- *zIdx* — Axial indices to evaluate (optional)

MCT(*mix*, *zIdx*)

MCT() Slip-induced momentum correction term [kg/s]

Computes the slip-induced momentum correction term per unit length.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

MFBT(*mix*, *zIdx*)

MFBT() Minimum Film Boiling Transition flag [-], wall dependent

Retrieves precomputed *MFBT()* flags by *Solvers.Mixture.Mixture.MFBT_CALC*, updated when *mix.H* is set.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to retrieve (optional)

MFLUX(*mix*, *zIdx*)

MFLUX() Mass flux [kg/m²/s]

Retrieves precomputed mass flux values by *Solvers.Mixture.Mixture.MFLUX_CALC*, updated when *mix.W* is set.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to retrieve (optional)

MINT(*mix*, *zIdx*)

MINT() Linear interfacial mass transfer rates [kg/s/m]

Computes condensation and evaporation mass transfer rates based on deviation from equilibrium vapor flow and time relaxation.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object with flow and relaxation data
- *zIdx* — Axial indices to evaluate (optional; defaults to full axial range)

Notes:

- Uses *mix.MRM.WV* for equilibrium deviation

- U_{Veq} is the approximated vapor velocity

MINTCOND(*mix*, *zIdx*)

MINTCOND() Linear interfacial condensation rate [kg/s/m]

Extracts the condensation component from *Solvers.Mixture.Mixture.MINT*.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

MINTEVAP(*mix*, *zIdx*)

MINTEVAP() Linear interfacial evaporation rate [kg/s/m]

Extracts the evaporation component from *Solvers.Mixture.Mixture.MINT*.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

MTOT(*mix*, *zIdx*)

MTOT() Total linear vapor mass transfer rate [kg/s/m]

Computes the total vapor mass transfer rate by summing interfacial condensation, evaporation, and wall boiling contributions.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

MTRANSV(*mix*, *zIdx*)

MTRANSV() Linear transversal mass exchange [kg/s/m]

Models transverse vapor mass exchange between wall regions using a time relaxation approach. Currently not used.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Uses a fixed relaxation time for now (RELAXT = 0.01 s)

MU(*mix*, *zIdx*)

MU() Dynamic viscosity [Pa·s]

Computes the mixture dynamic viscosity using vapor and liquid phase contributions weighted by vapor quality.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object with fluid properties
- *zIdx* — Axial indices to evaluate (optional)

MWALEVAP(*mix*, *zIdx*)

MWALEVAP() Linear wall mass evaporation rate [kg/s/m]

Computes the wall boiling mass transfer rate using latent heat and wall heat generation rate.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object with heat transfer and boiling data
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Latent heat is computed based on *Inputs.Model.INTTRANSH* setting

OAFIDX(*mix*)

OAFIDX() Onset of annular flow node.

Returns the axial node index corresponding to the onset of annular flow, determined by comparing the equilibrium quality to the onset threshold. The result is cached in *mix.oafidx_const* to avoid recomputation.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object containing flow quality and onset threshold

Notes:

- If no node satisfies the condition, the most upstream node (1) is returned

OAFWL(*mix*)

OAFWL() Liquid mass flow rate at onset of annular flow.

Returns the liquid mass flow rate at the axial index corresponding to the onset of annular flow, as defined by *Solvers.Mixture.Mixture.OAFIDX*.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object containing liquid flow data

OAFX(*mix*, *zIdx*)

OAFX() Onset of annular flow equilibrium quality.

Computes the equilibrium quality at the onset of annular flow using a model selected in *Inputs.Model.OAF*. The quality is evaluated over the specified axial indices.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object containing model, geometry, and fluid data
- *zIdx* — Axial indices to evaluate (optional; defaults to full axial range)

Supported Models:

- WALLIS: Full Wallis model
- WALLIS_SIMP: Simplified Wallis model
- LEVITAN: Levitan model

OAFZ(*mix*)

OAFZ() Onset of annular flow elevation.

Returns the axial elevation (in meters) corresponding to the onset of annular flow, as defined by *Solvers.Mixture.Mixture.OAFIDX*.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object containing axial grid data

PHI2F(*mix*, *zIdx*)

PHI2F() Two-phase wall friction multiplier [-]

Computes the two-phase wall friction multiplier using selected *Inputs.Model.TPFM* model.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

Supported models

- HOMOGENEOUS: Homogeneous model
- SLIP: Constant velocity slip ratio
- EPRI: EPRI multiplier

PHI2K(*mix*, *zIdx*)

PHI2K() Two-phase local pressure drop multiplier [-]

Computes the multiplier for local pressure drop in two-phase flow using selected *Inputs.Model.TPKM* model.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

Supported models

- HOMOGENEOUS: Homogeneous model
- SLIP: Constant velocity slip ratio
- ROMIE: Romie multiplier

RE(*mix*, *zIdx*)

RE() Reynolds number [-]

Computes the Reynolds number based on total mass flow rate and mixture viscosity.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

REL(*mix*, *zIdx*)

REL() Liquid-equivalent Reynolds number [-]

Computes the Reynolds number using liquid viscosity and total mass flow.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to evaluate (optional)

RELAXT(*mix*, *zIdx*)

RELAXT() Interfacial time relaxation [s]

Computes interfacial time relaxation based on sign of deviation from equilibrium vapor flow.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object with flow and relaxation data
- *zIdx* — Axial indices to evaluate (optional; defaults to full axial range)

Notes:

- Uses *mix.MRM.WV* for equilibrium deviation
- Vapor mass deviation from equilibrium, *Wint*, must be consistent with MINT method

RELAXTCOND(*mix*, *zIdx*)

RELAXTCOND() Time relaxation for interfacial condensation [s]

Retrieves precomputed time relaxation values for condensation by *Solvers.Mixture.Mixture.RELAXTCOND_CALC*, updated when *mix.H* is set.

Inputs:

- *mix* — *Solvers.Mixture.Mixture* object
- *zIdx* — Axial indices to retrieve (optional)

RELAXTEVAP(*mix*, *zIdx*)

RELAXTEVAP() Time relaxation for interfacial evaporation [s]

Retrieves precomputed time relaxation values for evaporation by *Solvers.Mixture.Mixture.RELAXTEVAP_CALC*, updated when *mix.H* is set.

Inputs:

- `mix` — [Solvers.Mixture.Mixture](#) object
- `zIdx` — Axial indices to retrieve (optional)

RHO(*mix*, *zIdx*)

[RHO\(\)](#) Density [kg/m³]

Retrieves precomputed mixture density values by `Solvers.Mixture.Mixture.RHO_CALC`, updated when `mix.H` is set.

Inputs:

- `mix` — [Solvers.Mixture.Mixture](#) object
- `zIdx` — Axial indices to retrieve (optional)

T(*mix*, *zIdx*)

[T\(\)](#) Temperature [K]

Returns the fluid temperature corresponding to the local enthalpy.

Inputs:

- `mix` — [Solvers.Mixture.Mixture](#) object with enthalpy data
- `zIdx` — Axial indices to evaluate (optional)

TAUW(*mix*, *zIdx*)

[TAUW\(\)](#) Wall shear stress [N/m²], wall dependent

Computes the wall shear stress for each wall segment using the friction factor, mass flux and two-phase multiplier. Supports multiple walls and adjusts for boiling transition conditions using [Inputs.Model.BTMTM](#) model.

Inputs:

- `mix` — [Solvers.Mixture.Mixture](#) object containing flow and geometry data
- `zIdx` — Axial indices to evaluate (optional)

Supported boiling transition models

- TPFM: Two-phase friction multiplier
- VAPOR: Shear stress to vapor phase

TWALL(*mix*, *zIdx*)

[TWALL\(\)](#) Wall temperature [K]

Computes the wall temperature based on heat flux and wall heat transfer coefficient. Uses liquid bulk temperature pre-CBT and vapor bulk temperature post-CBT.

Inputs:

- `mix` — [Solvers.Mixture.Mixture](#) object
- `zIdx` — Axial indices to evaluate (optional)

U(*mix*, *zIdx*)

[U\(\)](#) Static velocity [m/s]

Computes the mass-centered mean mixture velocity.

Inputs:

- `mix` — [Solvers.Mixture.Mixture](#) object
- `zIdx` — Axial indices to evaluate (optional)

UP(*mix*, *zIdx*)

[UP\(\)](#) Advection velocity [m/s]

Retrieves precomputed mixture (advection) velocity values by `Solvers.Mixture.Mixture.UP_CALC`, updated when `mix.H` is set.

Inputs:

- `mix` — *Solvers.Mixture.Mixture* object
- `zIdx` — Axial indices to retrieve (optional)

VF(*mix*, *zIdx*)

VF() Void fraction [-]

Retrieves precomputed void fraction values by `Solvers.Mixture.Mixture.VF_CALC`, updated when `mix.H` is set.

Inputs:

- `mix` — *Solvers.Mixture.Mixture* object
- `zIdx` — Axial indices to retrieve (optional)

WALEVAPRATIO(*mix*, *zIdx*)

WALEVAPRATIO() Wall mass evaporation ratio [-]

Computes the fraction of liquid mass undergoing wall boiling based on equilibrium quality and boiling onset thresholds *Inputs.Model.WBOILINGXSUB* and *Inputs.Model.WBOILINGXSAT*.

Inputs:

- `mix` — *Solvers.Mixture.Mixture* object with boiling model parameters
- `zIdx` — Axial indices to evaluate (optional)

Notes:

- Ratio is clamped between 0 and 1
- CBT and MFBT flags suppress boiling at transition nodes

WWALL(*mix*, *zIdx*)

WWALL() Mixture mass flow distribution per wall [kg/s]

Computes the wall-distributed mass flow rate by scaling the total flow with the wall perimeter ratio.

Inputs:

- `mix` — *Solvers.Mixture.Mixture* object with geometry and flow data
- `zIdx` — Axial indices to evaluate (optional; defaults to full axial range)

X(*mix*, *zIdx*)

X() Vapor quality [-]

Retrieves precomputed vapor quality values by `Solvers.Mixture.Mixture.X_CALC`, updated when `mix.H` is set.

Inputs:

- `mix` — *Solvers.Mixture.Mixture* object
- `zIdx` — Axial indices to retrieve (optional)

XEQ(*mix*, *zIdx*)

XEQ() Equilibrium quality [-]

Retrieves precomputed equilibrium quality values by `Solvers.Mixture.Mixture.XEQ_CALC`, updated when `mix.H` is set.

Inputs:

- `mix` — *Solvers.Mixture.Mixture* object
- `zIdx` — Axial indices to retrieve (optional)

class `Solvers.Mixture.Liquid`(*mix*)

Bases: *Solvers.AbstractPhase*

LIQUID Represents the liquid phase in a mixture solver simulation.

The `Liquid` class provides access to liquid-specific properties and calculations derived from the mixture solution. It encapsulates methods for computing flow, thermodynamic, and transport properties of the liquid phase at each axial node.

Responsibilities:

- Compute liquid mass fraction and volumetric fraction
- Calculate liquid velocity, enthalpy, and temperature
- Evaluate wall heat flux contributions to the liquid phase
- Support derived quantities such as Reynolds number and mass flux

Notes:

- All methods assume access to a valid `Solvers.Mixture.Mixture` object
- Axial indexing is optional; defaults to full axial domain
- Velocity and enthalpy calculations include fallback logic to handle single-phase vapor regions and numerical stability

Constructor Summary

Liquid(*mix*)

LIQUID Constructor

Initializes the `Liquid()` object from one or more instances of the `Solvers.Mixture.Mixture` class. Each `Liquid()` instance stores a reference to its corresponding Mixture object and the number of axial nodes (NZ).

Input:

- *mix* — Array of `Solvers.Mixture.Mixture` objects representing the simulation state

Notes:

- Supports vectorized initialization for transient simulations
- Assumes each `Solvers.Mixture.Mixture` object is fully initialized

Method Summary

C(*liquid*, *zIdx*)

`C()` Liquid mass fraction [-]

H(*liquid*, *zIdx*)

`H()` Liquid enthalpy [J/kg]

Calculated based on mixture and vapor enthalpies, and vapor quality.

HFLUX(*liquid*, *zIdx*)

`HFLUX()` Wall heat flux to liquid phase [W/m²]

HWALL(*liquid*, *twall*, *zIdx*)

`HWALL()` Single-phase liquid wall heat transfer coefficient [W/m²/K]

Computes the wall heat transfer coefficient using liquid thermal conductivity and Nusselt number.

MFLUX(*liquid*, *zIdx*)

`MFLUX()` Liquid mass flux [kg/m²/s]

NU(*liquid*, *twall*, *zIdx*)

`NU()` Liquid wall Nusselt number [-]

Computes the Nusselt number for wall heat transfer to liquid based on `Inputs.Model.SPHTM` model.

Q(*liquid*, *zIdx*)
Q() Liquid volumetric flow rate [m³/s]

RE(*liquid*, *zIdx*)
RE() Liquid Reynolds number [-]

T(*liquid*, *zIdx*)
T() Liquid temperature [K]

TIME(*liquid*)
TIME() Time series [s]

U(*liquid*, *zIdx*)
U() Liquid velocity [m/s]

VF(*liquid*, *zIdx*)
VF() Liquid volumetric fraction [-]

W(*liquid*, *zIdx*)
W() Liquid mass flow rate [kg/s]

X(*liquid*, *zIdx*)
X() Liquid mass flow fraction [-]

Z(*liquid*, *zIdx*)
Z() Axial node positions [m]

5.4.3 Three-field solver

The `Solvers.ThreeField` module in **OpenSTREAM** provides classes and tools for simulating annular two-phase flows using a three-field approach. In this method, the two liquid fields (film and droplets) are treated separately, each governed by its own set of conservation equations for mass, momentum, and energy. The vapor phase is solved using the `Solvers.Mixture` module. This framework enables explicit modeling of interactions between the film and droplet liquid fields, slip velocities, and hydrodynamic non-equilibrium effects, making it well-suited for applications involving annular boiling two-phase flow regimes where field separation and interfacial dynamics are significant.

This module includes:

- Solver class for three-field-based flow simulations: `Solvers.ThreeField.ThreeFieldSolver`
- Film and drop field classes: `Solvers.ThreeField.Film`, `Solvers.ThreeField.Drop`
- Three-field primary properties: `Solvers.ThreeField.Film.W`, `Solvers.ThreeField.Film.U`, `Solvers.ThreeField.Film.H`, `Solvers.ThreeField.Drop.W`, `Solvers.ThreeField.Drop.U`, `Solvers.ThreeField.Drop.H`
- Key solver class methods: `Solvers.ThreeField.ThreeFieldSolver.initializeSolver`, `Solvers.ThreeField.ThreeFieldSolver.solve()`, `Solvers.ThreeField.ThreeFieldSolver.plotz`
- Field class methods for calculating secondary three-field parameters.

class `Solvers.ThreeField.ThreeFieldSolver`(*inputSet*, *mixSolver*)

Bases: `Solvers.AbstractSolver`

THREEFIELDSOLVER Solver for initializing, solving, and visualizing three-field flow.

The `ThreeFieldSolver` class handles the setup, execution, and visualization of thermal-hydraulic simulations using a two-phase three-field approach.

Responsibilities:

- Initializes solver parameters from an *Inputs.InputSet* object
- Constructs *Solvers.ThreeField.Film* and *Solvers.ThreeField.Drop* objects for transient and steady-state analysis
- Provides plotting utilities for spatial and temporal distributions

Key Components:

- Liquid field construction (film/drop)
- Field mass/momentum/energy transport modeling
- Pressure drop gradient obtained from mixture solver
- Vapor solution obtained from mixture solver
- Visualization of results across time and axial domains

Constructor Summary

ThreeFieldSolver(*inputSet*, *mixSolver*)

THREEFIELDSOLVER Constructor for the ThreeFieldSolver() class.

Creates a new instance of the ThreeFieldSolver() class using the provided *Inputs.InputSet*. This constructor initializes the solver by calling the abstract *Solvers.AbstractSolver* superclass constructor and then sets up all necessary solver parameters and internal data structures via *Solvers.ThreeField.ThreeFieldSolver.initializeSolver*

Input:

- *inputSet* — An *Inputs.InputSet* object containing model configuration, geometry, boundary conditions, and solver options.
- *mixSolver* — An optional solved *Solvers.Mixture.MixtureSolver* object

Notes:

- This constructor assumes *Inputs.InputSet* is fully validated.
- A solved mixture-model solution is required for initialization. If a solved mixture solver is supplied, its solution is reused. If unsolved or if no mixture solver is supplied, one is created and solved automatically.
- Solver initialization includes liquid and vapor objects setup.
- Time/space discretization and boundary condition interpolation initialized by *Solvers.Mixture.MixtureSolver* are used

Property Summary

DT = 0

Time step size [s] from *Inputs.Options.TSTEP*

DZ = 0

Axial step size [m]

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from *Inputs.Model.NNODES*

TIME = 0

Time series [s]

Z = 1.

Elevation [m]

boundaryConditions

Boundary conditions object [Inputs.BoundaryConditions](#)

drop

Drop object

dropInit

Null transient drop object

film

Film object

filmInit

Null transient film object

fluid

Fluid object [Inputs.FluidProperties](#)

inputSet

Input set object [Inputs.InputSet](#)

mixSolver

Mixture object [Solvers.Mixture.Mixture](#)

Method Summary**EQUIL**(*tfSolver, flm, drp, mix, zIdx*)

EQUIL() Find entrained ratio at film/drop equilibrium state [-]

Iteratively computes the entrained liquid ratio at the onset of annular flow by balancing droplet deposition and film entrainment rates.

Inputs:

- *tfSolver* — [Solvers.ThreeField.ThreeFieldSolver](#) object
- *flm* — Film field object ([Solvers.ThreeField.Film](#))
- *drp* — Droplet field object ([Solvers.ThreeField.Drop](#))
- *mix* — Mixture object ([Solvers.Mixture.Mixture](#))
- *zIdx* — Axial index for evaluation

Algorithm:

- Starts with an initial guess for droplet mass flow (50% of liquid mass)
- Updates droplet and film flow rates iteratively
- Computes difference between deposition and entrainment rates
- Stops when error < 1e-4 kg/s/m or after 100 iterations

Notes:

- Enhanced drop deposition is disabled
- Uses interpolation for improved convergence
- Falls back to ad-hoc update if interpolation fails
- Method update film and drop mass flow rates. Use *flm.copy()* and *drp.copy()* as input arguments if used in post-process

initializeSolver(*tfSolver*)

INITIALIZESOLVER Initializes solver parameters and constructs film/drop objects.

Sets up the solver's internal state using the provided [Inputs.InputSet](#) and [Solvers.ThreeField.ThreeFieldSolver](#). This includes initialization of data structures for transient and steady-state simulations based on the mixture solver solutions.

Operations performed:

- Initializes film and drop arrays for each time step

- Initializes fluid property objects
- Sets up mixture object
- Assigns initial conditions for phase mass flow rates, velocities, enthalpies

Notes:

- The function assumes uniform axial discretization.
- Wall heat flux is computed from interpolated boundary conditions.

plott(*tfSolver, zIdx, opt*)

PLOTT Plots temporal distributions of three-field parameters at a given axial location.

Generates time-series plots of selected three-field parameters at a specified axial index. The method supports multiple display modes, wall selections, and plot arrangements.

Inputs:

- *tfSolver* — [Solvers.ThreeField.ThreeFieldSolver](#) object containing simulation data
- *zIdx* — Axial index (scalar, positive integer)
- *opt.display* — Parameters to display (e.g., 'HFLUX', 'W', 'U', etc.)
- *opt.solveMode* — Solve mode: 'REAL' or 'NULL'
- *opt.wall* — Wall index(es) to plot
- *opt.tIdx* — Time indices to include in the plot
- *opt.reverseTime* — Logical flag to reverse time axis
- *opt.unitTemp* — Temperature unit: 'K' or 'C'
- *opt.arrangement* — Plot arrangement: 'flow', 'vertical', or 'horizontal'
- *opt.resize* — Resize factor for plot scaling

Notes:

- If *opt.display* is set to 'ALL', all supported parameters are plotted.
- The function validates that at least two time indices are provided.
- Temperature unit conversion is applied if 'C' is selected.
- Each wall is plotted in a separate tile with appropriate legends and axis scaling.

plotz(*tfSolver, tIdx, opts*)

PLOTZ Plots spatial (axial) distributions of three-field parameters.

Generates axial plots of selected three-field parameters at specified time indices. The function supports multiple display modes, wall selections, and optional animation over time.

Inputs:

- *tfSolver* — [Solvers.ThreeField.ThreeFieldSolver](#) object containing simulation data
- *tIdx* — Time index or indices (vector of positive integers)
- *opts.display* — Parameters to display (e.g., 'HFLUX', 'W', 'U', etc.)
- *opts.solveMode* — Solve mode: 'REAL' or 'NULL'
- *opts.wall* — Wall index(es) to plot
- *opts.zIdx* — Axial indices to include in the plot
- *opts.obstructions* — Logical flag to display obstruction locations
- *opts.annular* — Logical flag to plot annular two-phase flow parameters only from the onset of annular flow
- *opts.unitTemp* — Temperature unit: 'K' or 'C'
- *opts.arrangement* — Plot arrangement: 'flow', 'vertical', or 'horizontal'
- *opts.resize* — Resize factor for plot scaling

Notes:

- If multiple time indices are provided, the function creates an animated plot.
- At least two axial indices must be specified to enable spatial plotting.
- Temperature unit conversion is applied if 'C' is selected.
- Obstruction locations are plotted if *opts.obstructions* is true.
- Each wall is plotted in a separate tile with appropriate legends and axis scaling.

plotzt(*tfSolver, opt*)

PLOTZT Plots 2D time/elevation distributions of three-field parameters.

Generates surface plots of selected three-field related parameters over time and axial (elevation) positions. It supports plotting for film and drop fields, and can handle both real and null (initial) transient data.

Inputs:

- *tfSolver* — [Solvers.ThreeField.ThreeFieldSolver](#) object containing simulation data
- *opt.display* — Parameter(s) to display (e.g., 'HFLUX', 'U', etc.)
- *opt.label* — Corresponding labels for display parameters
- *opt.unit* — Units for each parameter
- *opt.field* — Field to plot: 'film' or 'drop'
- *opt.solveMode* — Solve mode: 'REAL' or 'NULL'
- *opt.wall* — Wall index(es) to plot
- *opt.zIdx* — Axial indices (must include at least 2)
- *opt.tIdx* — Time indices (must include at least 2)
- *opt.reverseTime* — Logical flag to reverse time axis
- *opt.shading* — Surface shading style: 'faceted', 'flat', or 'interp'
- *opt.view* — View angle for 3D plot [azimuth elevation]

Notes:

- If 'ALL' is passed to *opt.display*, all supported parameters are plotted.
- The function validates that at least two axial and time indices are provided to enable meaningful 2D plotting.
- Each wall is plotted in a separate figure window with appropriate titles and subplot grouping.

solve(*tfSolver*)

SOLVE Executes the three-field solver for steady-state and transient simulations.

Runs the full solution process for the [Solvers.ThreeField.ThreeFieldSolver](#) object. It manages initialization, time stepping, axial sweeps, inner iterations, convergence checks, and logging.

Workflow:

- Enables logging and opens persistent log file
- Validates solver state before execution
- Runs steady-state initialization (*solveINIT* = true)
- If initial step converges, proceeds with transient simulation
- Handles exceptions and ensures proper log closure

Notes:

- Uses internal *solver* function to handle both steady-state and transient modes
- Applies relaxation factors for phase flows, velocities, and enthalpies
- Supports thermal non-equilibrium modeling
- Logs progress and outputs to session directory

class [Solvers.ThreeField.Drop](#)(*inputSet, fluid*)

Bases: [Solvers.AbstractField](#)

DROP Class for modeling drop field in three-field solver

This class encapsulates the physical and numerical properties of the drop field, including flow variables and phase interactions. It supports multiple solver models and provides methods for computing derived quantities.

Constructor Summary

Drop(*inputSet*, *fluid*)

DROP Constructor for *Drop()* class

Initializes the drop field object with input configuration and fluid properties.

Inputs:

- *inputSet* — *Inputs.InputSet* object containing model, geometry, and boundary conditions
- *fluid* — *Inputs.FluidProperties* object containing thermophysical fluid data

Property Summary

DT = 0

Time step size [s] from *Inputs.Options.TSTEP*

H = 1E6

Enthalpy [J/kg]

ITR

Iteration tracking

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from *Inputs.Model.NNODES*

TIDX = 1

Time step index [-]

TIME = 0

Time series [s]

U = 1.

Velocity [m/s]

W = 1.

Mass flow rate [kg/s]

Z = 1.

Elevation [m]

mix = NaN

Solvers.Mixture.Mixture object

Method Summary

AI(*drop*, *zIdx*)

AI() Drop volumetric interfacial area [m⁻¹]

Calculates the interfacial area per unit volume for drops.

Inputs:

- *drop* — *Solvers.ThreeField.Drop* object
- *zIdx* — Axial indices to evaluate (optional)

AREA(*drop*, *zIdx*)

AREA() Drop interfacial area [m²]

Calculates the surface area of a spherical drop.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- zIdx — Axial indices to evaluate (optional)

CONC(drop, zIdx)

CONC() Drop concentration [kg/m³]

Calculates the local drop-field concentration based on mass flow and vapor-phase properties.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- zIdx — Axial indices to evaluate (optional)

DENSITY(drop, zIdx)

DENSITY() Drop number density [m⁻³]

Calculates the number of drops per unit volume based on flow properties and geometry.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- zIdx — Axial indices to evaluate (optional)

DIAM(drop, zIdx)

DIAM() Drop diameter [m]

Returns the drop diameter based on *Inputs.Model.DROPDIAM*.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- zIdx — Axial indices to evaluate (optional)

DRAG(drop, zIdx)

DRAG() Drop drag coefficient [-]

Calculates drag coefficient using selected correlation based on *Inputs.Model.DROPDRAG*.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- zIdx — Axial indices to evaluate (optional)

FBUOY(drop, zIdx)

FBUOY() Drop buoyancy force [N/m³]

Calculates buoyancy force based on pressure gradient.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- zIdx — Axial indices to evaluate (optional)

FDRAG(drop, zIdx)

FDRAG() Vapor drag force on drops [N/m³]

Calculates shear force exerted by vapor on drops.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- zIdx — Axial indices to evaluate (optional)

FENT(drop, film, zIdx)

FENT() Film entrainment shear force [N/m³]

Calculates shear force due to film entrainment acting on drops.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- film — *Solvers.ThreeField.Film* object
- zIdx — Axial indices to evaluate (optional)

FGRV(drop, zIdx)

FGRV() Gravitational force on drops [N/m³]

Calculates gravity force acting on drops based on channel inclination angle.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- zIdx — Axial indices to evaluate (optional)

FTOT(drop, film, zIdx, opt)

FTOT() Total force acting on drops [N/m³]

Calculates combined forces (drag, buoyancy, gravity, entrainment) based on selected option.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- film — *Solvers.ThreeField.Film* object
- zIdx — Axial indices to evaluate (optional)
- opt — Option flag (0: full balance, 1: simplified)

KENH(drop, zIdx)

KENH() Drop deposition enhancement factor [-]

Calculates axial variation of deposition enhancement using Windecker correlation or other models based on *Inputs.Model.DEPENHANCEMENT*.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- zIdx — Axial indices to evaluate (optional)

MDEP(drop, zIdx, enhanced)

MDEP() Drop deposition mass flux [kg/m²/s]

Implements various deposition models selected based on *Inputs.Model.DEPOSITION*. Applies enhancement factors, and restricts deposition to annular flow region. The enhancement flag allow to enable/disable deposition enhancement

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- zIdx — Axial indices to evaluate (optional)
- enhanced — Flag indicating enhanced deposition downstream obstructions (optional, default = true)

RE(drop, zIdx)

RE() Drop Reynolds number [-]

Calculates the Reynolds number for the drop field based on mass flow rate, viscosity, and channel perimeter.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- zIdx — Axial indices to evaluate (optional)

REV(drop, zIdx)

REV() Drop Reynolds number with respect to vapor properties [-]

Calculates Reynolds number using vapor density, viscosity, and relative phase velocity.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- zIdx — Axial indices to evaluate (optional)

UALGEBR(*drop, film, zIdx*)

UALGEBR() Algebraic drop velocity [m/s]

Calculates drop velocity consistent with mixture model based on void fraction and geometry.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- film — *Solvers.ThreeField.Film* object
- zIdx — Axial indices to evaluate (optional)

UEQUIL(*drop, film, zIdx, opt*)

UEQUIL() Drop velocity based on equilibrium model ($F_{tot} = 0$) [m/s]

Iteratively solves for drop velocity such that the total force acting on the drop phase is zero (force balance). Uses secant method with relaxation for convergence.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- film — *Solvers.ThreeField.Film* object
- zIdx — Axial indices to evaluate (optional)
- **opt** — **Option flag for force balance:**
0 = full force balance (drag, buoyancy, gravity, entrainment) 1 = simplified (drag + buoyancy only)

USLIP(*drop, zIdx*)

USLIP() Slip velocity model for drops [m/s]

Calculates drop velocity using slip ratio model based on *Inputs.Model.DROPSLIP*.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- zIdx — Axial indices to evaluate (optional)

VOLUME(*drop, zIdx*)

VOLUME() Drop volume [m³]

Calculates the volume of a spherical drop.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- zIdx — Axial indices to evaluate (optional)

VR(*drop, zIdx*)

VR() Drop local relative velocity [m/s]

Calculates the velocity difference between vapor and drop fields.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- Only AREAMEAN option is implemented.

X(*drop, zIdx*)

X() Vapor quality in the bulk (drop) region [-]

Vapor quality in the bulk region, considering only the droplet for the liquid phase. The bulk region is complementary of the near-wall region, defined in the mixture solver.

Inputs:

- drop — *Solvers.ThreeField.Drop* object
- zIdx — Axial indices to evaluate (optional)

class *Solvers.ThreeField.Film*

Bases: *Solvers.AbstractFilm*

FILM Class for modeling liquid film in three-field solver

This class encapsulates the physical and numerical properties of the film field, including flow variables and phase interactions.

Property Summary

DT = 0

Time step size [s] from *Inputs.Options.TSTEP*

H = 1E6

Enthalpy [J/kg]

HFLUX = 1.

Film heat flux [W/m²]

ITR

Iteration tracking

MEVAP = -1.

Evaporation mass flux [kg/s/m²]

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from *Inputs.Model.NNODES*

TIDX = 1

Time step index [-]

TIME = 0

Time series [s]

U = 1.

Velocity [m/s]

W = 1.

Mass flow rate [kg/s]

Z = 1.

Elevation [m]

5.4.4 Two-fluid solver

The *Solvers.TwoFluid* module in **OpenSTREAM** provides classes and tools for simulating two-phase flows using a two-fluid approach. In this method, each phase (liquid and vapor) is modeled as an interpenetrating continuum with its own set of conservation equations for mass, momentum, and energy. This framework enables explicit representation

of phase interactions, slip velocities, and non-equilibrium effects, making it well-suited for applications where phase separation, interfacial dynamics, and transient boiling phenomena are significant.

This module includes:

- Solver class for two-fluid-based flow simulations: `Solvers.TwoFluid.TwoFluidSolver`
 - Liquid and vapor field classes: `Solvers.TwoFluid.Liquid`, `Solvers.TwoFluid.Vapor`
 - Mixture class: `Solvers.TwoFluid.Mixture`
 - Two-fluid primary properties: `Solvers.TwoFluid.Liquid.W`, `Solvers.TwoFluid.Liquid.U`, `Solvers.TwoFluid.Liquid.H`, `Solvers.TwoFluid.Vapor.W`, `Solvers.TwoFluid.Vapor.U`, `Solvers.TwoFluid.Vapor.H`
 - Key solver class methods: `Solvers.TwoFluid.TwoFluidSolver.initializeSolver`, `Solvers.TwoFluid.TwoFluidSolver.solve()`, `Solvers.TwoFluid.TwoFluidSolver.plotz`
 - Field class methods for calculating secondary two-fluid parameters.
-

class `Solvers.TwoFluid.TwoFluidSolver`(*inputSet*, *mixSolver*)

Bases: `Solvers.AbstractSolver`

TWOFLUIDSOLVER Solver for initializing, solving, and visualizing two-fluid flow.

The `TwoFluidSolver` class handles the setup, execution, and visualization of thermal-hydraulic simulations using a two-phase two-fluid approach.

Responsibilities:

- Initializes solver parameters from an `Inputs.InputSet` object
- Constructs `Solvers.TwoFluid.Liquid` and `Solvers.TwoFluid.Vapor` objects for transient and steady-state analysis
- Provides plotting utilities for spatial and temporal distributions

Key Components:

- Phase construction (liquid/vapor)
- Phase mass/momentum/energy transport modeling
- Pressure drop gradient obtained from mixture solver
- Support for relaxation models
- Visualization of results across time and axial domains

Constructor Summary

TwoFluidSolver(*inputSet*, *mixSolver*)

TWOFLUIDSOLVER Constructor for the `TwoFluidSolver()` class.

Creates a new instance of the `TwoFluidSolver()` class using the provided `Inputs.InputSet`. This constructor initializes the solver by calling the abstract `Solvers.AbstractSolver` superclass constructor and then sets up all necessary solver parameters and internal data structures via `Solvers.TwoFluid.TwoFluidSolver.initializeSolver`

Input:

- *inputSet* — An `Inputs.InputSet` object containing model configuration, geometry, boundary conditions, and solver options.
- *mixSolver* — An optional solved `Solvers.Mixture.MixtureSolver` object

Notes:

- This constructor assumes *Inputs.InputSet* is fully validated.
- A solved mixture-model solution is required for initialization. If a solved mixture solver is supplied, its solution is reused. If unsolved or if no mixture solver is supplied, one is created and solved automatically.
- Solver initialization includes liquid and vapor objects setup.
- Time/space discretization and boundary condition interpolation initialized by *Solvers.Mixture.MixtureSolver* are used

Property Summary

DT = 0

Time step size [s] from *Inputs.Options.TSTEP*

DZ = 0

Axial step size [m]

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from *Inputs.Model.NNODES*

TIME = 0

Time series [s]

Z = 1.

Elevation [m]

boundaryConditions

Boundary conditions object *Inputs.BoundaryConditions*

fluid

Fluid object *Inputs.FluidProperties*

inputSet

Input set object *Inputs.InputSet*

liquid

Liquid object

liquidInit

Null transient liquid object

mixSolver

Mixture object *Solvers.Mixture.Mixture*

vapor

Vapor object

vaporInit

Null transient vapor object

Method Summary

initializeSolver(*twfSolver*)

INITIALIZESOLVER Initializes solver parameters and constructs liquid/vapor objects.

Sets up the solver's internal state using the provided [Inputs.InputSet](#) and [Solvers.TwoFluid.TwoFluidSolver](#). This includes initialization of data structures for transient and steady-state simulations based on the mixture solver solutions.

Operations performed:

- Initializes liquid and vapor arrays for each time step
- Initializes fluid property objects
- Sets up mixture object
- Assigns initial conditions for phase mass flow rates, velocities, enthalpies

Notes:

- The function assumes uniform axial discretization.
- Wall heat flux is computed from interpolated boundary conditions.

plott(*twfSolver*, *zIdx*, *opt*)

PLOTT Plots temporal distributions of two-fluid parameters at a given axial location.

Generates time-series plots of selected two-fluid parameters at a specified axial index. The method supports multiple display modes, wall selections, and plot arrangements.

Inputs:

- twfSolver — [Solvers.TwoFluid.TwoFluidSolver](#) object containing simulation data
- zIdx — Axial index (scalar, positive integer)
- opt.display — Parameters to display (e.g., 'HFLUX', 'W', 'U', etc.)
- opt.solveMode — Solve mode: 'REAL' or 'NULL'
- opt.wall — Wall index(es) to plot
- opt.tIdx — Time indices to include in the plot
- opt.reverseTime — Logical flag to reverse time axis
- opt.unitTemp — Temperature unit: 'K' or 'C'
- opt.arrangement — Plot arrangement: 'flow', 'vertical', or 'horizontal'
- opt.resize — Resize factor for plot scaling

Notes:

- If opt.display is set to 'ALL', all supported parameters are plotted.
- The function validates that at least two time indices are provided.
- Temperature unit conversion is applied if 'C' is selected.
- Each wall is plotted in a separate tile with appropriate legends and axis scaling.

plotz(*twfSolver*, *tIdx*, *opts*)

PLOTZ Plots spatial (axial) distributions of two-fluid parameters.

Generates axial plots of selected two-fluid parameters at specified time indices. The function supports multiple display modes, wall selections, and optional animation over time.

Inputs:

- twfSolver — [Solvers.TwoFluid.TwoFluidSolver](#) object containing simulation data
- tIdx — Time index or indices (vector of positive integers)
- opts.display — Parameters to display (e.g., 'HFLUX', 'W', 'U', etc.)
- opts.solveMode — Solve mode: 'REAL' or 'NULL'
- opts.wall — Wall index(es) to plot
- opts.zIdx — Axial indices to include in the plot
- opts.obstructions — Logical flag to display obstruction locations
- opts.unitTemp — Temperature unit: 'K' or 'C'
- opts.arrangement — Plot arrangement: 'flow', 'vertical', or 'horizontal'
- opts.resize — Resize factor for plot scaling

Notes:

- If multiple time indices are provided, the function creates an animated plot.
- At least two axial indices must be specified to enable spatial plotting.
- Temperature unit conversion is applied if 'C' is selected.
- Obstruction locations are plotted if `opts.obstructions` is true.
- Each wall is plotted in a separate tile with appropriate legends and axis scaling.

plotzt(*twfSolver*, *opt*)

PLOTZT Plots 2D time/elevation distributions of two-fluid parameters.

Generates surface plots of selected two-fluid related parameters over time and axial (elevation) positions. It supports plotting for liquid and vapor fields, and can handle both real and null (initial) transient data.

Inputs:

- `twfSolver` — [Solvers.TwoFluid.TwoFluidSolver](#) object containing simulation data
- `opt.display` — Parameter(s) to display (e.g., 'HFLUX', 'U', etc.)
- `opt.label` — Corresponding labels for display parameters
- `opt.unit` — Units for each parameter
- `opt.field` — Field to plot: 'liquid' or 'vapor'
- `opt.solveMode` — Solve mode: 'REAL' or 'NULL'
- `opt.wall` — Wall index(es) to plot
- `opt.zIdx` — Axial indices (must include at least 2)
- `opt.tIdx` — Time indices (must include at least 2)
- `opt.reverseTime` — Logical flag to reverse time axis
- `opt.shading` — Surface shading style: 'faceted', 'flat', or 'interp'
- `opt.view` — View angle for 3D plot [azimuth elevation]

Notes:

- If 'ALL' is passed to `opt.display`, all supported parameters are plotted.
- The function validates that at least two axial and time indices are provided to enable meaningful 2D plotting.
- Each wall is plotted in a separate figure window with appropriate titles and subplot grouping.

solve(*twfSolver*)

SOLVE Executes the two-fluid solver for steady-state and transient simulations.

Runs the full solution process for the [Solvers.TwoFluid.TwoFluidSolver](#) object. It manages initialization, time stepping, axial sweeps, inner iterations, convergence checks, and logging.

Workflow:

- Enables logging and opens persistent log file
- Validates solver state before execution
- Runs steady-state initialization (`solveINIT = true`)
- If initial step converges, proceeds with transient simulation
- Handles exceptions and ensures proper log closure

Notes:

- Uses internal *solver* function to handle both steady-state and transient modes
- Applies relaxation factors for phase flows, velocities, and enthalpies
- Supports thermal non-equilibrium modeling
- Logs progress and outputs to session directory

class `Solvers.TwoFluid.REGIMES`

Bases: `double`

REGIMES Enumeration of two-phase flow regimes

This class defines the available two-phase flow regimes for constitutive model selection, used by the two-fluid solver.

The flow regimes are defined as an enum class to increase computational efficiency while increasing readability of code, particularly when different calculations are carried out for different flow regimes.

Flow regimes:

- LIQUID — Single-phase liquid
- BUBBLY_SUBCOOLED — Bubbly flow in subcooled region
- BUBBLY_SATURATED — Bubbly flow in saturated region
- INTERMEDIATE — Intermediate flow region
- ANNULAR — Annular flow
- DFFB — Dispersed Flow Film Boiling

class `Solvers.TwoFluid.Vapor`(*inputSet*, *fluid*)

Bases: [Solvers.AbstractField](#)

VAPOR Class for modeling vapor field in two-fluid solver

This class encapsulates the physical and numerical properties of the vapor phase, including flow variables, phase interactions, heat transfer, and relaxation models. It supports multiple solver models and provides methods for computing derived quantities and handling flow regime transitions.

Constructor Summary

Vapor(*inputSet*, *fluid*)

VAPOR Constructor for [Vapor\(\)](#) class

Initializes the vapor field object with input configuration and fluid properties.

Inputs:

- *inputSet* — [Inputs.InputSet](#) object containing model, geometry, and boundary conditions
- *fluid* — [Inputs.FluidProperties](#) object containing thermophysical fluid data

Property Summary

DT = 0

Time step size [s] from [Inputs.Options.TSTEP](#)

H = 1E6

Enthalpy [J/kg]

ITR

Iteration properties

NTIME = 0

Number of time steps [-]

NZ = 0

Number of axial steps [-] from [Inputs.Model.NNODES](#)

TIDX = 1

Time step index [-]

TIME = 0

Time series [s]

U = 1.

Velocity [m/s]

W = 1.

Mass flow rate [kg/s]

Z = 1.

Elevation [m]

mix = NaN

Solvers.TwoFluid.Mixture object

Method Summary

AREA(*vapor, liquid, zIdx*)

AREA() Vapor cross-section area [m²]

Computes the effective cross-sectional area occupied by the vapor phase.

Inputs:

- *vapor* — *Solvers.TwoFluid.Vapor* object
- *liquid* — *Solvers.TwoFluid.Liquid* object
- *zIdx* — Axial indices to evaluate (optional)

CD(*vapor, liquid, zIdx*)

CD() Drag coefficient for dispersed gas [-]

Computes the drag coefficient for dispersed vapor based on *Inputs.Model.DROPDRAG* model.

Inputs:

- *vapor* — *Solvers.TwoFluid.Vapor* object
- *liquid* — *Solvers.TwoFluid.Liquid* object
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Values are clamped to a maximum of 1

FBUOY(*vapor, liquid, zIdx*)

FBUOY() Vapor buoyancy force [N/m]

Computes the buoyancy force acting on the vapor phase due to pressure gradient.

Inputs:

- *vapor* — *Solvers.TwoFluid.Vapor* object
- *liquid* — *Solvers.TwoFluid.Liquid* object
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- The pressure gradient from the mixture solver is used

FDRAG(*vapor, liquid, zIdx*)

FDRAG() Interfacial drag force [N/m]

Computes the interfacial drag force acting on the vapor phase by reversing the sign of the liquid-phase drag (*Solvers.TwoFluid.Liquid.FDRAG()*).

Inputs:

- *vapor* — *Solvers.TwoFluid.Vapor* object
- *liquid* — *Solvers.TwoFluid.Liquid* object
- *zIdx* — Axial indices to evaluate (optional)

FGRAV(*vapor, liquid, zIdx*)*FGRAV()* Vapor gravity force [N/m]

Computes the gravitational force acting on the vapor phase.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

FINTEVAP(*vapor, liquid, zIdx*)*FINTEVAP()* Momentum exchange through interfacial evaporation [N/m]

Computes the momentum exchange due to interfacial evaporation.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

FLOWREGIME(*vapor, liquid, zIdx*)*FLOWREGIME()* Categorical two-phase flow regimes

Returns the flow regime classification based on liquid-phase criteria (*Solvers.TwoFluid.Liquid.FLOWREGIME()*).

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

Returns:

- flowregime — Flow regime category (from *Solvers.TwoFluid.REGIMES*)

FTOT(*vapor, liquid, zIdx*)*FTOT()* Total force [N/m]

Computes the total force acting on the vapor phase, including gravity, buoyancy, wall shear, drag, and interfacial momentum exchanges.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

FWALEVAP(*vapor, liquid, zIdx*)*FWALEVAP()* Momentum exchange through wall evaporation [N/m]

Computes the momentum exchange due to wall boiling evaporation.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

FWALL(*vapor, liquid, zIdx*)*FWALL()* Wall shear force [N/m]

Computes the wall shear force acting on the vapor phase, weighted by void fraction.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

HFLUX(vapor, zIdx)

HFLUX() Wall heat flux to vapor phase [W/m²]

Computes the net wall heat flux transferred to the vapor phase.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- Uses *Solvers.TwoFluid.Vapor.HWALHEAT()* divided by wall perimeter

HFLUXWALEVAP(vapor, liquid, zIdx)

HFLUXWALEVAP() Wall boiling heat flux [W/m²]

Computes the portion of wall heat flux attributed to boiling (evaporation) for the vapor phase, based on liquid-phase calculations (*Solvers.TwoFluid.Liquid.HFLUXWALEVAP()*).

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

HINT(vapor, liquid, zIdx)

HINT() Linear interfacial heat transfer rate [W/m]

Computes the interfacial heat transfer rate for the vapor phase based on mass transfer and enthalpy difference (based on *Inputs.Model.INTTRANS* model).

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

Returns:

- Hcond — Condensation heat transfer rate [W/m]
- Hevap — Evaporation heat transfer rate [W/m]

HINTCOND(vapor, liquid, zIdx)

HINTCOND() Linear interfacial heat condensation rate [W/m]

Extracts the condensation component from *Solvers.TwoFluid.Vapor.HINT()*.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

HINTEVAP(vapor, liquid, zIdx)

HINTEVAP() Linear interfacial heat evaporation rate [W/m]

Extracts the evaporation component from *Solvers.TwoFluid.Vapor.HINT()*.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

HTOT(*vapor, liquid, zIdx*)

HTOT() Total linear vapor heat rate [W/m]

Computes the total heat transfer to the vapor phase, including interfacial and wall contributions.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

HWALEVAP(*vapor, liquid, zIdx*)

HWALEVAP() Linear wall heat evaporation (boiling) rate [W/m]

Computes the wall boiling heat transfer rate using latent heat (based on *Inputs.Model.INTTRANSH* model) and wall mass evaporation.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

HWALHEAT(*vapor, zIdx*)

HWALHEAT() Linear wall heat to vapor rate [W/m]

Returns the wall heat generation rate transferred to the vapor phase.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- Set to zero before CBT

INTAREA(*vapor, liquid, zIdx*)

INTAREA() Volumetric interfacial area [m⁻¹]

Computes the interfacial area concentration using liquid-phase logic (from *Solvers.TwoFluid.Liquid.INTAREA()*).

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

INTHFLUX(*vapor, liquid, zIdx*)

INTHFLUX() Interfacial heat flux [W/m²]

Computes the interfacial heat flux due to evaporation and condensation (delegated to liquid-phase calculation in *Solvers.TwoFluid.Liquid.INTHFLUX()*).

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

Returns:

- inthflux_evap — Evaporation heat flux [W/m²]
- inthflux_cond — Condensation heat flux [W/m²]

Notes:

- Positive toward liquid phase

INTNU(*vapor, liquid, zIdx*)*INTNU()* Interfacial Nusselt number for dispersed vapor [-]Computes the interfacial Nusselt number based on *Inputs.Model.INTNU* model.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

J(*vapor, zIdx*)*J()* Vapor superficial velocity [m/s]

Computes the superficial velocity of the vapor phase.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

L(*vapor, zIdx*)*L()* Dispersed vapor interfacial length scale [m]

Returns the characteristic length scale for dispersed vapor (e.g., bubble diameter).

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- Currently uses a constant model

MINT(*vapor, liquid, zIdx*)*MINT()* Linear interfacial mass transfer rates [kg/s/m]Computes condensation and evaporation mass transfer rates for the vapor phase by reversing the sign of liquid-phase values (*Solvers.TwoFluid.Liquid.MINT()*).

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

Returns:

- Mcond — Condensation mass transfer rate [kg/s/m]
- Mevap — Evaporation mass transfer rate [kg/s/m]

MINTCOND(*vapor, liquid, zIdx*)*MINTCOND()* Linear interfacial condensation mass transfer [kg/s/m]Extracts the condensation component from *Solvers.TwoFluid.Vapor.MINT()*.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

MINTEVAP(*vapor, liquid, zIdx*)*MINTEVAP()* Linear interfacial evaporation mass transfer [kg/s/m]Extracts the evaporation component from *Solvers.TwoFluid.Vapor.MINT()*.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

MTOT(*vapor, liquid, zIdx*)

MTOT() Total linear mass transfer [kg/s/m]

Computes the total vapor mass transfer rate by reversing the sign of the liquid-phase total (*Solvers.TwoFluid.Liquid.MTOT()*).

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

MWALEVAP(*vapor, liquid, zIdx*)

MWALEVAP() Linear wall mass evaporation rate [kg/s/m]

Computes the wall boiling mass transfer rate for vapor by reversing the sign of the liquid-phase value (*Solvers.TwoFluid.Liquid.MWALEVAP()*).

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

RE(*vapor, zIdx*)

RE() Vapor Reynolds number [-]

Computes the vapor-phase Reynolds number based on flow rate and viscosity.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

REL(*vapor, liquid, zIdx*)

REL() Dispersed vapor Reynolds number [-]

Computes the Reynolds number for dispersed vapor using liquid properties and relative velocity.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

S(*vapor, liquid, zIdx*)

S() Phase slip ratio [-]

Returns the slip ratio between vapor and liquid phases.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

T(*vapor, zIdx*)

T() Vapor temperature [K]

Computes the vapor temperature from enthalpy.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object

- *zIdx* — Axial indices to evaluate (optional)

TAUW(*vapor*, *zIdx*)

TAUW() Wall shear stress [N/m²]

Returns the wall shear stress acting on the vapor phase.

Inputs:

- *vapor* — *Solvers.TwoFluid.Vapor* object
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Uses mixture-level shear stress for consistency

USLIP(*vapor*, *liquid*, *zIdx*)

USLIP() Vapor velocity based on input phase slip [m/s]

Computes the vapor velocity using slip ratio and liquid velocity.

Inputs:

- *vapor* — *Solvers.TwoFluid.Vapor* object
- *liquid* — *Solvers.TwoFluid.Liquid* object
- *zIdx* — Axial indices to evaluate (optional)

VF(*vapor*, *liquid*, *zIdx*)

VF() Vapor volumetric fraction [-]

Computes the vapor volumetric (i.e., void) fraction using slip ratio and densities.

Inputs:

- *vapor* — *Solvers.TwoFluid.Vapor* object
- *liquid* — *Solvers.TwoFluid.Liquid* object
- *zIdx* — Axial indices to evaluate (optional)

VISC(*vapor*, *liquid*, *zIdx*)

VISC() Viscosity number for dispersed gas [-]

Computes the dimensionless viscosity number using liquid and vapor properties.

Inputs:

- *vapor* — *Solvers.TwoFluid.Vapor* object
- *liquid* — *Solvers.TwoFluid.Liquid* object
- *zIdx* — Axial indices to evaluate (optional)

WALEVAPRATIO(*vapor*, *liquid*, *zIdx*)

WALEVAPRATIO() Wall evaporation mass ratio [-]

Returns the fraction of liquid mass undergoing wall boiling.

Inputs:

- *vapor* — *Solvers.TwoFluid.Vapor* object
- *liquid* — *Solvers.TwoFluid.Liquid* object
- *zIdx* — Axial indices to evaluate (optional)

Returns:

- *walevapratio* — Wall boiling mass ratio [-]

X(*vapor*, *zIdx*)

X() Vapor mass fraction [-]

Computes the vapor mass fraction relative to the mixture flow.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

XTR_ANN(vapor, liquid)

XTR_ANN() Quality at onset of annular flow region

XTR_CBT(vapor, liquid)

XTR_CBT() Quality at Critical Boiling Transition [-]

Returns the equilibrium quality threshold for the onset of critical boiling transition.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object

XTR_ITM(vapor, liquid)

XTR_ITM() Quality at onset of intermediate region [-]

Returns the equilibrium quality threshold for the onset of the intermediate flow regime.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object

XTR_SAT(vapor, liquid)

XTR_SAT() Quality at end of subcooled boiling transition [-]

Returns the equilibrium quality threshold for the onset of saturated boiling.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object

XTR_SUB(vapor, liquid)

XTR_SUB() Quality at onset of subcooled boiling transition [-]

Returns the equilibrium quality threshold for the onset of subcooled boiling.

Inputs:

- vapor — *Solvers.TwoFluid.Vapor* object
- liquid — *Solvers.TwoFluid.Liquid* object

class *Solvers.TwoFluid.Mixture*(mixture, liquid, vapor)

Bases: *Solvers.AbstractPhase*

MIXTURE Class for two-fluid mixture properties in thermal-hydraulic solver

Represents combined properties of liquid and vapor phases in a two-fluid solver. Provides methods to compute mixture-level quantities such as mass flow, density, velocity, heat flux, and thermodynamic quality.

Responsibilities:

- Aggregate liquid and vapor properties into mixture-level results
- Compute derived quantities (void fraction, equilibrium quality, CHF)
- Provide access to geometry and fluid properties for calculations

Key features:

- Supports axial indexing for spatially resolved calculations
- Interfaces with liquid and vapor sub-objects for phase-specific data

Constructor Summary

Mixture(*mixture, liquid, vapor*)

MIXTURE Constructor for *Mixture()* class

Initializes the mixture object by linking liquid and vapor phase objects and copying solver configuration and fluid properties.

Inputs:

mixture — *Solvers.Mixture.Mixture* object liquid — *Solvers.TwoFluid.Liquid* object

vapor — *Solvers.TwoFluid.Vapor* object

Method Summary

AFDISTR(*mix, param1, param2, zIdx*)

AFDISTR() Annular flow distribution function [-]

Computes the annular flow distribution function for given parameters.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object param1 — First parameter for distribution function

param2 — Second parameter for distribution function zIdx — Axial indices to evaluate (optional)

CBT(*mix, zIdx*)

CBT() Critical Boiling Transition flag [-]

Returns the *CBT()* flag for each axial location.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object zIdx — Axial indices to evaluate (optional)

CHF(*mix, zIdx*)

CHF() Critical Heat Flux [W/m²]

Returns the *CHF()* value for each axial location.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object zIdx — Axial indices to evaluate (optional)

DPDZ(*mix, zIdx*)

DPDZ() Pressure gradient [Pa/m]

Computes the axial pressure gradient from mixture pressure drop.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object zIdx — Axial indices to evaluate (optional)

H(*mix, zIdx*)

H() Mixture enthalpy [J/kg]

Calculates the mixture enthalpy as a mass-weighted average of phase enthalpies.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object zIdx — Axial indices to evaluate (optional)

HFLUX(*mix, zIdx*)

HFLUX() Wall heat flux [W/m²]

Returns the wall heat flux distribution along the axial domain.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object zIdx — Axial indices to evaluate (optional)

LHGR(*mix, zIdx*)

LHGR() Linear heat generation rate [W/m]

Computes the linear heat generation rate from wall heat flux and perimeter.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object zIdx — Axial indices to evaluate (optional)

MFLUX(mix, zIdx)

MFLUX() Mixture mass flux [kg/m²/s]

Calculates the mixture mass flux using total flow and channel area.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object zIdx — Axial indices to evaluate (optional)

OAFX(mix, zIdx)

OAFX() Onset of annular flow equilibrium quality [-]

Returns the OAF quality from the mixture object.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object zIdx — Axial indices to evaluate (optional)

RELAXTCOND(mix, zIdx)

RELAXTCOND() Time relaxation for interfacial condensation [s]

Returns the relaxation time applied to interfacial condensation processes in the two-fluid mixture model.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object zIdx — Axial indices to evaluate (optional)

RELAXTEVAP(mix, zIdx)

RELAXTEVAP() Time relaxation for interfacial evaporation [s]

Returns the relaxation time applied to interfacial evaporation processes in the two-fluid mixture model.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object zIdx — Axial indices to evaluate (optional)

RHO(mix, zIdx)

RHO() Mixture density [kg/m³]

Computes the mixture density based on void fraction and phase densities.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object zIdx — Axial indices to evaluate (optional)

TAUW(mix, zIdx)

TAUW() Wall shear stress [N/m²]

Returns the wall shear stress from the mixture object.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object zIdx — Axial indices to evaluate (optional)

TIME(mix)

TIME() Time series [s]

Returns the time array from the mixture object.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object

U(mix, zIdx)

U() Mixture velocity [m/s]

Calculates the mixture velocity as a mass-weighted average of phase velocities.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object zIdx — Axial indices to evaluate (optional)

VF(*mix*, *zIdx*)

VF() Void fraction [-]

Computes the vapor volume fraction in the mixture.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)

W(*mix*, *zIdx*)

W() Mixture mass flow rate [kg/s]

Computes the total mass flow rate as the sum of liquid and vapor flows.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)

WALEVAPRATIO(*mix*, *zIdx*)

WALEVAPRATIO() Wall mass evaporation ratio [-]

Returns the ratio of wall heat flux converted to evaporation.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)

XEQ(*mix*, *zIdx*)

XEQ() Equilibrium thermodynamic quality [-]

Returns the equilibrium quality from the mixture object.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)

Z(*mix*, *zIdx*)

Z() Axial nodes [m]

Returns the axial positions for the mixture domain.

Inputs:

mix — *Solvers.TwoFluid.Mixture* object *zIdx* — Axial indices to evaluate (optional)

class *Solvers.TwoFluid.Liquid*(*inputSet*, *fluid*)

Bases: *Solvers.AbstractField*

LIQUID Class for modeling liquid field in two-fluid solver

This class encapsulates the physical and numerical properties of the liquid phase, including flow variables, phase interactions, heat transfer, and relaxation models. It supports multiple solver models and provides methods for computing derived quantities and handling flow regime transitions.

Constructor Summary

Liquid(*inputSet*, *fluid*)

LIQUID Constructor for *Liquid()* class

Initializes the liquid field object with input configuration and fluid properties.

Inputs:

- *inputSet* — *Inputs.InputSet* object containing model, geometry, and boundary conditions
- *fluid* — *Inputs.FluidProperties* object containing thermophysical fluid data

Property Summary

DT = 0

Time step size [s] from *Inputs.Options.TSTEP*

H = 1E6
Enthalpy [J/kg]

ITR
Iteration tracking

NTIME = 0
Number of time steps [-]

NZ = 0
Number of axial steps [-] from *Inputs.Model.NNODES*

TIDX = 1
Time step index [-]

TIME = 0
Time series [s]

U = 1.
Velocity [m/s]

W = 1.
Mass flow rate [kg/s]

Z = 1.
Elevation [m]

mix = NaN
Solvers.TwoFluid.Mixture object

Method Summary

AREA(*liquid, vapor, zIdx*)
AREA() Liquid cross-section area [m²]
Computes the effective cross-sectional area occupied by the liquid phase.
Inputs:

- *liquid* — *Solvers.TwoFluid.Liquid* object
- *vapor* — *Solvers.TwoFluid.Vapor* object
- *zIdx* — Axial indices to evaluate (optional)

CD(*liquid, vapor, zIdx*)
CD() Drag coefficient for dispersed liquid [-]
Computes the drag coefficient for dispersed liquid based on *Inputs.Model.DROPDRAG* model.
Inputs:

- *liquid* — *Solvers.TwoFluid.Liquid* object
- *vapor* — *Solvers.TwoFluid.Vapor* object
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Values are clamped to a maximum of 1

FBUOY(*liquid, vapor, zIdx*)
FBUOY() Liquid buoyancy force [N/m]
Computes the buoyancy force acting on the liquid phase due to pressure gradient.
Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- The pressure gradient from the mixture solver is used

FDRAG(*liquid, vapor, zIdx*)

FDRAG() Interfacial drag force [N/m]

Computes the interfacial drag force acting on the liquid phase.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- Uses smooth transition between dispersed gas and liquid drag based on *Inputs.Model.DISPGAS2DISPLIQ* model.

FGRAV(*liquid, vapor, zIdx*)

FGRAV() Liquid gravity force [N/m]

Computes the gravitational force acting on the liquid phase.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

FINTCOND(*liquid, vapor, zIdx*)

FINTCOND() Momentum exchange through interfacial condensation [N/m]

Computes the momentum exchange due to interfacial condensation.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

FLOWID(*liquid, zIdx*)

FLOWID() Two-phase flow regime ID

Returns the integer ID corresponding to the flow regime classification.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

FLOWREGIME(*liquid, zIdx*)

FLOWREGIME() Categorical two-phase flow regimes

Classifies flow regime based on equilibrium quality and boiling transition flags.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

Returns:

- flowregime — Categorical flow regime (from *Solvers.TwoFluid.REGIMES*)
- id — Integer regime ID

FTOT(*liquid*, *vapor*, *zIdx*)*FTOT*() Total force [N/m]

Computes the total force acting on the liquid phase, including gravity, buoyancy, wall shear, drag, and interfacial condensation.

Inputs:

- *liquid* — *Solvers.TwoFluid.Liquid* object
- *vapor* — *Solvers.TwoFluid.Vapor* object
- *zIdx* — Axial indices to evaluate (optional)

FWALL(*liquid*, *vapor*, *zIdx*)*FWALL*() Liquid wall shear force [N/m]

Computes the wall shear force acting on the liquid phase, weighted by void fraction.

Inputs:

- *liquid* — *Solvers.TwoFluid.Liquid* object
- *vapor* — *Solvers.TwoFluid.Vapor* object
- *zIdx* — Axial indices to evaluate (optional)

HFLUX(*liquid*, *zIdx*)*HFLUX*() Wall heat flux to liquid phase [W/m²]

Computes the net wall heat flux transferred to the liquid phase, combining direct wall heating and boiling contributions.

Inputs:

- *liquid* — *Solvers.TwoFluid.Liquid* object
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Combines *Solvers.TwoFluid.Liquid.HWALHEAT* and *Solvers.TwoFluid.Liquid.HFLUXWALEVAP*
- Uses wall perimeter from geometry

HFLUXWALEVAP(*liquid*, *zIdx*)*HFLUXWALEVAP*() Wall evaporation (boiling) heat flux [W/m²]

Computes the portion of wall heat flux attributed to evaporation (e.g., boiling), based on the wall evaporation mass ratio.

Inputs:

- *liquid* — *Solvers.TwoFluid.Liquid* object
- *zIdx* — Axial indices to evaluate (optional)

HINT(*liquid*, *vapor*, *zIdx*)*HINT*() Linear interfacial heat transfer rate [W/m]

Computes the interfacial heat transfer rate using mass transfer and enthalpy difference (based on *Inputs.Model.INTTRANSH* model).

Inputs:

- *liquid* — *Solvers.TwoFluid.Liquid* object
- *vapor* — *Solvers.TwoFluid.Vapor* object
- *zIdx* — Axial indices to evaluate (optional)

Returns:

- *Hcond* — Condensation heat transfer rate [W/m]
- *Hevap* — Evaporation heat transfer rate [W/m]

HINTCOND(*liquid, vapor, zIdx*)*HINTCOND()* Linear interfacial heat condensation rate [W/m]Extracts the condensation component from *Solvers.TwoFluid.Liquid.HINT*.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

HINTEVAP(*liquid, vapor, zIdx*)*HINTEVAP()* Linear interfacial heat evaporation rate [W/m]Extracts the evaporation component from *Solvers.TwoFluid.Liquid.HINT*.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

HTOT(*liquid, vapor, zIdx*)*HTOT()* Total linear vapor heat rate [W/m]

Computes the total heat transfer to the vapor phase, including interfacial and wall contributions.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

HWALEVAP(*liquid, vapor, zIdx*)*HWALEVAP()* Linear wall heat evaporation (boiling) rate [W/m]Computes the wall boiling heat transfer rate using latent heat (based on *Inputs.Model.INTTRANSH* model) and wall mass evaporation.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

HWALHEAT(*liquid, zIdx*)*HWALHEAT()* Linear wall heat to liquid rate [W/m]

Returns the wall heat generation rate transferred to the liquid phase.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- Set to zero beyond CBT

INTAREA(*liquid, vapor, zIdx*)*INTAREA()* Volumetric interfacial area [m⁻¹]

Computes the interfacial area concentration based on flow regime and phase topology.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

Notes:

- Uses a smooth transition between dispersed gas and dispersed liquid models

INTHFLUX(*liquid*, *vapor*, *zIdx*)

INTHFLUX() Interfacial heat flux [W/m²]

Computes the interfacial heat flux due to evaporation and condensation based on *Inputs.Model.INTAREA* interfacial area model.

Inputs:

- *liquid* — *Solvers.TwoFluid.Liquid* object
- *vapor* — *Solvers.TwoFluid.Vapor* object
- *zIdx* — Axial indices to evaluate (optional)

Returns:

- *inthflux_evap* — Evaporation heat flux [W/m²]
- *inthflux_cond* — Condensation heat flux [W/m²]

Notes:

- Uses interfacial Nusselt number and length scale
- Driven by deviation from saturated temperature

INTNU(*liquid*, *vapor*, *zIdx*)

INTNU() Interfacial Nusselt number for dispersed liquid [-]

Computes the interfacial Nusselt number based on *Inputs.Model.INTNU* model.

Inputs:

- *liquid* — *Solvers.TwoFluid.Liquid* object
- *vapor* — *Solvers.TwoFluid.Vapor* object
- *zIdx* — Axial indices to evaluate (optional)

J(*liquid*, *zIdx*)

J() Liquid superficial velocity [m/s]

Computes the superficial velocity of the liquid phase.

Inputs:

- *liquid* — *Solvers.TwoFluid.Liquid* object
- *zIdx* — Axial indices to evaluate (optional)

L(*liquid*, *zIdx*)

L() Dispersed liquid interfacial length scale [m]

Returns the characteristic length scale for dispersed liquid (e.g., drop Sauter mean diameter).

Inputs:

- *liquid* — *Solvers.TwoFluid.Liquid* object
- *zIdx* — Axial indices to evaluate (optional)

Notes:

- Currently uses a constant model

MINT(*liquid*, *vapor*, *zIdx*)

MINT() Linear interfacial mass transfer rates [kg/s/m]

Computes condensation and evaporation mass transfer rates using either heat flux-based or relaxation-based models.

Inputs:

- *liquid* — *Solvers.TwoFluid.Liquid* object
- *vapor* — *Solvers.TwoFluid.Vapor* object
- *zIdx* — Axial indices to evaluate (optional)

Returns:

- Mcond — Condensation mass transfer rate [kg/s/m]
- Mevap — Evaporation mass transfer rate [kg/s/m]

Notes:

- Supports ‘CONSTANT’, ‘RANZMARSHALL’, and ‘RELAXATION’ models based on [Inputs.Model.INTNU](#) model
- Applies wall-wise distribution

MINTCOND(*liquid, vapor, zIdx*)

[MINTCOND\(\)](#) Linear interfacial condensation mass transfer [kg/s/m]

Extracts the condensation component from [Solvers.TwoFluid.Liquid.MINT](#).

Inputs:

- liquid — [Solvers.TwoFluid.Liquid](#) object
- vapor — [Solvers.TwoFluid.Vapor](#) object
- zIdx — Axial indices to evaluate (optional)

MINTEVAP(*liquid, vapor, zIdx*)

[MINTEVAP\(\)](#) Linear interfacial evaporation mass transfer [kg/s/m]

Extracts the evaporation component from [Solvers.TwoFluid.Liquid.MINT](#).

Inputs:

- liquid — [Solvers.TwoFluid.Liquid](#) object
- vapor — [Solvers.TwoFluid.Vapor](#) object
- zIdx — Axial indices to evaluate (optional)

MTOT(*liquid, vapor, zIdx*)

[MTOT\(\)](#) Total linear mass transfer [kg/s/m]

Computes the total vapor mass transfer rate by summing interfacial condensation, evaporation, and wall boiling contributions.

Inputs:

- liquid — [Solvers.TwoFluid.Liquid](#) object
- vapor — [Solvers.TwoFluid.Vapor](#) object
- zIdx — Axial indices to evaluate (optional)

MWALEVAP(*liquid, vapor, zIdx*)

[MWALEVAP\(\)](#) Linear wall mass evaporation rate [kg/s/m]

Computes the wall boiling mass transfer rate using latent heat (based on [Inputs.Model.INTTRANSH](#) model) and wall heat flux.

Inputs:

- liquid — [Solvers.TwoFluid.Liquid](#) object
- vapor — [Solvers.TwoFluid.Vapor](#) object
- zIdx — Axial indices to evaluate (optional)

RE(*liquid, zIdx*)

[RE\(\)](#) Liquid Reynolds number [-]

Computes the liquid-phase Reynolds number based on flow rate and viscosity.

Inputs:

- liquid — [Solvers.TwoFluid.Liquid](#) object
- zIdx — Axial indices to evaluate (optional)

REV(*liquid, vapor, zIdx*)

REV() Dispersed liquid Reynolds number [-]

Computes the Reynolds number for dispersed liquid using vapor properties and relative velocity.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

S(*liquid, vapor, zIdx*)

S() Phase vapor/liquid slip ratio [-]

Computes the slip ratio between vapor and liquid velocities.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

T(*liquid, zIdx*)

T() Liquid temperature [K]

Computes the temperature of the liquid phase from enthalpy.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

TAUW(*liquid, zIdx*)

TAUW() Liquid wall shear stress [N/m²]

Returns the wall shear stress acting on the liquid phase.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

USLIP(*liquid, vapor, zIdx*)

USLIP() Liquid velocity based on input phase slip [m/s]

Computes the liquid velocity using a slip-based void fraction model.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

VF(*liquid, vapor, zIdx*)

VF() Liquid volumetric fraction [-]

Computes the liquid volumetric fraction as the complement of vapor **VF()**.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

VISC(*liquid, vapor, zIdx*)

VISC() Viscosity number for dispersed liquid [-]

Computes the dimensionless viscosity number using vapor properties and interfacial tension.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

VR(*liquid, vapor, zIdx*)

VR() Local relative velocity [m/s]

Computes the relative velocity between vapor and liquid phases using various models (AREAMEAN, SCALED, DRIFT, FLOWREGIME) based on *Inputs.Model.LOCRELVEL* model.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- vapor — *Solvers.TwoFluid.Vapor* object
- zIdx — Axial indices to evaluate (optional)

WALEVAPRATIO(*liquid, zIdx*)

WALEVAPRATIO() Wall evaporation mass ratio [-]

Returns the fraction of liquid mass undergoing wall boiling.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

X(*liquid, zIdx*)

X() Liquid mass fraction [-]

Computes the liquid mass fraction relative to the mixture flow.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object
- zIdx — Axial indices to evaluate (optional)

XTR_ANN(*liquid*)

XTR_ANN() Quality at onset of annular flow region [-]

Returns the equilibrium quality threshold for the onset of annular flow.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object

Notes:

- Uses minimum value from mix.OAFX

XTR_ITM(*liquid*)

XTR_ITM() Quality at onset of intermediate region [-]

Returns the equilibrium quality threshold for the onset of the intermediate flow regime.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object

XTR_SAT(*liquid*)

XTR_SAT() Quality at saturated boiling transition [-]

Returns the equilibrium quality threshold for the onset of saturated boiling.

Inputs:

- liquid — *Solvers.TwoFluid.Liquid* object

XTR_SUB(*liquid*)

XTR_SUB() Quality at subcooled boiling transition [-]

Returns the equilibrium quality threshold for the onset of subcooled boiling.

Inputs:

- *liquid* — *Solvers.TwoFluid.Liquid* object

The following is a list of associated superclasses, each with its respective properties and methods. These foundational components provide shared functionality and structure across solver implementations.

- Abstract solver: *Solvers.AbstractSolver*
 - Abstract field: *Solvers.AbstractField*
 - Abstract phase: *Solvers.AbstractPhase*
 - Abstract film: *Solvers.AbstractFilm*
 - Solver plotter: *Solvers.SolverPlotter*
 - Solver state: *Solvers.SolverState*
-

class *Solvers.AbstractFilm*(*inputSet*, *fluid*)

Bases: *Solvers.AbstractField*

ABSTRACTFILM Base class for liquid film models in three- and four-field solvers

This abstract class defines shared methods and properties for liquid film modeling. It includes calculations for film thickness, entrainment, shear forces, and velocity models. Specific entrainment models (e.g., Okawa, Govan) and friction models are implemented.

Constructor Summary

AbstractFilm(*inputSet*, *fluid*)

AbstractFilm() Constructor for *AbstractFilm()* class

Initializes *Inputs.InputSet* and *Inputs.FluidProperties*

Property Summary

DZ = 0

Axial step size [m]

fluid

Inputs.FluidProperties object containing *fluid* thermophysical properties

inputSet

Inputs.InputSet object containing geometry, model, and boundary conditions

mix = NaN

Solvers.Mixture.Mixture object of the mixture solver

Method Summary

CV(*absfilm*, *zIdx*)

CV() Film/vapor interfacial friction factor [-]

Based on selected *Inputs.Model.VAPORFRIC* model.

CV_WALLISTHICK_CALC(*absfilm*, *thick*, *C*)

CV_WALLISTHICK_CALC() Private method to calculate the interfacial shear factor [-]

Based on the *Inputs.Model.VAPORFRIC* = *WALLISTHICK* model.

CW(*absfilm*, *zIdx*)

CW() Film wall friction factor [-]

Based on selected *Inputs.Model.THINFILMFRIC* model.

CW_LAM_CALC(*absfilm*, *zIdx*, *C*)

CW_LAM_CALC() Private method to calculate the laminar wall friction factor [-]

CW_TURB_CALC(*absfilm*, *zIdx*, *C*)

CW_TURB_CALC() Private method to calculate the turbulent wall friction factor [-]

ENTNUM(*absfilm*, *zIdx*)

ENTNUM() Private method to calculate the entrainment number [-]

Based on Okawa model assumptions.

FBUOY(*absfilm*, *zIdx*)

FBUOY() Film buoyancy force [N/m^2]

FDEP(*absfilm*, *drop*, *zIdx*)

FDEP() Drop deposition shear force [N/m^2]

FGRAV(*absfilm*, *zIdx*)

FGRAV() Gravitational force on film [N/m^2]

FTOT(*absfilm*, *drop*, *zIdx*)

FTOT() Total film forces per unit wall area [N/m^2]

FVAPOR(*absfilm*, *zIdx*)

FVAPOR() Vapor shear stress on film [N/m^2]

FWALL(*absfilm*, *zIdx*)

FWALL() Film wall shear stress [N/m^2]

MENT(*absfilm*, *zIdx*)

MENT() Film entrainment mass flux [kg/m^2/s]

Based on selected *Inputs.Model.ENTRAINMENT* model.

MTOT(*absfilm*, *drop*, *zIdx*)

MTOT() Total film mass transfer [kg/m^2/s]

OKAWACOEFS(*absfilm*, *entnum*, *coefs*)

OKAWACOEFS() Private method to determine the coefficients used in *Inputs.Model.ENTRAINMENT* = *OKAWA* entrainment models

Based on the calculated entrainment number

OKAWAMENT(*absfilm*, *zIdx*, *coefs*)

OKAWAMENT() Private method to calculate the entrainment mass flux [kg/m^2/s]

Based on Okawa models.

RE(*absfilm*, *zIdx*)

RE() Film Reynolds number [-]

THICK(*absfilm*, *zIdx*)

THICK() Film thickness [m]

UALGEBR(*absfilm*, *zIdx*)

UALGEBR() Film velocity using simple algebraic model [m/s]

From Adamsson and Le Corre [2014] (eq. 21 or 23).

UEQUIL(*absfilm*, *drop*, *zIdx*)

UEQUIL() Film velocity using full force equilibrium model [m/s]

UEQUILS(*absfilm*, *zIdx*)

UEQUILS() Film velocity assuming equilibrium between wall and vapor shear [m/s]

UWALL(*absfilm*, *zIdx*)

UWALL() Film wall friction velocity [m/s]

WL(*absfilm*, *zIdx*)

WL() Film mass flow rate per unit perimeter [kg/s/m]

X(*absfilm*, *zIdx*)

X() Vapor quality in the near-wall region [-]

Vapor quality in the near-wall region, considering only the film for the liquid phase. The near-wall region is defined in the mixture solver.

Inputs:

- *absfilm* — *Solvers.AbstractFilm* object
- *zIdx* — Axial indices to evaluate (optional)

YPLUS2THICK(*absfilm*, *yplus*, *zIdx*)

YPLUS2THICK() Thickness based on wall unit value [m]

class *Solvers.SolverPlotter*(*titles*, *WallIdxs*, *opts*)

Bases: *handle*

SOLVERPLOTTER Visualization framework for solver results.

This class provides a flexible plotting interface for visualizing solver results. It supports tiled layouts, animated series, and customizable plot styles. It also includes UI controls for animation playback and export.

Constructor Summary

SolverPlotter(*titles*, *WallIdxs*, *opts*)

SOLVERPLOTTER Constructor method to initialize plotter objects and layout

Property Summary

FontSize = 13

Font size of text in plots

Grid = 'on'

Grid display option

Title = ""

Title of the figure

Ts = []

Time series [s]

WallIdx = 1

Wall index for multi-wall simulations

Zs = []

Axial positions [m]

currentAhIdx = NaN

Index of current active axes

Method Summary

addTile(*plotters, opts*)

ADDTILE Method to add a new tile to the layout or retrieve existing one by title

static fieldName2plotStyle(*fieldName*)

FIELDNAME2PLOTSTYLE Static method to map field names to plot styles (color, line, marker)

gca(*plotters*)

GCA Get Method to retrieve current axes handle

gcf(*plotters*)

GCF Method to retrieve current plotter handle

legend(*plotters, varargin*)

LEGEND Control legends on current tile, a specific tile, or all tiles

Usage:

plotter.legend('Location','northwest') plotter.legend('Tile',2,'Location','best') plotter.legend('AllTiles',true,'Location','south') plotter.legend('Axes',ah,'off')

newTile(*plotters, opts*)

NEWTILE Method to create a new tile with labels and formatting

plotK(*plotters, klocZ, ah*)

PLOTK Method to plot obstruction elevation indicators

plotOAF(*plotters, oafZ*)

PLOT OAF Method to plot onset of annular flow boundary indicator

plotz(*plotters, YData, fieldName, opts*)

PLOTZ Method to plot spatial or temporal data (YData) on the current axes

resizeFigure(*plotters, scaleFactor*)

[resizeFigure\(\)](#) Resizes the figure based on tiledlayout rows/columns Keeps top-left fixed and goes full-screen if size exceeds screen resolution.

Usage:

- `plotters.resizeFigure()` — Default `scaleFactor` = 1
- `plotters.resizeFigure(1.2)` — Tiles 20% larger
- `plotters.resizeFigure(0.8)` — Tiles 20% smaller

setZs(*plotters, Zs*)

SETZS Method to assign the parameter(space or time) on the plotter x-axis

xlim(*plotters, newLim*)

[xlim\(\)](#) Method to get or set x-axis limits for current axes

ylabel(*plotters, labels*)

YLABEL Method to set y-axis tick labels

ylim(*plotters, newLim*)

YLIM Method to get or set y-axis limits for current axes

class Solvers.**AbstractSolver**(*inputSet*)

Bases: handle

ABSTRACTSOLVER Defines the base interface and shared functionality for all solver classes.

Subclasses must implement the abstract methods and properties defined here.

Constructor Summary

AbstractSolver(*inputSet*)

ABSTRACTSOLVER Constructor for *AbstractSolver()*

Initializes the solver with a given *Inputs.InputSet* and applies solver-specific properties.

Property Summary

STATE

Solver state, defined by *Solvers.SolverState* enumeration

inputSet

Input configuration object of type *Inputs.InputSet*

Method Summary

static CreateITR(*NZ, ITRFields*)

CreateITR() Create and initialize the inner iteration storage structure

Inputs:

- *NZ* — Number of axial nodes (scalar)
- *ITRFields* — (string array) Names of fields to include in the struct

Output:

- *ITR* — Struct with fields initialized to zero vectors of length *NZ*

log(*solver, varargin*)

LOG Log messages to the session *log()*

save(*solver, opts*)

SAVE Save the solver object to a .mat file with a customizable name.

Inputs:

- *solver* — The solver object to be saved.
- *opts.name* — (string) Name of the variable under which to *save()* the solver. Default set to solver name
- *opts.showpath* — (logical) Whether to display the *save()* path. Default: true.

solverName(*solver*)

SOLVERNAME Returns the name of the solver

warning(*solver, varargin*)

LOG Log *warning()* messages to the session log

class Solvers.**AbstractField**

Bases: matlab.mixin.Copyable

ABSTRACTFIELD Base class for field definitions across solvers

This abstract class defines shared properties and methods for all field-type classes used in solver implementations. It provides mechanisms for transient data extraction, plotting, memoization, struct conversion, and property copying.

Constructor Summary

AbstractField()

ABSTRACTFIELD Constructor for *AbstractField()* class

Property Summary

DT

Time step size [s]

ITR

Iteration properties

NTIME

Number of time steps [-]

NZ

Number of axial steps [-]

TIDX

Time step index [-]

TIME

Time series [s]

Z

Elevation [m]

Method Summary

copyFlowProperties(*srcObj, targetObj, opts*)

COPYFLOWPROPERTIES Copies flow properties from source to target field object

Supports full or partial copying depending on 'opts.all' flag.

memoizeFunction(*methodStr, methodHandle, varargin*)

MEMOIZEDMETHOD Registers and retrieves memoized function handles

Avoids redundant computation by caching results.

Adapted from <https://stackoverflow.com/a/75037451>

plotzt(*param, ylabelText, ylabelUnit, k, opt, subobj, annular, time*)

PLOTZT 2D Generates a 2D space-time distribution plot for a given parameter

struct()

STRUCT Converts field object to a structured array

Includes TIME, ITR, and flow properties.

transient(*param, subobj, opt*)

TRANSIENT Extracts *transient()* distribution array for a given parameter

Supports wall-wise and axial/time slicing with optional subobject access.

class Solvers.AbstractPhase

Bases: handle

ABSTRACTPHASE Base class for phase definitions across solvers

This abstract class serves as a common interface for all phase-type classes used in solver implementations. It is intended to provide a foundation for shared behavior and structure.

class Solvers.SolverState

Bases: uint16

SOLVERSTATE Enumeration of solver execution states

This enumeration defines the possible states a solver can be in during or after execution. It is used to track solver execution and convergence status.

States:

- UNSOLVED — Solver has not yet run
- SOLVEDCONVERGED — Solver completed and converged successfully
- SOLVEDNOTCONVERGED — Solver completed but did not converge
- INITIALSTEPCONVERGED — Initial steady-state step completed and converged
- INITIALSTEPNOTCONVERGED — Initial steady-state step completed but did not converge

EXTERNAL PACKAGES

6.1 CoolPropWrapper

The CoolPropWrapper package is designed to serve as a MATLAB interface to the CoolProp thermophysical property library, which is a Python-based library for calculating fluid properties.

More information about the CoolPropWrapper can be found at <https://github.com/mfval/CoolPropWrapper>.

Information about CoolProp can be found at <https://coolprop.org>.

COMMUNITY

7.1 Authors

The **OpenSTREAM** solvers have been developed and validated through a dynamic and growing international collaboration that began in 2023 between **Westinghouse Electric Sweden** (WSE) and the **University of Wisconsin–Madison** (UWM). Since its inception, the initiative has expanded to include partnerships with the **Naval Nuclear Laboratory** (NNL), the **Royal Institute of Technology** (KTH), the **Massachusetts Institute of Technology** (MIT), the **Swiss Federal Institute of Technology Zurich** (ETHZ), and the **French Alternative Energies and Atomic Energy Commission** (CEA). In addition, the solvers are being utilized in research programs at other leading institutions, including the **Paul Scherrer Institut** (PSI) and **McMaster University**.

This effort is driven by the OpenSTREAM code development team, with active contributions from MSc and PhD students as part of their research and coursework under the guidance of their academic supervisors. The collaboration continues to grow, with the goal of engaging more academic and research institutions working on advanced two-phase flow modeling and thermal-hydraulic simulation.

Maintainers and contributors/users listed alphabetically by first name, last name.

7.1.1 Maintainers

These are the primary maintainers of OpenSTREAM, actively supporting development, reviewing contributions, and guiding the project forward:

- [Jason Chan](#) (UWM)
- [Jean-Marie Le Corre](#) (WSE)
- [Roman Morse](#) (MIT)

7.1.2 Contributors and users

OpenSTREAM is built through the collaborative efforts of many talented individuals. Here are some of the contributors and users who have helped shape the project:

- [Allison Mahvi](#) (UWM)
- [Cole Dunbar](#) (UWM)
- [Elisabeth Walter](#) (ETHZ)
- [Evan Hurlburt](#) (NNL)
- [Fabrice Francois](#) (CEA)
- [Haipeng Li](#) (KTH)

- [Juliana Duarte](#) (UWM)
- [Tyler Lee](#) (UWM)
- You! — Contributions are welcome. Join us and help improve OpenSTREAM!

7.2 Contribute to the code

Contributions to the **OpenSTREAM** solvers, documentation, tutorials, tests, and examples are welcome. Following the guidelines below helps ensure that changes can be reviewed, tested, and integrated efficiently.

7.2.1 How to contribute

1. Fork the repository

Open the [OpenSTREAM repository](#) and select **Fork** to create a copy under your GitHub account.

2. Clone your fork

```
git clone https://github.com/<your-username>/openstream.git
cd openstream
```

3. Create a development branch

Create a branch with a concise name describing the intended change:

```
git checkout -b feat/my-new-feature
```

Other appropriate prefixes may include `fix/`, `docs/`, `test/`, and `refactor/`.

4. Make your changes

- Follow the coding and documentation standards described below.
- Add or update tests when functionality or numerical behavior changes.
- Update user documentation when the change affects visible behavior, inputs, outputs, workflows, or examples.
- Avoid committing generated output files.

5. Run the automated tests

From the `tests` folder, run:

```
runOpenSTREAMTests
```

Confirm that the complete available test suite passes before pushing the change.

6. Commit and push the changes

Review the staged files before committing to ensure that generated outputs, temporary files, and unrelated changes are not included:

```
git status
git diff --cached
```

Commit messages must follow the [OpenSTREAM commit message style guide](#).

Organize the changes into focused commits. For example:

```
git add <files>
git commit -m "feat(scope): describe the change"
git push origin feature/my-new-feature
```

Avoid using `git add .` without first reviewing all modified and untracked files. Separate functional changes from formatting, documentation, test-reference, and unrelated maintenance changes where practical.

7. Open a pull request

Open a pull request against the original OpenSTREAM repository.

The pull-request description should:

- Explain the purpose of the change.
- Summarize the implementation.
- Summarize the focused commits included in the pull request.
- Identify affected models, solvers, inputs, tests, or documentation.
- Describe any intentional numerical changes.
- Explain and justify any changes to approved reference solutions.
- Reference related issues when applicable.

7.2.2 Coding standards

MATLAB

- Use clear and descriptive variable, property, method, and class names.
- Follow the existing OpenSTREAM package and class organization.
- Document public classes, properties, functions, and methods.
- Add comments where they clarify numerical methods, physical assumptions, or non-obvious implementation choices.
- Avoid hard-coded repository paths.
- Use `fullfile` when constructing file-system paths.
- Preserve the existing coding style unless a broader refactoring is part of the proposed change.
- Avoid unrelated formatting changes in the same commit as a functional modification.

Python

- Follow PEP 8 conventions where practical.
- Document externally visible functions and classes.
- Keep interactions between MATLAB and Python explicit and reproducible.
- Update dependency specifications when adding or modifying Python package requirements.

General practices

- Keep commits focused and descriptive.
- Follow the [OpenSTREAM commit message style guide](#).
- Prefer automated tests over manual verification whenever practical.

- Avoid duplicating existing functionality.
- Preserve backward compatibility unless an incompatible change is intentional, documented, and reviewed.
- Separate functional, documentation, test-reference, formatting, and repository-maintenance changes where practical.
- Review staged and untracked files before every commit.
- Do not commit generated outputs, temporary files, local sandbox scripts, MATLAB autosave files, or obsolete copies.

7.2.3 Documentation

Update the documentation when a contribution changes:

- User-visible functionality.
- Input parameters or accepted values.
- Default model or numerical selections.
- Solver behavior or calculated outputs.
- Public classes, methods, or properties.
- Installation or dependency requirements.
- Tutorials or recommended workflows.
- File names, links, or repository organization.

Documentation changes should be reviewed in both the generated website and PDF documentation when the affected content is included in both formats.

Changes that affect reported numerical results should preserve the information required by the reproducibility checklist.

7.2.4 Testing

OpenSTREAM includes automated environment, unit, integration, and regression verification. The tests are organized under:

```
tests/environment/  
tests/unit/  
tests/integration/
```

The complete available test suite can be run from the `tests` folder:

```
runOpenSTREAMTests
```

The runner recursively discovers the available test classes, displays detailed diagnostics, produces a summary table, reports the total test-suite execution time, and raises an error if any test fails or does not complete.

After execution, the MATLAB test results and generated summary table are available in the workspace as `testResults` and `testSummary`.

The current test categories include:

- Environment tests that verify the MATLAB-Python interface, required Python packages, and basic CoolProp functionality.
- Unit tests that verify focused OpenSTREAM components, currently including input handling.

- Integration tests that construct and solve representative OpenSTREAM cases, perform physical-consistency checks, compare selected numerical results with approved references, and verify solver persistence.

The automated test suite is under active development and does not yet exercise every OpenSTREAM class, method, physical model, numerical option, solver path, error condition, or post-processing capability. A successful test run confirms that the currently implemented tests passed. It does not constitute complete verification or validation of OpenSTREAM.

Testing contributions

Contributors should:

- Add focused unit tests for new testable classes, functions, and methods.
- Add integration tests when a change affects solver workflows, physical consistency, or numerical results.
- Add environment tests when introducing new external dependencies.
- Verify that all existing tests continue to pass.
- Review numerical differences from approved references.
- Update tests when expected behavior changes intentionally.
- Update the testing documentation when the framework, test organization, or execution procedure changes.

Reference solutions

Approved solver-reference objects are stored below:

```
tests/integration/references/<session-directory>/
```

These files form part of the numerical regression baseline and are version-controlled.

Reference solutions should be updated only when:

- A defect has been corrected.
- A model formulation has intentionally changed.
- Numerical behavior has intentionally changed and the new behavior has been reviewed.
- A justified change to the approved test environment changes the calculated reference results.

Reference solutions should not be replaced merely to make a failing test pass. Before updating a reference, identify and understand the numerical differences and document why the new result is correct.

Generated outputs

Some tests generate temporary files, logs, session directories, CSV files, and saved solver objects under category-specific `outputs` directories.

Generated outputs are excluded from version control and should not normally be committed. Approved reference solutions are stored separately and should be committed only as part of a justified and reviewed update to the regression baseline.

Continuous integration

The complete available test suite is executed automatically by the MATLAB CI workflow when changes are pushed to the repository.

After pushing a change:

1. Open the corresponding GitHub Actions run.

2. Confirm that the MATLAB CI workflow completes successfully.
3. Review the diagnostic output if a test fails or does not complete.
4. Reproduce failures locally using `runOpenSTREAMTests`.
5. Push a correction only after rerunning the complete local test suite.

The documentation build and MATLAB CI are separate workflows. A successful documentation build or GitHub Pages deployment does not indicate that the MATLAB tests passed. Review both workflow results independently.

7.2.5 Reporting issues

Use the [OpenSTREAM issue tracker](#) to report defects, request enhancements, or propose changes.

An issue report should include, where applicable:

- A clear description of the problem.
- Steps required to reproduce the behavior.
- Expected and actual behavior.
- The relevant OpenSTREAM version or commit.
- MATLAB and Python versions.
- Relevant dependency versions.
- Input files or a minimal reproducible case.
- Solver output, logs, screenshots, or error messages.
- Any temporary workaround that has been identified.

Before opening a new issue, review existing issues to determine whether the topic has already been reported or discussed.

7.2.6 Code of conduct

Contributors should communicate respectfully and constructively. Technical disagreements should focus on the implementation, evidence, assumptions, and expected behavior rather than on individuals.

7.2.7 Communication

Use [GitHub Discussions](#) for general questions, ideas, and community discussions.

For major changes, open an issue before beginning implementation. Early discussion is particularly useful for:

- New solver frameworks.
- New conservation equations or closure-model families.
- Changes to established model formulations.
- Changes to public interfaces or input formats.
- Changes that affect backward compatibility.
- Changes that require substantial updates to regression references.

7.2.8 Contributor checklist

Before submitting a pull request, confirm that:

- The change is limited to the intended scope.
- The code follows the existing OpenSTREAM style and organization.
- Changes are organized into focused commits.
- Commit messages follow the [OpenSTREAM commit message style guide](#)

<https://github.com/OpenSTREAM-solvers/openstream/wiki> - Staged and untracked files have been reviewed before committing. - New or modified functionality includes appropriate tests. - The complete local test suite passes. - Numerical differences from approved references have been reviewed. - Reference solutions have been updated only when justified. - User-visible behavior is documented. - Relevant tutorials or examples have been updated. - Documentation links and cross-references remain valid. - Generated outputs and temporary files are not committed. - The MATLAB CI workflow passes after the changes are pushed. - Documentation workflows pass when documentation has been modified.

7.3 Contribute to the documentation

The OpenSTREAM documentation is created using [Sphinx](#) with the [Read the Docs theme](#).

This guide explains how to prepare a local documentation environment, write and review reStructuredText content, build the documentation, and submit documentation changes.

Any suitable text editor or integrated development environment can be used. This guide includes optional instructions for configuring Sublime Text, but contributors may instead use Visual Studio Code, Vi, Nano, or another preferred editor.

Parts of the editor setup guidance are based on the [Sublime Text and Sphinx guide](#).

7.3.1 Set up Sublime Text as a text editor

Any text editor, such as Vi, Nano, Sublime Text, or Visual Studio Code, or integrated development environment can be used to write and manage the documentation source files.

For convenience, Sublime Text is used in this section.

1. Download and install [Sublime Text](#).
2. Open Sublime Text.
3. Install Package Control.

On the top menu bar, select **Tools > Command Palette**, or press Ctrl+Shift+P on Windows or Linux, or Cmd+Shift+P on macOS.

Enter `Install Package Control` and select the matching command.

4. Open the Command Palette again and select **Package Control: Install Package**.
5. Install the following packages, when useful:
 - `RestructuredText` Improved for syntax highlighting.
 - `OmnimarkPreviewer` for quick browser previews.
 - `Sublime RST Completion` for reStructuredText snippets and completion.

A quick editor preview can be useful while writing, but it does not replace a complete Sphinx build. Sphinx directives, cross-references, citations, generated API documentation, and the complete navigation structure must be reviewed using the actual documentation build.

7.3.2 Configure Python and Sphinx

Python and the required Sphinx packages must be installed before building the documentation locally.

Set up Python

Install a supported Python version from the [official Python website](#).

Depending on the operating system and installation method, the Python installation directory and its `Scripts` directory may need to be added to the system path.

Verify the installation:

```
python --version
python -m pip --version
```

Use `python3` instead of `python` if required by the local operating system or Python installation.

Create a virtual environment

A virtual environment isolates the documentation dependencies from other Python projects.

Change to the OpenSTREAM docs folder:

```
cd <path-to-openstream>/docs
```

Create a virtual environment named `.venv`:

```
python -m venv .venv
```

The `.venv` directory is local generated content and must not be committed.

Activate the environment

On Windows Command Prompt:

```
.venv\Scripts\activate
```

On Windows PowerShell:

```
.\.venv\Scripts\Activate.ps1
```

On Git Bash, Linux, or macOS:

```
source .venv/Scripts/activate
```

On Linux or macOS, the activation path may instead be:

```
source .venv/bin/activate
```

After activation, the command prompt normally indicates that the virtual environment is active.

Install Sphinx and dependencies

Install the documentation dependencies from the requirements file:

```
python -m pip install --upgrade pip
python -m pip install -r source/requirements.txt
```

The requirements file should contain the Sphinx version and extensions needed to reproduce the documentation build.

If installation fails, retain the complete error message and record:

- The operating system.
- The Python version.
- The pip version.
- The failing package and version.
- The command used to install the requirements.

7.3.3 Start writing

The documentation is written in reStructuredText and compiled using Sphinx.

Useful references include:

- The [Sphinx reStructuredText primer](#).
- The [Sphinx tutorial](#).
- The [reStructuredText cheat sheet](#).

Documentation source files should use the established OpenSTREAM structure, terminology, and formatting conventions.

7.3.4 Writing guidelines

Use clear, concise technical language. Documentation should distinguish among:

- Implemented software capability.
- Theoretical assumptions.
- Numerical behavior.
- Verification evidence.
- Experimental validation.
- Known limitations.
- Recommended user practice.

Avoid statements that imply broader validation or applicability than is supported by the available evidence.

Terminology

Use terminology consistently across theory pages, tutorials, package references, application projects, and publications.

In particular, distinguish between:

- OpenSTREAM and OpenSTREAM-database.
- A physical model and the solver implementing that model.
- The mixture model and the Mixture solver.
- The two-fluid model and the TwoFluid solver.
- The three-field model and the ThreeField solver.
- The four-field model and the FourField solver.
- Pointwise convergence, pseudo-time convergence, and physical-time advancement.

- Direct experimental measurements, derived quantities, and calculated quantities.
- Boiling transition, critical heat flux, dryout, and film depletion.
- Thermal nonequilibrium and hydrodynamic nonequilibrium.

Use lowercase descriptive forms when discussing physical formulations and code-style names when referring to specific software interfaces.

Units

Use SI units throughout the documentation unless original units are required to explain an experimental source or conversion.

Absolute temperatures must be expressed in kelvin in OpenSTREAM inputs and stored results.

When original experimental units are presented, state the corresponding SI representation and document the conversion.

Code examples

Code examples should:

- Reflect the current public interface.
- Use clear and descriptive variable names.
- Avoid hard-coded local paths.
- Use `fullfile` for constructed file-system paths.
- Include only the options relevant to the example.
- Use comments above logical sections.
- State any nondefault physical or numerical settings.
- Identify required working-directory or MATLAB-path assumptions.
- Avoid including generated session paths from an individual computer.

MATLAB code blocks should use:

```
.. code-block:: matlab

inputSet = Inputs.InputSet(...);
```

Shell commands should use:

```
.. code-block:: bash

git status
```

Links

Use standard reStructuredText links:

```
`OpenSTREAM repository
<https://github.com/OpenSTREAM-solvers/openstream>`_
```

Do not paste HTML elements such as `<a href=...>` into reStructuredText files.

Prefer internal Sphinx cross-references for pages within the documentation:

```
:doc: `getting-started guide <../Usage/gettingStarted>`
```

Use descriptive link text rather than displaying a long URL.

Headings

Use a consistent heading hierarchy within each file.

A typical hierarchy is:

Page title

=====

Main section

Subsection

~~~~~

**Lower-level section**

AAAAAAAAAAAAAAAAAAAAA

The underline must be at least as long as the heading text.

Avoid adding unnecessary intermediate headings when the main topics can be represented directly as page sections.

## Lists and directives

Leave a blank line before and after lists, code blocks, directives, tables, and other block-level elements.

Indent directive content consistently. For example:

```
.. note::
```

```
    This text belongs to the note directive.
```

For list tables:

```
.. list-table::
   :header-rows: 1
   :widths: 30 70
```

```
   * - Column one
     - Column two
   * - Value one
     - Value two
```

## Cross-references and navigation

When adding a new page:

1. Add the source file under the appropriate documentation folder.
2. Add the page to the relevant `toctree`.
3. Add cross-references from related pages where useful.
4. Confirm that the page appears at the intended navigation level.

5. Confirm that the page title and navigation label are appropriate.
6. Avoid duplicating substantial guidance already maintained elsewhere.

Use one authoritative location for detailed guidance and link to it from related pages.

### Citations and bibliographies

Use the configured BibTeX file and Sphinx citation roles for publications.

An inline citation can use:

```
:cite:p:`CitationKey`
```

To display a specific full reference in a page:

```
.. bibliography::  
   :filter: False  
  
   CitationKey
```

Confirm that:

- The citation key exists in the configured BibTeX file.
- The bibliography is rendered correctly.
- Duplicate citation warnings are understood and resolved.
- The bibliographic information matches the authoritative publication.
- Publication links are valid.

### Images and figures

Images should:

- Contribute directly to the explanation.
- Be legible at the rendered page width.
- Include meaningful alternative text where supported.
- Use descriptive filenames.
- Be stored in the appropriate static-image folder.
- Have documented origin and reuse permission.
- Avoid including confidential, proprietary, or copyrighted material without permission.

Figure captions should identify the quantities, conditions, relevant model configuration, and source where appropriate.

Generated figures included in the documentation should be reproducible from a maintained script, tutorial, test, or project workflow whenever practical.

## 7.3.5 Build the documentation

A quick editor preview is useful for basic reStructuredText formatting, but a complete Sphinx build is required before submitting a documentation change.

### Quick preview

When using Sublime Text with OmniMarkupPreviewer, preview the current document using `Ctrl+Alt+O` on Windows or Linux, or `Cmd+Option+O` on macOS.

Alternatively, open the Command Palette and select **OmniMarkupPreviewer: Preview Current Markup in Browser**.

This preview does not process the complete Sphinx configuration, `toctree` entries, API references, citations, or all Sphinx directives.

### Full HTML build

Build the complete HTML documentation using the repository's established Sphinx build command.

When invoking `sphinx-build` directly, the general command structure is:

```
sphinx-build -b html source build/html
```

Review the actual source and output paths configured by the repository before running the command.

A clean build is useful after adding, renaming, or removing pages:

```
sphinx-build -E -a -b html source build/html
```

The `-E` option rebuilds the Sphinx environment, and `-a` writes all output files.

Open the generated HTML entry page in a browser and review the affected pages in the complete site context.

### PDF build

When a change affects content included in the printable documentation, build and review the PDF documentation using the repository's established PDF build procedure.

The PDF build can expose issues not visible in HTML, including:

- Overwide tables.
- Long code lines.
- Poor page breaks.
- Misplaced figures.
- Citation or bibliography placement.
- Heading and navigation inconsistencies.
- Unsupported interactive content.

A successful HTML build does not guarantee a successful or readable PDF build.

### Warnings

Review all Sphinx warnings.

Do not ignore a warning solely because the generated page appears acceptable. Warnings can indicate:

- Broken cross-references.
- Duplicate labels.
- Missing files.
- Missing citations.
- Invalid directives.

- Documents not included in a toctree.
- Malformed tables.
- Inconsistent indentation.
- Unsupported markup.

If an existing warning is unrelated to the contribution, confirm that the change does not introduce additional warnings and document the existing condition in the pull request when relevant.

### 7.3.6 Documentation-maintenance checklist

Use the following checklist when adding or revising OpenSTREAM documentation.

#### Source review

- Confirm that the change is made in the authoritative source file.
- Confirm that generated HTML, LaTeX, or PDF files are not edited directly.
- Review related pages for duplicated, conflicting, or outdated information.
- Use one authoritative location for detailed guidance and add cross-references elsewhere.
- Confirm that terminology and capitalization match the current OpenSTREAM conventions.
- Confirm that solver, model, property, method, and option names match the current source code.
- Confirm that filenames, folder names, and repository paths are current.
- Remove copied interface markup, malformed links, and obsolete comments.

#### Technical accuracy

- Confirm that capability statements match the current implementation.
- Distinguish software capability from verification and validation.
- State assumptions and limitations where they affect interpretation.
- Confirm that physical-model descriptions match the corresponding theory and implementation.
- Confirm that default and nondefault settings are identified correctly.
- Confirm that units are correct and consistently expressed.
- Use kelvin for absolute temperatures.
- Distinguish direct measurements, derived quantities, and calculated quantities.
- Avoid implying general validation from a limited set of comparisons.
- Confirm that publication companion workflows are not presented as complete records of the underlying research activities.

#### Code and command review

- Confirm that MATLAB examples follow the current public interface.
- Confirm that command-line examples use valid syntax.
- Confirm that paths are portable and do not identify a local computer or network location.
- Use `fullfile` in MATLAB path-construction examples.
- Avoid unreviewed `git add .` examples.

- Confirm that generated outputs, session directories, and local sandbox files are not presented as maintained repository content.
- Run representative examples when practical.
- Confirm that example output is consistent with the documented command.

### Structure and navigation

- Confirm that heading levels are consistent.
- Confirm that heading underlines are long enough.
- Confirm that new pages are included in the correct `toctree`.
- Confirm that renamed or moved pages have updated cross-references.
- Confirm that navigation labels are concise and unambiguous.
- Check that no page is unintentionally orphaned.
- Confirm that internal `:doc:` and `:ref:` links resolve.
- Confirm that page titles are consistent with existing navigation and do not change unnecessarily.

### Links and citations

- Check all added or modified external links.
- Use standard reStructuredText links rather than embedded HTML.
- Confirm that internal links use appropriate Sphinx cross-references.
- Confirm that BibTeX keys exist.
- Confirm that citations render in both HTML and PDF where applicable.
- Review duplicate-citation and duplicate-label warnings.
- Confirm that bibliographic details match the authoritative source.
- Confirm that publication PDFs and other copyrighted materials are linked or cited appropriately and are not redistributed without permission.

### Figures and tables

- Confirm that figures are legible in the HTML build.
- Confirm that figures are legible and correctly placed in the PDF build.
- Check figure captions, labels, units, and source attribution.
- Confirm that generated figures can be reproduced from a maintained workflow where practical.
- Check that tables fit within the rendered page width.
- Review list-table column widths in both HTML and PDF.
- Confirm that long code lines and equations do not extend beyond the printable page.
- Confirm that static exports do not imply preservation of unsupported interactive behavior.

### HTML review

- Build the complete HTML documentation.
- Review all modified pages in a browser.
- Check the navigation hierarchy and previous or next page links.
- Check headings, lists, notes, tables, code blocks, figures, and citations.
- Review pages at a narrow browser width for avoidable layout problems.
- Confirm that external links open the intended destination.
- Confirm that code can be copied without interface-generated markup.
- Confirm that no local file paths or temporary build information appear in the rendered pages.

### PDF review

- Build the complete PDF documentation when the changed content is included in the printable version.
- Review the table of contents and section hierarchy.
- Check page breaks, table widths, figure placement, and code wrapping.
- Confirm that citations and bibliography entries are present.
- Confirm that links and cross-references are understandable in print.
- Confirm that interactive-only content is described appropriately.
- Check that no blank or nearly blank pages are introduced unnecessarily.

### Build diagnostics

- Review the complete Sphinx output.
- Resolve warnings introduced by the change.
- Confirm that the build does not rely on untracked or ignored local files.
- Perform a clean build after adding, renaming, or deleting pages.
- Confirm that generated API documentation remains synchronized with the source code.
- Confirm that the documentation build works from a clean repository checkout when practical.

### Final repository review

- Review `git status` before committing.
- Review the staged diff.
- Confirm that only intended source files are staged.
- Confirm that generated build directories are not staged.
- Confirm that the local virtual environment is not staged.
- Confirm that temporary files, editor backups, and MATLAB autosave files are not staged.
- Confirm that source PDFs or copyrighted publications are not included unintentionally.
- Organize documentation and unrelated functional changes into focused commits where practical.
- Follow the [OpenSTREAM commit message style guide](#).

### 7.3.7 Submitting documentation changes

Documentation contributions follow the same fork, branch, commit, and pull request workflow as code contributions.

Before committing:

```
git status
git diff
git diff --cached
```

Stage only the intended source files:

```
git add <documentation-files>
```

Commit messages must follow the [OpenSTREAM commit message style guide](#).

The pull-request description should identify:

- The purpose of the documentation change.
- The pages added, removed, or revised.
- Related code, interface, solver, model, or workflow changes.
- The HTML and PDF builds performed.
- Sphinx warnings introduced or resolved.
- Links, citations, tutorials, or examples reviewed.
- Any remaining documentation limitations or follow-up work.

Documentation-only changes should normally use the docs commit type as described in the OpenSTREAM commit-message guidance.

### 7.3.8 Continuous integration

The documentation build can be executed automatically when changes are pushed to the repository.

After pushing a documentation change:

1. Open the corresponding GitHub Actions run.
2. Confirm that the documentation workflow completes successfully.
3. Review the complete diagnostic output.
4. Correct warnings or errors introduced by the change.
5. Confirm that the deployed documentation corresponds to the intended branch and source revision.

The documentation build and MATLAB continuous-integration workflow are separate. A successful documentation build does not indicate that the MATLAB tests passed, and a successful MATLAB test run does not indicate that the documentation built correctly.

When a contribution changes both code and documentation, review both workflow results.

### 7.3.9 Reporting documentation issues

Use the [OpenSTREAM issue tracker](#) to report documentation errors, outdated examples, missing explanations, broken links, build failures, or navigation problems.

A documentation issue should include, where applicable:

- The page title and link.

- A description of the problem.
- The expected correction or clarification.
- The relevant OpenSTREAM version or commit.
- A screenshot or copied

## 7.4 OpenSTREAM FAQ

This page provides concise answers to common questions about the purpose, distinctive features, capabilities, assumptions, simplifications, and limitations of **OpenSTREAM**. It also provides guidance on installation, configuration, solver selection, simulation setup, numerical convergence, result interpretation, post-processing, troubleshooting, model development, and contribution to the project.

Select a question to display its answer.

### 7.4.1 Getting started

#### What is OpenSTREAM?

**OpenSTREAM**, short for *Open Solvers for Two-phase flow Research, Engineering Analysis and Modeling*, is an open-source, object-oriented MATLAB environment for simulating one-dimensional, multi-field, liquid-vapor two-phase flows with phase change in straight channels.

OpenSTREAM includes four solver frameworks:

- a mixture solver with thermal non-equilibrium capabilities;
- a two-fluid solver that represents the liquid and vapor phases separately;
- a three-field solver for annular two-phase flow;
- a four-field solver that separates the liquid film into base-film and disturbance-wave fields.

Depending on the selected solver and closure models, OpenSTREAM can represent wall boiling, interfacial evaporation and condensation, and the associated exchanges of mass, momentum, and energy between phases and fields.

The solver frameworks provide different levels of physical detail and support education, model development, performance evaluation, and validation.

#### Why should I use OpenSTREAM instead of another available code?

OpenSTREAM is not intended as a replacement for established thermal-hydraulic codes. It is designed primarily as an open, transparent, and extensible environment for education, fundamental model development, closure-model implementation, controlled numerical experiments, and validation of one-dimensional two-phase flow formulations.

OpenSTREAM may be particularly useful when you need:

- **Full access to the source code.** The governing equations, closure models, numerical algorithms, input processing, and post-processing methods can be inspected and modified directly.
- **A transparent modeling environment.** Primary variables, secondary quantities, source terms, field exchanges, and convergence histories are accessible through the MATLAB solver objects. This facilitates interpretation, debugging, and conservation checks.
- **Several solver frameworks within a common architecture.** OpenSTREAM includes mixture, two-fluid, three-field, and four-field formulations. The same geometry, boundary conditions, fluid-property interface, and general workflow can therefore be used to compare different levels of physical resolution.

- **State-of-the-art annular-flow modeling.** The three-field solver separates the liquid into wall-film and entrained-droplet fields. The four-field solver further separates the liquid film into base-film and disturbance-wave fields and includes wave-frequency transport.
- **A practical platform for closure-model development.** Physical models are selected through documented input enumerations and implemented in dedicated phase or field classes. A developer can add, modify, test, and compare closure models without working within a large proprietary code base.
- **Rapid prototyping in MATLAB.** MATLAB provides an interactive environment for inspecting objects, modifying equations, visualizing results, performing parameter studies, and developing new models.
- **Reproducible research and education.** Input sets, tutorials, documentation, references, and source code can be distributed together, allowing users to inspect the assumptions behind a calculation and reproduce the workflow.
- **A computationally efficient one-dimensional model.** OpenSTREAM is useful when cross-section-averaged axial behavior is sufficient and the computational expense and geometric detail of a multidimensional CFD calculation are not required.

Other codes may be more appropriate when the application requires:

- qualified or extensively validated models for a specific industrial application;
- complete reactor-system or plant-network simulation;
- closed-loop systems, including natural-circulation loops;
- simultaneous solution of multiple connected flow paths and their flow distribution;
- dedicated component models for pumps, valves, vessels, separators, or heat exchangers;
- multidimensional flow resolution;
- complex geometries, crossflow, or connected flow networks;
- compressible pressure-wave dynamics;
- multi-component fluids or non-condensable gases;
- safety or licensing analysis.

The appropriate code therefore depends on the purpose of the analysis. OpenSTREAM is especially well suited to understanding, developing, and evaluating models in a transparent one-dimensional framework. A more established application code may be preferable when a broad component library, an application-specific validation basis, or a qualified engineering workflow is required.

Selecting OpenSTREAM does not remove the need for numerical verification, closure-model assessment, applicability review, and validation against appropriate reference data.

### What is unique about OpenSTREAM, and what are its main strengths?

OpenSTREAM combines advanced one-dimensional two-phase flow models with an open, modern, and extensible software architecture. Its purpose is not only to perform simulations, but also to make the governing equations, closure models, numerical methods, and calculated source terms accessible for inspection, modification, and collaborative development.

Its main distinctive features and strengths include:

- **Advanced models rarely available in open-source thermal-hydraulic codes.** To the best of the developers' knowledge, OpenSTREAM provides the only openly available implementations of its four-field annular-flow model and Mixture Relaxation Model within a general two-phase flow simulation environment.

The four-field model explicitly represents vapor, entrained droplets, the base liquid film, and disturbance waves. It also includes transport of wave frequency, allowing the evolution of disturbance-wave behavior to be studied.

The model therefore provides a level of annular-flow detail that is not commonly available in one-dimensional thermal-hydraulic codes.

The Mixture Relaxation Model extends the conventional mixture formulation by solving additional vapor mass and energy conservation equations. It provides a computationally efficient framework for representing thermal non-equilibrium through relaxation of interfacial mass and energy transfer.

- **Several solver frameworks within one consistent environment.** OpenSTREAM includes mixture, two-fluid, three-field, and four-field solvers. These frameworks share the same general input system, geometry definition, fluid-property interface, session management, and post-processing approach.

This common structure allows users to compare different levels of physical resolution without moving between unrelated codes or reconstructing the complete simulation workflow.

- **A modern object-oriented architecture.** OpenSTREAM is implemented using MATLAB classes that represent solvers, phases, fields, inputs, fluid properties, sessions, and plotting utilities. Shared behavior is organized through inheritance and common base classes, while solver-specific behavior is implemented in dedicated modules.

This architecture supports modular development and helps isolate changes to a particular physical model, field, or solver. It also makes primary variables, secondary quantities, source terms, and solver methods directly accessible through the corresponding objects.

- **Direct access to primary and secondary variables.** Users can inspect the primary solution variables, such as mass flow rate, pressure, velocity, and enthalpy, together with derived quantities such as void fraction, film thickness, interfacial area, flow regime, and disturbance-wave frequency, where applicable to the selected solver.

The stored pseudo-time convergence histories also allow users to examine how the steady-state solution was obtained. This level of access is useful for interpretation, debugging, numerical diagnostics, and verification.

- **Direct access to field exchange terms.** The mass, momentum, and energy exchange terms calculated by the selected solver are readily accessible through the corresponding phase and field objects.

This includes wall and interfacial transfer terms, such as evaporation, condensation, entrainment, deposition, drag, shear, and heat transfer, where applicable to the selected solver. Access to the individual contributions facilitates interpretation of the governing equations, conservation checks, closure-model evaluation, and debugging.

- **Direct access to pressure-drop contributions.** The individual pressure-drop components calculated by the mixture solver are readily available for inspection and post-processing.

These contributions can be examined separately to determine the relative effects of gravity, wall friction, spatial acceleration, temporal acceleration, and local pressure losses, as applicable to the simulated case. This decomposition helps users understand the physical origin of the calculated pressure distribution.

- **Integrated plotting and animation capabilities.** Each solver provides dedicated methods for visualizing its principal variables and calculated exchange terms. Axial distributions, time histories, and, where implemented, time-elevation distributions can be generated directly from the solver object.

The plotting methods support solver-specific quantities, selected axial and time indices, multiple plot arrangements, and animation of transient results. This allows users to inspect a calculation interactively without first developing separate post-processing routines.

- **Designed for model development and comparison.** Physical models are selected through documented input options and implemented in dedicated phase or field classes. New closure models can therefore be introduced, verified, and compared with existing models without modifying an opaque or monolithic code base.

The shared solver architecture is particularly useful for studying how assumptions made at the mixture, phase, or field level influence the calculated results.

- **Comprehensive and integrated documentation.** The OpenSTREAM documentation includes installation instructions, theory descriptions, governing equations, notation, a glossary, package and class references, tutorials, publications, contribution guidance, and this FAQ.

Solver-specific MATLAB Live Scripts provide executable examples that users can inspect and modify. The documentation is generated from the same repository as the source code, helping to keep implementation details and user guidance together.

- **Interactive MATLAB workflow.** MATLAB provides an environment in which users can run a case, inspect solver objects, modify models, visualize intermediate quantities, perform parameter studies, and develop post-processing routines interactively.

MATLAB Live Scripts combine formatted explanations, executable code, figures, and calculated output in a single document, making them useful for education, demonstrations, and reproducible technical studies.

- **Open and modifiable source code.** OpenSTREAM is distributed under the MIT License. Users can inspect, use, modify, and redistribute the code in accordance with the license terms.

Open development enables independent scrutiny of the modeling basis and supports collaboration among researchers and institutions. The source code, documentation, input cases, and references can be considered together when evaluating a model.

- **Computational efficiency.** The one-dimensional formulation and streamlined numerical methods provide a computationally efficient framework for steady-state and transient calculations.

This efficiency facilitates rapid model development, repeated calculations, closure-model comparisons, parameter studies, sensitivity analyses, and validation exercises.

These strengths do not mean that OpenSTREAM is more appropriate than other thermal-hydraulic codes. Established system, subchannel, and CFD codes may provide broader component libraries, more complex geometries, multidimensional resolution, or a more extensive validation basis for particular applications.

OpenSTREAM is most distinctive when transparency, access to the governing models, advanced annular-flow modeling, thermal non-equilibrium model development, and controlled comparison of solver frameworks are central to the objective of the study.

### What should I read first?

A suggested learning path is:

1. Follow the installation instructions in the Usage section.
2. Run **Tutorial #1: OpenSTREAM Quick Start**.
3. Continue with the solver-specific tutorial that matches your needs:
  - Tutorial #2 for the mixture solver;
  - Tutorial #3 for the two-fluid solver;
  - Tutorial #4 for the three-field solver;
  - Tutorial #5 for the four-field solver.
4. Consult the theory pages for governing equations and assumptions.
5. Use the package reference to inspect model options, object properties, and methods.

### How do I make OpenSTREAM available in MATLAB?

Add the OpenSTREAM root folder to the MATLAB search path. For example, when running a tutorial from the tutorials folder:

```
osp = './..';  
addpath(osp);
```

If MATLAB cannot find a class such as `InputSet` or `MixtureSolver`, verify that the OpenSTREAM root folder is correct and present on the MATLAB path.

You can inspect the resolved location of a class with:

```
which Inputs.InputSet  
which Solvers.Mixture.MixtureSolver
```

### Why does OpenSTREAM require Python?

OpenSTREAM uses the Python version of **CoolProp** to calculate fluid thermophysical properties. `CoolPropWrapper` provides the interface between MATLAB and Python.

MATLAB must therefore use a compatible Python installation. Check the active Python environment with:

```
pyenv
```

You can test the property interface with:

```
cp = CoolPropWrapper.CoolPropWrapper();
```

If Python or CoolProp cannot be loaded, follow the MATLAB-Python compatibility and CoolPropWrapper setup instructions in the installation section.

### Which MATLAB and Python versions should I use?

Use one of the MATLAB/Python combinations listed in the tested compatibility table. Other combinations may work but are not necessarily exercised by the continuous-integration workflow.

When reproducing or reporting a result, record the MATLAB release with:

```
version('-release')
```

Also record the Python and CoolProp versions when fluid-property behavior is relevant.

### Which CoolProp version should I install?

The CI-tested environment currently installs CoolProp==8.0.0 and nanobind. However, other CoolProp versions may work.

### Where can I find working examples?

The `tutorials` folder contains MATLAB Live Scripts that can be run and modified directly. The documentation also provides completed HTML versions for reference.

Start with **Tutorial #1: OpenSTREAM Quick Start**, then continue with the solver-specific tutorial that matches your application.

## 7.4.2 Running simulations

### What is the minimum workflow for running a case?

A basic mixture-solver calculation consists of four steps:

1. import the required packages;
2. construct an `InputSet`;
3. construct the solver;
4. call `solve`.

For example:

```
import Inputs.*
import Solvers.*
import Solvers.Mixture.*

% Load the inputSet.
inputSet = InputSet( ...
    modelFilePath      = './inputs/models.inp', ...
    modelID            = 'TUTORIAL1', ...
    optionsFilePath    = './inputs/options.inp', ...
    optionsID          = 'DEFAULT', ...
    geometryFilePath   = './inputs/geom.inp', ...
    geometryID         = 'TUTORIAL1', ...
    bcFilePath         = './inputs/tutorial1.inp', ...
    sessionParentDir   = fullfile(pwd, 'outputs'), ...
    overwriteSessionFiles = true, ...
    LOGMODE            = 'BOTH');

% Create and solve the three-field solver.
mixSolver = MixtureSolver(inputSet);
mixSolver.solve();

% Generate defaults axial distribution plots
mixSolver.plotz();
```

### How are OpenSTREAM inputs organized?

An OpenSTREAM case is assembled from four separate input categories. Each category is normally defined in a dedicated input file:

- **Model inputs** select the physical models and closure models.
- **Numerical options** control time steps, iteration limits, relaxation factors, and convergence criteria.
- **Geometry inputs** define the channel length, cross-sectional flow area, wall perimeters, and orientation.
- **Boundary conditions** define the system pressure, inlet enthalpy, inlet mass flow rate, total power, and axial power distribution.

The paths to the four input files are provided separately when constructing an `InputSet`, for example:

```
inputSet = InputSet( ...
    modelFilePath      = './inputs/models.inp', ...
    modelID            = 'TUTORIAL1', ...
```

(continues on next page)

(continued from previous page)

```
optionsFilePath = './inputs/options.inp', ...
optionsID       = 'DEFAULT', ...
geometryFilePath = './inputs/geom.inp', ...
geometryID      = 'TUTORIAL1', ...
bcFilePath      = './inputs/tutorial1.inp');
```

The model, numerical-option, and geometry files may each contain several input sets. An identifier such as TUTORIAL1 or DEFAULT selects the required set from the corresponding file.

A boundary-condition file may contain the information required for a particular steady-state or transient case. Depending on how the study is organized, separate boundary-condition files can be used for different operating conditions or transients.

This separation allows physical models, numerical options, geometries, and boundary conditions to be combined and reused without duplicating complete case definitions. For example, one geometry can be evaluated using several physical-model sets, or one physical-model set can be applied to several boundary-condition cases.

### Can I specify inlet temperature instead of inlet enthalpy?

No. The boundary-condition input interface requires the inlet specific enthalpy,  $H_{IN}$ , in J/kg. Direct specification of inlet temperature is not supported and is not currently planned.

Specific enthalpy provides a more general definition of the inlet thermodynamic state. At a specified pressure, temperature can define a single-phase subcooled-liquid or superheated-vapor state, but it cannot uniquely define a saturated two-phase state. At saturation, the liquid, vapor, and all intermediate two-phase states have the same saturation temperature but different specific enthalpies.

In contrast, inlet specific enthalpy can represent:

- a subcooled-liquid inlet;
- a saturated-liquid inlet;
- a saturated two-phase inlet with a specified vapor quality;
- a saturated-vapor inlet;
- a superheated-vapor inlet.

If the inlet condition is known as temperature and pressure for a single-phase state, first calculate the corresponding specific enthalpy for the selected fluid. Enter the resulting value as  $H_{IN}$  in the boundary-condition file:

```
PRESSURE    ! System pressure [Pa]           > ...
HIN         ! Inlet specific enthalpy [J/kg] > ...
```

For a saturated two-phase inlet, determine  $H_{IN}$  from the system pressure and the required inlet vapor quality. A temperature-only input would not contain enough information to distinguish among saturated liquid, a two-phase mixture, and saturated vapor.

### What input-file formats are supported?

OpenSTREAM supports its native .inp format and the JSON format. The examples and tutorials generally use the native .inp format.

### What happens if I do not specify a model or numerical option?

Unspecified entries use the default values defined by the corresponding input class, principally `Inputs.Model` and `Inputs.Options`.

Defaults make basic cases easier to configure, but they should not be treated as universally appropriate. For research or validation work, review and document all influential physical and numerical selections.

### Why does MATLAB print many default-value warnings?

The warnings identify properties that were not explicitly specified and therefore use their class defaults.

These messages are useful while developing or reviewing a case. Tutorials may temporarily suppress them to keep the displayed output concise:

```
warning off
inputSet = InputSet(...);
warning on
```

Do not suppress warnings routinely until the configuration has been reviewed. An unexpected warning can reveal an omitted or unsupported input.

### Why does OpenSTREAM report that an input entry was not used?

Common causes include:

- a misspelled keyword;
- a keyword that does not belong to the selected input class;
- an obsolete keyword;
- an entry intended for another model configuration.

Check the keyword against `Inputs.Model`, `Inputs.Options`, `Inputs.Geometry`, `Inputs.BoundaryConditions`, or the relevant package-reference page.

An entry reported as unused is not applied to the corresponding input object. Whether this affects the simulation depends on whether the corresponding valid property is used by the selected solver and model configuration. If it is required, OpenSTREAM may instead use the default value, and the simulation may differ from what was intended. If the property is not used by the selected solver or active models, the unused entry may have no effect on the results.

### How do I define a steady-state calculation?

A boundary-condition file containing a single time entry defines a steady-state case.

The solver obtains the steady-state solution through pseudo-time advancement. The pseudo-time sequence is a numerical procedure used to approach a stationary solution and should not be interpreted as a physical transient.

### How do I define a transient calculation?

Provide boundary-condition values at multiple physical times. Depending on the boundary-condition interface, pressure, inlet enthalpy, inlet mass flow, power, and axial power distribution can vary with time.

The solver first calculates the initial steady state. The physical transient proceeds only after the initial solution has converged.

The physical time step is controlled through the numerical options. A time-step sensitivity study is recommended when transient timing or peak values are important.

### Why is the mixture solver run before the other solvers?

The two-fluid, three-field, and four-field solvers require a solved mixture solution for initialization. They also reuse information from the mixture solution, including the pressure-gradient solution.

A solved mixture solver can be passed explicitly:

```
mixSolver = MixtureSolver(inputSet);
mixSolver.solve();

twfSolver = TwoFluidSolver(inputSet, mixSolver);
tfSolver = ThreeFieldSolver(inputSet, mixSolver);
ffSolver = FourFieldSolver(inputSet, mixSolver);
```

If a mixture solver is not supplied, the advanced solver constructor creates and solves one when required. Passing an existing solved object avoids repeating the same mixture calculation.

### Can I call solve more than once on the same solver object?

A solver object represents a specific initialized calculation. Calling `solve` again on an already solved object produces an error stating that the solver must be reinitialized.

For a new calculation, construct a new `InputSet` and solver object. This keeps each simulation configuration and its stored results clearly separated.

### How do I reduce the amount of information printed in the MATLAB Command Window?

Set `LOGMODE` to `'LOGTOFILEONLY'` when constructing the `InputSet`:

```
inputSet = InputSet( ...
    ...
    LOGMODE = 'LOGTOFILEONLY');
```

Solver progress remains available in the session log while most Command Window output is suppressed.

## 7.4.3 Choosing a solver

### Which solver should I use?

The appropriate solver depends on the physical phenomena and level of detail required.

#### Mixture solver

Use for robust initialization, efficient calculations with mixture quantities, hydrodynamically coupled phases, and mixture-based thermal non-equilibrium models.

#### Two-fluid solver

Use when separate liquid and vapor mass, momentum, and energy behavior is needed. The solver can represent different phase velocities, enthalpies, and temperatures.

#### Three-field solver

Use for annular two-phase flow when the liquid distribution between a wall film and entrained droplets is important.

#### Four-field solver

Use for annular two-phase flow when the liquid film must be separated into base-film and disturbance-wave fields and wave transport is part of the analysis.

Increasing the number of fields does not automatically make a simulation more accurate. The selected solver assumptions and closure models must be appropriate for the application.

### What is the difference between a phase and a field?

A **phase** is a thermodynamic state of matter, such as liquid or vapor.

A **field** is a computationally resolved constituent with its own transported variables or conservation equations. One phase can be represented by more than one field. For example:

- The three-field solver represents the liquid phase through a wall-film field and a droplet field.
- The four-field solver further divides the wall film into a base-film field and a disturbance-wave field.

### Are the three-field and four-field solvers thermal non-equilibrium models?

The current three-field and four-field formulations assume thermal equilibrium. Their additional detail concerns the hydrodynamic distribution and transport of liquid fields.

Use the mixture relaxation model or the two-fluid solver when separate thermal behavior is required, subject to the assumptions and implemented closure models of the selected framework.

### Where are the three-field and four-field equations applied?

Although the separate annular-flow field equations are numerically solved from the channel inlet, their results are physically meaningful only from the onset of annular flow. Upstream of this location, the conservation equations are formulated so that the annular-flow field variables evolve smoothly toward the prescribed conditions at the onset of annular flow.

Solving the field equations upstream of the onset of annular flow ensures that preceding-time-step values are available for all required field variables if the onset location moves upstream during a transient calculation.

The onset-of-annular-flow model, the initial film-droplet split, and, for the four-field solver, the initial base-film and disturbance-wave split can therefore influence the solution near the transition.

### Why can results near the onset of annular flow be sensitive?

The initial liquid distribution among film, droplets, base film, and waves depends on the selected onset and field-splitting models. Entrainment and deposition then redistribute liquid as the solution develops downstream.

When the region near annular-flow onset is important, examine sensitivity to:

- the onset-of-annular-flow model;
- the initial entrained-droplet fraction;
- the initial base-film and wave split;
- entrainment and deposition models;
- spatial resolution.

### Should entrainment and deposition models be selected together?

Yes. Entrainment and deposition are coupled processes that exchange liquid between the film and droplet fields. The corresponding models are generally developed and calibrated together to provide a consistent representation of this exchange.

When available, use entrainment and deposition models from the same model family unless there is a documented justification for combining models from different families.

## 7.4.4 Numerical convergence

### What is the difference between point convergence and steady-state convergence?

At each axial node and time step, OpenSTREAM performs local point iterations for the nonlinear equations. **Point convergence** refers to the change in solved variables between these local iterations.

For a steady-state calculation, OpenSTREAM also compares solutions between successive pseudo-time steps. **Steady-state convergence** refers to these temporal changes becoming smaller than the specified steady-state criteria.

A calculation therefore needs adequately converged local iterations and an adequately converged pseudo-time solution.

### What does solveMode='NULL' mean in plotting methods?

The NULL solve mode displays the pseudo-time history used to obtain the initial steady-state solution. It does not represent the physical transient.

For example:

```
solver.plott( ...  
    solver.NZ, ...  
    'display', {'W','U'}, ...  
    'solveMode', 'NULL');
```

The default REAL mode is used to display the stored physical solution.

### What should I do if the steady-state calculation does not converge?

Start with the information printed by the solver:

- identify the node with the largest point-iteration count;
- identify which variable has the largest residual;
- inspect the final solver state;
- plot the pseudo-time history with solveMode='NULL';
- check model, geometry, and boundary-condition warnings;
- verify that input values and units are physically reasonable.

Numerical options that may affect convergence include:

- maximum point iterations;
- maximum steady-state pseudo-time iterations;
- pseudo-time step;
- variable-specific relaxation factors;
- pointwise convergence criteria;
- steady-state convergence criteria.

Change one numerical setting at a time and document each modification. After convergence is achieved, check that the converged solution is not excessively sensitive to the selected relaxation factors, iteration limits, pseudo-time step, or convergence criteria. Do not obtain apparent convergence merely by relaxing the convergence criteria without verifying the resulting residuals and physical solution.

### Why was my transient skipped?

The physical transient is not calculated when the initial steady-state solution fails to converge. The solver log indicates that the transient is being skipped.

Inspect the initial pseudo-time solution, correct any input or model problems, and obtain a converged initial state before interpreting transient results.

### Can a converged calculation still be inaccurate?

Yes. Numerical convergence indicates that the implemented discrete equations have been solved to the selected tolerances. It does not establish that:

- the spatial mesh is sufficiently refined;
- the physical time step is sufficiently small;
- the selected closure models are valid for the conditions;
- the one-dimensional assumptions are appropriate;
- the model has been validated for the application.

Mesh sensitivity, time-step sensitivity, conservation checks, and comparison with appropriate reference data remain necessary.

### Is OpenSTREAM limited in terms of mesh size or time-step size?

OpenSTREAM does not impose a hard-coded maximum number of axial nodes, `NNODES`, or a predefined set of allowable physical time-step values, `TSTEP`. Within the applicable input requirements, users can select the spatial and temporal discretizations appropriate for their calculations.

The absence of hard-coded limits does not mean that any mesh or time step is numerically appropriate. Practical limits arise from available memory, computational effort, convergence behavior, numerical accuracy, and the spatial and temporal scales of the modeled problem.

The axial mesh is defined by `NNODES` in the physical-model inputs. Increasing `NNODES` improves the spatial resolution of the one-dimensional solution but also increases the number of calculated values, memory use, and computational effort.

A coarse axial mesh may adequately represent smooth distributions but can smooth localized variations, shift the predicted location of a transition, or fail to capture a local maximum or minimum. Finer meshes may be required near:

- abrupt changes in wall heat flux;
- local geometric perturbations or pressure losses;
- boiling or flow-regime transitions;
- the onset of annular flow;
- film dryout or regeneration;
- other regions containing strong axial gradients.

The physical time step is defined by `TSTEP` in the numerical-option inputs. OpenSTREAM does not impose a hard-coded minimum or maximum physical time-step size. However, `TSTEP` must be selected according to the duration and shortest relevant time scale of the physical transient.

OpenSTREAM uses fully implicit backward Euler time integration, which provides robust time advancement. Robustness or numerical stability does not, however, guarantee temporal accuracy.

A large physical time step may reproduce the overall transient trend but smooth rapid changes, underestimate or miss peak values, or shift the calculated timing of an event. A smaller time step provides greater temporal resolution but requires more calculated time steps and greater computational effort.

Important prescribed changes in the boundary conditions should also be represented with sufficient temporal resolution. A time step that is adequate for slowly varying quantities may still be too large for the accurate calculation of peak values, peak timing, or short-duration behavior.

The physical time step, `TSTEP`, should not be confused with the steady-state pseudo-time step, `SSTSTEP`. `TSTEP` controls advancement of the physical transient, whereas `SSTSTEP` controls the numerical progression from the solver-assigned initialization state toward the steady-state solution.

A smaller pseudo-time step can make the point iterations easier to converge but may require more pseudo-time steps. A larger pseudo-time step may accelerate progression toward steady state but can make the local nonlinear iterations more difficult or produce less regular convergence.

Mesh and time-step adequacy should be demonstrated through sensitivity studies. Successive spatial or temporal refinements should produce progressively smaller changes in the quantities of interest. The finest mesh or smallest time step used in a study is a reference solution and should not automatically be treated as exact.

### Can OpenSTREAM use a non-uniform axial mesh?

No. The current OpenSTREAM implementation uses a uniform axial mesh.

The user specifies the total channel length, `LENGTH`, and the number of axial nodes, `NNODES`. OpenSTREAM then distributes the nodes uniformly along the channel. The current input interface does not provide an option for specifying individual axial-node locations or variable node lengths.

Consequently, local mesh refinement cannot currently be applied only in selected regions, such as near:

- an abrupt change in wall heat flux;
- a local geometric perturbation or pressure loss;
- boiling or flow-regime transitions;
- the onset of annular flow;
- film dryout or regeneration;
- another region containing a strong axial gradient.

When additional spatial resolution is required, `NNODES` must be increased for the complete channel. This reduces the uniform axial node spacing but also increases the number of calculated values and the computational effort throughout the domain.

The absence of a non-uniform mesh option does not necessarily prevent accurate calculations. A uniform mesh can be adequate when the axial behavior is sufficiently smooth or when enough nodes are used to resolve the shortest relevant axial length scale. Mesh adequacy should be assessed through a spatial-sensitivity study.

Support for user-defined axial-node locations or variable node lengths may be considered in a future development, but it is not currently implemented.

### Which numerical options control steady-state convergence?

Relevant options include the pseudo-time step, the maximum number of steady-state iterations, variable-specific relaxation factors, local point-iteration tolerances, and steady-state temporal convergence criteria.

The exact property names and defaults are listed in `Inputs.Options`. Use the solver-specific tutorials and package reference when changing these values.

## 7.4.5 Results and post-processing

### How do I display results?

The solver classes provide plotting methods for common result types.

#### **plotz**

Plots axial distributions. Multiple physical time indices can produce an animated axial plot.

#### **plott**

Plots time histories at a selected axial location.

#### **plotzt**

Plots time-elevation distributions where implemented.

For example:

```
mixSolver.plotz( ...
    'display', {'W','VR'}, ...
    'arrangement', 'horizontal', ...
    'resize', 1);
```

Available display names depend on the selected solver. Consult the corresponding solver class in the package reference.

### Can I access results without using the built-in plots?

Yes. Solver objects retain their solution objects and arrays. Examples include:

- mixture variables in `mixSolver.mixture`;
- liquid and vapor variables in `twfSolver.liquid` and `twfSolver.vapor`;
- film and droplet variables in `tfSolver.film` and `tfSolver.drop`;
- base-film and wave variables through the four-field film object.

The package reference lists the available properties and methods. Check property dimensions before processing because arrays may vary over axial position, time, and wall index.

### How do I save a solver object?

Use the solver save method:

```
mixSolver.save( ...
    'name', 'mixSolver', ...
    'showpath', true);
```

The solver object contains the imported inputs, solution objects, and associated simulation information. Saving it allows later post-processing without repeating the calculation.

### How do I access the base-film and disturbance-wave objects in a four-field solution?

In the four-field solver, the base-film and disturbance-wave objects are contained within the liquid-film object. They are therefore accessed through the `film` property of the four-field solver:

```
base = ffSolver.film.base;
wave = ffSolver.film.wave;
```

For a transient calculation, `ffSolver.film` is an array containing one film object for each stored physical time step. A specific time step can therefore be accessed using its time index:

```
tIdx = 1;

base = ffSolver.film(tIdx).base;
wave = ffSolver.film(tIdx).wave;
```

The base-film and disturbance-wave objects contain their separately calculated field quantities. For example:

```
baseMassFlow = ffSolver.film(tIdx).base.W;
baseVelocity = ffSolver.film(tIdx).base.U;

waveMassFlow = ffSolver.film(tIdx).wave.W;
waveVelocity = ffSolver.film(tIdx).wave.U;
waveFrequency = ffSolver.film(tIdx).wave.FREQUENCY;
```

The enclosing `film` object represents the total liquid film and also provides aggregate film quantities derived from the solved base-film and disturbance-wave fields. The droplet field is stored separately in `ffSolver.drop`, while the vapor-field solution is retained through the mixture solver.

The base-film and disturbance-wave objects used during the pseudo-time calculation of the initial steady state are similarly contained in the stored initialization film objects:

```
baseInit = ffSolver.filmInit(tIdx).base;
waveInit = ffSolver.filmInit(tIdx).wave;
```

### What is the minimum base film in the four-field solver?

In the four-field model, disturbance waves pass over a continuous and comparatively slow-moving base liquid film. The calculated base-film mass flow rate represents the condition immediately after a disturbance wave has passed and interacted with the base film.

Between two successive disturbance waves, the base film evolves without direct replenishment from a wave. During this wave-waiting period, wall evaporation removes liquid from the base film, while droplet deposition can add liquid to it. The **minimum base-film mass flow rate** represents the condition reached at the end of this period, immediately before the next disturbance wave arrives and replenishes the base film.

The minimum base-film mass flow rate is therefore a local, periodically varying quantity associated with the passage of disturbance waves. It should not be interpreted as the minimum value of the calculated base-film mass flow rate over the channel or over the complete transient.

Assuming that the base-film velocity remains constant during the wave-waiting period, the corresponding minimum base-film thickness is calculated from:

$$\delta_{b,\min} = \frac{W_{b,\min}}{\rho_{l,s} u_b \Pi}$$

where:

- $\delta_{b,\min}$  is the minimum base-film thickness;
- $W_{b,\min}$  is the minimum base-film mass flow rate;
- $\rho_{l,s}$  is the saturated-liquid density;
- $u_b$  is the base-film velocity;
- $\Pi$  is the corresponding wall perimeter.

The minimum base film is useful for evaluating intermittent base-film depletion between disturbance waves. If the minimum base-film mass flow rate or thickness reaches zero, the model predicts that the base film becomes locally dry before being reached by the next disturbance wave. This is referred to as **intermittent base-film dryout**.

The minimum base film differs from the equilibrium base film. The **equilibrium base-film thickness** is the target condition used by the base-film and disturbance-wave mass-exchange model, whereas the **minimum base-film thickness** describes the depletion of the base film during the interval between successive waves.

### What does it mean when the calculated liquid-film mass flow rate become negative?

The treatment of negative liquid-film quantities is controlled by the POSFILM physical-model input:

```
POSFILM      ! Positive film flow-rate/thickness model      > TRUE
```

POSFILM is set to TRUE by default. With this setting, the film mass flow rate and film thickness are restricted to non-negative values.

When POSFILM is set to FALSE, OpenSTREAM allows the calculated film mass flow rate and corresponding film thickness to become negative after the film has been completely depleted.

A negative film mass flow rate is not physically meaningful and does not represent liquid flowing in the opposite direction. It is an intentionally extended numerical quantity indicating that the integrated film mass balance has progressed beyond the predicted dryout condition.

The liquid-film mass conservation equation accounts for:

- liquid gain through droplet deposition;
- liquid loss through film entrainment;
- liquid loss through wall evaporation.

When the cumulative film losses exceed the available film inventory, the physical film mass flow rate reaches zero and film dryout occurs. With POSFILM set to FALSE, the film mass-balance equation continues to be integrated beyond this point, producing a negative value.

This continuation provides a signed measure of the margin to dryout:

- a positive film mass flow rate indicates that liquid film remains;
- a value close to zero indicates proximity to complete film dryout;
- a negative value indicates that the imposed conditions exceed the predicted dryout condition.

This behavior is useful in critical-power calculations and model-tuning studies. For example, power or a closure-model coefficient can be varied until the minimum film mass flow rate is sufficiently close to zero. The corresponding condition represents the predicted dryout threshold according to the selected models.

Allowing negative values also improves numerical robustness during parameter studies and power iterations. A trial calculation can continue after exceeding the dryout condition instead of terminating when the film first reaches zero.

Negative film values must be interpreted only as numerical dryout-margin indicators. They must not be interpreted as physical post-dryout film mass flow rates, negative film thicknesses, or reverse liquid-film flow. OpenSTREAM does not use this continuation as a physical post-dryout film model.

### What is the difference between calculated and equilibrium base-film and wave quantities?

The four-field solver distinguishes between the locally calculated base-film thickness and wave frequency and their corresponding equilibrium values.

The **calculated base-film thickness** is obtained from the transported base-film mass flow rate, base-film velocity, liquid density, and wall perimeter. It represents the current local state of the base-film field and may vary with axial position and time.

The **equilibrium base-film thickness** is obtained from the selected equilibrium base-film-thickness model. It represents the base-film thickness toward which the local base-film state tends under the current flow conditions. It is a closure-model target rather than a separately transported field variable.

The difference between the calculated and equilibrium base-film thicknesses is used in the relaxation model governing mass exchange between the base-film and disturbance-wave fields. In simplified form, the net source driving the base film toward equilibrium is proportional to:

$$\frac{\rho_{l,s} (\delta_{b,eq} - \delta_b)}{t_{b,relax}}$$

where:

- $\delta_b$  is the calculated base-film thickness;
- $\delta_{b,eq}$  is the equilibrium base-film thickness;
- $\rho_{l,s}$  is the saturated-liquid density;
- $t_{b,relax}$  is the base-film relaxation time.

If the calculated base-film thickness is smaller than its equilibrium value, the relaxation term tends to transfer liquid from the disturbance waves to the base film. If the calculated base-film thickness is larger than its equilibrium value, the relaxation term tends to transfer liquid from the base film to the disturbance waves.

The complete base-film and wave mass balances also include other mechanisms, such as droplet deposition, wall evaporation, film entrainment, and turbulent mixing. Consequently, the final base-film and wave mass-exchange rates are not determined by the thickness-relaxation term alone.

Similarly, the **calculated wave frequency** characterizes the current local disturbance-wave field. It is related to the transported wave number density and may differ from the frequency corresponding to local equilibrium conditions because of wave transport and hydrodynamic non-equilibrium.

The **equilibrium wave frequency** is obtained from the selected equilibrium wave-frequency model. It represents the wave frequency toward which the local disturbance-wave population tends under the current flow conditions.

The four-field solver transports the wave number density using a Boltzmann-type transport equation. Its source term represents the net effect of disturbance-wave interactions, including wave creation, merging, splitting, and dissipation. Using a relaxation-time approximation, the source term is expressed as:

$$\Phi_w = \frac{N_{w,eq} - N_w}{t_{w,relax}}$$

where:

- $N_w$  is the calculated wave number density;
- $N_{w,eq}$  is the equilibrium wave number density;
- $t_{w,relax}$  is the wave relaxation time;
- $\Phi_w$  is the net wave-number source or sink rate.

Wave frequency is often used as a more practical representation of wave number density. The corresponding frequency form similarly drives the calculated wave frequency toward its equilibrium value.

If the calculated wave frequency is smaller than its equilibrium value, the relaxation source is positive and tends to increase the local wave population. If the calculated wave frequency is larger than its equilibrium value, the source is negative and tends to reduce the local wave population.

The base-film and wave relaxation processes are related but distinct:

- the base-film-thickness relaxation controls mass redistribution between the base-film and disturbance-wave fields;
- the wave-frequency relaxation controls the source term in the wave number-density transport equation and therefore the evolution of the disturbance-wave population.

The equilibrium quantities should therefore be interpreted as local closure-model targets. Differences between calculated and equilibrium values are expected in developing or transient annular flow and represent the hydrodynamic non-equilibrium modeled by the four-field formulation.

### What is wave number density, and why is it transported in the four-field solver?

In the four-field model, disturbance waves are represented as a dispersed field of liquid structures moving over the continuous base film. Rather than tracking every individual disturbance wave, the model describes the wave population using cross-section-averaged macroscopic quantities.

One of these quantities is the **wave number density**,  $N_w$ . It is the number of disturbance waves per unit axial distance and is therefore a spatial measure of the local wave population.

The wave number density is defined as the inverse of the average wave spacing:

$$N_w = \frac{1}{\lambda_w}$$

where  $\lambda_w$  is the average axial distance between two successive disturbance waves.

Wave spacing is related to the wave velocity and wave frequency by:

$$\lambda_w = \frac{u_w}{f_w}$$

Consequently:

$$N_w = \frac{f_w}{u_w}$$

where  $u_w$  is the disturbance-wave velocity and  $f_w$  is the frequency at which disturbance waves pass a fixed axial location.

Wave number density and wave frequency describe related but different quantities. Wave number density describes how many waves are present per unit axial distance, whereas wave frequency describes how many waves pass a fixed location per unit time. Their relationship therefore depends on the wave velocity.

The four-field solver does not assume that the local wave population is always equal to its equilibrium value. Instead, it transports the wave number density using a one-dimensional wave number-density transport equation.

This equation is based on the concept of a Boltzmann transport equation applied to the number density of dispersed fluid particles. In the four-field model, disturbance waves are treated as the dispersed fluid structures. The equation is integrated over the possible wave sizes, so it transports the total wave number density rather than tracking separate populations for each wave size.

The transport equation can be written as:

$$\frac{\partial N_w}{\partial t} + \frac{\partial (u_w N_w)}{\partial z} = \Phi_w$$

The left-hand side represents:

- the local temporal change in wave number density; and
- the axial transport of waves at the disturbance-wave velocity.

The right-hand side,  $\Phi_w$ , represents the net source or sink of disturbance waves caused by interactions such as wave formation, merging, splitting, and dissipation.

In the current model, these interactions are represented using a relaxation-time approximation:

$$\Phi_w = \frac{N_{w,eq} - N_w}{t_{w,relax}}$$

where:

- $N_w$  is the calculated wave number density;
- $N_{w,eq}$  is the equilibrium wave number density obtained from the selected closure model;
- $t_{w,relax}$  is the wave relaxation time.

If the calculated wave number density is smaller than its equilibrium value, the source term is positive and increases the local wave population. This represents a net effect of processes that create or split waves.

If the calculated wave number density is larger than its equilibrium value, the source term is negative and decreases the local wave population. This represents a net effect of processes such as wave merging or dissipation.

The transport equation allows the disturbance-wave population to respond progressively to changing flow conditions instead of changing instantaneously to its local equilibrium value. It therefore represents hydrodynamic non-equilibrium associated with developing annular flow, phase change, inlet effects, geometric perturbations, and transient conditions.

Wave frequency is generally more convenient for interpretation and comparison with measurements. OpenSTREAM therefore commonly expresses and plots the transported wave population in terms of calculated and equilibrium wave frequencies. However, the underlying physical concept remains the transport of the spatial wave population represented by the wave number density.

The wave number-density transport equation is separate from the wave mass and momentum conservation equations:

- the wave mass equation determines how much liquid mass is carried by the disturbance-wave field;
- the wave momentum equation determines the disturbance-wave velocity;
- the wave number-density transport equation determines how the number of disturbance waves evolves.

Together with the wave shape model, these quantities are used to characterize wave spacing, width, amplitude, and other properties of the disturbance-wave field.

### Where are simulation outputs stored?

The `sessionParentDir` argument defines the parent output folder. Each solver uses the session infrastructure to store logs and requested results.

For example:

```
sessionParentDir = fullfile(pwd,'outputs');
```

Use a separate session directory or case name for parameter studies. Be careful with:

```
overwriteSessionFiles = true
```

because existing files for the corresponding session may be replaced.

### Why do my plots show results only from the onset of annular flow?

The three-field and four-field plotting functions can restrict field-specific results to the annular-flow region. This is appropriate because separate film, droplet, base-film, and wave equations apply from the onset of annular flow.

Where supported, set the annular plotting option to `false` to inspect initialization outside that region:

```
solver.plotz( ...  
    'display', 'W', ...  
    'annular', false);
```

Interpret pre-annular field values as initialization quantities rather than as a complete pre-annular multi-field model.

### How do I plot only part of the channel or selected times?

By default, `plotz` includes all axial elevations and all stored physical time steps. Similarly, `plott` includes all stored physical time steps at the selected axial location.

The plotting methods also allow selected axial and time indices to be specified. The same general syntax is used by all OpenSTREAM solvers.

To plot only part of the channel, first identify the corresponding axial indices and pass them using `zIdx`. For example, the following code plots results between elevations of 1 and 4 m:

```
zIdx = find(solver.Z >= 1 & solver.Z <= 4);

solver.plotz( ...
    'display', {'W','U'}, ...
    'zIdx', zIdx, ...
    'arrangement', 'horizontal', ...
    'resize', 1);
```

For a transient calculation, the first positional argument of `plotz` selects one or more stored time indices. For example, the following code plots the first, middle, and final stored states over the selected axial range:

```
tIdx = unique([1, round(solver.NTIME/2), solver.NTIME]);
zIdx = find(solver.Z >= 1 & solver.Z <= 4);

solver.plotz( ...
    tIdx, ...
    'display', {'W','U'}, ...
    'zIdx', zIdx, ...
    'arrangement', 'horizontal', ...
    'resize', 1);
```

When several time indices are selected, `plotz` generates an animated axial plot containing the selected states.

To plot a time history at one axial location, pass the axial index as the first argument of `plott` and select the stored time indices using `tIdx`. For example, the following code plots only the first half of the transient at the axial node closest to 3.5 m:

```
zTarget = 3.5;
[~, zIdx] = min(abs(solver.Z - zTarget));

tIdx = 1:ceil(solver.NTIME/2);

solver.plott( ...
    zIdx, ...
    'display', {'W','U'}, ...
    'tIdx', tIdx, ...
    'arrangement', 'horizontal', ...
    'resize', 1);
```

The same index-selection approach applies to the mixture, two-fluid, three-field, and four-field solvers. However, the available `display` values depend on the selected solver because each solver stores different physical quantities.

### How do I compare several model configurations?

Create one `InputSet` and solver object per model ID, store each object separately, and extract the same comparison quantities from every case.

Use distinct session directories to avoid overwriting results. A compact MATLAB table is useful for comparing scalar metrics, while solver objects can be retained separately for detailed post-processing.

## 7.4.6 Physical scope and limitations

### What are the main simplifications in OpenSTREAM, and why are they considered reasonable?

OpenSTREAM uses several deliberate simplifications to provide a transparent, computationally efficient, and numerically robust environment for developing and evaluating one-dimensional two-phase flow models. These simplifications define the intended scope of the code and must be considered when interpreting its results.

The main simplifications are:

- **One-dimensional, cross-section-averaged flow.** Flow variables vary only along the channel axis. Radial, azimuthal, and other local distributions are not resolved explicitly and must instead be represented through averaged quantities and closure models.

This approximation is reasonable for straight channels when axial transport dominates and detailed multidimensional effects, such as crossflow, flow separation, or strongly asymmetric mixing, are not essential to the quantity being studied. It also makes OpenSTREAM suitable for rapid model development, sensitivity studies, and comparison of closure models.

- **Straight channels with constant cross-sectional area.** The current geometry represents a straight flow path with constant flow area. Multiple wall perimeters and non-uniform wall heating can be defined, and local perturbations can be represented through dedicated models, but continuous geometric variation is not resolved.

This approximation covers many experimental channels and idealized thermal-hydraulic problems, including tubes, annuli, rectangular channels, and simplified small rod-bundle representations. It enables investigation of axial two-phase flow behavior without introducing additional geometric effects.

- **Mixture-based pressure-gradient solution.** The pressure-gradient solution obtained from the mixture solver is reused by the two-fluid, three-field, and four-field solvers rather than independently solving a fully coupled pressure-velocity system in each framework.

This treatment significantly improves numerical stability and provides a consistent pressure field for comparisons among the solver frameworks. It introduces some inconsistency between the pressure solution and the separate field momentum equations, but the available OpenSTREAM publications report that this inconsistency is negligible for many applications. Its acceptability must nevertheless be assessed for each new application.

- **Fluid properties evaluated at the system pressure.** Fluid thermophysical properties are currently evaluated using the prescribed system pressure rather than the calculated local pressure along the channel.

Consequently, an axial pressure decrease does not modify the local saturation properties and cannot, by itself, produce flashing. OpenSTREAM therefore does not currently represent flashing caused by local pressure drop along the channel.

Time-dependent changes in the prescribed system pressure are included in the fluid-property evaluation. OpenSTREAM can therefore represent flashing caused by a decrease in system pressure, subject to the assumptions and capabilities of the selected thermal non-equilibrium and phase-change models.

This simplification is considered reasonable for the high-pressure, relatively short channel applications targeted by the current OpenSTREAM frameworks, when the axial pressure variation is small relative to the system pressure. Under these conditions, the resulting variation in saturation properties along the channel is generally expected to be limited.

The approximation must be reassessed when the channel pressure drop is sufficiently large to affect the local saturation temperature, saturation enthalpies, phase densities, or phase-change behavior. It is not appropriate when pressure-drop-induced flashing is an important part of the physical problem.

- **Neglect of surface-tension forces in the conservation equations.** Surface tension may be used in closure quantities, but explicit surface-tension force contributions are neglected in the governing momentum equations.

This simplification is considered reasonable for the high-pressure channel-flow conditions targeted by the current implementations, where the retained pressure, inertia, gravity, and wall and interfacial momentum-transfer terms are generally more important at the one-dimensional field scale. It may not be appropriate for capillary-dominated flows, very small channels, or problems controlled by interface curvature.

- **Neglect of heating caused by friction.** Mechanical energy dissipated by wall or interfacial friction is not added explicitly to the fluid energy equations.

This contribution is considered small relative to the imposed wall heating and phase-change energy transfer for the high-pressure boiling applications for which the current models were developed. It should not be neglected without assessment in applications involving exceptionally large pressure losses or little external heating.

- **Neglect of temporal pressure-gradient contributions.** The current formulations omit minor energy or momentum contributions associated with the temporal pressure gradient.

This approximation is considered reasonable for the operational transients targeted by the current high-pressure boiling applications. It is not appropriate for rapid pressure-wave, depressurization, water-hammer, choking, or shock-wave problems, which are currently outside the intended scope of OpenSTREAM.

- **Neglect of spatial gradients of saturated-fluid enthalpies.** Spatial changes in saturated liquid and vapor enthalpies associated with the pressure distribution are neglected in the simplified conservation equations.

This approximation is considered reasonable for the high-pressure conditions and operational transients targeted by the current OpenSTREAM frameworks. Its adequacy should be reconsidered for cases with large pressure variations or strong depressurization.

- **Simplified flow-regime and interfacial-topology transitions.** The current two-fluid solver uses simplified assumptions to identify flow regimes and select the corresponding closure models.

This approach provides a practical framework for implementing and testing separate-phase conservation equations, but it limits the physical realism of complex transients involving repeated or strongly evolving changes in interfacial topology. The two-fluid results should therefore be interpreted according to the maturity and validation range of the selected transition and closure models.

- **Thermal equilibrium in the three-field and four-field solvers.** The film, droplet, base-film, disturbance-wave, and vapor fields are assumed to share the applicable saturation thermodynamic state. The solvers resolve hydrodynamic non-equilibrium among the fields but do not solve separate field energy equations for thermal non-equilibrium.

This approximation is reasonable for saturated annular-flow studies in which liquid-field mass and momentum transport, entrainment, deposition, film depletion, and disturbance-wave behavior are the primary phenomena of interest. It is not appropriate when separate field temperatures (e.g., post-dryout thermal non-equilibrium) are essential.

- **First-order numerical discretization.** The conservation equations are solved using first-order upwind spatial discretization and fully implicit backward Euler time integration. Nonlinear source terms and field couplings are treated through fixed-point iterations.

These methods are comparatively simple and numerically robust, which is useful for an open model-development platform and for obtaining steady-state solutions through pseudo-time advancement. Their numerical diffusion and first-order accuracy make mesh- and time-step-sensitivity studies necessary when spatial gradients, transient timing, or peak values are important.

These simplifications are reasonable only within the intended application domain and for quantities that are not controlled by the neglected phenomena. They make the governing equations easier to understand, modify, test, and compare while reducing computational cost and improving numerical robustness.

A simplification must not be interpreted as universally negligible. Before using OpenSTREAM for a new fluid, geometry, flow regime, or transient, assess whether the neglected terms and unresolved phenomena could materially affect the quantities of interest. Numerical convergence, sensitivity studies, conservation checks, and comparison with suitable reference data remain necessary.

### What are the current practical and technical limitations of OpenSTREAM?

OpenSTREAM is under active development. The current implementation is intended for one-dimensional, thermally expandable, single-component, liquid-vapor boiling flows in straight channels. The thermally expandable formulation accounts for changes in fluid properties with the local thermodynamic state, but it is not a general compressible-flow formulation.

The principal current limitations include:

- **No general compressible-flow or pressure-wave formulation.** OpenSTREAM accounts for thermodynamic density changes associated with heating, cooling, system pressure variation, and phase change. This thermally expandable treatment is suitable for the intended boiling-flow applications, but it is not a fully compressible-flow formulation.

In particular, the current implementation is not intended to resolve acoustic-wave propagation, rapid pressure waves, shock waves, expansion waves, choked flow, water hammer, or other phenomena for which compressible pressure-wave dynamics are essential.

- **No wall condensation model.** OpenSTREAM can represent interfacial condensation within the fluid, where supported by the selected solver and closure models, but it does not currently calculate condensation caused by heat transfer to a cooled wall.
- **No counter-current flow.** The current solution algorithms and boundary-condition treatment are intended for co-current flow in a single axial direction.

In addition, OpenSTREAM does not currently support:

- multi-component fluid mixtures;
- non-condensable gases;
- solid-particle flows, including liquid-solid, gas-solid, and liquid-gas-solid flows;
- pressure-drop-induced flashing. Fluid properties are evaluated at the prescribed system pressure rather than the calculated local pressure;
- wall heat conduction or conjugate heat transfer;
- continuously varying channel area;
- bends, junctions, plena, or connected flow networks;
- crossflow between neighboring channels;
- multidimensional flow resolution;
- general-purpose component models such as pumps, valves, tanks, or heat exchangers;
- thermal non-equilibrium in the three-field and four-field solvers;
- fully coupled pressure-velocity solution in the advanced solvers;

Some currently unsupported capabilities may be considered for future development.

### Is OpenSTREAM a CFD code?

No. OpenSTREAM is not a Computational Fluid Dynamics (CFD) code in the conventional engineering sense.

OpenSTREAM solves one-dimensional, cross-section-averaged conservation equations for steady-state and transient two-phase flows. Depending on the selected solver, these equations describe mixture, phase, or field mass, momentum, and energy transport. Physical processes that are not resolved explicitly are represented through closure models.

In contrast, a conventional CFD code discretizes a multidimensional computational domain and resolves spatial variations of the flow within that domain. OpenSTREAM does not resolve radial, azimuthal, or other multidimensional distributions of velocity, pressure, temperature, void fraction, or phase structure.

OpenSTREAM also does not explicitly resolve:

- local velocity and temperature profiles across the channel;
- multidimensional turbulence structures;
- flow separation, recirculation, or secondary flow;
- circumferential transport around the channel;
- crossflow between neighboring channels;
- detailed three-dimensional geometrical effects.

Instead, OpenSTREAM uses cross-section-averaged quantities and closure models for wall friction, interfacial transfer, phase change, entrainment, deposition, and other unresolved processes.

This one-dimensional formulation is intentional. OpenSTREAM was developed to support the formulation, implementation, assessment, and validation of the types of one-dimensional thermal-hydraulic models used in established system codes and subchannel-analysis codes. Its purpose is to provide an open and transparent environment in which these models can be inspected, modified, compared, and extended.

The one-dimensional approach also provides a computationally efficient environment for model development, sensitivity studies, and validation. It is particularly suitable for straight-channel problems in which axial transport is the principal behavior of interest and multidimensional effects are not essential to the quantities being studied.

OpenSTREAM should therefore be described as a **one-dimensional, multi-field, two-phase flow simulation environment**, rather than as a CFD code.

### Can OpenSTREAM simulate flashing caused by a pressure decrease?

OpenSTREAM can represent flashing caused by a time-dependent decrease in the prescribed system pressure, subject to the assumptions and capabilities of the selected phase-change model.

However, OpenSTREAM does not currently represent flashing caused by the calculated local pressure drop along the channel. Fluid thermophysical properties, including saturation properties, are evaluated using the prescribed system pressure rather than the calculated local pressure. Therefore, the axial pressure decrease does not locally modify the saturation state used by the fluid-property calculations.

This approximation is considered reasonable for the high-pressure, relatively short channel applications targeted by the current OpenSTREAM frameworks when the axial pressure variation is small relative to the system pressure. Under these conditions, the corresponding axial variation in saturation properties is expected to remain limited.

The approximation is not suitable when pressure-drop-induced flashing or axial variations in saturation properties are important to the physical problem.

### What geometries can OpenSTREAM represent?

OpenSTREAM represents one-dimensional flow in straight channels. Cross-sectional geometry is described through averaged quantities such as flow area and wall perimeter.

The formulation supports multi-wall channel representations, allowing different wall surfaces and heating distributions to be considered. Examples may include tubes, annuli, rectangular channels, and simplified small rod-bundle representations when the one-dimensional approximation is appropriate.

OpenSTREAM does not resolve bends, crossflow, or detailed three-dimensional velocity and temperature distributions.

### How are multi-wall geometries represented in OpenSTREAM?

OpenSTREAM represents a multi-wall geometry as a single one-dimensional flow channel bounded by two or more wall surfaces. The channel has one common cross-sectional flow area, while the perimeter of each wall is specified separately using the PERIM geometry input.

For example, an annular channel can be represented by defining separate perimeters for the inner and outer walls. Other idealized configurations, such as rectangular channels or simplified subchannels, can be represented in the same way when the one-dimensional approximation is appropriate.

A simplified annular-channel geometry could, for example, use:

```
ID      ! Geometry identifier           > ANNULUS
LENGTH ! Channel length [m]           > ...
AREA    ! Common cross-sectional flow area [m2] > ...
PERIM   ! Inner and outer wall perimeters [m] > ... ...
END
```

The number of walls is determined automatically from the number of perimeter values specified in PERIM. OpenSTREAM also calculates the contribution of each wall to the total channel perimeter.

When several walls are present, the boundary-condition power distribution must be consistent with the number of wall perimeters. Wall-specific results can be inspected using the wall-selection options provided by the applicable solver plotting methods.

Wall-dependent quantities are stored separately for each wall where supported by the selected solver. These quantities may include:

- imposed wall heat flux;
- liquid-film mass flow rate and velocity;
- film thickness;
- wall boiling or film evaporation;
- wall shear stress;
- film entrainment and droplet deposition;
- base-film and disturbance-wave quantities in the four-field solver.

This wall-level formulation allows different walls to have different heating conditions. Heated and unheated walls, or walls with different power distributions, can therefore be represented within the same one-dimensional channel.

The bulk or core flow remains shared by all walls. Depending on the selected solver, this includes mixture quantities, vapor quantities, and the pressure distribution. Wall-specific fields interact through this common bulk-flow solution and the implemented closure models.

Multi-wall treatment does not create separate connected flow channels. OpenSTREAM does not currently calculate crossflow between neighboring channels, independent flow redistribution among parallel channels, or multidimensional

transport around the channel cross-section. These configurations require a different system or subchannel representation.

### Can the wall-specific solutions interact with one another?

Yes, but only indirectly through the common bulk-flow solution.

In a multi-wall geometry, OpenSTREAM represents all walls as bounding surfaces of the same one-dimensional flow channel. Each wall can have its own perimeter, heating conditions, and wall-specific quantities. In the three-field and four-field solvers, the liquid-film equations are formulated separately for each wall. The four-field solver similarly calculates separate base-film and disturbance-wave quantities for each wall.

The wall-specific fields do not exchange liquid directly with one another. For example, there is no model that transfers liquid laterally from the film on one wall directly to the film on another wall.

However, the wall solutions are coupled indirectly through the shared mixture, vapor, and droplet fields. A wall can therefore influence the other walls through mechanisms such as:

- wall heating and evaporation, which affect the common vapor and mixture solution;
- film entrainment, which transfers liquid from a wall film to the common entrained-droplet field;
- droplet deposition, which redistributes liquid from the common droplet field back to the wall films;
- wall and interfacial momentum exchanges, which contribute to the common flow solution;

Consequently, changing the heat flux or film behavior on one wall can affect the bulk-flow conditions and thereby modify the conditions experienced by the other walls. This is an indirect interaction through the shared flow, rather than a direct exchange between neighboring wall fields. The multi-wall formulation is specifically intended to represent straight channels with multiple independent wall surfaces and potentially different heating rates.

In the three-field and four-field formulations, entrainment and deposition provide an important indirect coupling mechanism. Liquid entrained from one wall enters the common droplet population and may subsequently deposit on another wall, depending on the implemented deposition-partition model. The wall equations themselves nevertheless remain wall-specific. The four-field equations additionally include direct exchanges between the base-film and disturbance-wave fields on the same wall, but these are not exchanges between different walls.

This treatment does not resolve circumferential transport, lateral liquid-film flow, or multidimensional mixing between walls. OpenSTREAM also does not represent separate neighboring channels or crossflow between them. When these effects are important, a multidimensional or subchannel formulation is required.

### What units should I use?

OpenSTREAM uses SI units unless a specific model or input explicitly states otherwise. Common units include:

- length in metres;
- time in seconds;
- pressure in pascals;
- temperature in kelvin;
- mass flow rate in kilograms per second;
- velocity in metres per second;
- specific enthalpy in joules per kilogram;
- wall heat flux in watts per square metre.

Check comments in the example input files and the relevant property documentation before entering a value.

### Can OpenSTREAM be used for safety or licensing analysis?

No. **OpenSTREAM must not be used for safety or licensing analysis.**

OpenSTREAM is intended for research, education, model development, performance evaluation, and validation activities. It is not a licensed safety-analysis code and has not been qualified for use in regulatory or licensing applications.

Results produced with OpenSTREAM must not be used as the basis for safety decisions, licensing submissions, regulatory compliance demonstrations, or the operation of safety-related systems.

### Which fluids can be simulated with OpenSTREAM?

OpenSTREAM is designed for single-component, liquid-vapor fluids whose required thermophysical properties are available through CoolProp. The fluid is selected using its CoolProp identifier in the FLUID model input.

The CoolProp interface can be tested in MATLAB before constructing a complete simulation:

```
cp = CoolPropWrapper.CoolPropWrapper('WATER');
```

CoolProp compatibility alone does not demonstrate that an OpenSTREAM calculation is physically applicable or validated. The selected closure models must also be appropriate for the fluid and operating conditions. Many current models and example cases have been developed or evaluated primarily for boiling water and steam.

OpenSTREAM does not currently support multi-component fluid mixtures or non-condensable gases. Pseudo-pure fluids, predefined mixtures, and incompressible solutions available in CoolProp must not be assumed to be compatible without verification of the full property and model requirements.

### How are fluid properties calculated?

Fluid properties are calculated using CoolProp through CoolPropWrapper. The fluid is selected using the FLUID model input, while the PROPERTIES model input defines how the phase properties are evaluated.

Two fluid-property options are available:

- **SATURATED:** Phase properties are evaluated at saturated conditions using the prescribed system pressure. Saturated liquid properties are used for the liquid phase, and saturated vapor properties are used for the vapor phase. The properties therefore do not vary with the local phase enthalpy.
- **PSYSTEM:** Phase properties are evaluated using the prescribed system pressure and the local phase enthalpy. This allows liquid properties to vary between subcooled and saturated-liquid conditions and vapor properties to vary between saturated-vapor and superheated conditions.

**SATURATED** is the default option and provides a simple property treatment that is appropriate when variations within the subcooled liquid and superheated vapor regions are not important. **PSYSTEM** is more appropriate when thermal non-equilibrium causes the liquid or vapor enthalpy to differ significantly from its saturation value.

The option is selected in the physical-model input file. For example:

```
FLUID      ! CoolProp fluid identifier      > WATER
PROPERTIES ! Fluid property assumptions    > PSYSTEM
```

For both options, the saturation temperature, saturated liquid and vapor enthalpies, latent heat, saturated phase densities, and other saturation properties are calculated at the prescribed system pressure.

The current implementation does not evaluate properties using the calculated local pressure along the channel. Consequently, selecting **PSYSTEM** accounts for variations with local phase enthalpy, but it does not enable flashing caused by the local axial pressure drop.

### Does a more detailed solver always give a better result?

No. Greater field resolution introduces additional equations, closure models, numerical parameters, and applicability limits. A more detailed solver is useful only when its modeled phenomena and validation basis match the application.

Select the simplest solver that represents the required physics and document the sensitivity to influential assumptions.

### What should I verify before trusting a result?

At minimum, check:

- input values, IDs, units, and warnings;
- point and steady-state convergence;
- mass and energy consistency;
- spatial-mesh sensitivity;
- physical time-step sensitivity for transients;
- sensitivity to influential closure models;
- applicability and validation of the selected models.

## 7.4.7 Extending and contributing

### How do I select another closure model?

Closure models are selected in the physical-model input file. Valid values are defined by classes in the `InputEnums` package.

Model selections may control, for example:

- void fraction;
- wall friction;
- interfacial heat transfer;
- phase or field momentum;
- entrainment and deposition;
- onset of annular flow;
- base-film thickness;
- disturbance-wave frequency.

Consult the relevant `InputEnums` class and `Inputs.Model` property before adding an entry. If a selection is not explicitly provided, the documented default is used.

### How do I add a new closure model?

Adding a closure model normally requires coordinated changes to:

1. the relevant `InputEnums` class;
2. any new model properties and defaults in `Inputs.Model`;
3. the solver or field method that evaluates the closure;
4. model documentation and references;
5. input examples;

6. verification and regression tests.

The implementation should document equations, units, assumptions, validity range, and source references. Verify the closure independently before using it in validation or engineering analysis.

### How should I report a bug?

Use the OpenSTREAM GitHub issue tracker. Before opening an issue:

- search for an existing report;
- provide the OpenSTREAM revision;
- provide the MATLAB, Python, and CoolProp versions;
- include the smallest input case that reproduces the problem;
- include the complete error message and stack trace;
- distinguish unexpected code behavior from a model-validity question.

Issue tracker:

- <https://github.com/OpenSTREAM-solvers/openstream/issues>

### How can I request a feature?

Open a GitHub issue describing:

- the use case and physical or workflow need;
- the proposed behavior;
- which solver or package is affected;
- any relevant equations, references, or examples;
- how the feature could be verified.

Search existing issues first to avoid duplicating an active request.

### How can I contribute?

Fork the repository, create a focused branch, implement and test the change, and submit a pull request. Follow the coding, testing, documentation, and code-of-conduct guidance in the Community section.

Keep changes focused and include documentation and tests when behavior or interfaces change.

### How do I contribute to the documentation?

Edit the reStructuredText source files, build the documentation locally when possible, and verify both HTML and PDF outputs. Add or update links, references, examples, and package documentation as needed.

### Where can I get additional help?

Consult, in this order:

1. the Quick Start and solver-specific tutorials;
2. the theory overview;
3. the glossary and notation guide;
4. the package reference;
5. existing GitHub issues;

6. the GitHub issue tracker for a new question or reproducible problem.

When requesting help, include enough information for another user to reproduce the calculation.

## 7.5 Support

Need help with **OpenSTREAM**? The best way to get assistance is by using the issue [tracker](#) on GitHub. *Maintainers* are notified of new issues and will do their best to respond promptly.

**Ask a question, report a bug, or suggest a feature here:**

<https://github.com/OpenSTREAM-solvers/openstream/issues>



## BIBLIOGRAPHY

- [1] Carl Adamsson and Jean-Marie Le Corre. Modeling and validation of a mechanistic tool (MEFISTO) for the prediction of critical power in BWR fuel assemblies. *Nuclear Engineering and Design*, 241(8):2843–2858, 2011. URL: <https://www.sciencedirect.com/science/article/pii/S0029549311000847>, doi:10.1016/j.nucengdes.2011.01.033.
- [2] Carl Adamsson and Jean-Marie Le Corre. Transient dryout prediction using a computationally efficient method for sub-channel film-flow analysis. *Nuclear Engineering and Design*, 280:316–325, 2014. URL: <https://www.sciencedirect.com/science/article/pii/S0029549314005081>, doi:10.1016/j.nucengdes.2014.09.012.
- [3] D. Bestion. The physical closure laws in the CATHARE code. *Nuclear Engineering and Design*, 124(3):229–245, 1990. URL: <https://www.sciencedirect.com/science/article/pii/0029549390902948>, doi:10.1016/0029-5493(90)90294-8.
- [4] Yu. A. Bezrukov, V. I. Astakhov, and V. G. Brantov. Experimental investigation and statistical analysis of data on burnout in rod bundles for water-moderated water-cooled reactors. *Teploenergetika*, 23:67–70, 1976. English translation available in Thermal Engineering.
- [5] L. Biasi, G. C. Clerici, S. Garribba, R. Sala, and A. Tozzi. Studies on burnout. part 3: a new correlation for round ducts and uniform heating and its comparison with world data. *Energ. Nucleare (Milan)*, 14(9):530–536, September 1966.
- [6] A. A. Bishop, R. O. Sandberg, and L. S. Tong. Forced convection heat transfer at high pressure after the critical heat flux. In *ASME-AIChE Heat Transfer Conference*, number Preprint 65-HT-31. ASME, 1965.
- [7] F. W. Dittus and L. M. K. Boelter. Heat transfer in automobile radiators of the tubular type. *Publications in Engineering*, 2:443–461, 1930. URL: <https://www.sciencedirect.com/science/article/abs/pii/073519338590003X>, doi:10.1016/0735-1933(85)90003-X.
- [8] R. S. Dougall and Warren M. Rohsenow. Film boiling on the inside of vertical tubes with upward flow of the fluid at low qualities. Technical Report MIT Report No. 9079-26, Massachusetts Institute of Technology, Cambridge, MA, 1963. URL: <https://dspace.mit.edu/entities/publication/89b0bb7b-cca2-49f3-9619-97b555e49447>.
- [9] V. Gnielinski. New equations for heat and mass transfer in turbulent pipe and channel flow. *International Chemical Engineering*, 16(2):359–368, 1976.
- [10] D. C. Groeneveld and G. G. J. Delorme. Prediction of thermal non-equilibrium in the post-dryout regime. *Nuclear Engineering and Design*, 36:17–26, 1976.
- [11] D. C. Groeneveld and E. O. Moeck. An investigation of heat transfer in the liquid deficient regime. Technical Report, Atomic Energy of Canada Ltd., Chalk River (Ontario). Chalk River Nuclear Labs., Canada, December 1969. URL: <https://www.osti.gov/biblio/4747993>.

- [12] G. F. Hewitt and A. H. Govan. Phenomenological modelling of non-equilibrium flows with phase change. *International Journal of Heat and Mass Transfer*, 33(2):229–242, February 1990. URL: [https://doi.org/10.1016/0017-9310\(90\)90094-B](https://doi.org/10.1016/0017-9310(90)90094-B), doi:10.1016/0017-9310(90)90094-B.
- [13] Jean-Marie Le Corre. Phenomenological model of disturbance waves in annular two-phase flow. *International Journal of Multiphase Flow*, 151:104057, 2022a. URL: <https://www.sciencedirect.com/science/article/pii/S0301932222000696>, doi:10.1016/j.ijmultiphaseflow.2022.104057.
- [14] Jean-Marie Le Corre. Non-equilibrium modeling of disturbance waves with phase change in fuel bundle. In *Proceedings of the 19th International Topical Meeting on Nuclear Reactor Thermal Hydraulics (NURETH-19)*. Brussels, Belgium, March 6–11 2022b.
- [15] Jean-Marie Le Corre. Mechanistic prediction of Westinghouse TRITON11® BWR fuel critical power with MEFISTO-T subchannel analysis code. *Nuclear Engineering and Design*, 429:113613, 2024. URL: <https://www.sciencedirect.com/science/article/pii/S0029549324007131>, doi:10.1016/j.nucengdes.2024.113613.
- [16] Jean-Marie Le Corre, Jason Chan, Elisabeth Walter, Evan T. Hurlburt, and Roman W. Morse. Open-STREAM: an open-source platform for two-phase flow modeling and simulation. In *Proceedings of the 21st International Topical Meeting on Nuclear Reactor Thermal Hydraulics (NURETH-21)*. Busan, Korea, August 31–September 5 2025a. URL: [https://openstream-solvers.github.io/openstream/\\_static/publications/LeCorre2025OpenSTREAM.pdf](https://openstream-solvers.github.io/openstream/_static/publications/LeCorre2025OpenSTREAM.pdf).
- [17] Jean-Marie Le Corre, Jason Chan, Elisabeth Walter, Evan T. Hurlburt, and Roman W. Morse. Open-STREAM: a new open-source platform for two-phase flow model development. In *Proceedings of the 12th International Conference on Multiphase Flow (ICMF 2025)*. Toulouse, France, May 12–16 2025b. URL: [https://openstream-solvers.github.io/openstream/\\_static/publications/LeCorre2025ICMF.pdf](https://openstream-solvers.github.io/openstream/_static/publications/LeCorre2025ICMF.pdf).
- [18] Jean-Marie Le Corre, Fabrice Francois, and Henryk Anglart. A mixture relaxation model for thermal non-equilibrium in convective flow boiling. In *Proceedings of the 15th International Topical Meeting on Nuclear Reactor Thermal-Hydraulic, Operation and Safety (NUTHOS-15)*. Kashiwa-no-ha, Japan, September 2026.
- [19] G. S. Lellouche and B. A. Zolotar. A mechanistic model for predicting two-phase void fraction for water in vertical tubes, channels and rod bundles. Technical Report EPRI NP-2246-SR, Electric Power Research Institute, Palo Alto, California, February 1982. URL: <https://www.nrc.gov/docs/ML2001/ML20010E663.pdf>.
- [20] L. L. Levitan and L. Y. A. Borevsky. Holography of steam-water flows. *Moskwa: Energoatomizdat*, 1989.
- [21] William H. McAdams. *Heat Transmission*. McGraw-Hill, New York, NY, 1942. Includes correlation for turbulent single-phase convection.
- [22] D. G. Morris, C. B. Mullins, and G. L. Yoder. Analysis of transient film boiling of high-pressure water in a rod bundle. Technical Report NUREG/CR-2469; ORNL/NUREG-85, Oak Ridge National Laboratory and U.S. Nuclear Regulatory Commission, Oak Ridge, TN, USA, March 1982. ON: DE82013953. URL: <https://www.osti.gov/biblio/5330166>.
- [23] Tomio Okawa, Akio Kotani, Isao Kataoka, and Masanori Naito. Prediction of critical heat flux in annular flow using a film flow model. *Journal of Nuclear Science and Technology*, 40(6):388–396, June 2003. URL: <https://doi.org/10.1080/18811248.2003.9715370>, doi:10.1080/18811248.2003.9715370.
- [24] Tomio Okawa, Akio Kotani, Isao Kataoka, and Masanori Naitoh. Prediction of the critical heat flux in annular regime in various vertical channels. *Nuclear Engineering and Design*, 229(2):223–236, 2004. URL: <https://www.sciencedirect.com/science/article/pii/S0029549304000512>, doi:10.1016/j.nucengdes.2004.01.005.
- [25] W. E. Ranz and W. R. Marshall. Evaporation from drops. part i. *Chemical Engineering Progress*, 48(3):141–146, March 1952.

- [26] D. G. Reddy and others. Two-phase friction multiplier for high pressure steam-water flow. Technical Report EPRI-NP-2522, Electric Power Research Institute, Palo Alto, California, July 1982.
- [27] Warren M. Rohsenow, James P. Hartnett, and Young I. Cho, editors. *Handbook of Heat Transfer*. McGraw-Hill, New York, 3 edition, 1998. ISBN 0-07-0535558. A comprehensive handbook edited by Rohsenow, Hartnett, and Cho.
- [28] P. Saha and N. Zuber. Point of net vapor generation and vapor void fraction in subcooled boiling. In *Proceedings of the International Heat Transfer Conference (IHTC-5)*. Tokyo, Japan, December 1973. Georgia Institute of Technology, Atlanta, Scripta Book Co. Conference paper; OSTI ID 4195144. URL: <https://www.osti.gov/biblio/4195144>.
- [29] L. Sanmiguel Gimeno, S. P. Walker, G. F. Hewitt, Jean-Marie Le Corre, A. Dasgupta, and M. Ahmad. Validation and cross-verification of three mechanistic codes for annular two-phase flow simulation and dryout prediction. In *Proceedings of NURETH-16 (16th International Topical Meeting on Nuclear Reactor Thermalhydraulics)*. Chicago, IL, USA, 2015.
- [30] E. N. Sieder and G. E. Tate. Heat transfer and pressure drop of liquids in tubes. *Industrial and Engineering Chemistry*, 28(12):1429–1435, 1936. URL: <https://doi.org/10.1021/ie50324a027>, doi:10.1021/ie50324a027.
- [31] G. B. Wallis. *One-Dimensional Two-Phase Flow*. McGraw-Hill, New York, 1st edition, 1969. ISBN 9780070679429.
- [32] Elisabeth K. Walter. Derivation and open source implementation of a simplified one-dimensional two-fluid model. Master's thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2024. URL: <https://www.diva-portal.org/smash/record.jsf?pid=diva2:1888131>.
- [33] U. Windecker. Spacer model for prediction of axial dryout locations in annular flow. Technical Report SECRC/B/TR-99/047E, ABB (ASEA Brown Boveri), Sweden, 1999.
- [LCCW+5a] Jean-Marie Le Corre, Jason Chan, Elisabeth Walter, Evan T. Hurlburt, and Roman W. Morse. Open-STREAM: an open-source platform for two-phase flow modeling and simulation. In *Proceedings of the 21st International Topical Meeting on Nuclear Reactor Thermal Hydraulics (NURETH-21)*. Busan, Korea, August 31–September 5 2025a. URL: [https://openstream-solvers.github.io/openstream/\\_static/publications/LeCorre2025OpenSTREAM.pdf](https://openstream-solvers.github.io/openstream/_static/publications/LeCorre2025OpenSTREAM.pdf).



## MATLAB MODULE INDEX

### i

[InputEnums](#), 91

[Inputs](#), 101

### S

[Session](#), 118

[Solvers](#), 194

[Solvers.FourField](#), 124

[Solvers.Mixture](#), 142

[Solvers.ThreeField](#), 164

[Solvers.TwoFluid](#), 173



## A

AbstractField (*class in Solvers*), 198  
 AbstractField() (*Solvers.AbstractField method*), 199  
 AbstractFilm (*class in Solvers*), 194  
 AbstractFilm() (*Solvers.AbstractFilm method*), 194  
 AbstractPhase (*class in Solvers*), 199  
 AbstractSolver (*class in Solvers*), 198  
 AbstractSolver() (*Solvers.AbstractSolver method*), 198  
 addTile() (*Solvers.SolverPlotter method*), 197  
 ADIAM() (*Inputs.Geometry method*), 111  
 AFDISTR() (*Solvers.Mixture.Mixture method*), 146  
 AFDISTR() (*Solvers.TwoFluid.Mixture method*), 183  
 AFFNC() (*Solvers.Mixture.Mixture method*), 146  
 AI() (*Solvers.ThreeField.Drop method*), 165  
 ALPHAF (*Inputs.FluidProperties attribute*), 108  
 ALPHAG (*Inputs.FluidProperties attribute*), 108  
 ALPHAL() (*Inputs.FluidProperties method*), 109  
 ALPHAV() (*Inputs.FluidProperties method*), 109  
 AMPLITUDE() (*Solvers.FourField.Wave method*), 128  
 ANGLE (*Inputs.Geometry attribute*), 111  
 Annular flow, 46  
 applySolverDependentProperties() (*Inputs.Input method*), 117  
 applySolverDependentProps() (*Inputs.InputSet method*), 108  
 AREA (*Inputs.Geometry attribute*), 111  
 AREA() (*Solvers.ThreeField.Drop method*), 165  
 AREA() (*Solvers.TwoFluid.Liquid method*), 186  
 AREA() (*Solvers.TwoFluid.Vapor method*), 175  
 AXIALINTERP (*Inputs.Options attribute*), 114  
 axialInterpolate() (*Solvers.Mixture.MixtureSolver method*), 139

## B

Base (*class in Solvers.FourField*), 133  
 base (*Solvers.FourField.Film attribute*), 126  
 Base() (*Solvers.FourField.Base method*), 133  
 BASEEQTHICK (*class in InputEnums*), 96  
 BASEEQTHICK (*Inputs.Model attribute*), 101  
 BASEEQTHICKCOEF (*Inputs.Model attribute*), 101  
 BASEYPLUS (*Inputs.Model attribute*), 101  
 bc (*Inputs.InputSet attribute*), 107  
 BETA() (*Solvers.FourField.Base method*), 134

BETA() (*Solvers.FourField.Wave method*), 128  
BETAP() (*Solvers.FourField.Base method*), 134  
BETAP() (*Solvers.FourField.Wave method*), 128  
Boundary condition, [46](#)  
BoundaryConditions (*class in Inputs*), 112  
boundaryConditions (*Solvers.Mixture.MixtureSolver attribute*), 139  
boundaryConditions (*Solvers.ThreeField.ThreeFieldSolver attribute*), 161  
boundaryConditions (*Solvers.TwoFluid.TwoFluidSolver attribute*), 171  
BoundaryConditions() (*Inputs.BoundaryConditions method*), 112  
BHTM (*class in InputEnums*), 93  
BHTM (*Inputs.Model attribute*), 101  
BTMT (*class in InputEnums*), 91  
BTMT (*Inputs.Model attribute*), 101  
BUBBLEDRAE (*class in InputEnums*), 95  
BUBBLEDRAE (*Inputs.Model attribute*), 101  
BUBBLEDRAECOEF (*Inputs.Model attribute*), 102

## C

C() (*Solvers.Mixture.Liquid method*), 159  
C() (*Solvers.Mixture.Vapor method*), 143  
CBT (*class in InputEnums*), 97  
CBT (*Inputs.Model attribute*), 102  
cbt (*Solvers.Mixture.Mixture attribute*), 145  
CBT() (*Solvers.Mixture.Mixture method*), 146  
CBT() (*Solvers.TwoFluid.Mixture method*), 183  
CBTELEVATION (*Inputs.Model attribute*), 102  
CBTIDX() (*Solvers.Mixture.Mixture method*), 146  
CBTKEFFECT (*Inputs.Model attribute*), 102  
CBTMULT (*Inputs.Model attribute*), 102  
CBTZ() (*Solvers.Mixture.Mixture method*), 147  
CD() (*Solvers.TwoFluid.Liquid method*), 186  
CD() (*Solvers.TwoFluid.Vapor method*), 175  
CHF() (*Solvers.Mixture.Mixture method*), 147  
CHF() (*Solvers.TwoFluid.Mixture method*), 183  
closeLog() (*Session.Log method*), 120  
Closure model, [46](#)  
CONC() (*Solvers.ThreeField.Drop method*), 166  
Conservation equation, [47](#)  
convert2JSON() (*Inputs.Input static method*), 117  
copyFlowProperties() (*Solvers.AbstractField method*), 199  
copyFlowProperties() (*Solvers.FourField.Base method*), 137  
copyFlowProperties() (*Solvers.FourField.Film method*), 126  
CPF (*Inputs.FluidProperties attribute*), 108  
CPG (*Inputs.FluidProperties attribute*), 108  
CPL() (*Inputs.FluidProperties method*), 109  
CPV() (*Inputs.FluidProperties method*), 110  
CreateITR() (*Solvers.AbstractSolver static method*), 198  
CSTWAVEFREQ (*Inputs.Model attribute*), 102  
currentAhIdx (*Solvers.SolverPlotter attribute*), 197  
CV() (*Solvers.AbstractFilm method*), 194  
CV\_WALLISTHICK\_CALC() (*Solvers.AbstractFilm method*), 194  
CW() (*Solvers.AbstractFilm method*), 195  
CW\_LAM\_CALC() (*Solvers.AbstractFilm method*), 195  
CW\_TURB\_CALC() (*Solvers.AbstractFilm method*), 195

## D

[defaultValueUsedReport\(\)](#) (*Inputs.Input method*), 117  
[delete\(\)](#) (*Session.Log method*), 120  
[DELTA\\_FREQ\(\)](#) (*Solvers.FourField.Wave method*), 128  
[DENSITY\(\)](#) (*Solvers.ThreeField.Drop method*), 166  
[DEPENHANCEMENT](#) (*class in InputEnums*), 96  
[DEPENHANCEMENT](#) (*Inputs.Model attribute*), 102  
[Deposition](#), 47  
[DEPOSITION](#) (*class in InputEnums*), 93  
[DEPOSITION](#) (*Inputs.Model attribute*), 102  
[DHIN\(\)](#) (*Inputs.BoundaryConditions method*), 113  
[DIAM\(\)](#) (*Solvers.ThreeField.Drop method*), 166  
[diaryFilePath](#) (*Session.Log attribute*), 119  
[diaryIsOn](#) (*Session.Log attribute*), 119  
[diaryOff\(\)](#) (*Session.Log method*), 120  
[diaryOn\(\)](#) (*Session.Log method*), 120  
[directory](#) (*Session.Session attribute*), 118  
[dirName](#) (*Session.Session attribute*), 118  
[distributeOAFW\(\)](#) (*Solvers.FourField.Film method*), 126  
[Disturbance wave](#), 47  
[DITTUSBOELTERCOEF](#) (*Inputs.Model attribute*), 102  
[DP](#) (*Solvers.Mixture.Mixture attribute*), 144  
[DPACCS\(\)](#) (*Solvers.Mixture.Mixture method*), 147  
[DPACCT\(\)](#) (*Solvers.Mixture.Mixture method*), 147  
[DPAC CZ\(\)](#) (*Solvers.Mixture.Mixture method*), 147  
[DPDZ\(\)](#) (*Solvers.TwoFluid.Mixture method*), 183  
[DPGRAV\(\)](#) (*Solvers.Mixture.Mixture method*), 147  
[DPK\(\)](#) (*Solvers.Mixture.Mixture method*), 147  
[DPPARTS\(\)](#) (*Solvers.Mixture.Mixture method*), 148  
[DPSUM](#) (*Solvers.Mixture.Mixture attribute*), 145  
[DPTOT\(\)](#) (*Solvers.Mixture.Mixture method*), 148  
[DPWALL\(\)](#) (*Solvers.Mixture.Mixture method*), 148  
[DRAG\(\)](#) (*Solvers.ThreeField.Drop method*), 166  
[DRAGCOEF\(\)](#) (*Solvers.FourField.Wave method*), 128  
[Drop](#) (*class in Solvers.FourField*), 124  
[Drop](#) (*class in Solvers.ThreeField*), 164  
[drop](#) (*Solvers.ThreeField.ThreeFieldSolver attribute*), 162  
[Drop\(\)](#) (*Solvers.ThreeField.Drop method*), 164  
[DROPDIAM](#) (*Inputs.Model attribute*), 102  
[DROPDRA G](#) (*class in InputEnums*), 96  
[DROPDRA G](#) (*Inputs.Model attribute*), 102  
[DROPDRA GCOEF](#) (*Inputs.Model attribute*), 102  
[dropInit](#) (*Solvers.ThreeField.ThreeFieldSolver attribute*), 162  
[Droplet field](#), 47  
[DROPSLIP](#) (*Inputs.Model attribute*), 102  
[DT](#) (*Solvers.AbstractField attribute*), 199  
[DT](#) (*Solvers.FourField.Base attribute*), 133  
[DT](#) (*Solvers.FourField.Film attribute*), 125  
[DT](#) (*Solvers.FourField.Wave attribute*), 127  
[DT](#) (*Solvers.Mixture.Mixture attribute*), 145  
[DT](#) (*Solvers.Mixture.MixtureSolver attribute*), 139  
[DT](#) (*Solvers.ThreeField.Drop attribute*), 165  
[DT](#) (*Solvers.ThreeField.Film attribute*), 169  
[DT](#) (*Solvers.ThreeField.ThreeFieldSolver attribute*), 161

DT (*Solvers.TwoFluid.Liquid attribute*), 185  
DT (*Solvers.TwoFluid.TwoFluidSolver attribute*), 171  
DT (*Solvers.TwoFluid.Vapor attribute*), 174  
DTIN() (*Inputs.BoundaryConditions method*), 113  
DTMFB (*Inputs.Model attribute*), 102  
DZ (*Solvers.AbstractFilm attribute*), 194  
DZ (*Solvers.Mixture.Mixture attribute*), 145  
DZ (*Solvers.Mixture.MixtureSolver attribute*), 139  
DZ (*Solvers.ThreeField.ThreeFieldSolver attribute*), 161  
DZ (*Solvers.TwoFluid.TwoFluidSolver attribute*), 171

## E

ECT() (*Solvers.Mixture.Mixture method*), 148  
ENTNUM() (*Solvers.AbstractFilm method*), 195  
ENTNUMMAXITER (*Inputs.Options attribute*), 114  
ENTNUMTOL (*Inputs.Options attribute*), 114  
Entrainment, 47  
ENTRAINMENT (*class in InputEnums*), 97  
ENTRAINMENT (*Inputs.Model attribute*), 102  
EPSILON() (*Solvers.FourField.Base method*), 134  
EPSILON() (*Solvers.FourField.Wave method*), 128  
EQFREQUENCY() (*Solvers.FourField.Wave method*), 129  
EQPERIOD() (*Solvers.FourField.Wave method*), 129  
EQSTROUHAL (*class in InputEnums*), 91  
EQSTROUHAL (*Inputs.Model attribute*), 102  
EQSTROUHAL() (*Solvers.FourField.Wave method*), 129  
EQSTROUHALCOEF (*Inputs.Model attribute*), 102  
EQTHICK() (*Solvers.FourField.Base method*), 134  
EQUIL() (*Solvers.FourField.Film method*), 126  
EQUIL() (*Solvers.FourField.FourFieldSolver method*), 122  
EQUIL() (*Solvers.ThreeField.ThreeFieldSolver method*), 162  
ERRORFW (*Inputs.Options attribute*), 114  
ERRORH (*Inputs.Options attribute*), 114  
ERRORP (*Inputs.Options attribute*), 114  
ERRORU (*Inputs.Options attribute*), 114  
ERRORUD (*Inputs.Options attribute*), 114  
ERRORUF (*Inputs.Options attribute*), 114  
ERRORW (*Inputs.Options attribute*), 114  
ERRORWF (*Inputs.Options attribute*), 114  
ETA() (*Solvers.FourField.Base method*), 134  
ETA() (*Solvers.FourField.Wave method*), 129  
extra (*Inputs.Input attribute*), 117

## F

FBASE() (*Solvers.FourField.Wave method*), 129  
FBASEMASS() (*Solvers.FourField.Wave method*), 129  
FBASEVAPOR() (*Solvers.FourField.Base method*), 134  
FBUOY() (*Solvers.AbstractFilm method*), 195  
FBUOY() (*Solvers.ThreeField.Drop method*), 166  
FBUOY() (*Solvers.TwoFluid.Liquid method*), 186  
FBUOY() (*Solvers.TwoFluid.Vapor method*), 175  
FDEP() (*Solvers.AbstractFilm method*), 195  
FDEP() (*Solvers.FourField.Base method*), 135  
FDEP() (*Solvers.FourField.Wave method*), 130

FDRAG() (*Solvers.FourField.Wave method*), 130  
 FDRAG() (*Solvers.ThreeField.Drop method*), 166  
 FDRAG() (*Solvers.TwoFluid.Liquid method*), 187  
 FDRAG() (*Solvers.TwoFluid.Vapor method*), 175  
 FDRY() (*Solvers.FourField.Base method*), 135  
 FENT() (*Solvers.ThreeField.Drop method*), 166  
 FGRAV() (*Solvers.AbstractFilm method*), 195  
 FGRAV() (*Solvers.ThreeField.Drop method*), 167  
 FGRAV() (*Solvers.TwoFluid.Liquid method*), 187  
 FGRAV() (*Solvers.TwoFluid.Vapor method*), 175  
 Field, 47  
 fieldName2plotStyle() (*Solvers.SolverPlotter static method*), 197  
 Film (*class in Solvers.FourField*), 124  
 Film (*class in Solvers.ThreeField*), 169  
 film (*Solvers.FourField.Wave attribute*), 128  
 film (*Solvers.ThreeField.ThreeFieldSolver attribute*), 162  
 Film dryout, 47  
 Film() (*Solvers.FourField.Film method*), 125  
 filmInit (*Solvers.ThreeField.ThreeFieldSolver attribute*), 162  
 FINTCOND() (*Solvers.TwoFluid.Liquid method*), 187  
 FINTEVAP() (*Solvers.TwoFluid.Vapor method*), 176  
 FLOWID() (*Solvers.TwoFluid.Liquid method*), 187  
 FLOWREGIME() (*Solvers.TwoFluid.Liquid method*), 187  
 FLOWREGIME() (*Solvers.TwoFluid.Vapor method*), 176  
 FLUID (*Inputs.FluidProperties attribute*), 108  
 FLUID (*Inputs.Model attribute*), 102  
 fluid (*Solvers.AbstractFilm attribute*), 194  
 fluid (*Solvers.Mixture.Mixture attribute*), 145  
 fluid (*Solvers.Mixture.MixtureSolver attribute*), 139  
 fluid (*Solvers.ThreeField.ThreeFieldSolver attribute*), 162  
 fluid (*Solvers.TwoFluid.TwoFluidSolver attribute*), 171  
 FLUIDPROPERTIES (*class in InputEnums*), 92  
 FluidProperties (*class in Inputs*), 108  
 FluidProperties() (*Inputs.FluidProperties method*), 108  
 FontSize (*Solvers.SolverPlotter attribute*), 196  
 Four-field model, 47  
 FourFieldSolver (*class in Solvers.FourField*), 121  
 FourFieldSolver() (*Solvers.FourField.FourFieldSolver method*), 121  
 FOURIERCOND() (*Solvers.Mixture.Mixture method*), 148  
 FOURIEREVAP() (*Solvers.Mixture.Mixture method*), 149  
 FREQUENCY (*Solvers.FourField.Wave attribute*), 127  
 FRICTION (*Inputs.Model attribute*), 102  
 FSHEAR() (*Solvers.FourField.Wave method*), 130  
 FTOT() (*Solvers.AbstractFilm method*), 195  
 FTOT() (*Solvers.FourField.Base method*), 135  
 FTOT() (*Solvers.FourField.Wave method*), 130  
 FTOT() (*Solvers.ThreeField.Drop method*), 167  
 FTOT() (*Solvers.TwoFluid.Liquid method*), 187  
 FTOT() (*Solvers.TwoFluid.Vapor method*), 176  
 FVAPOR() (*Solvers.AbstractFilm method*), 195  
 FVAPOR() (*Solvers.FourField.Wave method*), 130  
 FW() (*Solvers.Mixture.Mixture method*), 149  
 FW() (*Solvers.Mixture.Vapor method*), 143  
 FWALEVAP() (*Solvers.TwoFluid.Vapor method*), 176

`FWALL()` (*Solvers.AbstractFilm method*), 195  
`FWALL()` (*Solvers.FourField.Wave method*), 130  
`FWALL()` (*Solvers.TwoFluid.Liquid method*), 188  
`FWALL()` (*Solvers.TwoFluid.Vapor method*), 176  
`FWAVE()` (*Solvers.FourField.Base method*), 135  
`FWAVEMASS()` (*Solvers.FourField.Base method*), 135  
`FWL()` (*Solvers.Mixture.Mixture method*), 149

## G

`G` (*Inputs.Model attribute*), 103  
`gca()` (*Solvers.SolverPlotter method*), 197  
`gcf()` (*Solvers.SolverPlotter method*), 197  
`Geometry` (*class in Inputs*), 111  
`geometry` (*Inputs.InputSet attribute*), 107  
`Geometry()` (*Inputs.Geometry method*), 111  
Governing equation, 47  
`Grid` (*Solvers.SolverPlotter attribute*), 196

## H

`H` (*Solvers.FourField.Base attribute*), 133  
`H` (*Solvers.FourField.Film attribute*), 125  
`H` (*Solvers.FourField.Wave attribute*), 127  
`H` (*Solvers.Mixture.Mixture attribute*), 145  
`H` (*Solvers.ThreeField.Drop attribute*), 165  
`H` (*Solvers.ThreeField.Film attribute*), 169  
`H` (*Solvers.TwoFluid.Liquid attribute*), 185  
`H` (*Solvers.TwoFluid.Vapor attribute*), 174  
`HC()` (*Inputs.FluidProperties method*), 110  
`HC()` (*Solvers.Mixture.Liquid method*), 159  
`HC()` (*Solvers.Mixture.Vapor method*), 143  
`HC()` (*Solvers.TwoFluid.Mixture method*), 183  
`HDIAM()` (*Inputs.Geometry method*), 111  
`HF` (*Inputs.FluidProperties attribute*), 108  
`HF()` (*Inputs.BoundaryConditions method*), 113  
`HFG` (*Inputs.FluidProperties attribute*), 108  
`HFLUX` (*Solvers.FourField.Base attribute*), 133  
`HFLUX` (*Solvers.FourField.Film attribute*), 125  
`HFLUX` (*Solvers.FourField.Wave attribute*), 127  
`HFLUX` (*Solvers.Mixture.Mixture attribute*), 145  
`HFLUX` (*Solvers.ThreeField.Film attribute*), 169  
`HFLUX()` (*Solvers.Mixture.Liquid method*), 159  
`HFLUX()` (*Solvers.Mixture.Vapor method*), 143  
`HFLUX()` (*Solvers.TwoFluid.Liquid method*), 188  
`HFLUX()` (*Solvers.TwoFluid.Mixture method*), 183  
`HFLUX()` (*Solvers.TwoFluid.Vapor method*), 177  
`HFLUXWALEVAP()` (*Solvers.Mixture.Vapor method*), 143  
`HFLUXWALEVAP()` (*Solvers.TwoFluid.Liquid method*), 188  
`HFLUXWALEVAP()` (*Solvers.TwoFluid.Vapor method*), 177  
`HG` (*Inputs.FluidProperties attribute*), 108  
`HG()` (*Inputs.BoundaryConditions method*), 113  
`HIN` (*Inputs.BoundaryConditions attribute*), 112  
`HINT()` (*Solvers.Mixture.Mixture method*), 149  
`HINT()` (*Solvers.TwoFluid.Liquid method*), 188  
`HINT()` (*Solvers.TwoFluid.Vapor method*), 177

[HINTCOND\(\) \(Solvers.Mixture.Mixture method\), 150](#)  
[HINTCOND\(\) \(Solvers.TwoFluid.Liquid method\), 188](#)  
[HINTCOND\(\) \(Solvers.TwoFluid.Vapor method\), 177](#)  
[HINTEVAP\(\) \(Solvers.Mixture.Mixture method\), 150](#)  
[HINTEVAP\(\) \(Solvers.TwoFluid.Liquid method\), 189](#)  
[HINTEVAP\(\) \(Solvers.TwoFluid.Vapor method\), 177](#)  
[HM\(\) \(Solvers.Mixture.Mixture method\), 150](#)  
[HTOT\(\) \(Solvers.Mixture.Mixture method\), 150](#)  
[HTOT\(\) \(Solvers.TwoFluid.Liquid method\), 189](#)  
[HTOT\(\) \(Solvers.TwoFluid.Vapor method\), 177](#)  
[HVTOT\(\) \(Solvers.Mixture.Mixture method\), 150](#)  
[HWALEVAP\(\) \(Solvers.Mixture.Mixture method\), 151](#)  
[HWALEVAP\(\) \(Solvers.TwoFluid.Liquid method\), 189](#)  
[HWALEVAP\(\) \(Solvers.TwoFluid.Vapor method\), 178](#)  
[HWALHEAT\(\) \(Solvers.Mixture.Mixture method\), 151](#)  
[HWALHEAT\(\) \(Solvers.TwoFluid.Liquid method\), 189](#)  
[HWALHEAT\(\) \(Solvers.TwoFluid.Vapor method\), 178](#)  
[HWALL\(\) \(Solvers.Mixture.Liquid method\), 159](#)  
[HWALL\(\) \(Solvers.Mixture.Mixture method\), 151](#)  
[HWALL\(\) \(Solvers.Mixture.Vapor method\), 143](#)  
[HWALLBOIL\(\) \(Solvers.Mixture.Mixture method\), 151](#)  
[HWALLPOSTCBT\(\) \(Solvers.Mixture.Mixture method\), 151](#)  
[Hydrodynamic equilibrium, 47](#)  
[Hydrodynamic non-equilibrium, 47](#)

**I**

[ID \(Inputs.Geometry attribute\), 111](#)  
[ID \(Inputs.Model attribute\), 103](#)  
[ID \(Inputs.Options attribute\), 114](#)  
[initializeBaseAndWave\(\) \(Solvers.FourField.Film method\), 126](#)  
[initializeSolver\(\) \(Solvers.FourField.FourFieldSolver method\), 122](#)  
[initializeSolver\(\) \(Solvers.Mixture.MixtureSolver method\), 140](#)  
[initializeSolver\(\) \(Solvers.ThreeField.ThreeFieldSolver method\), 162](#)  
[initializeSolver\(\) \(Solvers.TwoFluid.TwoFluidSolver method\), 171](#)  
[Input \(class in Inputs\), 116](#)  
[Input\(\) \(Inputs.Input method\), 117](#)  
[InputEnums \(module\), 91](#)  
[Inputs \(module\), 101](#)  
[InputSet \(class in Inputs\), 107](#)  
[inputSet \(Solvers.AbstractFilm attribute\), 194](#)  
[inputSet \(Solvers.AbstractSolver attribute\), 198](#)  
[inputSet \(Solvers.Mixture.Mixture attribute\), 145](#)  
[inputSet \(Solvers.Mixture.MixtureSolver attribute\), 139](#)  
[inputSet \(Solvers.ThreeField.ThreeFieldSolver attribute\), 162](#)  
[inputSet \(Solvers.TwoFluid.TwoFluidSolver attribute\), 171](#)  
[InputSet\(\) \(Inputs.InputSet method\), 107](#)  
[INTAREA \(class in InputEnums\), 91](#)  
[INTAREA \(Inputs.Model attribute\), 103](#)  
[INTAREA\(\) \(Solvers.TwoFluid.Liquid method\), 189](#)  
[INTAREA\(\) \(Solvers.TwoFluid.Vapor method\), 178](#)  
[Interfacial area, 47](#)  
[Interfacial transfer, 47](#)  
[interpBoundaryConditions\(\) \(Solvers.Mixture.MixtureSolver method\), 140](#)  
[INTHFLUX\(\) \(Solvers.TwoFluid.Liquid method\), 190](#)

INTHFLUX() (*Solvers.TwoFluid.Vapor method*), 178  
INTLENGTH (*class in InputEnums*), 95  
INTLENGTH (*Inputs.Model attribute*), 103  
INTLENGTHLCST (*Inputs.Model attribute*), 103  
INTLENGTHVCST (*Inputs.Model attribute*), 103  
INTNU (*class in InputEnums*), 100  
INTNU (*Inputs.Model attribute*), 103  
INTNU() (*Solvers.TwoFluid.Liquid method*), 190  
INTNU() (*Solvers.TwoFluid.Vapor method*), 178  
INTNULCST (*Inputs.Model attribute*), 103  
INTNUVCST (*Inputs.Model attribute*), 103  
INTTRANSH (*class in InputEnums*), 92  
INTTRANSH (*Inputs.Model attribute*), 103  
isLegalPath() (*in module Session*), 118  
ITR (*Solvers.AbstractField attribute*), 199  
ITR (*Solvers.FourField.Base attribute*), 133  
ITR (*Solvers.FourField.Film attribute*), 125  
ITR (*Solvers.FourField.Wave attribute*), 127  
ITR (*Solvers.Mixture.Mixture attribute*), 145  
ITR (*Solvers.ThreeField.Drop attribute*), 165  
ITR (*Solvers.ThreeField.Film attribute*), 169  
ITR (*Solvers.TwoFluid.Liquid attribute*), 186  
ITR (*Solvers.TwoFluid.Vapor attribute*), 174

## J

J() (*Solvers.TwoFluid.Liquid method*), 190  
J() (*Solvers.TwoFluid.Vapor method*), 179  
JG() (*Solvers.Mixture.Mixture method*), 152  
JL() (*Solvers.Mixture.Mixture method*), 152

## K

KBLOCKRATIO (*Inputs.Model attribute*), 103  
KDIST() (*Solvers.Mixture.Mixture method*), 152  
KENH() (*Solvers.ThreeField.Drop method*), 167  
KF (*Inputs.FluidProperties attribute*), 108  
KG (*Inputs.FluidProperties attribute*), 109  
KIDX() (*Solvers.Mixture.Mixture method*), 152  
KL() (*Inputs.FluidProperties method*), 110  
KLOC (*Inputs.Model attribute*), 103  
KLOCZ() (*Solvers.Mixture.Mixture method*), 152  
KLOSS (*Inputs.Model attribute*), 103  
KLOSS() (*Solvers.Mixture.Mixture method*), 152  
KTRELAX (*Inputs.Model attribute*), 103  
KTRELAX() (*Solvers.Mixture.Mixture method*), 153  
KTUNING (*Inputs.Model attribute*), 103  
KV() (*Inputs.FluidProperties method*), 110

## L

L() (*Solvers.TwoFluid.Liquid method*), 190  
L() (*Solvers.TwoFluid.Vapor method*), 179  
legend() (*Solvers.SolverPlotter method*), 197  
LENGTH (*Inputs.Geometry attribute*), 111  
LHGR() (*Solvers.Mixture.Mixture method*), 153  
LHGR() (*Solvers.TwoFluid.Mixture method*), 183

Liquid (class in *Solvers.Mixture*), 158  
 Liquid (class in *Solvers.TwoFluid*), 185  
 liquid (*Solvers.Mixture.Mixture* attribute), 146  
 liquid (*Solvers.TwoFluid.TwoFluidSolver* attribute), 171  
 Liquid film, 47  
 Liquid() (*Solvers.Mixture.Liquid* method), 159  
 Liquid() (*Solvers.TwoFluid.Liquid* method), 185  
 liquidInit (*Solvers.TwoFluid.TwoFluidSolver* attribute), 171  
 listInputProperties() (*Inputs.Input* method), 117  
 LOCRELVEL (class in *InputEnums*), 99  
 LOCRELVEL (*Inputs.Model* attribute), 103  
 Log (class in *Session*), 119  
 log (*Session.Session* attribute), 118  
 Log() (*Session.Log* method), 119  
 log() (*Session.Log* method), 120  
 log() (*Solvers.AbstractSolver* method), 198  
 LOGFID (*Session.Log* attribute), 119  
 logFileName (*Session.Log* attribute), 119  
 logFilePath (*Session.Log* attribute), 120  
 LogMode (class in *Session*), 120  
 LOGMODE (*Session.Log* attribute), 119

## M

makeSessionDirectory() (*Session.Session* method), 119  
 MAXITER (*Inputs.Options* attribute), 114  
 MBASE() (*Solvers.FourField.Wave* method), 130  
 MCT() (*Solvers.Mixture.Mixture* method), 153  
 MDEP() (*Solvers.FourField.Base* method), 136  
 MDEP() (*Solvers.FourField.Wave* method), 131  
 MDEP() (*Solvers.ThreeField.Drop* method), 167  
 MDER (*Solvers.Mixture.Mixture* attribute), 145  
 memoizeFunction() (*Solvers.AbstractField* method), 199  
 MENT() (*Solvers.AbstractFilm* method), 195  
 MENT() (*Solvers.FourField.Base* method), 136  
 MENT() (*Solvers.FourField.Wave* method), 131  
 MEVAP (*Solvers.FourField.Film* attribute), 125  
 MEVAP (*Solvers.ThreeField.Film* attribute), 169  
 MEVAP() (*Solvers.FourField.Base* method), 136  
 MEVAP() (*Solvers.FourField.Wave* method), 131  
 MFBT (class in *InputEnums*), 92  
 MFBT (*Inputs.Model* attribute), 103  
 mfbt (*Solvers.Mixture.Mixture* attribute), 146  
 MFBT() (*Solvers.Mixture.Mixture* method), 153  
 MFLOW (*Inputs.BoundaryConditions* attribute), 112  
 MFLUX() (*Solvers.Mixture.Liquid* method), 159  
 MFLUX() (*Solvers.Mixture.Mixture* method), 153  
 MFLUX() (*Solvers.Mixture.Vapor* method), 143  
 MFLUX() (*Solvers.TwoFluid.Mixture* method), 184  
 MINT() (*Solvers.Mixture.Mixture* method), 153  
 MINT() (*Solvers.TwoFluid.Liquid* method), 190  
 MINT() (*Solvers.TwoFluid.Vapor* method), 179  
 MINTCOND() (*Solvers.Mixture.Mixture* method), 154  
 MINTCOND() (*Solvers.TwoFluid.Liquid* method), 191  
 MINTCOND() (*Solvers.TwoFluid.Vapor* method), 179

`MINTEVAP()` (*Solvers.Mixture.Mixture method*), 154  
`MINTEVAP()` (*Solvers.TwoFluid.Liquid method*), 191  
`MINTEVAP()` (*Solvers.TwoFluid.Vapor method*), 179  
`mix` (*Solvers.AbstractFilm attribute*), 194  
`mix` (*Solvers.ThreeField.Drop attribute*), 165  
`mix` (*Solvers.TwoFluid.Liquid attribute*), 186  
`mix` (*Solvers.TwoFluid.Vapor attribute*), 175  
`mixSolver` (*Solvers.ThreeField.ThreeFieldSolver attribute*), 162  
`mixSolver` (*Solvers.TwoFluid.TwoFluidSolver attribute*), 171  
`Mixture` (*class in Solvers.Mixture*), 144  
`Mixture` (*class in Solvers.TwoFluid*), 182  
`mixture` (*Solvers.Mixture.MixtureSolver attribute*), 139  
`Mixture model`, 47  
`Mixture()` (*Solvers.Mixture.Mixture method*), 144  
`Mixture()` (*Solvers.TwoFluid.Mixture method*), 183  
`mixtureInit` (*Solvers.Mixture.MixtureSolver attribute*), 139  
`MixtureSolver` (*class in Solvers.Mixture*), 138  
`MixtureSolver()` (*Solvers.Mixture.MixtureSolver method*), 138  
`Model` (*class in Inputs*), 101  
`model` (*Inputs.InputSet attribute*), 107  
`Model()` (*Inputs.Model method*), 101  
`MOECKCOEF` (*Inputs.Model attribute*), 103  
`MOMENTBASE` (*class in InputEnums*), 95  
`MOMENTBASE` (*Inputs.Model attribute*), 103  
`MOMENTDROP` (*class in InputEnums*), 95  
`MOMENTDROP` (*Inputs.Model attribute*), 104  
`MOMENTFILM` (*class in InputEnums*), 98  
`MOMENTFILM` (*Inputs.Model attribute*), 104  
`MOMENTGAS` (*class in InputEnums*), 100  
`MOMENTGAS` (*Inputs.Model attribute*), 104  
`MOMENTLIQUID` (*class in InputEnums*), 100  
`MOMENTLIQUID` (*Inputs.Model attribute*), 104  
`MOMENTWAVE` (*class in InputEnums*), 94  
`MOMENTWAVE` (*Inputs.Model attribute*), 104  
`MRM` (*Solvers.Mixture.Mixture attribute*), 145  
`MRMTIMEINT` (*class in InputEnums*), 98  
`MRMTIMEINT` (*Inputs.Options attribute*), 114  
`MTOT()` (*Solvers.AbstractFilm method*), 195  
`MTOT()` (*Solvers.FourField.Base method*), 136  
`MTOT()` (*Solvers.FourField.Wave method*), 131  
`MTOT()` (*Solvers.Mixture.Mixture method*), 154  
`MTOT()` (*Solvers.TwoFluid.Liquid method*), 191  
`MTOT()` (*Solvers.TwoFluid.Vapor method*), 180  
`MTRANSV()` (*Solvers.Mixture.Mixture method*), 154  
`MTURB()` (*Solvers.FourField.Base method*), 136  
`MTURB()` (*Solvers.FourField.Wave method*), 131  
`MU()` (*Solvers.Mixture.Mixture method*), 154  
`MUF` (*Inputs.FluidProperties attribute*), 109  
`MUG` (*Inputs.FluidProperties attribute*), 109  
`MUL()` (*Inputs.FluidProperties method*), 110  
`Multi-field model`, 48  
`Multi-wall channel`, 48  
`MUV()` (*Inputs.FluidProperties method*), 110  
`MWALEVAP()` (*Solvers.Mixture.Mixture method*), 154

MWALEVAP() (*Solvers.TwoFluid.Liquid method*), 191  
 MWALEVAP() (*Solvers.TwoFluid.Vapor method*), 180  
 MWAVE() (*Solvers.FourField.Base method*), 136

## N

N() (*Solvers.FourField.Wave method*), 131  
 name (*Session.Session attribute*), 118  
 NEARWALL (*Solvers.Mixture.Mixture attribute*), 145  
 NEARWALLEQOAF (*Inputs.Model attribute*), 104  
 NEARWALLH (*class in InputEnums*), 92  
 NEARWALLH (*Inputs.Model attribute*), 104  
 NEARWALLHRATIO (*Inputs.Model attribute*), 104  
 NEARWALLRATIO (*Inputs.Model attribute*), 104  
 NEARWALLRELAX (*class in InputEnums*), 93  
 NEARWALLRELAX (*Inputs.Model attribute*), 104  
 NEARWALLRELAXCOEF (*Inputs.Model attribute*), 104  
 NEARWALLRELAXT (*Inputs.Model attribute*), 104  
 NEARWALLRELAXX (*Inputs.Model attribute*), 104  
 newTile() (*Solvers.SolverPlotter method*), 197  
 NNODES (*Inputs.Model attribute*), 104  
 Non-equilibrium, 48  
 NTIME (*Solvers.AbstractField attribute*), 199  
 NTIME (*Solvers.FourField.Base attribute*), 133  
 NTIME (*Solvers.FourField.Film attribute*), 125  
 NTIME (*Solvers.FourField.Wave attribute*), 127  
 NTIME (*Solvers.Mixture.Mixture attribute*), 145  
 NTIME (*Solvers.Mixture.MixtureSolver attribute*), 139  
 NTIME (*Solvers.ThreeField.Drop attribute*), 165  
 NTIME (*Solvers.ThreeField.Film attribute*), 169  
 NTIME (*Solvers.ThreeField.ThreeFieldSolver attribute*), 161  
 NTIME (*Solvers.TwoFluid.Liquid attribute*), 186  
 NTIME (*Solvers.TwoFluid.TwoFluidSolver attribute*), 171  
 NTIME (*Solvers.TwoFluid.Vapor attribute*), 174  
 NU() (*Solvers.Mixture.Liquid method*), 159  
 NU() (*Solvers.Mixture.Vapor method*), 143  
 NWALL() (*Inputs.Geometry method*), 111  
 NZ (*Solvers.AbstractField attribute*), 199  
 NZ (*Solvers.FourField.Base attribute*), 133  
 NZ (*Solvers.FourField.Film attribute*), 125  
 NZ (*Solvers.FourField.Wave attribute*), 127  
 NZ (*Solvers.Mixture.Mixture attribute*), 145  
 NZ (*Solvers.Mixture.MixtureSolver attribute*), 139  
 NZ (*Solvers.ThreeField.Drop attribute*), 165  
 NZ (*Solvers.ThreeField.Film attribute*), 169  
 NZ (*Solvers.ThreeField.ThreeFieldSolver attribute*), 161  
 NZ (*Solvers.TwoFluid.Liquid attribute*), 186  
 NZ (*Solvers.TwoFluid.TwoFluidSolver attribute*), 171  
 NZ (*Solvers.TwoFluid.Vapor attribute*), 174

## O

OAF (*class in InputEnums*), 98  
 OAF (*Inputs.Model attribute*), 104  
 OAFBASERATIO (*Inputs.Model attribute*), 104  
 OAFDROPRATIO (*Inputs.Model attribute*), 104

OAFENTRAINED (*class in InputEnums*), 93  
OAFENTRAINED (*Inputs.Model attribute*), 104  
OAFEQUILMAXITER (*Inputs.Options attribute*), 114  
OAFEQUILTOL (*Inputs.Options attribute*), 114  
OAFFILMSPLIT (*class in InputEnums*), 98  
OAFFILMSPLIT (*Inputs.Model attribute*), 104  
OAFIDX() (*Solvers.Mixture.Mixture method*), 155  
OAFTRANSITION (*Inputs.Model attribute*), 105  
OAFWL() (*Solvers.Mixture.Mixture method*), 155  
OAFX() (*Solvers.Mixture.Mixture method*), 155  
OAFX() (*Solvers.TwoFluid.Mixture method*), 184  
OAFZ() (*Solvers.Mixture.Mixture method*), 155  
OKAWACOEFS (*Inputs.Model attribute*), 105  
OKAWACOEFS() (*Solvers.AbstractFilm method*), 195  
OKAWAMENT() (*Solvers.AbstractFilm method*), 195  
One-dimensional model, 48  
openLog() (*Session.Log method*), 120  
Options (*class in Inputs*), 113  
options (*Inputs.InputSet attribute*), 107  
Options() (*Inputs.Options method*), 113  
overwriteFiles (*Session.Session attribute*), 118

## P

P (*Solvers.Mixture.Mixture attribute*), 145  
parentDir (*Session.Session attribute*), 119  
PCRIT (*Inputs.FluidProperties attribute*), 109  
PERIM (*Inputs.Geometry attribute*), 111  
PERIOD() (*Solvers.FourField.Wave method*), 132  
Phase, 48  
Phase change, 48  
PHI2F() (*Solvers.Mixture.Mixture method*), 155  
PHI2K() (*Solvers.Mixture.Mixture method*), 155  
plot() (*Inputs.BoundaryConditions method*), 113  
plot() (*Inputs.FluidProperties method*), 110  
plotK() (*Solvers.SolverPlotter method*), 197  
plotOAF() (*Solvers.SolverPlotter method*), 197  
plott() (*Solvers.FourField.FourFieldSolver method*), 123  
plott() (*Solvers.Mixture.MixtureSolver method*), 140  
plott() (*Solvers.ThreeField.ThreeFieldSolver method*), 163  
plott() (*Solvers.TwoFluid.TwoFluidSolver method*), 172  
plotz() (*Solvers.FourField.FourFieldSolver method*), 123  
plotz() (*Solvers.Mixture.MixtureSolver method*), 141  
plotz() (*Solvers.SolverPlotter method*), 197  
plotz() (*Solvers.ThreeField.ThreeFieldSolver method*), 163  
plotz() (*Solvers.TwoFluid.TwoFluidSolver method*), 172  
plotzt() (*Solvers.AbstractField method*), 199  
plotzt() (*Solvers.FourField.FourFieldSolver method*), 123  
plotzt() (*Solvers.Mixture.MixtureSolver method*), 141  
plotzt() (*Solvers.ThreeField.ThreeFieldSolver method*), 163  
plotzt() (*Solvers.TwoFluid.TwoFluidSolver method*), 173  
POSFILM (*Inputs.Model attribute*), 105  
POWER (*Inputs.BoundaryConditions attribute*), 112  
PRANDTLF (*Inputs.FluidProperties attribute*), 109  
PRANDTLG (*Inputs.FluidProperties attribute*), 109

PRANDTLTLL() (*Inputs.FluidProperties method*), 110  
 PRANDTLV() (*Inputs.FluidProperties method*), 110  
 PRESSURE (*Inputs.BoundaryConditions attribute*), 112  
 PRESSURE (*Inputs.FluidProperties attribute*), 109  
 PROPERTIES (*Inputs.FluidProperties attribute*), 109  
 PROPERTIES (*Inputs.Model attribute*), 105  
 PTRIPLE (*Inputs.FluidProperties attribute*), 109

## Q

Q() (*Solvers.Mixture.Liquid method*), 159  
 Q() (*Solvers.Mixture.Vapor method*), 144  
 QK (*Inputs.FluidProperties attribute*), 109

## R

RANZMARSHALLCST (*Inputs.Model attribute*), 105  
 RANZMARSHALLVCST (*Inputs.Model attribute*), 105  
 RE() (*Solvers.AbstractFilm method*), 195  
 RE() (*Solvers.Mixture.Liquid method*), 160  
 RE() (*Solvers.Mixture.Mixture method*), 156  
 RE() (*Solvers.Mixture.Vapor method*), 144  
 RE() (*Solvers.ThreeField.Drop method*), 167  
 RE() (*Solvers.TwoFluid.Liquid method*), 191  
 RE() (*Solvers.TwoFluid.Vapor method*), 180  
 REGIMES (*class in Solvers.TwoFluid*), 173  
 REL() (*Solvers.Mixture.Mixture method*), 156  
 REL() (*Solvers.TwoFluid.Vapor method*), 180  
 Relaxation model, 48  
 Relaxation time, 48  
 RELAXCONDCOEFF (*Inputs.Model attribute*), 105  
 RELAXCONDFO (*Inputs.Model attribute*), 105  
 RELAXCONDFOCOEFF (*Inputs.Model attribute*), 105  
 RELAXCONDT (*Inputs.Model attribute*), 105  
 RELAXEVAPCOEFF (*Inputs.Model attribute*), 105  
 RELAXEVAPFO (*Inputs.Model attribute*), 105  
 RELAXEVAPFOCOEFF (*Inputs.Model attribute*), 105  
 RELAXEVAPT (*Inputs.Model attribute*), 105  
 RELAXFW (*Inputs.Options attribute*), 114  
 RELAXHL (*Inputs.Options attribute*), 115  
 RELAXHM (*Inputs.Options attribute*), 115  
 RELAXHV (*Inputs.Options attribute*), 115  
 RELAXPM (*Inputs.Options attribute*), 115  
 RELAXT() (*Solvers.Mixture.Mixture method*), 156  
 RELAXTB (*Inputs.Model attribute*), 105  
 RELAXTCOND() (*Solvers.Mixture.Mixture method*), 156  
 RELAXTCOND() (*Solvers.TwoFluid.Mixture method*), 184  
 RELAXTEVAP() (*Solvers.Mixture.Mixture method*), 156  
 RELAXTEVAP() (*Solvers.TwoFluid.Mixture method*), 184  
 RELAXTW (*Inputs.Model attribute*), 105  
 RELAXUB (*Inputs.Options attribute*), 115  
 RELAXUD (*Inputs.Options attribute*), 115  
 RELAXUF (*Inputs.Options attribute*), 115  
 RELAXUL (*Inputs.Options attribute*), 115  
 RELAXUV (*Inputs.Options attribute*), 115  
 RELAXUW (*Inputs.Options attribute*), 115

RELAXWB (*Inputs.Options attribute*), 115  
RELAXWF (*Inputs.Options attribute*), 115  
RELAXWL (*Inputs.Options attribute*), 115  
RELAXWM (*Inputs.Options attribute*), 115  
RELAXWV (*Inputs.Options attribute*), 115  
RELAXWW (*Inputs.Options attribute*), 115  
RELAXX (*Inputs.Model attribute*), 105  
RELVELCST (*Inputs.Model attribute*), 105  
resizeFigure() (*Solvers.SolverPlotter method*), 197  
REV() (*Solvers.FourField.Wave method*), 132  
REV() (*Solvers.ThreeField.Drop method*), 167  
REV() (*Solvers.TwoFluid.Liquid method*), 191  
RHO() (*Solvers.Mixture.Mixture method*), 157  
RHO() (*Solvers.TwoFluid.Mixture method*), 184  
RHOF (*Inputs.FluidProperties attribute*), 109  
RHOG (*Inputs.FluidProperties attribute*), 109  
RHOL() (*Inputs.FluidProperties method*), 110  
RHOV() (*Inputs.FluidProperties method*), 110  
RWALL() (*Inputs.Geometry method*), 111

## S

S() (*Solvers.TwoFluid.Liquid method*), 192  
S() (*Solvers.TwoFluid.Vapor method*), 180  
save() (*Solvers.AbstractSolver method*), 198  
Session (*class in Session*), 118  
session (*Inputs.InputSet attribute*), 107  
Session (*module*), 118  
Session() (*Session.Session method*), 118  
setupLog() (*Session.Session method*), 119  
setZs() (*Solvers.SolverPlotter method*), 197  
SHAPEFACTOR() (*Solvers.FourField.Wave method*), 132  
SHAPEFACTORCOEF (*Inputs.Model attribute*), 105  
showWarnings (*Session.Log attribute*), 120  
showWarnings (*Session.Session attribute*), 119  
SIGMA (*Inputs.FluidProperties attribute*), 109  
SLIP (*Inputs.Model attribute*), 106  
solve() (*Solvers.FourField.FourFieldSolver method*), 124  
solve() (*Solvers.Mixture.MixtureSolver method*), 142  
solve() (*Solvers.ThreeField.ThreeFieldSolver method*), 164  
solve() (*Solvers.TwoFluid.TwoFluidSolver method*), 173  
Solver, [48](#)  
SOLVERDEPENDENTPROPS (*Inputs.Model attribute*), 106  
solverName() (*Solvers.AbstractSolver method*), 198  
SolverPlotter (*class in Solvers*), 196  
SolverPlotter() (*Solvers.SolverPlotter method*), 196  
Solvers (*module*), [194](#)  
Solvers.FourField (*module*), 124  
Solvers.Mixture (*module*), 142  
Solvers.ThreeField (*module*), 164  
Solvers.TwoFluid (*module*), 173  
SolverState (*class in Solvers*), 200  
Source term, [48](#)  
SPACING() (*Solvers.FourField.Wave method*), 132  
SPHTM (*class in InputEnums*), 96

SPHTM (*Inputs.Model* attribute), 106  
 SPMTM (*class in InputEnums*), 92  
 SPMTM (*Inputs.Model* attribute), 106  
 SSCONVFW (*Inputs.Options* attribute), 115  
 SSCONVH (*Inputs.Options* attribute), 115  
 SSCONVP (*Inputs.Options* attribute), 115  
 SSCONVU (*Inputs.Options* attribute), 116  
 SSCONVUD (*Inputs.Options* attribute), 116  
 SSCONVUF (*Inputs.Options* attribute), 116  
 SSCONVW (*Inputs.Options* attribute), 116  
 SSCONVWF (*Inputs.Options* attribute), 116  
 SSMAXITER (*Inputs.Options* attribute), 116  
 SSTSTEP (*Inputs.Options* attribute), 116  
 STATE (*Solvers.AbstractSolver* attribute), 198  
 Straight channel, 48  
 struct() (*Solvers.AbstractField* method), 199

## T

T() (*Inputs.FluidProperties* method), 110  
 T() (*Solvers.Mixture.Liquid* method), 160  
 T() (*Solvers.Mixture.Mixture* method), 157  
 T() (*Solvers.Mixture.Vapor* method), 144  
 T() (*Solvers.TwoFluid.Liquid* method), 192  
 T() (*Solvers.TwoFluid.Vapor* method), 180  
 TAUW() (*Solvers.Mixture.Mixture* method), 157  
 TAUW() (*Solvers.Mixture.Vapor* method), 144  
 TAUW() (*Solvers.TwoFluid.Liquid* method), 192  
 TAUW() (*Solvers.TwoFluid.Mixture* method), 184  
 TAUW() (*Solvers.TwoFluid.Vapor* method), 181  
 TBASE() (*Solvers.FourField.Base* method), 137  
 TDRY() (*Solvers.FourField.Base* method), 137  
 Thermal equilibrium, 48  
 Thermal non-equilibrium, 48  
 THERMALNONEQ (*class in InputEnums*), 100  
 THERMALNONEQ (*Inputs.Model* attribute), 106  
 THERMALRELAX (*class in InputEnums*), 99  
 THERMALRELAX (*Inputs.Model* attribute), 106  
 THICK() (*Solvers.AbstractFilm* method), 195  
 THICK() (*Solvers.FourField.Wave* method), 132  
 THICKMIN() (*Solvers.FourField.Base* method), 137  
 THINFILMFRIC (*class in InputEnums*), 94  
 THINFILMFRIC (*Inputs.Model* attribute), 106  
 THINFILMTHICK (*Inputs.Model* attribute), 106  
 THINWAVETHICK (*Inputs.Model* attribute), 106  
 Three-field model, 48  
 ThreeFieldSolver (*class in Solvers.ThreeField*), 160  
 ThreeFieldSolver() (*Solvers.ThreeField.ThreeFieldSolver* method), 161  
 TIDX (*Solvers.AbstractField* attribute), 199  
 TIDX (*Solvers.FourField.Base* attribute), 133  
 TIDX (*Solvers.FourField.Film* attribute), 125  
 TIDX (*Solvers.FourField.Wave* attribute), 127  
 TIDX (*Solvers.Mixture.Mixture* attribute), 145  
 TIDX (*Solvers.ThreeField.Drop* attribute), 165  
 TIDX (*Solvers.ThreeField.Film* attribute), 169

TIDX (*Solvers.TwoFluid.Liquid attribute*), 186  
TIDX (*Solvers.TwoFluid.Vapor attribute*), 174  
TIME (*Inputs.BoundaryConditions attribute*), 112  
TIME (*Solvers.AbstractField attribute*), 199  
TIME (*Solvers.FourField.Base attribute*), 133  
TIME (*Solvers.FourField.Film attribute*), 125  
TIME (*Solvers.FourField.Wave attribute*), 127  
TIME (*Solvers.Mixture.Mixture attribute*), 145  
TIME (*Solvers.Mixture.MixtureSolver attribute*), 139  
TIME (*Solvers.ThreeField.Drop attribute*), 165  
TIME (*Solvers.ThreeField.Film attribute*), 169  
TIME (*Solvers.ThreeField.ThreeFieldSolver attribute*), 161  
TIME (*Solvers.TwoFluid.Liquid attribute*), 186  
TIME (*Solvers.TwoFluid.TwoFluidSolver attribute*), 171  
TIME (*Solvers.TwoFluid.Vapor attribute*), 174  
TIME() (*Solvers.Mixture.Liquid method*), 160  
TIME() (*Solvers.Mixture.Vapor method*), 144  
TIME() (*Solvers.TwoFluid.Mixture method*), 184  
TIMEINTERP (*Inputs.Options attribute*), 116  
timeInterpolate() (*Solvers.Mixture.MixtureSolver method*), 142  
TIN() (*Inputs.BoundaryConditions method*), 113  
Title (*Solvers.SolverPlotter attribute*), 196  
TPFM (*class in InputEnums*), 94  
TPFM (*Inputs.Model attribute*), 106  
TPHTM (*class in InputEnums*), 95  
TPHTM (*Inputs.Model attribute*), 106  
TPKM (*class in InputEnums*), 97  
TPKM (*Inputs.Model attribute*), 106  
transient() (*Inputs.FluidProperties method*), 110  
transient() (*Solvers.AbstractField method*), 199  
Ts (*Solvers.SolverPlotter attribute*), 196  
TSAT (*Inputs.FluidProperties attribute*), 109  
TSAT() (*Inputs.BoundaryConditions method*), 113  
TSTEP (*Inputs.Options attribute*), 116  
TWALL() (*Solvers.Mixture.Mixture method*), 157  
Two-fluid model, [49](#)  
Two-phase flow, [49](#)  
TwoFluidSolver (*class in Solvers.TwoFluid*), 170  
TwoFluidSolver() (*Solvers.TwoFluid.TwoFluidSolver method*), 170

## U

U (*Solvers.FourField.Base attribute*), 133  
U (*Solvers.FourField.Film attribute*), 125  
U (*Solvers.FourField.Wave attribute*), 127  
U (*Solvers.ThreeField.Drop attribute*), 165  
U (*Solvers.ThreeField.Film attribute*), 169  
U (*Solvers.TwoFluid.Liquid attribute*), 186  
U (*Solvers.TwoFluid.Vapor attribute*), 174  
U() (*Solvers.Mixture.Liquid method*), 160  
U() (*Solvers.Mixture.Mixture method*), 157  
U() (*Solvers.Mixture.Vapor method*), 144  
U() (*Solvers.TwoFluid.Mixture method*), 184  
UALGEBR() (*Solvers.AbstractFilm method*), 196  
UALGEBR() (*Solvers.ThreeField.Drop method*), 168

UC (*Inputs.FluidProperties attribute*), 109  
 UDEQUILMAXITER (*Inputs.Options attribute*), 116  
 UDEQUILTOL (*Inputs.Options attribute*), 116  
 UEQUIL() (*Solvers.AbstractFilm method*), 196  
 UEQUIL() (*Solvers.ThreeField.Drop method*), 168  
 UEQUILS() (*Solvers.AbstractFilm method*), 196  
 UFEQUILMAXITER (*Inputs.Options attribute*), 116  
 UFEQUILTOL (*Inputs.Options attribute*), 116  
 UP() (*Solvers.Mixture.Mixture method*), 157  
 USLIP() (*Solvers.ThreeField.Drop method*), 168  
 USLIP() (*Solvers.TwoFluid.Liquid method*), 192  
 USLIP() (*Solvers.TwoFluid.Vapor method*), 181  
 UWALL() (*Solvers.AbstractFilm method*), 196

## V

validateFunctionHandleInput() (*Inputs.Input static method*), 117  
 validateInputEntry() (*Inputs.Input method*), 117  
 Vapor (*class in Solvers.Mixture*), 142  
 Vapor (*class in Solvers.TwoFluid*), 174  
 vapor (*Solvers.Mixture.Mixture attribute*), 146  
 vapor (*Solvers.TwoFluid.TwoFluidSolver attribute*), 171  
 Vapor() (*Solvers.Mixture.Vapor method*), 143  
 Vapor() (*Solvers.TwoFluid.Vapor method*), 174  
 VAPORFRIC (*class in InputEnums*), 99  
 VAPORFRIC (*Inputs.Model attribute*), 106  
 VAPORFRICCST (*Inputs.Model attribute*), 106  
 vaporInit (*Solvers.TwoFluid.TwoFluidSolver attribute*), 171  
 VF() (*Solvers.Mixture.Liquid method*), 160  
 VF() (*Solvers.Mixture.Mixture method*), 158  
 VF() (*Solvers.Mixture.Vapor method*), 144  
 VF() (*Solvers.TwoFluid.Liquid method*), 192  
 VF() (*Solvers.TwoFluid.Mixture method*), 184  
 VF() (*Solvers.TwoFluid.Vapor method*), 181  
 VISC() (*Solvers.TwoFluid.Liquid method*), 192  
 VISC() (*Solvers.TwoFluid.Vapor method*), 181  
 VOID (*class in InputEnums*), 98  
 VOID (*Inputs.Model attribute*), 106  
 VOLUME() (*Solvers.ThreeField.Drop method*), 168  
 VR() (*Solvers.ThreeField.Drop method*), 168  
 VR() (*Solvers.TwoFluid.Liquid method*), 193

## W

W (*Solvers.FourField.Base attribute*), 133  
 W (*Solvers.FourField.Film attribute*), 126  
 W (*Solvers.FourField.Wave attribute*), 127  
 W (*Solvers.Mixture.Mixture attribute*), 145  
 W (*Solvers.ThreeField.Drop attribute*), 165  
 W (*Solvers.ThreeField.Film attribute*), 169  
 W (*Solvers.TwoFluid.Liquid attribute*), 186  
 W (*Solvers.TwoFluid.Vapor attribute*), 175  
 W() (*Solvers.Mixture.Liquid method*), 160  
 W() (*Solvers.Mixture.Vapor method*), 144  
 W() (*Solvers.TwoFluid.Mixture method*), 185  
 WALEVAPRATIO() (*Solvers.Mixture.Mixture method*), 158

WALEVAPRATIO() (*Solvers.TwoFluid.Liquid method*), 193  
WALEVAPRATIO() (*Solvers.TwoFluid.Mixture method*), 185  
WALEVAPRATIO() (*Solvers.TwoFluid.Vapor method*), 181  
Wall boiling, 49  
Wall friction, 49  
Wall heat flux, 49  
WallIdx (*Solvers.SolverPlotter attribute*), 196  
warning() (*Session.Log method*), 120  
warning() (*Solvers.AbstractSolver method*), 198  
warnings (*Inputs.Input attribute*), 117  
Wave (*class in Solvers.FourField*), 127  
wave (*Solvers.FourField.Film attribute*), 126  
Wave() (*Solvers.FourField.Wave method*), 127  
WAVEBASEINT (*class in InputEnums*), 92  
WAVEBASEINT (*Inputs.Model attribute*), 106  
WAVEDRAGCOEF (*Inputs.Model attribute*), 106  
WAVEFREQUENCY (*class in InputEnums*), 94  
WAVEFREQUENCY (*Inputs.Model attribute*), 106  
WAVEMIXCOEF (*Inputs.Model attribute*), 106  
WBOILINGN (*Inputs.Model attribute*), 107  
WBOILINGXSAT (*Inputs.Model attribute*), 107  
WBOILINGXSUB (*Inputs.Model attribute*), 107  
WIDTH() (*Solvers.FourField.Wave method*), 132  
WL() (*Solvers.AbstractFilm method*), 196  
WL() (*Solvers.FourField.Wave method*), 132  
WMESH (*Inputs.BoundaryConditions attribute*), 112  
WMIN() (*Solvers.FourField.Base method*), 137  
WMINL() (*Solvers.FourField.Base method*), 137  
WPOWER (*Inputs.BoundaryConditions attribute*), 112  
writeInputFile() (*Inputs.BoundaryConditions static method*), 113  
writeInputFile() (*Inputs.Geometry static method*), 111  
writeInputFile() (*Inputs.Input static method*), 117  
writeInputFile() (*Inputs.Model static method*), 107  
writeInputFile() (*Inputs.Options static method*), 116  
WWALL() (*Solvers.Mixture.Mixture method*), 158

## X

X() (*Solvers.AbstractFilm method*), 196  
X() (*Solvers.Mixture.Liquid method*), 160  
X() (*Solvers.Mixture.Mixture method*), 158  
X() (*Solvers.Mixture.Vapor method*), 144  
X() (*Solvers.ThreeField.Drop method*), 168  
X() (*Solvers.TwoFluid.Liquid method*), 193  
X() (*Solvers.TwoFluid.Vapor method*), 181  
XEQ() (*Solvers.Mixture.Mixture method*), 158  
XEQ() (*Solvers.TwoFluid.Mixture method*), 185  
XIN() (*Inputs.BoundaryConditions method*), 113  
xlim() (*Solvers.SolverPlotter method*), 197  
XTR\_ANN() (*Solvers.TwoFluid.Liquid method*), 193  
XTR\_ANN() (*Solvers.TwoFluid.Vapor method*), 182  
XTR\_CBT() (*Solvers.TwoFluid.Vapor method*), 182  
XTR\_ITM() (*Solvers.TwoFluid.Liquid method*), 193  
XTR\_ITM() (*Solvers.TwoFluid.Vapor method*), 182  
XTR\_SAT() (*Solvers.TwoFluid.Liquid method*), 193

XTR\_SAT() (*Solvers.TwoFluid.Vapor method*), [182](#)

XTR\_SUB() (*Solvers.TwoFluid.Liquid method*), [193](#)

XTR\_SUB() (*Solvers.TwoFluid.Vapor method*), [182](#)

## Y

ylabels() (*Solvers.SolverPlotter method*), [197](#)

ylim() (*Solvers.SolverPlotter method*), [197](#)

YPLUS2THICK() (*Solvers.AbstractFilm method*), [196](#)

## Z

Z (*Solvers.AbstractField attribute*), [199](#)

Z (*Solvers.FourField.Base attribute*), [134](#)

Z (*Solvers.FourField.Film attribute*), [126](#)

Z (*Solvers.FourField.Wave attribute*), [128](#)

Z (*Solvers.Mixture.Mixture attribute*), [145](#)

Z (*Solvers.Mixture.MixtureSolver attribute*), [139](#)

Z (*Solvers.ThreeField.Drop attribute*), [165](#)

Z (*Solvers.ThreeField.Film attribute*), [169](#)

Z (*Solvers.ThreeField.ThreeFieldSolver attribute*), [161](#)

Z (*Solvers.TwoFluid.Liquid attribute*), [186](#)

Z (*Solvers.TwoFluid.TwoFluidSolver attribute*), [171](#)

Z (*Solvers.TwoFluid.Vapor attribute*), [175](#)

Z() (*Solvers.Mixture.Liquid method*), [160](#)

Z() (*Solvers.Mixture.Vapor method*), [144](#)

Z() (*Solvers.TwoFluid.Mixture method*), [185](#)

Zs (*Solvers.SolverPlotter attribute*), [197](#)